

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

Поняття алгоритму. Визначення його часової та просторової (за обсягом пам'яті) складності

- Алгоритм – послідовність, система, набір систематизованих правил виконання обчислювального процесу, що обов'язково призводить до розв'язання певного класу завдань після скінченного числа операцій.
- Алгоритм – це кінцевий набір правил, що визначає послідовність операцій для рішення конкретної множини задач і володіє п'ятьма важливими рисами: скінченність, визначеність, введення, виведення, ефективність.
- Часова складність алгоритму — характеристика продуктивності алгоритму, що визначається кількістю елементарних операцій, які потрібно виконати для реалізації алгоритму. При цьому вважають, що кожна елементарна операція виконується за однаковий час.
- Асимптотична оцінка дозволяє оцінити, наскільки швидко зростає час роботи алгоритму зі збільшенням розміру вхідних даних.
- Просторова складність визначає те, скільки додаткової пам'яті (оперативної) буде потрібно алгоритму під час виконання, коли збільшується розмір вхідних даних.

Поняття алгоритму. Визначення його часової та просторової (за обсягом пам'яті) складності

Приклади асимптотичної складності алгоритмів

Складність	Коментар	Приклади
$O(1)$	Сталий час роботи не залежно від розміру задачі	Пошук у хеш-таблиці
$O(\log \log n)$	Дуже повільне зростання необхідного часу	Очікуваний час роботи інтерполюючого пошуку n елементів
$O(\log n)$	Логарифмічне зростання — подвоєння розміру задачі збільшує час роботи на сталу величину	Швидке обчислення x^n ; двійковий пошук у відсортованому масиві з n елементів
$O(n)$	Лінійне зростання — подвоєння розміру задачі подвоїть і необхідний час	Додавання/віднімання чисел з n цифр; лінійний пошук в масиві з n елементів
$O(n \log n)$	Лінеаритмічне зростання — подвоєння розміру задачі збільшить необхідний час трохи більше ніж вдвічі	Сортування злиттям або купою масиву з n елементів.
$O(n^2)$	Квадратичне зростання — подвоєння розміру задачі вчетверо збільшує необхідний час	Елементарні алгоритми сортування масивів з n елементів; Лінійний пошук у квадратній матриці розмірності n .
$O(n^3)$	Кубічне зростання — подвоєння розміру задачі збільшує необхідний час у вісім разів	Звичайне множення матриць
$O(a^n)$	Експоненціальне зростання — збільшення розміру задачі на 1 призводить до a -кратного збільшення необхідного часу; подвоєння	Деякі задачі комівояжера; Алгоритми пошуку повним перебором

Стеки, вектори, множини, мультимножини, кортежі

- Вектор (одновимірний масив) представляє собою структуру даних з фіксованим числом елементів одного і того ж типу. Кожен елемент вектора має свій унікальний індекс. З фізичної точки зору елементи вектора розміщуються в пам'яті у підряд розміщених комірках пам'яті.
- Стек — це структура даних, в якій елементи додаються і видаляються у порядку «останнім прийшов, першим пішов» (Last in first out – LIFO).

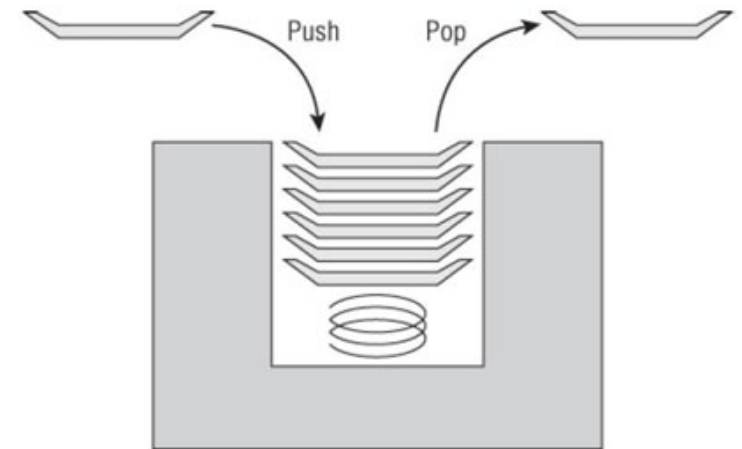
Операції:

- створення стека;
- включення елемента в стек;
- отримання елемента зі стека;
- видалення стека.

Множина – така структура, яка є набором даних одного і того ж типу, що не повторюються. Порядок слідування елементів множини не має принципового значення. Операції над множинами: перетин, об'єднання, різниця, симетрична різниця, перевірка входження елемента.

У мультимножині для кожного елемента запам'ятовується і саме входження, і кількість входжень.

Кортеж — впорядкований набір фіксованої множини.

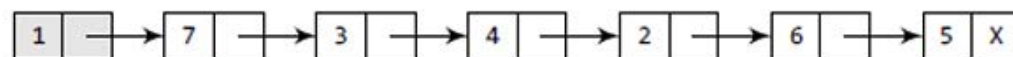


Списки

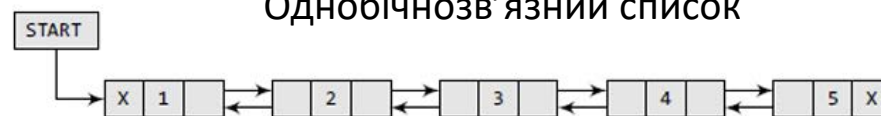
- Списки відрізняються від стеків, черг та деків тим, що мають можливість безліч разів проходити вздовж списку, отримувати доступ до будь-якого елемента, не змінюючи сам список. Зв'язний список не допускає прямого (випадкового) доступу до даних.

Операції над однозв'язним списком:

- створити список;
- визначення, чи порожній список;
- додати елемент у початок списку;
- взяти перший елемент списку;
- отримати хвіст списку.



Однобічнозв'язний список



Двобічнозв'язний список

СЛОВНИКИ

- Асоціативний масив або словник – це абстрактний тип даних, що дозволяє зберігати дані у вигляді набору пар ключ — значення та доступом до значень за їхніми ключами.

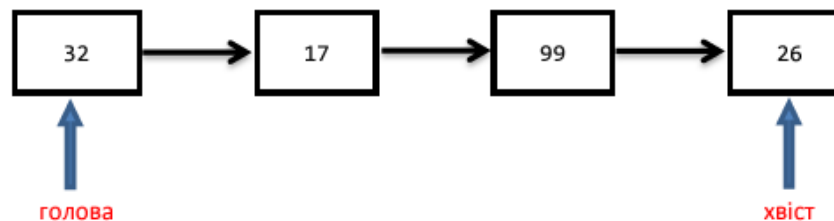
Основні операції:

- додавання нової пари (ключ, значення);
- пошук значень за ключем;
- видалення пари ключ — значення за ключем.

Хеш-таблиця – це структура даних, що реалізує інтерфейс асоціативного масиву, у якій додавання, пошук та видалення даних здійснюється за сталий час $O(1)$.

Хеш-функція – це функція, що перетворює кожний рядок у числове значення, що називається хеш-значенням.

Черги, черги з пріоритетами



- Черга— це змінюваний упорядкований (чи ні) набір елементів. Додавання елемента в чергу проводиться з одного кінця (хвоста черги, REAR), а вибірка - з іншого кінця (голови черги, FRONT) відповідно до правила "Першим прийшов - першим пішов" (FIFO: First Input - First Output).
- Часто використовуються черги з пріоритетами, в них більш пріоритетні елементи включаються ближче до голови черги, вибірка здійснюється, як правило, з голови черги.
- Основними операціями над чергою є: створення і видалення черги; включення в чергу нового елемента; вибірка елемента з черги.

Базові алгоритми та їх складність

- Класифікація за часом виконання (змінюється, якщо окремо розглядати найгірший, середній випадок):
- За час $O(n^2)$:
 - сортування вибором;
 - сортування вставкою;
 - сортування обміном
- За час $O(n \log n)$:
 - пірамідальне сортування;
 - швидке сортування;
 - сортування злиттям
- За час $O(n)$ з використанням додаткової інформації про елементи:
 - сортування підрахунком
- За час $O(n \log_2 n)$:
 - сортування деревом;
 - сортування Шелла.

Алгоритми сортування, пошуку

- Для знаходження інформації в неупорядкованому масиві потрібен послідовний пошук, що починається з першого елементу й закінчується при виявленні необхідних даних або при досягненні кінця масиву. Послідовний пошук у середньому переглядає $(n / 2)$ елементів. У найкращому разі він перевіряє тільки один елемент, а в найгіршому – n .
- Сортування вибором:
 - 1) знаходимо найменший елемент у масиві;
 - 2) міняємо його місцями з елементом, що знаходиться на першому місці;
 - 3) повторюємо процес із другої позиції й знайдений найменший елемент обмінюємо із другим елементом і так далі поки весь масив не буде відсортований.
- Сортування вставкою:
 - 1) зліва направо проходимо масив, порівнюючи сусідні елементи, поки не знайдемо елемент, що розташований не в порядку сортування;
 - 2) обмінюємо цей елемент з елементами розташованими ліворуч від нього, поки він не займе потрібну позицію;
 - 3) повторюємо перші дві дії, поки масив не буде відсортовано.
- Бульбашкове сортування (сортування обміном): здійснюється кілька проходів по списку, під час кожного з яких порівнюють пари сусідніх елементів. Якщо елементи стоять не правильно, вони міняються місцями.

Алгоритми сортування

- Швидке сортування: На кожній ітерації алгоритму обирається еталонний елемент списку, що називається опорним елементом (який як правило вибирається як центральний елемент списку). Далі вихідний список поділяється на дві частини, одна з яких містить всі елементи які менші за опорний, а інша – елементи що більші. Це досягається з допомогою перестановок елементів, так, щоб зліва від опорного елемента знаходилися елементи списку менші за нього, а справа – більші. Після чого правий маркер починає рух вліво доки не зустріне число, що є меншим або рівним за опорний елемент. Після того, як обидва маркери зупиняться – елементи на які вони вказують міняються місцями. Далі, лівий маркер починає знову рух вправо, поки не знайде елемент більший або рівний за опорний, а правий – вліво поки не знайде елемент менший або рівний за опорний. Міняємо їх місцями і знову продовжуємо процедуру змістивши лівий маркер на одну позицію вправо, а правий – на одну позицію вліво. Рух маркерів, разом з описаною вище процедурою, відбувається доти, доки лівий і правий маркери не почнуть вказувати на один і той же елемент або лівий маркер не займе позицію праворуч від правого.
- Сортування Шелла: виконується порівняння ключів, віддалених один від одного на деяку відстань d . Початковий розмір d звичайно вибирається рівним половині загального розміру сортуваної послідовності. Виконується сортування вставками з інтервалом порівняння d . Потім величина d зменшується удвічі і знов виконується сортування вставками, далі d зменшується ще удвічі і т.д. Останнє сортування вставками виконується при $d=1$.
- Сортування деревом: Побудувати двійкове дерево, вставляючи елементи вхідного масиву в двійкове дерево пошуку. Виконати обхід дерева, щоб отримати результуючий масив з впорядкованими елементами.

Алгоритми на графах

- Пошук у глибину: При пошуку в глибину відвідується перша вершина, потім необхідно йти уздовж ребер графа, до потрапляння в глухий кут. Вершина графа є глухим кутом, якщо всі суміжні з нею вершини вже відвідані. Після влучення в глухий кут потрібно вертатися назад уздовж пройденого шляху, поки не буде виявлена вершина, у якої є суміжна ще не відвідана вершина, а потім необхідно рухатися в цьому новому напрямку. Процес є завершеним при поверненні в початкову вершину, причому всі суміжні з нею вершини вже повинні бути відвідані. Часова складність залежить від представлення графа. Якщо застосовано матрицю суміжності, то часова складність дорівнює $O(n^2)$, а якщо нематричне представлення – $O(n+m)$.
- Пошук в ширину: після відвідування першої вершини, відвідуються всі сусідні з нею вершини. Потім відвідуються всі вершини, що перебувають на відстані двох ребер від початкової. При кожному новому кроці відвідуються вершини, відстань від яких до початкової на одиницю більша попередньої. Щоб запобігти повторному відвідуванню вершин, необхідно вести список відвіданих вершин. Для зберігання тимчасових даних, необхідних для роботи алгоритму, використовується черга – упорядкована послідовність елементів, у якій нові елементи додаються в кінець, а старі видаляються із початку. Складність пошуку в ширину при нематричному представленні графа дорівнює $O(n + m)$, тому що розглядаються всі n вершин і m ребер. Використання матриці суміжності приводить до оцінки $O(n^2)$.
- Пошук у глибину застосовується при пошуку зв'язних компонентів: викликається пошук у глибину, далі визначається транспонований граф (ребра змінюються на протилежні), а тоді виконується пошук у глибину, розглядаючи вершини за зменшенням часу завершення роботи з вершинами, отриманого за пешим виконанням пошуку у глибину. Кожне отримане дерево – сильно пов'язаний компонент.

Алгоритми на графах

- Побудова найкоротших шляхів з виділеної вершини виконується на основі алгоритмів Дейкстри та Беллмана-Форда.
- Алгоритм Дейкстри працює зі зваженим орієнтованим графом з невід'ємними ребрами. Він містить множину вершин, для яких обчислені остаточні ваги найкоротших шляхів. На кожному кроці обирається така вершина, якій відповідає найменша оцінка найкоротшого шляху. Після цього відбувається послаблення ребер, які виходять з неї (вибір між встановленою оцінкою вершини та оцінкою, що є сумою ваги ребра та оцінки обраної вершини). Складність $O(V \lg V + E)$.
- Алгоритм Беллмана-Форда може працювати з від'ємними вагами. Він визначає, чи присутній у графі від'ємний цикл. Для цього відбувається послаблення всіх ребер графа з повторенням цього процесу в кількості, що дорівнює кількості на 1 менше кількості вершин. Додатковий прохід перевіряє можливість виконання послаблення хоча би для одного ребра. Якщо така можливість є, то присутній від'ємний цикл. Складність $O(VE)$

Алгоритми на графах

- Кістяковим (або каркасним) деревом зв'язного неорієнтованого графа називається ациклічний зв'язний підграф цього графа, який містить усі його вершини.
- Для того щоб утворити каркасне дерево графа, потрібно з вихідного графа видалити максимальну кількість ребер, так, щоб при цьому граф лишався зв'язним.
- Ідея алгоритму Прима полягає у тому, що, починаючи з деякої вершини, на кожному кроці алгоритму до побудованого на попередніх кроках дерева додається ребро з найменшою вагою. Цей процес відбувається доти, доки дерево не стане містити всі вершини вихідного графа. На матриці суміжності складність алгоритму $O(V^2)$, на бінарній піраміді – $O(E \log V)$.
- Алгоритм пошуку A^* використовується для пошуку найкоротшого шляху між двома вершинами графу з додатніми вагами ребер. Алгоритм подібний до алгоритму Дейкстри, але при визначенні пріоритету вершин використовується додатково допоміжна функція (евристика), аби скеровувати напрям пошуку. Пріоритет тому визначається за допомогою функції $f(x) = g(x) + h(x)$, де $g(x)$ – функція, значення якої дорівнюють вартості шляху від початкової вершини до x (як у алгоритмі Дейкстри), $h(x)$ – евристична функція, яка оцінює вартість шляху від вершини x до кінцевої.

Стратегії розроблення алгоритмів

- Стратегія «розділяй та володарюй» передбачає поділ вихідної задачі на дрібніші задачі, розв'язання яких з алгоритмічної точки зору має меншу складність, на основі розв'язків яких можна легко отримати розв'язок вихідної задачі. При цьому, структура кожної з підзадач є подібною до структури вихідної, що в свою чергу дозволяє далі поділити їх на підзадачі аж доки не дійдемо до підзадач розв'язання яких є тривіальним. Реалізація концепції «розділяй та володарюй» має рекурсивний характер. Оцінка часу виконання: $T(n) = aT(n/b) + f(n)$, де a – кількість підзадач, n/b – розмір підзадач, $f(n)$ – час, що витрачається на декомпозицію задачі та об'єднання результатів розв'язків підзадач.
- Прикладами застосування цього методу є алгоритми сортування злиттям та бінарний пошук, підрахунок елементів послідовності чисел Фібоначчі, використовуючи рекурентні формули або рекурсивний опис.
- Метод балансування: при проєктуванні деяких алгоритмів доводиться йти на компроміси, тобто по можливості збалансувати обчислювальні витрати на використання різних частин алгоритму. Метод балансування можна розглядати як балансування дерев: дерево називається збалансованим, якщо висота лівого піддерева і висота правого піддерева для кожної вершини відрізняються не більше ніж на одиницю. Кращі зі збалансованих дерев є ідеальними. При включенні нової вершини в дерево може (але не обов'язково) збільшуватися висота одного з дерев.

Динамічне програмування

- Часом не вдається поділити задачу на невелику кількість задач меншого розміру, об'єднання розв'язків яких дозволить отримати рішення початкової задачі. У таких випадках пробують поділити задачу на стільки задач, скільки необхідно, потім кожен поділену задачу ділять на ще декілька менших і так далі. Якщо б весь алгоритм зводився саме до такої послідовності дій, то в результаті отримали б алгоритм з експоненціальним часом виконання.
- Але часто вдається отримати лише поліноміальну кількість задач меншого розміру і тому то чи іншу задачу доводиться вирішувати багаторазово. Якщо б замість того відслідковувати рішення кожної вирішеної задачі і просто шукати у випадку необхідності відповідний розв'язок, то отримують алгоритм з поліноміальним часом виконання.
- З точки зору реалізації, іноді, буває простіше створити таблицю розв'язків усіх задач меншого розміру, які доведеться вирішувати. Заповнюють таку таблицю незалежно від того, чи потрібна в реальному випадку конкретна задача для отримання загального розв'язку. Заповнення таблиці складових задач для отримання розв'язку певної задачі отримало назву динамічне програмування.
- Динамічним програмуванням (в найбільш загальній формі) називають процес покрокового розв'язку задачі, коли на кожному кроці вибирається один розв'язок з множини допустимих на цьому кроці, причому такий, який оптимізує задану цільову функцію або функцію критерію. В основі теорії динамічного програмування лежить принцип оптимальності Белмана.
- Прикладом застосування є задача про розв'язання стрижня.

Імперативний та декларативний підходи до програмування

- Імперативне програмування – стиль програмування, у якому програма складається з інструкцій, що виконуються послідовно, щоб змінити стан програми. Програміст описує послідовність дій, необхідних для досягнення бажаного результату. Однією з ключових особливостей імперативного програмування є явний опис того, як програма має працювати. Інструкції в імперативному програмуванні є прямими командами до виконання певної дії.
- Декларативне програмування – це стиль програмування, у якому програма описує, які завдання мають бути виконані, а не як саме їх виконати. Замість того щоб містити послідовність команд, декларативна програма визначає набір правил і обмежень, які мають бути задоволені. Декларативне програмування відрізняється від імперативного програмування тим, що не вимагає явної вказівки порядку виконання завдань.

Розв'язні, напіврозв'язні та нерозв'язні проблеми. Проблема зупинки

- Клас складності NP (Non-deterministic Polynomial time) – клас складності, до якого належать задачі, що можна розв'язати недетермінованими алгоритмами за поліноміальний час; тобто, недетермінованими алгоритмами в яких завжди існує шлях успішного обчислення за поліноміальний час відносно довжини вхідного рядка. Приклади: задача пакування рюкзака, задача комівояжера, розфарбування графа у три кольори, розкладання числа на прості множники.
- Клас складності P (Polynomial time) – клас задач, що можна розв'язати алгоритмами з поліноміальним часом. Приклад: стандартне множення матриць. Всі задачі з P можуть бути розв'язані поліномними алгоритмами.
- У загальному вигляді розв'язність проблеми – властивість проблеми володіти алгоритмом, який визначає за даною формулою, виводиться вона з множини аксіом даної проблеми (теорії) чи ні.
- Напіврозв'язність означає наявність алгоритму, який за скінченний час завжди підтвердить, що твердження істинне, якщо це дійсно так, але якщо це не так, то може працювати нескінченно.
- Алгоритмічна нерозв'язність – властивість деяких класів коректно поставлених задач, які допускають використання алгоритмів, яка полягає в тому, що задачі кожного із цих класів в принципі не мають якого-небудь загального універсального алгоритму вирішення, що об'єднує весь клас.
- Проблема зупинки є проблемою визначення, чи завершиться дана програма, отримавши певні вхідні дані за скінченний час, чи буде працювати нескінченно. Проблема зупинки - алгоритмічно нерозв'язна задача.

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка"
ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

Командна робота, підходи до розробки програмного забезпечення (ПЗ)

Класичні моделі розробки ПЗ: каскадна (водоспадна), ітераційна, інкрементна

Лекція 2 з дисципліни Управління ІТ-проєктами (с.12)

<https://drive.google.com/file/d/19z9Y3jutfap6afZzDqvQaGbUda2cW4wo/view?usp=sharing>

Промислові технології розробки ПЗ: RUP, MSF, Agile, Scrum, Extreme Programming (XP), Kanban

Agile, Scrum, Extreme Programming (XP), Kanban

Лекція 2 з дисципліни Управління ІТ-проєктами (с.23)

<https://drive.google.com/file/d/19z9Y3jutfap6afZzDqvQaGbUda2cW4wo/view?usp=sharing>

RUP, MSF

не викладалися

Ролі та обов'язки у програмній команді, переваги командної роботи, ризики та складність такої співпраці

Лекція 1 з дисципліни Управління ІТ-проєктами (с.29)

<https://drive.google.com/file/d/1uatgc26hmzUqkEtHqODqgXoMxENERBqr/view?usp=sharing>

Планування проєктів: діаграми GANTT та PERT

Лекція 4 з дисципліни Управління ІТ-проєктами (PERT с.30)

https://drive.google.com/file/d/1lwxo61Q7rJFKVVp6nYSWEK52omcJw8L-/view?usp=drive_link

Лабораторні роботи 8 та 9 дисципліни Управління ІТ-проєктами

https://drive.google.com/file/d/1Nf11tgQlhWxyO3D-J_K8gLOP4yrtTE7I/view?usp=drive_link

Історії користувачів

Лекція 2 з дисципліни Управління IT-проєктами (с.42)

<https://drive.google.com/file/d/19z9Y3jutfap6afZzDqvQaGbUda2cW4wo/view?usp=sharing>

Лабораторна робота 5 дисципліни Управління IT-проєктами (с.22)

https://drive.google.com/file/d/1Nf11tgQlhWxyO3D-J_K8gLOP4yrtTE7I/view?usp=drive_link

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

ФУНКЦІЇ БІНАРНОЇ ЛОГІКИ. БУЛЕВА АЛГЕБРА

Теоретичною основою комп'ютерної схемотехніки є алгебра логіки - наука, яка використовує математичні методи для розв'язання логічних задач. Алгебру логіки називають булевою на честь англійського математика Джорджа Буля, який вніс найбільший вклад у розвиток цієї науки.

Основним предметом булевої алгебри є **висловлювання** - просте твердження, про яке можна стверджувати: істинне воно (позначають символом 1) або хибне (позначають символом 0). За допомогою логічних зв'язок НЕ, АБО, І, ЯКЩО... ТО... будують складні висловлювання, які називають булевими (логічними) функціями.

У даний час головна задача алгебри логіки - аналіз, синтез і структурне моделювання будь-яких дискретних скінченних систем. Апарат булевої алгебри поширюється на об'єкти найрізноманітнішої природи безвідносно до їхньої суті, тільки б вони характеризувалися двома значеннями або станами: контакт увімкнений або вимкнений, наявність високого чи низького рівня електричної напруги, виконання або невиконання деякої умови роботи та ін.

Використання апарата алгебри логіки в комп'ютерній схемотехніці засновано на тому, що цифрові елементи характеризуються двома станами і через це можуть бути описані булевими функціями.

Змінну із скінченним числом значень (станів) називають **перемикальною**, а з двома значеннями - **булевою**. Функція, яка має як і кожна її змінна скінченне число значень, називається перемикальною (логічною). Логічна функція, число можливих значень якої і кожної її незалежної змінної дорівнює двом, є булевою. Таким чином, булева функція - це окремий випадок перемикальної.

Операція - це чітко визначена дія над одним або декількома операндами, яка створює новий об'єкт (результат). У булевій операції операнди і результат набувають "булевого значення 1" (далі просто значення 1) і "булевого значення 0" (далі просто значення 0).

Булеві функції можуть залежати від однієї, двох і в цілому від n змінних. Основними булевими операціями є заперечення (операція НЕ, інверсія), диз'юнкція (операція АБО, логічне додавання, об'єднання) і кон'юнкція (операція І, логічне множення).

Заперечення - це одномісна булева операція $F = X$ (читається «не X »), результатом якої є значення, протилежне значенню операнда.

Диз'юнкція - це булева операція $F = X_1 \vee X_2$ (читається «X1 чи X2»), результатом якої є значення нуль тоді і тільки тоді, коли обидва операнди мають значення нуль.

Кон'юнкція - це булева операція $F = X_1 \wedge X_2$ (читається «X1 і X2»), результатом якої є значення одиниця тоді і тільки тоді, коли значення кожного операнда дорівнює одиниці. У виразі $X_1 \cdot X_2$ крапку можна опускати; часто застосовують запис $X_1 \wedge X_2$ або $X_1 \& X_2$.

Операції заперечення, диз'юнкції і кон'юнкції можна задати за допомогою таблиць істинності, у яких зліва подані значення операндів, а справа - значення булевої функції.

X	$F = \bar{X}$
0	1
1	0

X_1	X_2	$F = X_1 \vee X_2$	$F = X_1 \cdot X_2$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

Для булевих операцій заперечення, диз'юнкції і кон'юнкції справедливі такі закони, властивості й тотожності:

- комутативність (переміщувальний закон):

$$X_1 \vee X_2 = X_2 \vee X_1; \quad X_1 \cdot X_2 = X_2 \cdot X_1;$$
- асоціативність (сполучний закон):

$$X_1 \vee X_2 \vee X_3 = (X_1 \vee X_2) \vee X_3; \quad X_1 \cdot X_2 \cdot X_3 = (X_1 \cdot X_2) \cdot X_3;$$
- дистрибутивність (розподільний закон):

$$X_1 (X_2 \vee X_3) = X_1 \cdot X_2 \vee X_1 \cdot X_3; \quad X_1 \vee X_2 \cdot X_3 = (X_1 \vee X_2) (X_1 \vee X_3);$$
- ідемпотентність (виключення повторення):

$$X \vee X \vee X = X; \quad X \cdot X \cdot X = X;$$
- закон поглинання:

$$X_1 \vee X_1 \cdot X_2 = X_1; \quad X_1 (X_1 \vee X_2) = X_1;$$
- закон склеювання:

$$X_1 \cdot X_2 \vee X_1 \cdot \bar{X}_2 = X_1; \quad (X_1 \vee X_2) (X_1 \vee \bar{X}_2) = X_1;$$
- закон де Моргана:

$$\overline{X_1 \vee X_2} = \bar{X}_1 \cdot \bar{X}_2; \quad \overline{X_1 \cdot X_2} = \bar{X}_1 \vee \bar{X}_2;$$
- властивості заперечення і констант:

$$X \vee \bar{X} = 1; \quad X \cdot \bar{X} = 0; \quad \overline{\bar{X}} = X; \quad \bar{1} = 0; \quad \bar{0} = 1;$$

$$X \vee 0 = X; \quad X \vee 1 = 1; \quad X \cdot 1 = X; \quad X \cdot 0 = 0;$$
- тотожності:

$$X_1 \vee \bar{X}_1 \cdot X_2 = X_1 \vee X_2; \quad X_1 (\bar{X}_1 \vee X_2) = X_1 \cdot X_2.$$

Функції бінарної логіки:

X_1	0 0 1 1	Вираз	Назва операції
X_2	0 1 0 1		
F_0	0 0 0 0	$F_0 = 0$	Константа 0
F_1	0 0 0 1	$F_1 = X_1 X_2$	Кон'юнкція
F_2	0 0 1 0	$F_2 = \overline{X_1 X_2}$	Заборона по X_2
F_3	0 0 1 1	$F_3 = \overline{X_1}$	Повторення X_1
F_4	0 1 0 0	$F_4 = \overline{X_1} X_2$	Заборона по X_1
F_5	0 1 0 1	$F_5 = X_2$	Повторення X_2
F_6	0 1 1 0	$F_6 = X_1 \oplus X_2$	Сума за модулем 2
F_7	0 1 1 1	$F_7 = X_1 \vee X_2$	Диз'юнкція
F_8	1 0 0 0	$F_8 = X_1 \downarrow X_2$	Заперечення диз'юнкції
F_9	1 0 0 1	$F_9 = X_1 \sim X_2$	Еквівалентність
F_{10}	1 0 1 0	$F_{10} = \overline{X_2}$	Заперечення X_2
F_{11}	1 0 1 1	$F_{11} = X_1 \leftarrow X_2$	Імплікація від X_2 до X_1
F_{12}	1 1 0 0	$F_{12} = \overline{X_1}$	Заперечення X_1
F_{13}	1 1 0 1	$F_{13} = X_1 \rightarrow X_2$	Імплікація від X_1 до X_2
F_{14}	1 1 1 0	$F_{14} = X_1 / X_2$	Заперечення кон'юнкції
F_{15}	1 1 1 1	$F_{15} = 1$	Константа 1

Булеві функції одного і двох аргументів називають **елементарними**. Схему, яка здійснює елементарну логічну операцію, називають **логічним елементом (вентилем)**. Сукупність взаємозалежних логічних елементів з формальними методами опису називається **логічною схемою**.

ПРЕДСТАВЛЕННЯ ДАНИХ НА РІВНІ МАШИН

Системи числення, двійкове, вісімкове, шістнадцяткове числення.

Системою числення називається сукупність цифр і правил для записування чисел. Запис числа у деякій системі числення називається його **кодом**. Усі системи числення поділяють на позиційні й непозиційні.

У **непозиційних системах числення** значення кожної цифри не залежить від її позиції. Приклад: римські цифри.

Недоліком непозиційної системи є:

- відсутність нуля;
- відсутність формальних правил запису чисел і відповідно арифметичних дій з ними.

У **позиційних системах числення** значення кожної цифри визначається її зображенням і позицією в числі. Окремі позиції в записі числа називають розрядами, а номер позиції - номером розряду. Число розрядів у записі числа називається його **розрядністю** і збігається з довжиною числа.

Для запису чисел у **позиційній системі числення** використовують певну кількість графічних знаків (цифр і букв), які відрізняються один від одного. Число таких знаків q називається **основою позиційної системи числення**. В комп'ютерах використовують позиційні системи з різною основою.

Система числення повинна забезпечувати:

- можливість представлення будь-якого числа в заданому діапазоні;
- однозначність, стислість запису числа і простоту виконання арифметичних операцій;
- досягнення високої швидкодії машини в процесі оброблення інформації.

Число в позиційній системі можна подати поліномом

$$= \dots + (-1)^{m-1} \cdot q^{m-1} + (-1)^{m-2} \cdot q^{m-2} + \dots + (-1)^1 \cdot q^1 + (-1)^0 \cdot q^0 + (-1)^{-1} \cdot q^{-1} + \dots = \sum_{i=-m}^k a_i \cdot q^i = - \dots$$

де q - основа системи числення,

q^i - вага позиції, $i \in \{0, 1, \dots, (-1)\}$ - цифри в позиціях числа;

$0, 1, \dots, k$ - номери розрядів цілої частини числа;

$-1, -2, \dots, -m$ - номери розрядів дробової частини числа.

Позиційні системи з однаковою основою в кожному розряді, називаються **однорідними**. Оскільки на значення q немає ніяких обмежень, то теоретично можлива нескінченна множина позиційних систем числення.

На практиці застосовують скорочений запис полінома у вигляді послідовності цифр із знаком залежно від типу числа:

- для змішаного числа

$$= \pm \dots (-1)^{m-1} \cdot 1 \cdot 0, (-1)^{-1} \cdot \dots$$

- для цілого числа

$$= \pm \dots (-1)^{m-1} \cdot 1 \cdot 0;$$

- для правильного дробу

$$= \pm 0, (-1)^{-1} \cdot (-2) \cdot \dots$$

При однаковій розрядності в системах числення з більшою основою можна записати більше різних чисел.

Двійкова система числення є базовою в галузі обчислювальної техніки. На основі принципів побудови двійкового коду працюють майже всі електронні пристрої. Це зумовлено простотою роботи з ним як при зберіганні, так і при обробці інформації.

Перевагою двійкової системи є:

- простота виконання арифметичних операцій;
- наявність надійних мікроелектронних схем з двома стійкими станами (тригерів), призначених для зберігання значень двійкового розряду - цифр 0 або 1.

Двійкові цифри називають також бітами.

Взаємна відповідність різних систем числення має наступний вигляд.

Десяткове число	Двійкові розряди (біти)				Знаки систем числення	Системи числення			
0	0	0	0	0	0	Двійкова	Вісімкова	Десяткова	Шістнадцяткова
1	0	0	0	1	1				
2	0	0	1	0	2				
3	0	0	1	1	3				
4	0	1	0	0	4				
5	0	1	0	1	5				
6	0	1	1	0	6				
7	0	1	1	1	7				
8	1	0	0	0	8				
9	1	0	0	1	9				
10	1	0	1	0	A				
11	1	0	1	1	B				
12	1	1	0	0	C				
13	1	1	0	1	D				
14	1	1	1	0	E				
15	1	1	1	1	F				

ПРЕДСТАВЛЕННЯ ЧИСЕЛ У ЦІЛОЧИСЕЛЬНОМУ ФОРМАТІ ТА ФОРМАТІ ІЗ ПЛАВАЮЧОЮ КОМОЮ

Двійкові числа в комп'ютерах розміщуються в комірках пам'яті або в регістрах, які складаються із запам'ятовуючих елементів - тригерів. У комірці або тригері зберігається значення одного двійкового розряду - біту інформації. Розрядною сіткою комп'ютера називається сукупність запам'ятовуючих елементів для розміщення одного двійкового числа. Для різних класів комп'ютерів довжина розрядної сітки складає 8, 16, 32, 64 і більше розрядів. Форматом називається спосіб розміщення компонентів числа у розрядній сітці, тобто послідовність і позиції знака, мантиси, порядку та ін.

В комп'ютерах використовують дві **форми представлення числа**:

- з фіксованою комою перед старшим розрядом числа (для правильного дробу) або після молодшого (для цілого числа);

- з рухомою комою, місце положення якої задається порядком числа.

Місце коми в обох форматах розуміється неявно, без використання додаткових розрядів.

Абсолютне значення цілого числа A змінюється в межах:

$$-2^k \leq A \leq 2^k - 1; \quad 2^k = 2^{k-1} + 2^{k-1},$$

де k – число розрядів цифрової частини числа.

Із цього виразу випливає, що числа, за абсолютним значенням менші одиниці, сприймаються як "машинний нуль"; числа, більші за 2^{k-1} , викликають переповнення розрядної сітки.

Діапазон представлення цілих чисел з урахуванням симетрії відносно нуля числової осі:

$$-2^{k-1} \leq A \leq 2^{k-1} - 1; \quad 2^k = 2(2^{k-1} - 1) + 1.$$

Приклад: Розрахувати максимальне значення і діапазон подання цілого числа зі знаком у 16-розрядній сітці.

Рішення: Для $k=15$ одержуємо: $2^{15} - 1 = 32767$; $-2^{14} = -16384$

Абсолютне значення правильного дробу B змінюється в межах

$$-2^{-r} \leq B \leq 2^{-r}; \quad 2^{-r} = 1 - 2^{-r},$$

Правильний дріб, за модулем менший за 2^{-r} , сприймається як "машинний нуль"; числа, більші за одиницю, викликають переповнення розрядної сітки. Діапазон подання правильного дробу:

$$-2^{-r} \leq B \leq 2^{-r} \approx 2^{-r}, \text{ тому що } 1 \gg 2^{-r}.$$

У формі з рухомою комою числа подаються у вигляді добутків:

$$A = (-1)^p \cdot 2^X \cdot M$$

де q - основа системи числення;

P - порядок числа довжиною $k+1$ (ціле число зі знаком);

M - мантиса числа довжиною $r+1$ (правильний дріб із знаком);

X - характеристика числа.

Знак усього числа визначається знаком мантиси.

Мантиса називається **нормалізованою**, якщо її значення визначається нерівністю виду:

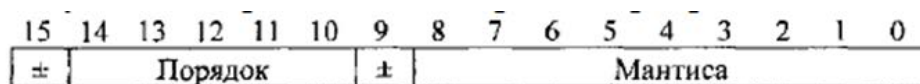
$$1/2 \leq |x| < 1,$$

тобто в старшому розряді модуля мантиси має бути записана одиниця.

Ілюстрація запису числа у форматі з рухомою комою

$$2 = 2^1 \cdot 110,111 = 2^2 \cdot 11,0111 = 2^4 \cdot 0,110111$$

Формат числа з рухомою комою в 16-розрядній сітці показаний на рис. Тут для модулів порядку і мантиси відведено відповідно п'ять і дев'ять розрядів. Кома в порядку розташовується (умовно) після молодшого розряду, а в мантисі - перед старшим. Знаки порядку і мантиси розміщуються перед їхніми старшими розрядами (рис.5).



Формат числа з рухомою комою в 16-розрядній сітці

Проведення арифметичних операцій з фіксованою комою алгоритмічно є простіше. Йому віддають перевагу при організації розрахунків.

Використання чисел з рухомою комою надає більшу гнучкість при розрахунках, але вимагає складніші алгоритми для їх виконання, реалізація яких можлива за умови використання додаткового процесора. Одним із шляхів прискорення таких розрахунків є організація конвеєрного обчислення.

КОДУВАННЯ ВІД'ЄМНИХ ЧИСЕЛ

Для записування знака числа, заміни операції віднімання чисел додаванням їхніх кодів, а також для визначення переповнення розрядної сітки використовують **прямий, обернений і доповняльний коди**, де для подання знака числа відводиться знаковий разряд, який розташовується зліва від числа і відокремлюється комою. У знаковий разряд записують нуль (для додатного числа) або одиницю (для від'ємного). Кома в машині у явному вигляді не зображується, а тільки припускається.

Числа, подані в прямому, оберненому і доповняльному кодах, називають **машинними зображеннями**. Вони складаються зі знакового розряду і цифрової частини (модуля числа). **Додатні числа у всіх кодах записуються однаково.**

Прямий код від'ємного числа представляє число в двійковій формі, в старшому розряді якого зазначено його знак – значення біту встановлено в одиницю.

Обернений код від'ємного числа утворюється з його прямого коду після інвертування значень розрядів цифрової частини, тобто заміною нуля на одиницю і одиниці - на нуль; значення знакового розряду не змінюється.

Доповняльний код від'ємного двійкового числа утворюється з його оберненого коду додаванням одиниці до молодшого розряду.

Приклад: Записати цілі двійкові числа $A=10112$, $B=-10112$, дробове число $C=-10112$ у прямих кодах.

Рішення:

$A = 1011,$	$B = -1011,$	$C = -1011$
$[A]_{\text{пр}} = 0,1011;$	$[B]_{\text{пр}} = 1,1011;$	$[C]_{\text{пр}} = 1,1011;$
$[A]_{\text{об}} = 0,1011;$	$[B]_{\text{об}} = 1,0100;$	$[C]_{\text{об}} = 1,0100;$
$[A]_{\text{доп}} = 0,1011;$	$[B]_{\text{доп}} = 1,0101;$	$[C]_{\text{доп}} = 1,0101;$

Із цих прикладів видно, що обернений і доповняльний коди цілих і дробових чисел за видом запису збігаються, розходження між ними відображені в алгоритмах обробки інформації.

У **модифікованих кодах** знак числа дублюється в двох знакових розрядах.

Приклад: Записати цілі двійкові числа $A=10112$, $B=-10112$ в модифікованих кодах.

Рішення:

$[A]_{\text{пр}} = [A]_{\text{об}} = [A]_{\text{доп}} = 00,1011;$		
$[B]_{\text{пр}} = 11,1011;$	$[B]_{\text{об}} = 11,0100;$	$[B]_{\text{доп}} = 11,0101.$

Для переходу від оберненого коду від'ємного числа до прямого коду треба інвертувати значення розрядів цифрової частини, не змінюючи значення знакового розряду. Для переходу від доповняльного коду від'ємного числа до прямого спочатку одержують його обернений код, а потім додають одиницю до молодшого розряду.

АРИФМЕТИЧНІ ОПЕРАЦІЇ ДОДАВАННЯ, ВІДНІМАННЯ

Всі операції в комп'ютері виконуються в арифметико-логічному пристрої (АЛП). Числа, які беруть участь в операціях, називаються операндами. Основною операцією в АЛП є додавання. Операція віднімання замінюється додаванням операндів в оберненому або доповняльному кодах. Операції множення і ділення зводяться до багатократних додавань і зсувів.

Правила виконання операцій додавання, віднімання, множення і додавання за модулем 2 у двійковій арифметиці наведені в табл. При додаванні двох одиниць виникає перенесення у старший розряд; при відніманні від нуля одиниці потрібна позика зі старшого розряду.

Таблиця 1 – Правила виконання деяких арифметичних операцій

Додавання	Віднімання	Множення	Модуль 2
0+0=0	0-0=0	0·0=0	0+0=0
0+1=1	1-0=1	0·1=0	0+1=1
1+0=1	1-1=0	1·0=0	1+0=1
1+1=10 ↑ Перенесення	0-1=11 ↑ Позика	1·1=1	1+1=0

В усіх комп'ютерах є команди додавання і віднімання чисел. Проте в суматорах реалізуються тільки операції додавання умовно додатних машинних зображень. В операціях віднімання знак другого операнда (який віднімається) автоматично змінюється на протилежний і після цього одержують його машинне зображення. Тому в наступних прикладах розглядаються тільки операції додавання.

У машинних зображеннях (для оберненого і доповняльного кодів) знаковий розряд і цифрова частина числа розглядаються як єдине ціле. Вони однаково беруть участь в операції додавання.

При додаванні двійкових n -розрядних чисел $= \dots, \dots, 1$ та $= \dots, \dots, 1$ результат в кожному розряді визначають за формулами:

$$a_i + b_i + Z_i = P_{i+1} + 2Z_{i+1}$$

$$P_{i+1} = \begin{cases} 0, & \text{при } a_i + b_i + Z_i \leq 1 \\ 1, & \text{при } a_i + b_i + Z_i \geq 2 \end{cases}$$

де a_i – значення i -х розрядів, Z_i – перенесення з молодшого розряду, P_{i+1} – результат, Z_{i+1} – перенесення у старший розряд.

При додаванні в **обернених кодах** перенесення із старшого знакового розряду результату подається на вхід перенесення молодшого розряду (*циклічне перенесення*). При додаванні в **доповняльних кодах** перенесення із старшого знакового розряду результату не враховується (*це перенесення передбачене при переході до доповняльного коду за рахунок додавання одиниці до молодшого розряду*), тому в суматорі ланцюг циклічного перенесення розривається. Знак результату при додаванні машинних зображень утворюється автоматично.

Приклад. Додати двійкові числа $A = -1010$ і $B = -0011$ в оберненому і доповняльному кодах.

Рішення.

$$\begin{array}{rcl}
 [A]_{\text{пр}} = 1,1010 & [A]_{\text{об}} = 1,0101 & [A]_{\text{доп}} = 1,0110 \\
 [B]_{\text{пр}} = 1,0011 & [B]_{\text{об}} = 1,1100 & [B]_{\text{доп}} = 1,1101 \\
 \\
 \begin{array}{r}
 [A]_{\text{об}} = 1,0101 \\
 + [B]_{\text{об}} = 1,1100 \\
 \hline
 [C]_{\text{об}} = 11,0001 \\
 +1 \\
 [C]_{\text{об}} = 1,0010
 \end{array} & & \begin{array}{r}
 [A]_{\text{доп}} = 1,0110 \\
 + [B]_{\text{доп}} = 1,1101 \\
 \hline
 [C]_{\text{доп}} = 1,0011
 \end{array} \\
 C = -1101_2 = -13_{10} & & C = -(1100_2 + 1_2) = -1101_2 = -13_{10}
 \end{array}$$

При додаванні чисел одного знаку можливе переповнення розрядної сітки, ознакою чого є розбіжність знака результату зі знаками операндів. В арифметико-логічному пристрої є спеціальні логічні схеми, які автоматично формують ознак переповнення.

Приклад. Додати двійкові числа $A = 1011$ і $B = 0111$ в оберненому і доповняльному кодах.

Рішення.

$$\begin{array}{rcl}
 [A]_{\text{пр}} = [A]_{\text{об}} = [A]_{\text{доп}} = 0,1011 & & \\
 [B]_{\text{пр}} = [B]_{\text{об}} = [B]_{\text{доп}} = 0,0111 & & \\
 \\
 \begin{array}{r}
 1110 \\
 [A]_{\text{об}} = 0,1011 \\
 + [B]_{\text{об}} = 0,0111 \\
 \hline
 [C]_{\text{об}} \neq 1,0010
 \end{array} & & \begin{array}{r}
 11110 \\
 [A]_{\text{доп}} = 0,1011 \\
 + [B]_{\text{доп}} = 0,0111 \\
 \hline
 [C]_{\text{доп}} \neq 1,0010
 \end{array}
 \end{array}$$

У цьому прикладі додавання додатних чисел призвело до додатного переповнення: операнди – додатні, результат – від’ємний.

Приклад. Додати двійкові числа $A = -1011$ і $B = -0111$ в оберненому і доповняльному кодах.

Рішення.

$$\begin{array}{rcl}
 [A]_{\text{пр}} = 1,1011 & [A]_{\text{об}} = 1,0100 & [A]_{\text{доп}} = 1,0101 \\
 [B]_{\text{пр}} = 1,0111 & [B]_{\text{об}} = 1,1000 & [B]_{\text{доп}} = 1,1001 \\
 \\
 \begin{array}{r}
 [A]_{\text{об}} = 1,0100 \\
 + [B]_{\text{об}} = 1,1000 \\
 \hline
 [C]_{\text{об}} \neq 10,1100
 \end{array} & & \begin{array}{r}
 10 \\
 [A]_{\text{доп}} = 1,0101 \\
 + [B]_{\text{доп}} = 1,1001 \\
 \hline
 [C]_{\text{доп}} \neq 10,1110
 \end{array}
 \end{array}$$

У цьому прикладі додавання від’ємних чисел призвело до від’ємного переповнення; операнди – від’ємні, результат – додатний.

У 1946 році Д. фон Нейман, Г. Голдстайн і А. Беркс в своїй спільній статті виклали нові принципи побудови і функціонування ЕОМ.

Принципи фон Неймана

1. **Використання двійкової системи числення в обчислювальних машинах.** Перевага перед десятковою системою числення полягає в тому, що пристрої можна робити досить простими, арифметичні і логічні операції в двійковій системі числення також виконуються досить просто.

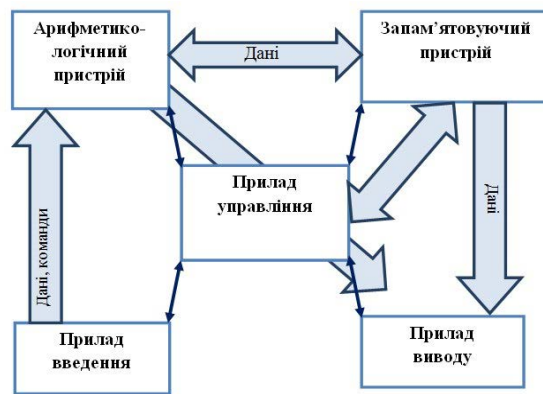
2. **Програмне управління ЕОМ.** Робота ЕОМ контролюється програмою, що складається з набору команд. Команди виконуються послідовно один за одним.

3. **Пам'ять комп'ютера використовується не тільки для зберігання даних, але і програм.** При цьому і команди програми і дані кодуються в двійковій системі числення, тобто їх спосіб запису однаковий. Тому в певних ситуаціях над командами можна виконувати ті ж дії, що і над даними.

4. **Комірки пам'яті ЕОМ мають адреси, які послідовно пронумеровані.** У будь-який момент можна звернутися до будь-якої комірки пам'яті за її адресою. Цей принцип відкрив можливість використовувати змінні в програмуванні.

5. **Можливість умовного переходу в процесі виконання програми.** Не дивлячись на те, що команди виконуються послідовно, в програмах можна реалізувати можливість переходу до будь-якої ділянки коду.

Головним наслідком цих принципів можна назвати те, що тепер програма вже не була постійною частиною машини (як наприклад, у калькулятора). Програму стало можливо легко змінити. А ось апаратура, звичайно ж, залишається незмінною, і дуже простою.



Працює процесор під управлінням програми, що знаходиться в оперативній пам'яті. Блок управління крім іншого відповідає за виклик чергової команди і визначення її типу.

Арифметико-логічний пристрій, отримавши дані і команду, виконує зазначену операцію і записує результат в один з вільних регістрів.

Поточна команда знаходиться в спеціально для неї відведеному **регістрі команд**. У процесі роботи з поточною командою збільшується значення так званого **лічильника команд**, який тепер вказує на наступну команду (якщо, звичайно, не було команди переходу або зупинки).

Часто команду представляють як структуру, що складається із запису операції (котру потрібно використати) і адрес комірки вихідних даних і результату. За адресами вказаними в команді беруться дані і поміщаються в звичайні регістри (в сенсі не в регістр команди), що вийшов результат теж спочатку виявляється в регістрі, а вже потім переміщається на свою адресу, вказану в команді.

Архітектура процесорів

Оскільки ядро МП покликане обробляти дані під управлінням команд, то очевидно ці дані і команди треба десь зберігати. І чим швидше ядро зможе отримати необхідні дані, і повернути результат, тим, очевидно, його робота буде ефективніше. У зв'язку з цим, розрізняють наступні архітектури МП, засновані на різному принципі взаємодії з операційним блоком:

- регістрова;
- стекова;
- архітектура, орієнтована на пам'ять.

Регістрова архітектура визначає наявність великого регістрового файлу всередині інтегральної схеми МП.

Переваги: висока швидкість роботи, використання скороченої адресації (менше довжина команд).

Недоліки: при частій зміні програм (мультипрограмування, завдання управління) ефективність падає, тому що при перемиканні необхідно зберігати і перевантажувати вміст регістрів; велику кількість регістрів важко розташувати на кристалі через меншу щільність розташування логічних схем, ніж схем пам'яті.

У регістровій реалізації машини структура даних, в яку поміщаються операнди, заснована на регістрах процесора. При цьому не потрібні операції PUSH або POP, але інструкції повинні явно містити адреси (регістри) в яких містяться операнди. Тобто, операнди для інструкцій, на відміну від стекової моделі, вказуються явно.

Команди виконуються швидше аналогічних команд стекової віртуальної машини. Інша перевага реєстрової моделі в тому, що вона дозволяє провести оптимізацію, яка не може бути виконана при стековому підході. Наприклад, кілька разів зустрічається вираз при регістровому підході може бути обчислено лише один раз і збережено в регістрі для подальшого використання, що економить час, необхідний для перерахунку вираження.

Але з іншого боку в середньому інструкція реєстрової машини довше ніж в стековій машині, так як в ній потрібна явна вказівка операндів.

Стекова архітектура використовує поле пам'яті з упорядкованою послідовністю запису і вибірки інформації.

Перевага: ефективна при роботі з підпрограмами (завдання управління).

Недолік: стек на кристалі малий і швидко переповнюється.

Як структури даних, куди поміщаються операнди, використовується стек. Операції отримують дані з стека, обробляють їх і заносять в стек результат за правилом LIFO (останній прийшов, перший пішов). Перевага стекової моделі в тому, що операнди задаються неявно покажчиком стека (на малюнку - SP). Це означає, що машині не потрібно явно вказувати адреси операндів, покажчик стека вказує на наступний операнд. У стекових машинах все арифметичні і логічні операції виконуються за допомогою отримання операндів і повернення результатів в стек.

Архітектура пам'ять - пам'ять забезпечує розташування регістрів і стека в ОП. Операнди, після вибірки з ОП, обробки в МП заносяться назад в пам'ять, а не зберігаються на регістрах. Оцінку цього способу необхідно проводити з урахуванням швидкодії МП і ОЗП.

Переваги: фактично необмежена свобода оперування розташуванням даних (велика кількість регістрів в ОЗП); швидке контекстне перемикання, тому що для перемикання реєстрового (контекстного) файлу необхідно тільки змінити регістр - показчик початкової адреси; спрощується зв'язок з іншими МП (багатопроцесорні системи, контролери ПУ).

Недолік: особливість - наявність двоадресних команд, що збільшує довжину команд. Архітектура пам'ять - пам'ять часто використовується в ОЕВМ (контролерів для роботи в реальному режимі часу, при великій кількості переривань, викликів підпрограм, мультипрограмування, і т.д.).

CISC і RISC архітектури

Зростаюча складність процедур обробки команд змушує вдаватися до мікропрограмного управління пристроями з керуючою пам'яттю замість більш швидкодіючого пристрою управління з "жорсткою" логікою.

Складний процесор з мікропрограмним управлінням важко реалізувати на одному кристалі, а це веде до збільшення довжини міжмодульних зв'язків, так влаштовані більшість процесорів з повним набором команд.

Навпаки, при скороченні кількості команд до деякого оптимального значення, можна скоротити довжину команд і спростити керуючий пристрій МП. Тому при проектуванні структури МП виділилося два напрямки щодо набору системи команд:

CISC (Complicated Instruction Set Computer – який використовує повний набір команд). Традиційна архітектура з широкою системою команд МП.

RISC (Reduced Instruction Set Computer) - архітектура з скороченим набором команд.

При використанні RISC архітектури команди набору виконувалися за один машинний цикл МП. Виконання більш складних, але операції, які рідко зустрічаються, забезпечують підпрограми, що складаються з набору простих команд.

Аналіз використання різними завданнями ресурсів МП показує, що в основному МП обробляє одні і ті ж інструкції з невеликої підмножини повної системи команд.

В першу чергу це команди читання / запису і команди переходів. Тому для прискорення роботи МП необхідно оптимізувати в першу чергу ці команди.

У ЕОМ з RISC-архітектурою **машинним циклом** називається час, протягом якого виробляється вибірка двох операндів з регістрів, виконання операції в ALU і

запам'ятовування результату в регістрі. Більшість команд в RISC-процесорах є швидкими командами типу регістр-регістр і виконуються без звернення до ОП. Звернення до пам'яті проводиться лише в командах завантаження регістрів з пам'яті і запам'ятовування їх в ОП.

Оскільки однією з головних завдань даних МП є зменшення кількості звернень до ОП, то характерна особливість RISC-процесорів - велика кількість регістрів.

Внаслідок скороченого набору команд (приблизно 50-100), невеликого числа способів адресації (2-3 і в основному реєстрова) спрощується керуючий пристрій МП, яке в цьому випадку обходиться без вбудованого управління і його пристрій управління може бути виконано на "жорсткій" логіці. Спрощення структури МП призводить до появи вільного місця на кристалі для реалізації додаткових схем.

CISC-архітектура	RISC-архітектура
Багатобайтові команди	Однобайтові команди
Мала кількість регістрів	Велика кількість регістрів
Складні команди	Прості команди
Одна або менш команд за один цикл процесора	Кілька команд за один цикл процесора
Традиційно один виконавчий пристрій	Кілька виконавчих пристроїв

Характеристики процесора

Тактова частота процесора на сьогоднішній день вимірюється в гігагерцах (ГГц), Раніше вимірювалося в мегагерцах (МГц). $1\text{МГц} = 1\text{ мільйону тактів в секунду}$.

Процесор «спілкується» з іншими пристроями (оперативною пам'яттю) за допомогою **шин даних, адреси і управління**. Розрядність шин завжди кратна 8 (зрОЗПміло чому, якщо ми маємо справу з байтами), мінлива в ході історичного розвитку комп'ютерної техніки і різна для різних моделей, а також не однакова для шини даних і адресної шини.

Розрядність шини даних говорить про те, яка кількість інформації (скільки байт) можна передати за раз (за такт). Від розрядності шини адреси залежить максимальний обсяг оперативної пам'яті, з яким процесор може працювати взагалі.

Організація центрального процесора

Поняття архітектури обчислювальної системи

Архітектура обчислювальної системи - концептуальна структура обчислювальної системи, яка визначає організацію процесу обробки інформації і принципи взаємодії технічних засобів і програмного забезпечення.

Архітектура обчислювальної системи включає як апаратну (hardware), так і програмну (software) складові.

Згідно архітектурі фон Неймана основними апаратними компонентами обчислювальної системи є: один центральний процесор, оперативна (основна) пам'ять і пристрої введення-виведення (периферійні пристрої).

Організація центрального процесорного пристрою

Ключовими компонентами центрального процесора є:

- реєстри;
- арифметико-логічний пристрій (АЛП);
- пристрій управління (УУ);
- зовнішні і внутрішні шини.

Регістри

Регістри являють собою швидкодіючу пам'ять, використовувану для зберігання проміжних результатів, констант і деяких команд управління. Як правило, всі реєстри мають однакову розрядність.

Регістри процесора Intel 8086, які використовуються для вибірки команд:

CS - реєстр, що містить адресу сегмента, в якому знаходиться фрагмент виконуваної програми.

PC - реєстр, що містить зсув (адреса) в межах сегмента коду виконуваної в даний момент команди.

IR - реєстр, який зберігає обрану з пам'яті команду, яка в даний момент виконується.

Арифметико-логічний пристрій

Арифметико-логічний пристрій здатний виконувати безліч найпростіших (типових) операцій над даними:

- | | | |
|----|------------------|------------------------|
| 1. | Установка | $RA := "1010"$ |
| 2. | Передача | $RA := RB$ |
| 3. | Зрушення | $RA := R_1(RA)$ |
| 4. | Інверсія | $RA := \text{not}(RA)$ |
| 5. | Логічні операції | $RA := RA \& RB$ |
| 6. | Розрахунок | $\text{inc}(RA)$ |

Організація АЛП залежить від способу представлення даних: з фіксованою або плаваючою комою, в ПК, ОК, ДК і т.д.

Арифметико-логічний пристрій має наступні характеристики:

1. Продуктивність - середнє число мікрооперацій за машинний такт. Висока продуктивність частіше характерна для схем дуже високої складності.
2. Швидкодія - величина, зворотна тривалості машинного такту роботи пристрою.
3. Апаратні витрати - кількість логічних вентилів або їх грошовий еквівалент.

Пристрій управління

Пристрій управління (ПУ) організовує автоматичне виконання програм і функціонування ЕОМ як єдиної системи. Основне завдання пристрою управління - вироблення керуючих сигналів і розподіл їх по ланцюгах управління.

Керуючий пристрій містить лічильник команд. Після завантаження програми і даних в пам'ять в лічильник команд записується адреса першої команди програми (стартову адресу). Пристрій управління зчитує з пам'яті вміст комірки пам'яті, адреса якої знаходиться в лічильнику команд, і поміщає його в регістр команд.

Шини

Шина являє собою безліч паралельних провідників, які забезпечують інформаційний обмін між різними блоками ЦПП (внутрішні шини), а також між ЦПП і пам'яттю, пристроями введення / виводу (зовнішні шини).

Передана по шинам інформація по типу може бути віднесена до одного з трьох класів:

- 1) адреса;
- 2) дані;
- 3) команда.

Етапи виконання команди центральним процесором

- вибірка команди з оперативної пам'яті і розміщення її в регістрі команд;
- (при необхідності) в залежності від коду операції обчислення виконавчих адрес операндів і вибірка їх з оперативної пам'яті;
- виконання операції;

- (при необхідності) обчислення виконавчого адреси результату і збереження його в оперативну пам'ять.

Організація взаємодії ЦПП і пам'яті (ОЗП)

- ЦПП виставляє на шину адреси номер комірки в ОЗП з потрібною командою, і на шину управління - сигнал читання з ОЗП.
- ОЗП записує номер комірки в MAR (memory address register) і вибирає з відповідної комірки потрібне значення, яке розміщує в MBR (memory buffer register). Про закінчення операції зчитування ОЗП сповіщає за допомогою спеціального сигналу.
- Зчитана команда по шині даних потрапляє в ЦПУ, де аналізується і виконується.

При необхідності з ОЗП по тій же схемі вибираються додаткові операнди або в ОЗП зберігається результат виконання операції.

Призначення ОЗП

- Зберігання даних і команд для подальшої їх передачі процесору для обробки. Інформація може надходити з оперативної пам'яті не відразу на обробку процесору, а в більш швидко, ніж ОЗП, кеш-пам'ять процесора.
- Зберігання результатів обчислень, вироблених процесором.
- Зчитування (або запис) вмісту осередків.

Особливості роботи ОЗП

Оперативна пам'ять може зберігати дані лише при включеному комп'ютері. Тому при його виключенні оброблювані дані слід зберігати на жорсткому диску або іншому носії інформації. При запуску програм інформація надходить в ОЗП, наприклад, з жорсткого диска комп'ютера. Поки йде робота з програмою вона присутня в оперативній пам'яті (зазвичай). Як тільки робота з нею закінчена, дані перезаписувати на жорсткий диск. Іншими словами, потоки інформації в оперативній пам'яті дуже динамічні.

ОЗП є оперативною пам'яттю. Це означає, що прочитати / записати дані можна з будь-якого елементу ОЗП в будь-який момент часу. Для порівняння, наприклад, магнітна стрічка є запам'ятовуючим пристроєм з послідовним доступом.

Логічний пристрій оперативної пам'яті

Оперативна пам'ять складається із осередків, кожна з яких має свою власну адресу. Всі осередки містять однакове число біт. Сусідні комірки мають послідовні адреси. Адреси пам'яті також як і дані виражаються в двійкових числах.

Зазвичай одна осередок містить 1 байт інформації (8 біт, те ж саме, що 8 розрядів) і є мінімальною одиницею інформації, до якої можливе звернення. Однак багато команд працюють з так званими словами. Слово являє собою область пам'яті, що складається з 4 або 8 байт (можливі інші варіанти).

Типи оперативної пам'яті

Прийнято виділяти два види оперативної пам'яті: статичну (SRAM) і динамічну (DRAM). **SRAM використовується в якості кеш-пам'яті процесора, а DRAM - безпосередньо в ролі оперативної пам'яті комп'ютера.**

SRAM складається з тригерів. Тригери можуть перебувати лише в двох станах: «включений» або «вимкнений» (зберігання біта). Тригер не зберігає заряд, тому перемикання між станами відбувається дуже швидко. Однак тригери вимагають більш складну технологію виробництва. Це неминуче відображається на ціні пристрою. По-друге, тригер, що складається з групи транзисторів і зв'язків між ними, займає багато місця (на мікрорівні), в результаті SRAM виходить досить великим пристроєм.

В **DRAM** немає тригерів, а біт зберігається за рахунок використання одного транзистора і одного конденсатора. Виходить дешевше і компактніше. Однак конденсатори зберігають заряд, а процес зарядки-розрядки більш тривалий, ніж перемикання тригера. Як наслідок, DRAM працює повільніше. Другий мінус - це мимовільна розрядка конденсаторів. Для підтримки заряду його регенерують через певні проміжки часу, на що витрачається додатковий час.

Зовні оперативна пам'ять персонального комп'ютера являє собою модуль з мікросхем (8 або 16 штук) на друкованій платі. Модуль вставляється в спеціальний роз'єм на материнській платі.

Static RAM (SRAM)

Статична оперативна пам'ять

Статична RAM - найбільш швидка з доступних на сьогоднішній день, вона має типовий час доступу близько 25 наносекунд. Статична RAM дорожча і дозволяє зберігати на чіп тільки чверть від того обсягу даних, який може зберігати DRAM, так як SRAM використовує два транзистора для зберігання одного біта проти одного у DRAM. На відміну від DRAM, статична RAM дозволяє зберігати дані до тих пір, поки

на чіп подається живлення. Транзистори попарно з'єднані таким чином, що тільки один з них може бути або включений, або вимкнений, в той час як інший завжди включений.

Динамічна оперативна пам'ять

Цей тип фізичної пам'яті використовується в більшості персональних комп'ютерів. Динамічна RAM використовує для зберігання даних конденсатори (поодинокі транзистори заряджають і розряджають їх), які з часом розряджаються і тому вимагають постійної підзарядки для збереження цілісності даних. Це здійснюється таким чином, що між періодами доступу до пам'яті надсилається електричний струм, що відновлює заряд на конденсаторах для підтримки цілісності даних, яка не може бути досягнута, якщо відбувається розрядка конденсаторів. Читання даних з DRAM розряджає конденсатори, тому дані повинні бути перезаписані відразу після читання.

Термін "динамічна" показує, що дані в пам'яті повинні постійно оновлюватися (перезаряджатися), інакше вони будуть втрачені. RAM (пам'ять з випадковим доступом, random-access memory) іноді іменується як DRAM, щоб відрізнити її від статичної пам'яті (SRAM). Статична RAM швидше і стабільніше, ніж динамічна RAM, але споживає велику потужність і дорожча.

Основні характеристики напівпровідникової пам'яті

Напівпровідникова пам'ять має велике число характеристик і параметрів, які необхідно враховувати при проектуванні систем:

1. Ємність пам'яті визначається числом біт інформації, що зберігається. Ємність кристала зазвичай виражається також в бітах і становить 1024 біта, 4 Кбіт, 16 Кбіт, 64 Кбіт і т.п. Важливою характеристикою кристала є інформаційна організація кристала пам'яті $M \times N$, де M - число слів, N - розрядність слова. Наприклад, кристал ємністю 16 Кбіт може мати різну організацію: $16 \text{ К} \times 1$, $4 \text{ К} \times 2$ $\text{К} \times 8$. При однаковому часу звернення пам'ять з більшою шириною вибірки володіє більшою інформаційною ємністю.

2. Часові характеристики пам'яті.

Час доступу - часовий інтервал, який визначається від моменту, коли центральний процесор виставив на шину адреси адресу необхідного елементу пам'яті і послав по шині управління наказ на читання або запис даних, до моменту здійснення зв'язку адресується осередку з шиною даних.

Час відновлення - це час, необхідний для приведення пам'яті в початковий стан після того, як ЦП зняв з ША - адреса, з ШУ - сигнал "читання" або "запис" і з ШД - дані.

3. Питома вартість пристрою, що запам'ятовує визначається відношенням його вартості до інформаційної ємкості, тобто визначається вартістю біта інформації, що зберігається.

4. Споживання енергії (або потужність, що розсіюється) наводиться для двох режимів роботи кристала: режиму пасивного зберігання інформації і активного режиму, коли операції запису і зчитування виконуються з номінальною швидкістю. Кристали динамічної МОП-пам'яті в резервному режимі споживають приблизно в десять разів менше енергії, ніж в активному режимі. Найбільше споживання енергії, яке не залежить від режиму роботи, характерно для кристалів біполярної пам'яті.

5. Щільність упаковки визначається площею запам'ятовуючого елемента і залежить від числа транзисторів в схемі елемента і використовуваної технології. Найбільша щільність упаковки досягнута в кристалах динамічної МОП-пам'яті.

Інтерфейси вводу/виводу

Інтерфейс- це сукупність програмних і апаратних засобів, призначених для передачі інформації між компонентами ЕОМ і включають в себе електронні схеми, лінії, шини і сигнали адрес, даних і управління, алгоритми передачі сигналів і правила інтерпретації сигналів пристроями.

Інтерфейси характеризуються такими параметрами:

- пропускна здатність - кількість інформації, яка може бути передана через *інтерфейс* в одиницю часу;
- максимальна частота передачі інформаційних сигналів через *інтерфейс*;
- максимально допустима відстань між пристроями, що сполучаються;
- загальне число проводів (ліній) в *інтерфейсі*;
- інформаційна ширина *інтерфейсу* - число біт або байт даних, що передаються паралельно через *інтерфейс*.

До динамічних параметрів інтерфейсу відноситься час передачі окремого слова і блоку даних з урахуванням тривалості процедур підготовки та завершення передачі.

Розробка систем введення-виведення вимагає вирішення цілої низки проблем, серед яких виділимо наступні:

- необхідно забезпечити можливість реалізації ЕОМ зі змінним складом устаткування, в першу чергу, з різним набором пристроїв введення-виведення, з тим, щоб користувач міг вибирати конфігурацію машини відповідно до її призначення, легко додавати нові пристрої і відключати ті, у використанні яких він не потребує;
- для ефективного і високопродуктивного використання обладнання комп'ютера слід реалізувати паралельну в часі роботу процесора над обчислювальною частиною програми і виконання периферійними пристроями процедур введення-виведення;
- необхідно спростити для користувача і стандартизувати програмування операцій введення-виведення, забезпечити незалежність програмування вводу-виводу від особливостей того чи іншого периферійного пристрою;
- в ЕОМ повинно бути забезпечено автоматичне розпізнавання і реакція процесора на різноманіття ситуацій, що виникають в ПВВ (готовність пристрою, відсутність носія, різні порушення нормальної роботи і ін.).

Головним напрямком вирішення зазначених проблем є **магістрально-модульний спосіб** побудови ЕОМ: всі пристрої, що становлять комп'ютер, включаючи і мікропроцесор, організовуються у вигляді модулів, які з'єднуються між собою загальною магістраллю. *Обмін інформацією* по магістралі задовольняє вимогам деякого загального *інтерфейсу*, встановленого для магістралі даного типу. Кожен *модуль* підключається до магістралі за допомогою спеціальних інтерфейсних схем.

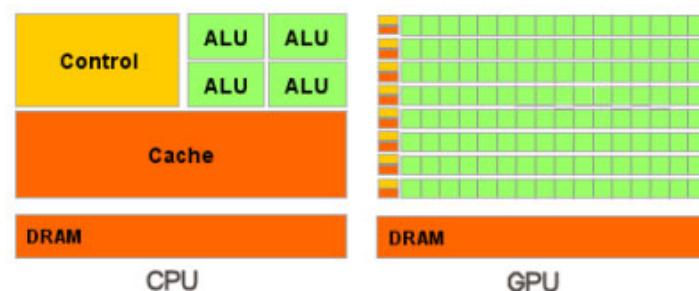
На інтерфейсні схеми модулів покладаються такі завдання:

- забезпечення функціональної та електричної сумісності сигналів і протоколів обміну модуля і системної магістралі;
- перетворення внутрішнього формату даних модуля у формат даних системної магістралі і назад;
- забезпечення сприйняття єдиних команд обміну інформацією і перетворення їх в послідовність внутрішніх керуючих сигналів.

Ці інтерфейсні схеми можуть бути досить складними і за своїми можливостями відповідати універсальним мікропроцесорам. Такі схеми прийнято називати контролерами.

Контролери мають високий ступінь автономності, що дозволяє забезпечити паралельну в часі роботу периферійних пристроїв і виконання програми обробки даних мікропроцесором.

Архітектура GPU (ядра відеокарти)



Основні відмінності між архітектурами CPU і GPU можна перелічити наступним чином:

CPU виконує 1–2 потоки обчислень на одне процесорне ядро, тоді як GPU можуть підтримувати до 1024 потоків на кожний мультипроцесор. Якщо перемикання з одного потоку на інший для CPU вимагає сотні тактів, то GPU може перемикає кілька потоків за один такт.

На відміну від сучасних універсальних CPU, GPU призначені для паралельних обчислень з великою кількістю арифметичних операцій. Значно більша кількість транзисторів GPU працює за прямим призначенням — обробці масивів даних, а не керує виконанням (контроль потоку) невеликої кількості послідовних обчислювальних потоків.

Виконання обчислень на GPU демонструє відмінні результати у алгоритмах, які використовують паралельну обробку даних. Це означає, що коли одну і ту ж послідовність математичних операцій застосовують до великого обсягу даних. При цьому кращі результати досягаються, якщо відношення кількості арифметичних інструкцій до кількості обробок пам'яті достатньо велике.

Основні застосування, в яких зараз використовуються обчислення на GPU: аналіз та обробка зображень і сигналів, обчислювальна математика, системи управління базами даних, криптографія, адаптивна променева терапія, астрономія,

комп'ютерне зороозріння, цифрове кіно та телебачення, електромагнітні симуляції, магнітно-резонансна томографія, нейромережі.

Технологія CUDA - це програмно-апаратна обчислювальна архітектура від NVIDIA, яка базується на розширенні мови C і надає можливість організації доступу до набору інструкцій графічного прискорювача та керування його пам'яттю при організації паралельних обчислень.

Скорочений конспект лекцій з дисципліни бази даних

Розробив: Євген Гофман, доцент каф. програмних засобів

Зміст

1. Загальні відомості про системи баз даних.....	2
Файлові системи та системи баз даних.....	2
Архітектура клієнт сервер.....	3
2. Теорія проектування реляційних баз даних	4
Побудова ER - діаграми	4
Об'єднання локальних подань у загальну ER - діаграму	6
Реляційна модель даних. Сім правил побудови ефективної моделі даних	7
Аномалії ненормалізованого відношення. Покроковий метод нормалізації баз даних	7
3. Мова структурованих запитів (SQL)	9
Загальні відомості	9
Створення таблиць і зв'язків	10
Створення індексів.....	10
Запити на вибірку даних (SELECT)	11
Запити на модифікацію даних (INSERT, UPDATE, DELETE).....	12
4. Безпека та цілісність даних в реляційних СКБД.....	13
Відновлення баз даних	13
Паралелізм.....	13
Привілеї.....	14
Контрольний слід виконання операцій.....	14
Безпека даних. Вибірче керування доступом.....	15
Цілісність даних. Мінімізація впливу людського фактору	16
4. Розподілені бази даних	17
Стратегії розподілу даних	17
Підтримка цілісності даних в розподілених базах даних	18
6. Концепція сховищ даних	19
7. Бази даних NoSQL.....	20
Бази даних «ключ-значення»	20
Бази даних документів	20
Графові бази даних	20
Переваги використання NoSQL баз даних.....	20

1. Загальні відомості про системи баз даних

Файлові системи та системи баз даних

Відомі два підходи до організації збереження інформації: **файлові системи та системи баз даних**.

З точки зору прикладної програми файл – це пойменована область зовнішньої пам'яті, до якої можна записувати або зчитувати дані. Така організація дозволяє досягти високої швидкості обробки інформації, але має цілу низку недоліків.

По-перше, така організація неодмінно викликає дублювання даних.

По-друге, дублювання даних при обробці може призвести до їхнього протиріччя між собою.

Наприклад, інформація про товарну продукцію може зберігатися у декількох відділах: відділі закупівель та відділі збуту. Оскільки зміна цих даних може здійснюватися різними особами, то відсутня можливість виявити порушення несуперечності інформації, що зберігається.

Крім того, формати даних жорстко зафіксовані в обробляючих програмах, що викликає виникнення можливих помилок, а також незручності в разі необхідності модифікації цих форматів.

Тому виникає необхідність відокремити дані від їх опису, визначити таку організацію зберігання даних із урахуванням зв'язків між ними, яка дозволила би використовувати ці дані одночасно для багатьох прикладних програм.

В результаті виникла нова концепція організації інформаційного забезпечення – концепція інтеграції даних, центральне місце в якій займає поняття «бази даних». При такому підході всі дані зберігаються в одному місці окремо від прикладних програм.

Централізований підхід щодо збереження та керування даними має наступні переваги:

1. Значно зменшується надлишок інформації.
2. Зменшення надлишку інформації дозволяє усунути протиріччя та забезпечити логічну цілісність даних.
3. Доступ до даних може здійснюватися одночасно багатьма прикладними програмами чи користувачами.
4. Завдяки повному контролю над базою даних з'являється можливість забезпечення заходів безпеки баз даних.

База даних – це певним чином організована сукупність даних, які відображають стан об'єктів деякої предметної області та відносини між цими об'єктами.
--

Предметною областю (ПО) називається сукупність об'єктів, предметів реального світу, що розглядаються у контексті задачі, що вирішується. У якості предмету реального світу в даному визначенні можуть бути як матеріальний об'єкт, так і процес, явище тощо.

Ціль створення БД міститься у відокремленні подання БД користувачу від її фізичного подання.

Виділяють зовнішній рівень – це вміст БД, яким бачить його окремий користувач (тобто для користувача зовнішнє подання саме й є база даних). Подання користувачем деякої задачі відображає його конкретні інформаційні потреби. Зовнішнє подання містить тільки ті об'єкти ПО, їхні характеристики та зв'язки між ними, що цікавлять користувача. Зовнішнє подання з точки зору прикладного програміста відображає елементи даних у їх взаємозв'язку.

Та внутрішній (фізичний) рівень пов'язаний з організацією зберігання даних на фізичних носіях інформації. З внутрішнім поданням зв'язується внутрішня схема БД, яка визначає типи структур даних, що використовуються, та їх взаємозв'язки. На цьому рівні здійснюється взаємодія СКБД з методами

доступу операційної системи з метою розміщення даних на запам'ятовуючих пристроях, створення індексів, вибірки даних тощо. На внутрішньому рівні зберігається наступна інформація:

- розподіл дискового простору для зберігання даних і індексів;
- опис подробиць збереження записів (із вказуванням реальних розмірів елементів даних);
- відомості про розміщення записів;
- відомості про стиснення даних і обраних методах їхнього шифрування.

Система баз даних – це система спеціальним чином організованих даних (баз даних), програмних, технічних, мовних, організаційно-методичних засобів, які призначені для забезпечення централізованого накопичення та колективного багатоцільового використання даних.

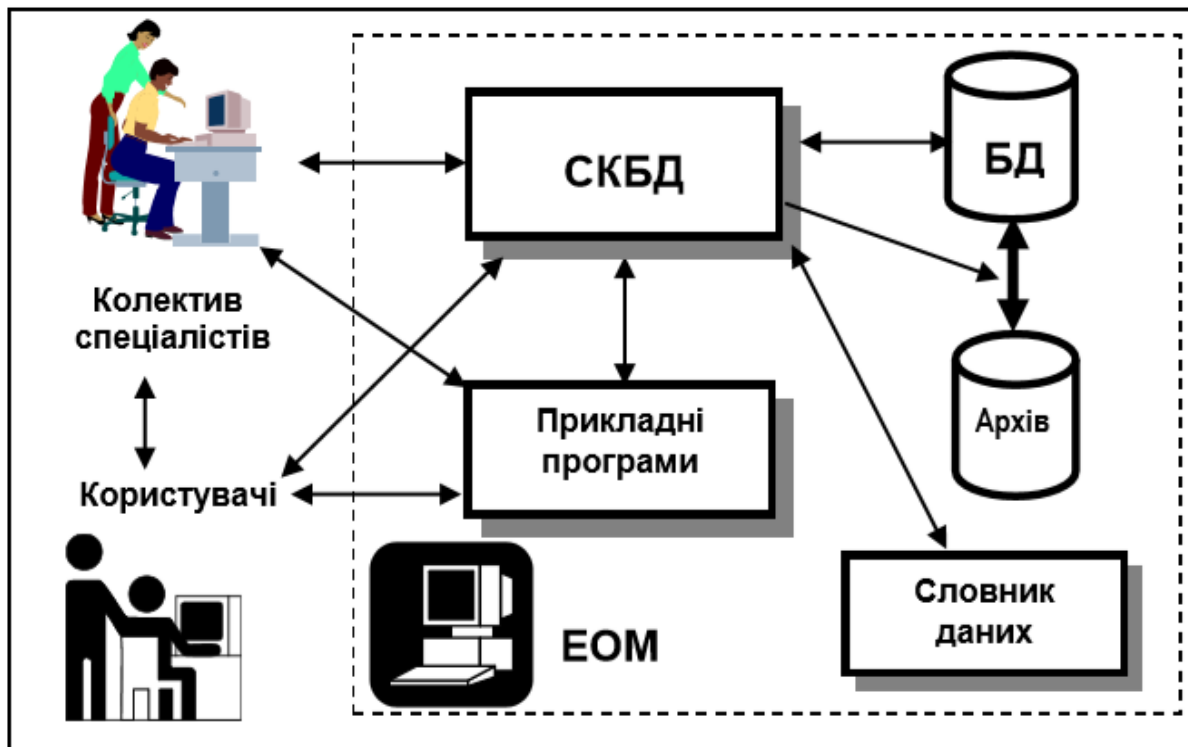


Рисунок 1.1 - Структура системи баз даних

СКБД – це комплекс програмних і лінгвістичних засобів, які забезпечують створення, завантаження, супроводження та експлуатацію бази даних

СКБД грає центральну роль у функціонуванні системи баз даних. Вона володіє засобами безпосереднього доступу до БД і надає кожній прикладній програмі інтерфейс для взаємодії з БД.

Архітектура клієнт сервер

При такому підході система баз даних складається з двох частин: набору клієнтів і сервера.

Клієнт (зазвичай називається локальним вузлом – local node), керує інтерфейсом користувача та логікою застосувань, діючи як складна робоча станція, на якій виконуються застосування користувача. Клієнт одержує від користувача запит, перевіряє синтаксис, генерує запит до бази даних на мові SQL або іншій мові бази даних і передає його серверу. Отримавши відповідь, клієнт форматує отримані дані для надання користувачеві.

Сервер (зазвичай його називають віддаленим вузлом – remote node) – це взагалі СКБД. Він знаходиться на спеціально виділених комп'ютерах і підтримує всі основні функції СКБД: визначення даних, обробка даних, захист і цілісність даних, адміністрування тощо. Сервер одержує та обробляє запити до бази даних, а потім передає отримані результати клієнту.

Таблиця 1.1 – Функції клієнта й сервера

Клієнт	Сервер
Керує інтерфейсом користувача	Приймає та обробляє запити з боку клієнта до бази даних
Приймає й перевіряє синтаксис запиту користувача	Перевіряє повноваження користувачів
Виконує застосування	Гарантує дотримання обмежень цілісності
Генерує запит до бази даних і передає його серверу	Виконує запити/оновлення й повертає результати клієнту
Відображає отримані результати користувачу	Підтримує системний каталог Забезпечує паралельний доступ до бази даних Забезпечує управління відновленням

2. Теорія проектування реляційних баз даних

Побудова ER - діаграми

На етапі концептуального проектування у якості моделі ПО використовується модель типу «сутність – зв’язок» (ER – Entity Relation), що була запропонована паном Ченом [33] у 1976 р.

Конструктивними елементами моделі служать сутності, атрибути та зв’язки. Основним конструктивним елементом являється сутність.

Користувач описує ті об’єкти ПО, що його зацікавили, за допомогою сутностей, потім визначає властивості сутностей, використовуючи атрибути, та, нарешті, описує відповідності між сутностями за допомогою зв’язків.

Сутність – це збиральне поняття, деяка абстракція реально існуючого об’єкта навколишнього світу, процесу чи явища.

Згідно з Ченом, об’єкти, що описуються сутностями, можна підрозділити на правильні об’єкти та слабкі об’єкти. Слабким називається об’єкт, який знаходиться в залежності від деякого іншого об’єкта, тобто він не може існувати, якщо не існує цей інший об’єкт.

Наприклад, сутність ПОСТАЧАЛЬНИК являється правильним об’єктом, а ПОСТАЧАЄМИЙ_ТОВАР – слабким об’єктом.

Атрибут – це поійменована характеристика сутності.

Атрибут являється засобом, за допомогою якого визначаються властивості сутності. Атрибути мають сенс тільки по відношенню до сутностей, яких вони визначають. Для ідентифікації конкретних екземплярів сутностей використовуються спеціальні атрибути – ідентифікатори (ключі). Це може бути один або декілька атрибутів, значення яких дозволяють беззаперечно відрізнати один екземпляр сутності від іншого.

Наприклад, ідентифікуючий атрибут (ключ) ТАБЕЛЬНИЙ_НОМЕР.

Зв’язки виступають у моделі в якості засобу, за допомогою якого представляються стосунки між двома й більш сутностями та атрибутами. Розрізняють три види зв’язків, що позначаються 1:1 (« один до одного »), 1:M (« один до багатьох ») та M:N (« багато до багатьох »).

Під час аналізу предметної області (ПО) можливе виявлення трьох типів зв’язку:

- сутність-сутність;
- сутність-атрибут;
- атрибут-атрибут.

Зв'язок «один до одного» (між двома типами сутностей)

При такому зв'язку кожному екземпляру сутності А відповідає один і тільки один екземпляр сутності В і, навпаки, кожному екземпляру сутності В відповідає один і тільки один екземпляр сутності А. При цьому ідентифікація екземплярів сутності унікальна в обох напрямках.

Зв'язок «один до багатьох» (між двома типами сутностей)

При такому зв'язку кожному екземпляру сутності А може відповідати 0, 1 або декілька екземплярів сутності В, але кожному екземпляру сутності В відповідає тільки один екземпляр сутності А. Це означає, що ідентифікація екземплярів сутності унікальна тільки в напрямку від В до А.

Наприклад, кожний продавець може обслуговувати декілька замовлень, але кожне замовлення обслуговується лише одним продавцем.

Зв'язок «багато до багатьох» (між двома типами сутностей)

Це відображення є найбільш складним видом зв'язку між сутностями. При цьому кожному екземпляру сутності А може відповідати 0, 1 або декілька екземплярів сутності В і навпаки, кожному екземпляру сутності В може відповідати 0, 1 чи декілька екземплярів сутності А, тобто ідентифікація екземплярів сутності неунікальна в обох напрямках.

Наприклад, зв'язок між сутностями ПРОДАВЕЦЬ і ПОКУПЕЦЬ.

Зв'язок «один до одного» (між двома атрибутами)

Якщо при описанні сутності використовуються два унікальних ідентифікатора, наприклад, КОД_КЛІЄНТА та НОМЕР_ПАСПОРТУ, то між ними існує зв'язок 1:1.

Зв'язок «один до багатьох» (між двома атрибутами)

Наприклад, між двома атрибутами НОМЕР_ГРУПИ й ПРИЗВИЩЕ_СТУДЕНТА існує зв'язок 1:М.

Зв'язок «багато до багатьох» (між двома атрибутами)

Наприклад, атрибути ВИКЛАДАЧ і ДИСЦИПЛІНА сутності ВИКЛАДАННЯ_ДИСЦИПЛІНИ пов'язані зв'язком М:М.

Сутності, атрибути і зв'язки є конструктивними елементами, з використанням яких створюється локальне подання – модель фрагменту предметної області.

Ключовий атрибут – це унікальний ідентифікатор екземпляру сутності. Розрізняють первинні і зовнішні ключі. Первинний ключ має містити множину унікальних значень, використовується для ідентифікації екземплярів сутності в окремій таблиці даних. Зовнішній ключ використовується для посилання на множину значень первинного ключа іншої таблиці, може містити повторювані значення, але тільки з переліку первинного ключа, на який посилається.

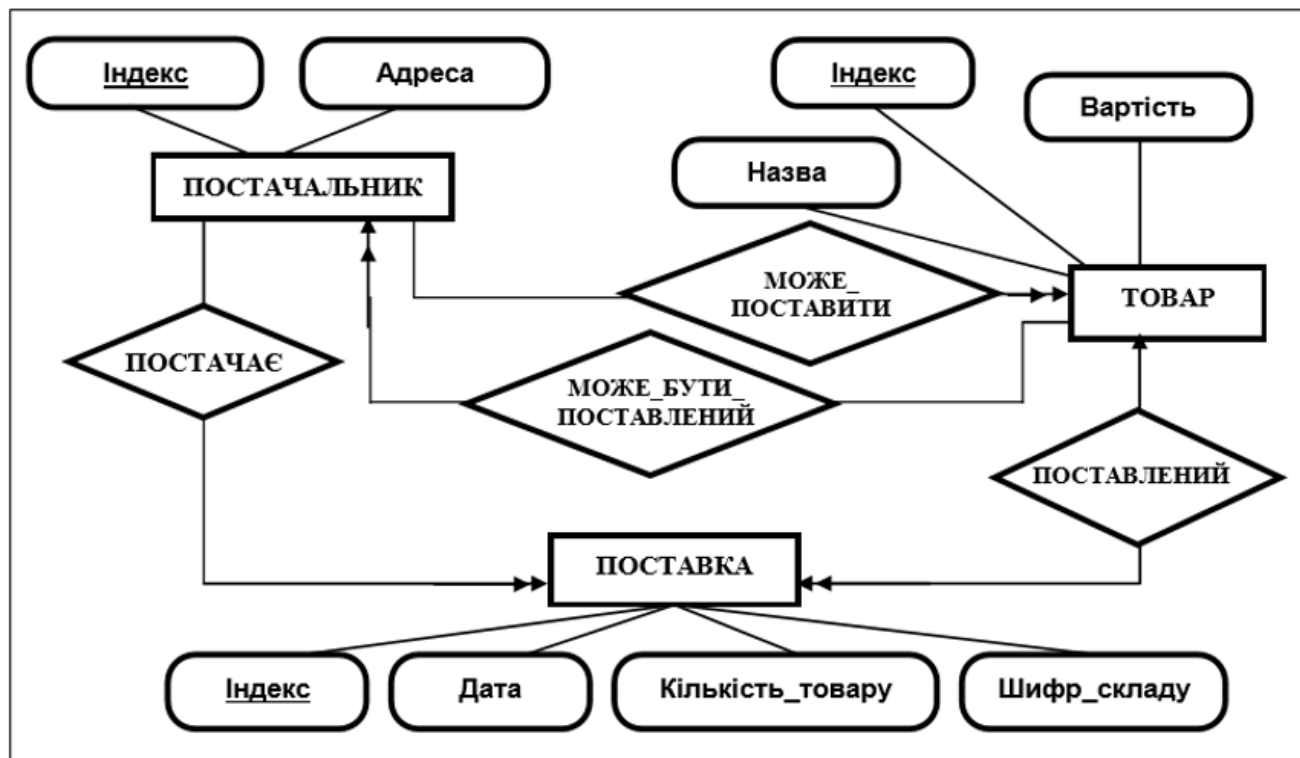


Рисунок 2.1 – приклад локального подання «Постачальник – Поставка – Товар»

Об'єднання локальних подань у загальну ER - діаграму

Після етапу формування локальних подань здійснюється їх об'єднання в єдину глобальну інформаційну структуру. Ця структура не тільки взаємно відповідає кожному поданню, але й уявляє собою інтегровану БД у стиснутому вигляді. При великій кількості подань (більше 5–6) зазвичай використовується бінарне об'єднання. При цьому результат об'єднання $N1$ об'єктів одного подання з $N2$ об'єктами другого подання дає $(N1 + N2 - X)$ об'єктів, де X – кількість об'єктів, які співпадають в об'єднаних поданнях.

Існують три основні концепції об'єднання: ідентичність, агрегація та узагальнення.

Два та більше елементів у моделі ідентичні, якщо вони мають однакове семантичне (змістове) значення. Але для ідентичних елементів зовсім необов'язково мати однакове синтаксичне представлення.

Агрегація дозволяє розглядати зв'язок між елементами моделі як новий елемент більш високого рівня.

Наприклад, елемент СЛУЖБОВЕЦЬ може бути сприйнятий як агрегація елементів ПРИЗВИЩЕ, НОМЕР_ПАСПОРТУ та АДРЕСА.

Узагальненням називається абстракція даних, яка дозволяє трактувати клас різних подібних об'єктів як один поименований узагальнений тип об'єкту.

Наприклад, клас об'єктів {собака, кішка, слон} може бути узагальнений в об'єкт ТВАРИНА.

Відношення – це підмножина декартового добутку доменів, або відношення = двомірна таблиця, де в стовпчиках визначаються атрибути, а у рядках екземпляри сутності.

Відношення поділяють на об'єктні та зв'язні.

Об'єктне відношення зберігає дані про об'єкти (екземпляри сутності).

Зв'язне відношення зберігає ключі двох або більше об'єктних відношень, тобто по ключам встановлюються зв'язки між об'єктами відношень.

Клас належності сутності повинний бути або **обов'язковим, або необов'язковим.**

Клас належності визначається по відношенню до зв'язаної сутності, залежить від того, чи можуть зберігатись екземпляри сутності, що не мають зв'язку з іншою сутністю.

Наприклад, клієнти, що нічого ще не купували, або товари, що ніколи ще не продавались.

Правило 1. Якщо ступінь бінарних зв'язків дорівнює 1:1 та клас приналежності обох сутностей є обов'язковим, то потрібне тільки одне відношення. Первинним ключем цього відношення може бути будь-який ключ із двох сутностей.
Правило 2. Якщо ступінь бінарного зв'язку дорівнює 1:1 і клас приналежності однієї сутності є обов'язковим, а іншої – необов'язковим, то необхідна побудова двох відношень – по одному для кожної сутності. При цьому ключ сутності повинний служити первинним ключем для відповідного відношення. Крім того, ключ сутності, для якої клас приналежності є необов'язковим, додається в якості зовнішнього ключа до відношення, яке виділене для сутності з обов'язковим класом приналежності.
Правило 3. Якщо ступінь бінарного зв'язку дорівнює 1:1 і клас належності жодної з сутностей не є обов'язковим, то необхідне використання трьох відношень: по одному для кожної сутності та одного для зв'язку.
Правило 4. Якщо ступінь бінарного зв'язку дорівнює 1:N і клас приналежності n-зв'язкової сутності є обов'язковим, то достатнім є використання двох відношень, по одному на кожну сутність, за умови, що ключ кожної сутності використовується в якості первинного ключа для відповідної сутності. Додатково ключ 1-зв'язкової сутності повинний бути доданий як атрибут (зовнішній ключ) до відношення, що відводиться n-зв'язковій сутності.
Правило 5. Якщо ступінь бінарного зв'язку дорівнює 1:n і клас приналежності n-зв'язкової сутності є необов'язковим, то необхідне формування трьох відношень: по одному на кожну сутність і одного відношення для зв'язку. Зв'язкове відношення повинне включати первинні ключі кожної сутності.
Правило 6. Якщо ступінь бінарних зв'язків дорівнює m:n, то для збереження даних потрібні три відношення поза залежністю від класу приналежності обох сутностей. Зв'язкове відношення повинно включати первинні ключі кожної сутності.
Правило 7. У випадку тристороннього зв'язку необхідно використовувати чотири відношення: по одному для кожної сутності й одне відношення для зв'язку. Зв'язкове відношення повинне включати первинні ключі кожної сутності. Аналогічно для n-стороннього зв'язку потрібно n+1 відношення.

Аномалії ненормалізованого відношення. Покроковий метод нормалізації баз даних

При проектуванні схеми реляційної БД атрибути групуються у відношення шляхом виводу реляційної схеми з розробленої ER-діаграми.

При експлуатації БД у відношенні можуть виникнути аномалії, пов'язані з виконанням операції підтримки його в актуальному стані. Виявити фактичний прояв тієї чи іншої аномалії можна, лише врахувавши конкретну семантику даних.

1. Аномалія оновлення (UPDATE). *Заміна керівника відділу призведе до необхідності внесення змін і модифікації по кожному працівнику даного відділу. Отже, для підтримки узгодженості даних необхідно виконати зміну не лише в кортежі бази, а цілий ряд змін, що може розглядатися як аномалія, оскільки характер самої зміни повинен стосуватися лише одного певного запису БД.*

2. Аномалія поповнення (INSERT). *Приймаючи на роботу нового співробітника, необхідно вносити до БД відомості не лише про нього, а й про керівника та тип контракту. Це також можна розглядати як аномалію, тому що не завжди можливо внести до БД відомості про відділ і контракт, оскільки часто співробітників беруть на роботу з випробувальним терміном і лише після його проходження підписують певний вид контракту та визначають відділ, де він працюватиме.*

3. Аномалія вилучення (DELETE). *Припустимо, що звільнились усі працівники певного відділу. Тоді разом з інформацією про них у БД буде втрачено інформацію про цей відділ. Якщо цю інформацію необхідно зберігати, то це також може розглядатися як аномалія.*

4. Надмірність. *Відомості про керівника відділу повторюються в ряді кортежів БД. Основні проблеми із зберіганням надлишку інформації пов'язані не лише з неефективним використанням пам'яті, а й із забезпеченням підтримки узгодженості цих даних.*

Усунути можливість аномалій або, принаймні, зменшити їхню кількість можна, використовуючи формальний метод, запропонований Е. Коддом і який дозволяє проектувальнику знаходити оптимальне угруповання атрибутів для кожного відношення схеми БД.

Термін функціональна залежність (ФЗ) означає: атрибут В відношення R функціонально залежить від атрибута А того ж відношення, якщо в кожному момент часу кожному значенню атрибута А відповідає тільки одне значення атрибута В, зв'язаного з А у відношенні R.

Якщо неключовий атрибут залежить тільки від частини ключа, то він знаходиться в **частковій функціональній залежності**.

Якщо для атрибутів А, В, С виконуються умови $A \rightarrow B$ і $B \rightarrow C$, але обернена залежність відсутня, то С знаходиться в **транзитивній залежності** від А.

У відношенні R атрибут В **багатозначно залежить** від атрибута А ($A \twoheadrightarrow B$), якщо кожному значенню А відповідає численність значень В, ніяк не зв'язаних з іншими атрибутами з R.

1НФ. Відношення знаходиться в першій нормальній формі (1НФ), якщо всі його неключові атрибути прості, тобто містять атомарні, неподільні значення.
2НФ. Відношення знаходиться в 2НФ, якщо воно знаходиться в 1НФ і в ньому відсутні часткові функціональні залежності описових атрибутів від атрибутів первинного ключа.
3НФ. Відношення знаходиться в 3НФ, якщо воно знаходиться в 2НФ і не містить транзитивних залежностей.
НФБК. Відношення знаходиться в НФБК тоді й тільки тоді, коли детермінанти є потенційними ключами. Іншими словами, на діаграмі функціональної залежності стрілки будуть починатися тільки з потенційних ключів.
4НФ. Відношення знаходиться в 4НФ, якщо воно знаходиться в НФБК і в ньому відсутні незалежні багатозначні залежності, тобто всі вони виділені (рознесені) в окремі відношення з тим самим ключем.

3. Мова структурованих запитів (SQL)

Загальні відомості

SQL може використовуватися для маніпуляції з даними (вибірка і модифікація), опрацювання структури бази даних (створення та вилучення таблиць та індексів) і адміністрування бази даних (визначення списку користувачів, призначення прав доступу). Використання SQL має багато переваг, найбільше важливими з яких є:

- SQL підтримується багатьма постачальниками СКБД. Програми, написані з використанням мови SQL, будуть працювати майже на будь-якому сервері бази даних;
- SQL простий у вивченні, оскільки в ньому для опису дій використовуються прості англійські слова, такі, як SELECT, INSERT, UPDATE, DELETE, COMMIT, ROLLBACK.

Таблиця 3.1 – Типи даних SQL

Тип даних	Розмір	Опис
1	2	3
BINARY	1 байт на знак	У полі цього типу можуть зберігатися дані будь-якого типу. Дані не перетворюються (наприклад, у текстові). Дані відображаються в тому ж вигляді, у якому вони вводяться в це поле.
BIT	1 байт	Значення «Так» (Yes) і «Ні» (No), а також поля, що містять одне з двох можливих значень.
TINYINT	1 байт	Ціле значення від 0 до 255.
MONEY	8 байтів	Ціле, що масштабується, від -922 337 203 685 477,5808 до 922 337 203 685 477,5807.
DATETIME (див. DOUBLE)	8 байтів	Дата або час; припустимий будь-який рік від 100 до 9999.
UNIQUEIDENTIFIER	128 бітів	Унікальний ідентифікатор, який використовується при викликах вилучених процедур.
REAL	4 байти	Число з крапкою, що плаває, та одинарною точністю від -3,402823E38 до -1,401298E-45 для негативних значень, від 1,401298E-45 до 3,402823E38 для позитивних значень або значення 0.

1	2	3
FLOAT	8 байтів	Число з крапкою, що плаває, та подвійною точністю від -1,79769313486232E-308 до -4,94065645841247E-324 для негативних значень, від 4,94065645841247E-324 до 1,79769313486232E308 для позитивних значень або значення 0.
SMALLINT	2 байти	Коротке ціле від -32 768 до 32 767 (див. «Примітки»).
INTEGER	4 байти	Довге ціле від -2 147 483 648 до 2 147 483 647.
DECIMAL	17 байтів	Тип даних для збереження точних числових значень від $-10^{28} - 1$ до $10^{28} - 1$. Точність (1–28) і фактор масштабування (від 0 до заданої точності) визначаються користувачем. За замовчуванням точність і фактор масштабування рівні відповідно 18 і 0.
TEXT	2 байти на знак	Від 0 до 2,14 Гбайт (поле MEMO)
IMAGE	Не обмежено	Від 0 до 2,14 Гбайт. Використовується для об'єктів OLE.
CHARACTER	2 байти на знак	Від 0 до 255 знаків.

Таблиця 3.2 – Команди мови SQL та їх призначення

Команда	Призначення
Команди визначення даних	
<i>alter table</i>	Змінює структуру таблиці
<i>create index</i>	Створює індекс
<i>create table</i>	Створює таблицю
<i>create view</i>	Створює подання
<i>Drop</i>	Вилучає таблицю, індекс, подання
Команди маніпуляції даними	
<i>Delete</i>	Вилучає запису таблиці
<i>Insert</i>	Додавляє запису в таблицю
<i>Update</i>	Змінює дані таблиці
Команда вибірки даних	
<i>Select</i>	Вибирає дані
Команди керування транзакціями	
<i>commit</i>	Робить зміни, проведені з початку транзакції, постійними
<i>rollback</i>	Відкочує всі проведені зміни до крапки зберігання або до початку транзакції
<i>savepoint</i>	Встановлює контрольну крапку, до якого згодом можна буде виконати відкат
Команди керування даними	
<i>check database</i>	Перевіряє цілісність бази даних
<i>grant</i>	Надає привілеї
<i>revoke</i>	Скасовує надані раніше привілеї

Створення таблиць і зв'язків

За допомогою команд визначення даних SQL можна створювати, вилучати та маніпулювати таблицями (добавляти, вилучати, переставляти стовпчики та змінювати їхні параметри). Щоб створити таблицю треба зробити, щонайменше, наступне: **задати ім'я таблиці, задати імена її стовпчиків, визначити тип даних для кожного стовпчика, визначити (або використовувати по умовчання) нульовий статус для кожного стовпчика** – припускається або забороняється використання в стовпчику нульових значень.

Створення баз даних

```
CREATE DATABASE ім'я_базу_даних;  
  
USE ім'я_базу_даних  
або  
DATABASE ім'я_базу_даних
```

Створення таблиць

```
create table customs  
(name varchar(25) not null,  
adress varchar(35) not null,  
phone char(8) null);
```

Обмеження на множину припустимих значень

```
create table salespeople  
(snum int not null primary key,  
sname char(10) not null,  
city char(10));
```

```
create table namefields  
(firstname char(10) not null,  
lastname char(10) not null,  
city char(10),  
primary key (firstname, lastname));
```

```
create table товар  
(код char(6) not null primary key  
check (код like '[A-Я] [A-Я] [0-9] [0-9] [0-9] [0-9]'),  
назва varchar(80) not null,  
тип char(12)  
default "невизначений" null  
check (тип in ("економіка", "психологія",  
"програмування", "невизначений")),  
ціна money null,  
кількість int null) ☞
```

Створення зв'язків між таблицями

```
create table покупці  
(ном integer not null primary key,  
прізвище char(15),  
ном_прод integer,  
foreign key (ном_прод) references продавці(ном_прод));
```

Рисунок 3.1 – приклад синтаксису SQL – команд, що використовуються для визначення даних

Створення індексів

У системах реляційних баз даних механізм індексації є засобом підвищення продуктивності, прискорюючи вибірку інформації з БД.

Індекс (index) – це упорядкований (в алфавітному або чисельному порядку) список вмісту стовпчиків або групи стовпчиків у таблиці.

Загальний вигляд команди

```
CREATE [UNIQUE] INDEX ім'я_індексу  
ON ім'я_таблиці (ім'я_стовпчика [,ім'я_стовпчика]..);
```

Приклади команд SQL

```
create index назва_інд  
on Товари (назва);
```

```
create index товар_інд  
on Товари (група, назва);
```

Рисунок 3.2 – приклад використання механізму індексації

Запити на вибірку даних (SELECT)

Для створення запитів на вибірку даних використовується конструкція SELECT. Результатом виконання запиту є таблиця, що зберігається в тимчасовому буфері серверу бази даних. Обрані дані можна використовувати для перегляду, друку звітів, формування графіків або, наприклад, додати їх до постійних (базових) таблиць бази даних.

Загальна форма конструкції select

```
SELECT [ALL|DISTINCT] список полів, що вибираються
FROM список таблиць
[WHERE умова вибірки]
[GROUP BY умова утворення
  [HAVING умова вибірки групи ] ]
[ORDER BY умова впорядкування];
```

Приклади запитів

```
select T.код, T.назва, T.ціна
from Товари [as] T;
```

```
select Товари.код, Товари.назва, Товари.ціна
from Товари;
```

Оператори завдання умов для речення where

Оператор, ключове слово	Приклад використання
оператори порівняння (=, <, >, >=, <=, <>, !=);	where ціна > 1000;
комбінації умовних і логічних операцій – (AND, OR, NOT)	where ціна < 5000 and ціна > 2000
діапазони (BETWEEN і NOT BETWEEN)	where ціна between 450 and 500
списки (IN, NOT IN)	where товар in («монітор», «принтер»)
невідомі значення (IS NULL і IS NOT NULL)	where телефон is null
відповідності символів (LIKE і NOT LIKE)	where телефон not like "415%"; where назва like "*БД*"

Приклади запитів

```
select прізвище, посада
from Посадовці
where прізвище > "Іваненко";
```

```
select *
from Продавці
where рейтинг > 100;
```

```
select *
from titles
where advance*2 > price + sales;
```

Рисунок 3.3 – приклад використання механізму відбору даних

Агрегатні функції

Функція	Результат
COUNT	Визначає кількість рядків або значень поля, обраних за допомогою запиту, що не є NULL-значеннями
SUM	Обчислює арифметичну суму всіх обраних значень даного поля
AVG	Обчислює середнє значення для всіх обраних значень даного поля
MAX	Обчислює найбільше зі всіх обраних значень даного поля
MIN	Обчислює найменше зі всіх обраних значень даного поля

Приклади

```
select avg(оклад*індекс)
from відомість;
```

```
select count (*)
from Покупці;
```

```
select count (all рейтинг)
from Покупці;
```

```
select назва, sum(сума)
from Продажі
where назва = "Монітор"
```

```
select count (distinct назва)
from Поставки;
```

Групування даних

```
SELECT список вибору
FROM список таблиць
[WHERE умови]
[GROUP BY список_групування];
```

Приклади

```
select музей, художник, count(картина)
from Музеї
group by музей, художник;
```

```
select художник
from Музеї
group by художник;
```

```
select distinct художник
from Музеї;
```

```
select музей, count(картина)
from Музеї
where painter = "Рубенс"
group by музей;
```

Рисунок 3.4 – приклади використання агрегатних функцій

Запити на модифікацію даних (INSERT, UPDATE, DELETE)

У SQL використовуються три команди модифікації даних:

- INSERT додає нові рядки до таблиці;
- UPDATE змінює існуючі в таблиці рядки;
- DELETE вилучає рядки з таблиці.

Оператор INSERT дозволяє додавати рядки за допомогою ключового слова VALUES або за допомогою оператора SELECT. Для кожного рядку, що додається, використовується свій оператор INSERT.

Порядок слідування стовпчиків в операторі INSERT може бути будь-яким, проте він повинний відповідати порядку слідування значень.

Якщо рядок, який додається, містить значення окремих стовпчиків таблиці, то стовпчики, що залишилися, (що не модифікуються) повинні припускати нульовий статус. У протилежному випадку команда буде заперечена.

Оператор SELECT у команді INSERT може містити агрегатні функції (Рисунок 3.5).

За допомогою команди DELETE із таблиці вилучається не окреме поле, а рядок цілком.

У предикаті команди DELETE можна використовувати підзапити.

Наприклад, для вилучення всіх покупців, що обслуговуються продавцями з Запоріжжя (Рисунок 3.5).

У предикаті команди UPDATE можна використовувати підзапити.

Наприклад, потрібно підвищити комісійні для всіх продавців, які обслуговують не менше двох покупців (Рисунок 3.5).

При відсутності речення WHERE зміни вносяться в усі значення стовпчиків, зазначених у SET.

– INSERT додає нові рядки до таблиці; – UPDATE змінює існуючі в таблиці рядки; – DELETE вилучає рядки з таблиці.	
Додавання нового рядку <pre>insert into authors select * from newauthors insert into Підсумки (дата, сума) select дата, sum(сума_замовлення) from Замовлення group by дата;</pre>	
Вилучення рядків <pre>delete from Замовлення where місто = «Донецьк»; delete from Склад_Замовлення where ном_заказа = any (select ном_заказа from Замовлення where місто = "Запоріжжя");</pre>	Зміна значень полів <pre>update Замовлення set ціна = ціна*2 where місто = «Київ»; update Продавці set ком = ком + 0.1 where 2 <= (select count(покуп_ном) from Покупці where Покупці.прод_ном=Продавці.прод_ном);</pre>

Рисунок 3.5 – приклади використання запитів на оновлення даних

4. Безпека та цілісність даних в реляційних СКБД

При експлуатації баз даних актуальною є проблема захисту інформації, пов'язана з можливістю виникнення наступних ситуацій:

- система може бути зруйнована під час виконання деяких програм, залишивши при цьому базу даних у непередбаченому стані;
- при виконанні конкуруючих програм між ними може виникнути конфлікт, який призведе до одержання неправильних результатів;
- дані можуть бути зіпсовані або змінені анонімним користувачем;
- оновлення можуть змінювати базу даних неприпустимим чином та інші.

Отже, система повинна забезпечувати функції захисту бази даних від подібних проблем, зокрема, відновлення, паралелізм, захист і цілісність.

Відновлення баз даних

Відновлення в системі баз даних означає, в першу чергу, відновлення самої бази даних, тобто повернення її в правильний стан, якщо якийсь збій зробив поточний стан неправильним або підозрілим.

Основний принцип, на якому будується таке відновлення, – це надмірність. Але ця надмірність будується на фізичному рівні та схована від користувача. Це означає, що, якщо будь-яка частина інформації, що міститься в базі даних, може бути реконструйована з іншої збереженої в системі надлишкової інформації, то база даних є такою, що відновлюється.

Транзакція – це логічна одиниця роботи, яка з точки зору відновлення даних розглядається та обробляється системою як єдина неподільна дія.

При цьому під транзакцією у загальному випадку розуміється не одиночна операція, а сукупність узгоджених операцій, що змінюють стан бази даних.

В стандарті ISO SQL/92 вказується, що транзакція автоматично запускається будь-яким оператором, який ініціює транзакцію та який виконується користувачем або програмою (наприклад, SELECT, INSERT або UPDATE). Зміни, внесені до бази даних в ході виконання даної транзакції, не будуть побачені будь-якими іншими транзакціями, що виконуються паралельно, доти, доки ця транзакція не буде явним чином завершена.

Системними називаються збої, що впливають на всі транзакції, що виконуються в даний момент, але не завдають фізичної шкоди самій базі даних. У випадку системного збою механізм відновлення системи баз даних повинний з'ясувати два питання:

– **Які транзакції не встигли завершитися до моменту збою та, отже, повинні бути скасовані.**

– **Які транзакції встигли завершитися до моменту збою, але відповідна інформація ще не переписалася із внутрішніх буферів системи до самої бази даних (фізичної) та, отже, повинні бути виконані повторно.**

Паралелізм

У більшості систем керування реляційними базами даних для вирішення проблем паралелізму застосовується механізм, який має назву блокування (locking). Принцип його роботи полягає в тому, що коли користувач вибирає якісь дані з БД, СКБД автоматично зачиняє їх «на замок» із тим, щоб ніякий інший користувач не міг оновлювати ці дані доти, доки блокування не буде зняте.

Існує два основні види блокування: блокування без взаємного доступу (монопольне блокування), що має також назву X-блокування (X lock – eXclusive lock) і блокування зі взаємним доступом (спільне блокування), що має назву S-блокування (S-lock – Shared lock).

X- і S-блокування називають також блокуваннями запису та читання, відповідно. Між цими видами блокування існують такі залежності:

- 1. Якщо транзакція А блокує кортеж р без можливості взаємного доступу (Х-блокування), то запит іншої транзакції В із блокуванням цього кортежу р буде скасований.**
- 2. Якщо транзакція А блокує кортеж р із можливістю взаємного доступу (S-блокування), то**
 - запит із боку деякої транзакції В на Х-блокування кортежу р буде заборонений;**
 - запит із боку деякої транзакції В на S-блокування кортежу р буде прийнятий (тобто транзакція В також буде блокувати кортеж р за допомогою S-блокування).**

Блокування в транзакціях зазвичай задаються неявним чином. Наприклад, запит «на витяг кортежу» є неявним запитом із S-блокуванням, а запит «на оновлення кортежу» – неявним запитом із Х-блокуванням відповідного кортежу. Зокрема, у стандарті SQL не передбачене явне завдання блокування.

Привілеї

Привілеї (privileges)– це сукупність дій, які дозволено виконувати користувачеві по відношенню до конкретної таблиці або подання.

В стандарті ISO визначаються наступний набір привілеїв:

- SELECT – право вибирати дані з таблиці;
- INSERT – право додавати нові рядки до таблиці;
- UPDATE – право змінювати дані в таблиці;
- DELETE – право вилучати рядки з таблиці;
- REFERENCES – право посилатися на стовпці вказаної таблиці в описах вимог підтримки цілісності даних;
- USAGE – право використання доменів, перевірки наборів символів і трансляції.

Привілеї (права або повноваження) пов'язані як с користувачами, так і з таблицями. Коли користувач створює таблицю або подання, він автоматично стає її володарем і отримує по відношенню до неї повний набір привілеїв. Для надання доступу до цих об'єктів іншим користувачам необхідно, щоб володар явним чином надав їм ці права.

Контрольний слід виконання операцій

Якщо, наприклад, суперечливість даних призводить до підозри, що зроблено несанкціоноване втручання до бази даних, то контрольний слід повинний бути використаний для прояснення ситуації й підтвердження того, що всі процеси знаходяться під контролем. Якщо це не так, то контрольний слід допоможе, принаймні, виявити порушника.

Для зберігання контрольного сліду зазвичай використовується особливий файл, у якому система автоматично записує всі операції, що виконані користувачем при роботі з БД. У деяких системах контрольний слід може фізично зберігатися у файлі відновлення даних, а в інших – в окремому файлі. У будь-якому випадку користувачі повинні мати можливість звертатися до контрольного сліду за допомогою традиційної мови реляційних запитів (звичайно, за умови, що такі запити санкціоновані!).

Типовий запис у файлі контрольного сліду може містити таку інформацію:

- запит (вихідний текст запиту);
- термінал, з якого була викликана операція;
- користувач, який задав операцію;
- дата й час запуску операції;
- базові відношення, кортежі й атрибути, до яких зверталися в процесі виконання операції;
- старі значення;
- нові значення.

Безпека даних. Вибірче керування доступом

У загальному випадку правила безпеки задаються п'ятьма компонентами (рисунок 4.1):

1. Ім'я правила, під яким воно буде зареєстровано в системному каталозі.
2. Одна або декілька привілеїв, які задані за допомогою директиви GRANT. У якості привілеїв можна використовувати одну з наведених на слайді:

- SELECT [(список_атрибутів_через_кому)] – витяг;
- INSERT – вставка;
- UPDATE [(список_атрибутів_через_кому)] – оновлення;
- DELETE – вилучення;
- ALL.

Специфікація ALL є скороченим записом використання всіх привілеїв – SELECT (із всіма атрибутами), INSERT, UPDATE (із всіма атрибутами) і DELETE.

3. Діапазон, до якого це правило застосовується та який є деякою підмножиною кортежів єдиного відношення. Він задається за допомогою директиви ON.
4. Один або декілька ідентифікаторів користувачів, які мають задані привілеї в заданому діапазоні, що вказується за допомогою директиви TO. Якщо задана директива ALL, то вона означає всіх відомих системі користувачів.
5. Реакція на порушення правил, яка диктує системі визначені дії, якщо користувач порушив правила безпеки. Вона задається директивою ON ATTEMPTED. По умовчання приймається відмова у виконанні дії, що запитувалася. Проте в більш складних ситуаціях може виявитися корисним застосувати іншу дію; наприклад, у деяких випадках може знадобитися завершити виконання програми або заблокувати термінал.

Формат команди	Список привілеїв
<i>CREATE SECURITYRULE</i> правило <i>GRANT</i> список_привілеїв_через_кому <i>ON</i> вираз <i>TO</i> список_користувачів_через_кому [<i>ON ATTEMPTED VIOLATION</i> дія];	• <i>SELECT</i> [(список_атрибутів_через_кому)] – витяг; • <i>INSERT</i> – вставка; • <i>UPDATE</i> [(список_атрибутів_через_кому)] – оновлення; • <i>DELETE</i> – вилучення; • <i>ALL</i>.
Приклад 1	Приклад 2
CREATE SECURITYRULE EX2 GRANT SELECT (S#, SNAME, CITY) ON S TO Іваненко, Петренко, Сидоренко;	CREATE SECURITYRULE EX3 GRANT SELECT, INSERT, UPDATE (код_товару, назва), DELETE ON Товар WHERE Товар. ціна < 100\$ TO ALL;

Рисунок 4.1 – визначення правил безпеки

Цілісність даних. Мінімізація впливу людського фактору

Термін цілісність використовується для опису точності та коректності даних у БД. Як при забезпеченні безпеки, так і при підтримці цілісності система повинна містити відомості про ті правила, які користувачу не можна порушувати. При цьому СКБД повинна стежити за дотриманням заданих правил при виконанні користувачем деяких операцій. Відмінністю обмежень цілісності від правил безпеки є те, що перші не залежать від користувача.

Обмеження цілісності зручно класифікувати за чотирма типами (рисунок 4.2):

1. Обмеження цілісності домену визначають множину значень, що утворюють цей домен, тобто просто перераховуються всі припустимі значення домену.
2. Обмеження цілісності атрибута задаються у вигляді окремої частини визначення цього атрибута при створенні відношення (таблиці). Ці обмеження завжди перевіряються негайно, тобто будь-яка спроба (за допомогою операцій вставки INSERT або оновлення UPDATE) ввести до БД некоректне значення атрибута буде заборонена.
3. Обмеження цілісності відношення є правилом, що задається тільки для одного якогось відношення. При цьому вони також перевіряються негайно. *Наприклад, можна задати, що постачальники зі Львову мають статус 20.*
4. Обмеження цілісності бази даних є правилом, що задається для двох і більш пов'язаних між собою відношень. *Наприклад, обмеження, що означає, що постачальники зі статусом менше 20 не можуть поставляти товари в кількості більш 500.*

Приклади, тип 1	Приклади, тип 3
<pre>CREATE DOMAIN Колір CHAR (15) FORALL Колір (Колір = «блакитний» OR Колір = «рожевий» OR Колір = «чорний» OR Колір = «зелений»)</pre> <pre>CREATE DOMAIN Колір CHAR (15) VALUES («блакитний», «рожевий», «чорний», «зелений»)</pre>	<pre>CREATE INTEGRITY RULE правило_5 FORALL Постачальник (IF Постачальник.місто= «Львів» THEN Постачальник. статус =20) ON ATTEMPTED VIOLATION REJECT;</pre>
Приклади, тип 4	
<pre>CREATE INTEGRITY RULE правило_6 FORALL Постачальник (FORALL Постачання (IF Постачальник.статус <20 AND Постачальник. Код_ постачальника = Постачання. Код_ постачальника THEN Постачання.Кількість ≤ 500)) [ON ATTEMPTED VIOLATION REJECT];</pre>	<pre>CREATE INTEGRITY RULE правило_6 IF Постачальник.статус <20 AND Постачальник.Код_ постачальника = Постачання. Код_ постачальника THEN Постачання.Кількість ≤ 500 [ON ATTEMPTED VIOLATION REJECT];</pre>

Рисунок 4.2 – приклади правил підтримки цілісності даних

4. Розподілені бази даних

У зв'язку з широким поширенням комп'ютерних мереж і хмарових сервісів намітилася тенденція до децентралізації збереження та обробки даних.

Децентралізований підхід відображає організаційну структуру компанії, що логічно складається з окремих підрозділів, які фізично розподілені по різних офісам, відділенням або філіям, причому кожна окрема структурна одиниця має справу з особистим набором даних, які обробляються.

Розробка розподілених баз даних, які відображають організаційні структури підприємств, дозволяють зробити загальнодоступними дані, що підтримуються кожним підрозділом, забезпечивши при цьому їх зберігання саме в тих місцях, де вони частіше використовуються. Такий підхід розширює можливості спільного використання даних, одночасно підвищуючи ефективність доступу до них.

Розподілена база даних (РБД) – це сукупність логічно взаємозв'язаних баз даних, фізично розподілених у комп'ютерній мережі.

У розподіленій БД (Distributed Database – DDB) не всі дані зберігаються централізовано; вони розподілені по мережі вузлів, які відділені географічно, але зв'язані комунікаційними лініями. Кожний вузол має свою власну базу даних; крім того, він може звертатися до даних, які зберігаються на інших вузлах.

Розподілена СКБД (РСКБД) – це програмний комплекс, який призначений для керування розподіленими базами даних і який дозволяє зробити розподіленість системи прозорою для користувача.

Прозорість означає незалежність даних у розподілених системах, яка передбачає, що користувач у цій системі працює з розподіленою базою даних як з логічно цілісною сукупністю даних, тобто на його роботу не повинно впливати те, як дані розподілені між вузлами мережі.

Стратегії розподілу даних

Розподілена стратегія без дублювання. За такої стратегії визначають дані, які потрібно зберігати в кожному вузлі мережі. При цьому розподілену базу даних проектують як неперетинні між собою підмножини даних, розподілені по вузлах мережі. Проектування даних за такої стратегії є складною задачею.

Ключовим фактором, який впливає на надійність і доступність бази даних, є так звана локалізація посилань. Якщо БД розподілена так, що дані, розміщені в цьому вузлі, викликаються винятково його користувачем, то це свідчить про високий ступінь локалізації посилань.

Якщо подібне розчленування даних здійснити неможливо та для виконання запитів користувача потрібно звертатись за інформацією до інших вузлів, то це свідчить про невисокий ступінь локалізації посилань.

Розглянута стратегія підходить для тих предметних областей, в яких практично немає дублювання даних у різних вузлах мережі; користувач кожного вузла працює із своїми файлами та досить рідко використовує дані інших вузлів мережі.

Економічні задачі за своїми інформаційними властивостями характеризуються дуже тісними інформаційними взаємозв'язками, тому для даного класу задач реалізація цієї стратегії досить складна, неефективна й недоцільна.

Розподілена стратегія з дублюванням. Ця стратегія полягає в тому, що база даних проектується як при централізованому підході, але фізично дублюється в кожному вузлі мережі. Кожний вузол має свою копію, продубльовану стільки разів, скільки вузлів у мережі.

Стратегія розподілу з дублюванням найбільш ефективно розв'язує проблеми доступу та вибірки даних із мінімальними витратами часу. Система досить проста при проектуванні.

Переваги цієї стратегії полягають у тому, що зменшуються витрати на передавання інформації та вірогідність виникнення черг, коли кілька користувачів одночасно звертаються до одного файлу БД. Але водночас цю стратегію важко контролювати з точки зору дублювання даних, чим ускладнюється реалізація проблеми узгодженості та цілісності даних. Значно складнішими є проблеми адміністрування та підтримування БД даних в актуальному стані.

Однак разом з перевагами цей підхід характеризується складністю адміністрування та розв'язання проблеми узгодженості файлів БД у різних вузлах мережі. Ця проблема узгодженості досить гостро може постати тоді, коли зв'язок у мережі порушується та в копіях в різних вузлах виникають розбіжності. У цьому разі потрібно розробити спеціальний механізм для узгодження деяких копій БД.

Змішана стратегія розподілу даних поєднує два підходи, пов'язані з розподілом без дублювання та з дублюванням даних, з метою використання їх переваг. Ця стратегія поділяє БД на багато логічних фрагментів, як це зроблено в стратегії розподілу без дублювання.

Крім того, вона повинна дозволяти мати довільну кількість фізичних копій кожного фрагмента. Такий підхід до створення РБД дає змогу дублювати дані довільну кількість разів і в кожному вузлі; водночас у кожному вузлі може міститися потрібна частина бази даних. Система, побудована за цією стратегією, допускає досить просту реалізацію паралельної обробки даних, що скорочує час відгуку системи. Ця стратегія також забезпечує дуже велику надійність даних, за рахунок дублювання даних їх легко можна відновити при помилках чи збоях обладнання. Однак, як і при стратегії дублювання, виникає проблема узгодженості копій бази даних у всіх вузлах мережі.

Локальні дані – це дані, що підтримуються у власній базі даних вузла мережі.
Глобальні дані – це дані, що підтримуються в базі даних, розташування якої відрізняється від розташування хоча б одного з її користувачів.
Агент – це процес, який відповідає за виконання транзакції на конкретному вузлі.
Локальна транзакція – це транзакція, що складається з одного агента.
Глобальна транзакція – це транзакція, що складається з декількох агентів.

Підтримка цілісності даних в розподілених базах даних

В РБД внаслідок розподілення даних по окремим вузлам проблема підтримки цілісності дещо ускладнюється. У розподілених системах виділяють чотири типи збоїв:

- збій транзакції,
- збій вузла,
- збій носія (диска),
- збій комунікаційної лінії.

У всіх перелічених ситуаціях збою транзакції необхідно перервати та виконати відкат бази даних. Для попередження «зависання» системи необхідний контроль за часом виконання транзакції. Якщо ліміт часу, відведеного для виконання транзакції, перевищено, то її потрібно відмітити як таку, що підлягає аварійному завершенню незалежно від результату початкових кроків, і всі файли, задіяні при її виконанні, перевести в початковий стан.

Для підтримання цілісності бази даних при різного роду збоях використовують протоколи журналізації, що вміщують інформацію про всі зміни, які вносяться до бази даних під час виконання транзакції. В одну транзакцію доцільно об'єднувати операції, що виконують логічно зв'язані між собою зміни файлів бази даних. Підтримка транзакцій – це основа забезпечення цілісності БД. У сучасних розподілених СКБД підтримку транзакцій можна виконувати за допомогою двох методів:

- 1. двофазною фіксацією транзакцій;**
- 2. реплікацією (дублюванням) даних.**

При використанні методу з двофазною фіксацією, транзакцію виконують у два етапи, спираючись на такі правила:

1. Якщо хоча б один вузол не може з будь-яких причин зафіксувати транзакцію, то розподілена транзакція припиняється на всіх інших вузлах, які беруть участь у її виконанні.
2. Якщо всі вузли погоджуються з фіксацією транзакції, то вона фіксується на всіх вузлах, які беруть участь в її реалізації.

При **двофазній фіксації** транзакції працюють одночасно з однією розподіленою базою даних. У випадку з **дублюванням даних** кожна транзакція має змогу копіювати необхідні для її виконання дані з розподіленої бази даних і працювати з нею як із своєю особистою базою даних, модифікувати ці дані, а потім після виконання транзакції повертати їх до розподіленої бази даних.

6. Концепція сховищ даних

Сховище даних (DW) – це предметно-орієнтований, інтегрований, прив'язаний до часу та незмінний набір даних, призначений для підтримки процесу прийняття рішень.

Ідея, покладена в основу технології DW, полягає в тому, що проводити оперативний аналіз безпосередньо на базі оперативних інформаційних систем неефективно.

Замість цього, всі необхідні для аналізу дані витягаються із декількох традиційних баз даних, а також зовнішніх джерел (наприклад, статистичних звітів), перетворюються та потім розміщуються в одне джерело даних – DW.

Є ще одна причина, що виправдовує появу окремого сховища – складні аналітичні запити до оперативної інформації гальмують поточну роботу компанії, надовго блокуючи таблиці та захоплюючи ресурси сервера.

Кінцевою метою створення сховища даних є інтеграція корпоративних даних у єдиному репозиторії, звертаючись до якого користувачі зможуть формувати запити, генерувати звіти й виконувати аналіз даних.

	Березень		
	Лютий		
	Січень		
	Україна	Грузія	Росія
Напої	10000	2000	1000
Продукти	5000	500	250
Інші товари	5000	500	250

Рисунок 6.1 – приклад побудови DW з використанням OLAP кубу

Дані, що надходять у сховище, можуть бути неузгодженими, фрагментованими, схильними до змін, містити дублікати або пропуски. Тому в процесі завантаження даних до сховища вони за допомогою менеджера завантаження:

- ❑ очищуються (усунення непотрібної інформації);
- ❑ агрегуються (обчислення сум, середніх значень, тощо);

- ❑ трансформуються (перетворення типів даних, реорганізація структур зберігання);
- ❑ поєднуються із зовнішніх і внутрішніх джерел (приведення до єдиних форматів);
- ❑ синхронізуються (відповідність одному моменту часу).

Доставка даних кінцевому користувачеві може бути або прямою, або через інформаційні вітрини – «магазини даних» (data mart – DM).

7. Бази даних NoSQL

Бази даних NoSQL використовують різноманітні моделі даних для доступу до інформації та управління нею. Бази даних таких типів оптимізовані спеціально для додатків, які потребують гнучких моделей даних, а також для роботи з частково структурованими або слабоструктурованими даними. Усе це досягається шляхом пом'якшення жорстких вимог до узгодженості даних, притаманних реляційних типів баз даних.

Залежно від моделі даних, реалізація може відрізнятися. Однак у багатьох базах даних NoSQL використовується текстовий формат обміну даними, що базується на мові Javascript (JSON), – відкритий формат обміну даними, що представляє дані у вигляді набору пар «ім'я-значення».

Існує кілька різних систем баз даних NoSQL у зв'язку з відмінностями у способах роботи та зберігання даних без схем.

Бази даних «ключ-значення»

База даних "ключ-значення" зберігає дані як сукупність пар "ключ-значення", в яких ключ служить унікальним ідентифікатором. Ключі і значення можуть бути будь-що: від простих до складних складових об'єктів.

Бази даних документів

У баз даних документів той самий формат моделі документа, який розробники використовують у кодї своїх додатків. Такі бази даних зберігають інформацію у вигляді гнучких, напівструктурованих та ієрархічних за своєю природою об'єктів JSON.

Графові бази даних

Графові бази даних призначені для спрощення розробки та запуску додатків, що працюють із наборами тісно пов'язаних даних. У таких базах даних використовуються вузли для зберігання сутностей інформації та ребра для зберігання взаємозв'язків між цими сутностями. Ребро завжди має початковий і кінцевий вузол, тип та напрямок. Ребра можуть описувати взаємозв'язки типу «предок-нащадок», дії, права володіння тощо.

Переваги використання NoSQL баз даних

NoSQL бази даних можна використовувати в додатках, в яких:

- потрібні гнучкі схеми, що забезпечують більш швидку та ітеративну розробку;
- надається перевага продуктивності, а не висока узгодженість даних і збереження зв'язків між таблицями даних (посилальна цілісність);
- потрібне горизонтальне масштабування шляхом сегментування між серверами;
- підтримується частково структуровані чи неструктуровані дані.

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

4.1 Складні та великі системи.

4.1.1 Властивості систем: емерджентність, адитивність, еквіфінальність.

Складна система - це множина взаємодіючих чи зв'язаних елементів, якій притаманна принаймні одна властивість, що не є характерною ні для жодного зі структурних елементів (так звана **системна властивість**).

В програмній інженерії **складною системою** вважають систему з багатьма рівнями абстрації (**Шар абстракції/Архітектурний шар**— спосіб розділення складних систем на простіші частини).

Складні системи мають наступні основні **властивості**:

1. **Загальність та абстрактність.** В якості системи можуть розглядатися всі без винятку предмети, явища, процеси незалежно від їх природи.
2. **Множинність.** Одна й та ж сукупність елементів є множиною систем, кожна з яких визначається конкретними системотворчими відношеннями та властивостями.
3. **Відносність і конкретність.** Поділ об'єктів на системи і не системи має сенс відносно конкретних заданих властивостей та відношень. Розгляд деякого об'єкта в якості системи безвідносно до конкретних властивостей та відношень не має сенсу.
4. **Цілісність.** Система поводить себе як одне ціле, якщо зміни однієї зі змінних викликають зміни інших змінних (підприємство — успіх кожної фази виробничого процесу обумовлений успіхом інших фаз).
5. **Емерджентність** - наявність у будь-якої системи особливих властивостей, не властивих її підсистемам і блокам, а також сумі елементів, не пов'язаних системоутвірними зв'язками; неможливість зведення властивостей системи до суми властивостей її компонентів.
6. **Еквіпотенційність.** Систему можна розглядати як підсистему системи вищого рівня, і навпаки — підсистему можна розглядати як систему зі своїм складом елементів та зв'язків між ними.
7. **Синергізм.** Ефективність сумісного функціонування елементів системи вища, ніж сумарна ефективність ізольованого функціонування цих же елементів.
8. **Еквіфінальність** - здатність складних систем досягати однакового кінцевого стійкого стану у процесі свого розвитку при різних стартових умовах і різними шляхами.
9. **Адитивність** - властивість систем, коли властивості системи дорівнюють сумі властивостей її компонентів.

4.1.2 Відкриті та закриті системи; класифікація за призначенням, походженням, видом елементів, способом організації. 4.1.3 Спільне та відмінності складних і великих систем

Класифікація систем може здійснюватися за різними ознаками:

- за призначенням (призначені для певної цілі (пасивні); здатні обирати ціль і до неї прагнути (активні));
- за походженням (природні, штучні, змішані);
- за видом елементів (матеріальні, абстрактні);
- за взаємодією із зовнішнім середовищем (**відкриті** ((постійно обмінюються речовиною, енергією або інформацією з середовищем)), **закриті** (з неї не виділяються речовина, енергія або інформація));
- за зміною стану (динамічні, статичні);
- за характером функціонування (детерміновані, стохастичні);
- за ступенем організованості (добре організовані, погано організовані, самоорганізуючі);
- за способом управління (управління ззовні, самоуправління, з комбінованим управлінням);
- за ступенем складності структури (надзвичайно складні, дуже складні, складні, прості).

Складні системи характеризуються тим, що вони одночасно інтегрують в собі природні і соціальні, природні і штучні складові. До основних характеристик складних систем відносять: велика кількість залежних між собою елементів; складність функцій; можливість розпадання системи на значну кількість підсистем; наявність ієрархічного управління, розгалужена інформаційна мережа; інтенсивні потоки інформації; залежність від середовища; функціонування в умовах випадкових факторів. Складність системи являє собою єдність складності складу, структури, функцій, організації, рівня і життєвого шляху системи. Розрізняють структурну, функціональну, динамічну, алгоритмічну види складності.

Від складних систем необхідно відрізняти **великі системи**. Під великою системою розуміють сукупність матеріальних ресурсів, засобів збору, передавання та оброблення інформації, людей-операторів, зайнятих на обслуговуванні цих засобів, і людей-керівників, наділених належними правами і відповідальністю для ухвалення рішень. До таких систем відносять економічні, організаційно-управлінські, біологічні, нейрофізіологічні системи тощо.

4.2 Моделі систем: склад і структура системи; моделі типу чорної та білої скриніки. Концептуальні, математичні та комп'ютерні моделі. Зв'язок між системою та моделлю; ізо- та гомоморфізм

До **складу системи** входять компоненти: елементи і підсистеми. Підсистема – це частина системи, що може бути піддана декомпозиції, тобто розкладена на інші підсистеми та елементи. Елемент – це компонент системи, що не розкладається.

Структура системи – це сукупність зв'язків між компонентами системи, що відбивають їх взаємодію.

Модель типу чорної скриньки - розглядається як така, що має сукупність «входів» та «виходів», що відображають зв'язки системи з навколишнім середовищем. При цьому залежності між входними та вихідними параметрами невідомі.

Модель типу білої скриньки – дозволяє дослідити структуру системи, процеси всередині системи, оскільки відомі відносини між входами та виходами системи.

Концептуальна модель (змістовна) — це абстрактна модель, що визначає структуру модельованої системи, властивості її елементів і причинно-наслідкові зв'язки, властиві системі й суттєві для досягнення мети моделювання.

Математична модель - це сукупність математичних співвідношень, рівнянь, нерівностей, що описують основні закономірності, властиві досліджуваному процесу, об'єкту або системі.

Під **комп'ютерною моделлю** найчастіше розуміють:

- умовний образ об'єкта чи деякої системи об'єктів (або процесів), описаних за допомогою взаємозалежних комп'ютерних таблиць, схем, діаграм, графіків, малюнків, анімаційних фрагментів, гіпертекстів і т. ін., що відбивають структуру та взаємозв'язки між елементами об'єкта чи системи. Комп'ютерні моделі такого типу називають структурно-функціональними;
- окрему програму, сукупність програм чи програмний комплекс, що дає змогу виконанням послідовності обчислень з подальшим графічним відображенням їх результатів відтворювати (імітувати) процеси функціонування об'єкта (системи об'єктів), що функціонує під впливом різних, як правило випадкових, факторів (імітаційну модель).

Модель – це “заступник оригіналу”(системи), що створюється на основі **теорії подібності**.

ІЗОМОРФІЗМ І ГОМОМОРФІЗМ - поняття, що виражають однаковість (ізоморфізм) або подобу (гомоморфізм) будови (структури) систем (множин, процесів, конструкцій).

4.3 Інформаційні системи. 4.3.1 Поняття, цілі, значення, класифікація за функціональністю, масштабом, сферою застосування. 4.3.2 Забезпечення інформаційних систем: організаційне, інформаційне, математичне, програмне, технічне, лінгвістичне, методичне, правове.

Інформаційна система (ІС) — сукупність організаційних і технічних засобів для збереження та обробки інформації з **ціллю** забезпечення інформаційних потреб користувачів. **ІС** — комунікаційна система, що забезпечує збирання, пошук, оброблення та пересилання інформації. **ІС** - це організаційно-технічна система, в якій реалізується технологія обробки інформації з використанням технічних і програмних засобів.

За **функціональним призначенням** можна виділити такі системи: керувальні; проєктувальні; наукового пошуку; діагностичні, моделювальні; система підтримки прийняття рішень.

За **функціональністю** розрізняють такі типи: інформаційно-пошукові; інформаційно-довідкові; інформаційно-управляючі (управлінські); інтелектуальні інформаційні системи; системи підтримки прийняття рішень.

За **масштабом реалізації**: глобальні; регіональні; локальні.

за **масштабністю охоплення завдань**: персональна; групова; корпоративна.

За **сферою застосування**: кожній предметній області відповідає свій тип інформаційної системи (медична, економічна, географічна, виробнича, військова, навчальна, адміністративна, екологічна та т.п.).

Види забезпечення ІС:

Організаційне - сукупність документів, що описують технологію функціонування ІС, методи вибору і застосування користувачами технологічних прийомів для одержання конкретних результатів при функціонуванні ІС.

Інформаційне - сукупність інформації, інформаційних ресурсів, засобів та методів ведення усієї інформаційної бази – об’єкта управління.

Математичне - сукупність математичних методів, моделей і алгоритмів розв’язування задач, які застосовуються в ІС.

Програмне забезпечення - сукупність програм на носіях даних і програмних документів, які призначені для налагодження, функціонування і перевірки працездатності ІС.

Технічне - сукупність усіх технічних засобів, використовуваних при функціонуванні комп’ютерної ІС.

Лінгвістичне - сукупність засобів і правил для формалізації природної мови, які використовуються при спілкуванні користувачів та експлуатаційного персоналу ІС з комплексом засобів автоматизації при функціонуванні ІС.

Методичне - сукупність документів, які описують технологію функціонування ІС, методи вибору і застосування користувачами технологічних прийомів для одержання конкретних результатів при функціонуванні ІС.

Правове - сукупність правових норм, які регламентують правові відносини при функціонуванні ІС та юридичний статус результатів такого функціонування.

4. ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.3. Аналіз вимог: джерела та методи збирання вимог

Вимоги визначають послуги, очікувані від системи (формулювання сервісів) та обмеження, яким система повинна підлягати (формулювання обмежень).

Основним джерелом вимог до інформаційної системи, безумовно, є міркування, висловлені представниками Замовника. У відповідність з ієрархічною моделлю вимог дана інформація структурується як мінімум на 2 рівні: бізнес-вимоги і вимоги користувачів.

Вимоги необхідно одержати від зацікавлених осіб. Цей вид діяльності називається виявленням вимог і здійснюється аналітиком бізнес-процесів (або системним аналітиком).

До **зацікавлених осіб в проєкті** відносяться:

- замовники;
- користувачі;
- аналітики вимог;
- розробники;
- тестери;
- технічні письменники;
- менеджер проєкту;
- співробітники юридичного відділу;
- представники промислових організацій;
- співробітники відділу продажів.

Потрібно виконати ретельний аналіз зібраних вимог для виявлення в них повторів і протиріч. Це незмінно приводить до перегляду вимог та повторного їх узгодження з зацікавленими особами.

Вимоги, задовільні з погляду зацікавлених осіб, документуються. При цьому вимогам дають визначення, класифікують, нумерують і привласнюють їм пріоритети. Структура документа, що описує вимоги, відповідає шаблону, обраному в організації для цієї мети.

Бізнес-аналітик виявляє вимоги до системи за допомогою консультацій, до участі в яких залучаються замовники, користувачі й експерти в проблемній області. У деяких випадках бізнес-аналітик має достатній досвід у проблемній області, і допомога експерта може не знадобитися. У цьому випадку бізнес-аналітик являє собою різновид експерта проблемної області.

Завдання бізнес-аналітика полягає в тому, щоб об'єднати два набори вимог у бізнес-моделі. Бізнес-модель містить модель бізнес-класів і модель бізнес-прецедентів. Модель бізнес-класів – це діаграма класів верхнього рівня, що ідентифікує й зв'язує між собою бізнес-об'єкти. Модель бізнес-прецедентів – це діаграма прецедентів верхнього рівня, що ідентифікує основні функціональні будівельні блоки системи.

Методи виявлення вимог розподіляються на традиційні та сучасні.

До традиційних належать:

- інтерв'ювання;
- анкетування;
- спостереження;
- вивчення документів програмних систем.

До сучасних:

- прототипування;
- спільне розроблення програмних застосунків (JAD-метод);
- швидке розроблення програмних застосунків (RAD-метод).

Більш детально: [АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ. Лекційний матеріал. 2.2 Встановлення вимог, 2.2.1 Виявлення вимог](#)

<https://is.qd/1N2d6o> або

https://moodle.zp.edu.ua/pluginfile.php/72286/mod_resource/content/3/%D0%90%D0%BD%D0%B0%D0%BB%D1%96%D0%B7%20%D0%B2%D0%B8%D0%BC%D0%BE%D0%B3%20%D0%BB%D0%B5%D0%BA%D1%86%D1%96%D0%B9%D0%BD%D0%B8%D0%B9%20%D0%BC%D0%B0%D1%82%D0%B5%D1%80%D1%96%D0%B0%D0%BB%202023.pdf

4. ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.3. Аналіз вимог: вимоги користувача (варіанти використання та історії користувачів)

Вимоги користувачів до інформаційної системи. Перший крок для визначення вимог є виявлення користувацьких властивостей програмного продукту.

Користувацькі властивості можуть бути:

- Функціональні властивості. Спроможність продукту вирішувати основну та додаткові проблеми.
- Експлуатаційні властивості. Кількість ресурсів, потрібних для роботи, надійність, діапазон робочих температур та багато іншого в залежності від рішення.
- Ергономічні та естетичні властивості. Зручність використання, можливість індивідуалізації користувача, дизайн тощо.
- Безпека використання.
- Соціальні властивості. Престижність, екологічність, виконання соціальних функцій тощо.
- Універсальність. Спроможність вирішувати декілька проблем.

Важливим компонентом збору вимог є визначення границь, в рамках яких буде розроблюватися програмний продукт. Границі проекту - інформація про події, які є початком і закінченням проекту, а також про роботи входять і не входять в проект. Границі не дозволяють "розмивати" проект, та визначати рамки відповідальності учасників проекту.

Модель бізнес-прецедентів (варіантів використання, use-case) являє собою модель прецедентів на верхньому рівні абстракції. Модель бізнес-прецедентів визначає узагальнені бізнес-процеси. Бізнес-прецедент відповідає тому, що іноді називають можливостями системи. (Можливості системи визначаються в документі, що описує бачення системи (system vision). Якщо при розробці системи наводиться документ опису бачення системи, він може використовуватись замість моделі бізнес-прецедентів).

Діаграма бізнес-прецедентів концентрується на архітектурі бізнес-процесів. Ця діаграма дає можливість глянути на передбачуване поведінку системи так сказати «з висоти пташиного польоту». Неформальний опис кожного з бізнес-прецедентів повинний бути коротким, орієнтованим на ділову сторону системи, і концентруватися на основних потоках видів діяльності.

На етапі аналізу бізнес-прецеденти перетворюються в прецеденти. Саме на цьому етапі визначаються детальні прецеденти, і неформальний опис розширюється за рахунок включення в нього підпроцесів і альтернативних процесів, деяких копій екранів, що демонструють GUI-інтерфейс, а також взаємозв'язків між введеними прецедентами.

Суб'єкти (actor) діаграми бізнес-прецедентів відрізняються від зовнішніх сутностей на діаграмі контексту. Суб'єкти активні. Вони керують процесом. Вони активізують прецеденти, відправляючи їм повідомлення про події. Прецеденти управляють подіями. Лінії, що зв'язують суб'єктів і прецеденти, — це не потоки даних. Ці лінії зв'язку являють собою потік подій, що виходять від суб'єктів і потік відгуків, що виходять від прецедентів.

Модель бізнес-класів — це модель класів. Як і у випадку з бізнес-прецедентами, різниця міститься в рівні абстрагування.

Існують різні шаблони опису варіантів використання. Розглядаються наступні основні стилі опису:

- Вільний формат
- Повний формат (запропонований А. Коберном)
- Таблиця в дві колонки
- Таблиця в три колонки
- Стиль RUP.

Крім того, іноді доцільно використовувати:

Більш детально: [АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ. Лекційний матеріал. 2.2 Встановлення вимог. 2.2.3 Рівні вимог. 2.2.4 Керування вимогами. 2.2.4 Бізнес-модель вимог](https://moodle.zp.edu.ua/pluginfile.php/72286/mod_resource/content/3/%D0%90%D0%BD%D0%B0%D0%BB%D1%96%D0%B7%20%D0%B2%D0%B8%D0%BC%D0%BE%D0%B3%20%D0%BB%D0%B5%D0%BA%D1%86%D1%96%D0%B9%D0%BD%D0%B8%D0%B9%20%D0%BC%D0%90%D1%82%D0%B5%D1%86%D1%8B%D0%BD%D0%BB%D0%BB%202023.pdf)

https://moodle.zp.edu.ua/pluginfile.php/72286/mod_resource/content/3/%D0%90%D0%BD%D0%B0%D0%BB%D1%96%D0%B7%20%D0%B2%D0%B8%D0%BC%D0%BE%D0%B3%20%D0%BB%D0%B5%D0%BA%D1%86%D1%96%D0%B9%D0%BD%D0%B8%D0%B9%20%D0%BC%D0%90%D1%82%D0%B5%D1%86%D1%8B%D0%BD%D0%BB%D0%BB%202023.pdf

- Діаграму активності UML
- Діаграму історії користувачів

Шаблон повного опису варіанту використання по А. Коберну
Назва <коротка фраза у вигляді дієслова в невизначеній формі доконаного виду, що відображає мету>

Контекст використання <уточнення мети, при необхідності — умови її нормального завершення>.

Зона дії <посилання на рамки проекту>. Наприклад — підсистема бухгалтерського обліку.

Рівень <одні з трьох: узагальнений, цілі користувача, підфункції>. Автор задає зумовлену тривірневу класифікацію вимог, в цілому відповідну класифікації вимог на бізнес-вимоги, вимоги користувачів і функціональні вимоги.

Основна дійова особа <ім'я ролі основного актора або його опис>.

Учасники і інтереси <список інших акторів-учасників прецеденту з вказівкою їх інтересів>.

Передумова <те, що очікується, вже має місце>.

Мінімальні гарантії <що гарантується акторам-учасникам>. Наприклад — у разі невдалої транзакції всі дані, що були в системі до її початку, зберігаються незмінними.

Гарантії успіху <що отримають актори-учасники у разі успішного досягнення мети>.

Тригер <те, що «запускає» варіант використання, зазвичай — подія в часі>.

Основний сценарій <тут перераховуються кроки основного сценарію, починаючи від тригера і аж до досягнення гарантії успіху>.

Формат опису: <Номер кроку> <Опис дії>

Розширення <тут послідовно описуються всі альтернативні сценарії>.

Кожна з альтернатив прив'язана до кроку основного сценарію.

Формат опису:

<Номер кроку.Номер розширення> <Умова> <Дія або посилання на підлеглий варіант використання>.

Будь-який з кроків основного сценарію може мати 1 або більше розгалужень. Кожне розгалуження оформляється у вигляді розширення. У блоці «Розширення» всі розширення описуються послідовно.

У випадку, якщо альтернативний сценарій не вдасться описати одним рядком — застосовується наступний формат.

Починаючи з рядка, наступного після опису розширення, йде опис його дій у форматі основного сценарію:

<Номер кроку.Номер розширення.Номер кроку розширення> <Дія>

Опис розширення закінчується описом виходу з розширення. Основні варіанти виходу з розширення: повернення до чергового по номеру кроку основного сценарію, закінчення прецеденту, перехід до іншого кроку основного сценарію.

4. ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.3. Аналіз вимог: вимоги користувача (варіанти використання та історії користувачів)

User Story (історія користувача) – це неформальне загальне пояснення функцій програмного забезпечення, написане з точки зору кінцевого користувача. Її мета полягає в тому, щоб сформулювати, яку цінність для замовника несе функціонал програмного забезпечення. Вона формулює не тільки бізнес-цінність, а й вимоги для розробки й тестування. User Story називається так, бо вона створюється через розповідь, як історія.

Можна також зустріти й інше визначення – це інструмент, завдяки якому замовник точно дає зрозуміти свої потреби.

Ключовим компонентом гнучкої розробки програмного забезпечення є ставлення людей на перше місце, а історія користувача ставить кінцевих користувачів у центр всього процесу. Ці історії не використовують технічну мову, щоб створити контекст для команди розробки продукту. Прочитавши історію користувача, команда розуміє, що вони роблять, чому вони це роблять і яку цінність має продукт, який вони створюють.

Сам формат можна викласти в короткому реченні на кшталт: «Як персона, я хочу, так що...» (As a [person], I [want to], [so that]):

«Персона» – для кого ми це створюємо?

«Хочу» – тут описуються наміри людей, а не функції, які вони використовують.

«Так що» – яка мета? Якої загальної вигоди потрібно досягти? Яку проблему необхідно вирішити?

Текст самої User Story повинен пояснювати роль/дії користувача в системі. Попри те, що історія користувача відіграє роль, яка раніше належала специфікаціям вимог, сценаріям використання тощо, вона все ж відчутно відрізняється рядом тонких, але критичних нюансів:

- User Story не є детальним описом вимог (тобто того, що система повинна робити), а являє собою скоріше обговорюване подання намірів (потрібно зробити щось на зразок цього);
- вона коротка та легко читається, зрозуміла розробникам, стейкхолдерам і користувачам;
- являє собою невеликі інкременти цінної функціональності, які можуть бути реалізовані в рамках декількох днів або тижнів;
- зусилля, що необхідні для реалізації, можуть бути швидко визначені;
- історія користувача не міститься у величезних, громіздких документах, а організована у списки, які легше впорядкувати й перепорядкувати з надходженням нової інформації;
- вона не деталізована на початку проекту. Таким чином можна уникнути затримок в розробці, нагромадження вимог і надмірно обмеженого формулювання задач;
- вона вимагає мінімум або зовсім не вимагає супроводу і може бути безпечно скасована після імплементації.

Історії користувачів фіксують лише найважливіші елементи вимоги:

- для кого це?
- чого очікує від системи?
- чому це важливо?

Практичні поради з написання історій користувача:

- краще написати багато історій менших, ніж кілька громіздких;
- краще писати цілою командою, щоб не загубити важливі для продукту, бізнесу і користувачів нюанси;
- кожна історія в ідеалі повинна бути написана без технічного жаргону;
- точно описати потреби користувача;
- історії повинні бути написані таким чином, щоб за ними можна було протестувати;
- тести повинні бути написані до коду;
- як можна довше варто уникати UI. Історія повинна виконуватися без прив'язки до конкретних елементів;
- кожна історія повинна містити оцінку складності, а також кількості часу для реалізації;
- історія повинна мати кінцівку – тобто завершуватися конкретним результатом;
- історія має бути зрозумілою для всіх учасників проекту;
- User Story повинна відповідати моделі INVEST.

Зазвичай юзер сторі пишуть за такими двома шаблонами:

1. As a <user/user role> I want <capability> so that <benefits description>.

<https://training.qatestlab.com/blogs/technical-articles/user-story/>
<https://lampm.club.ua/blog/2020/05/20/10-use-cases-for-non-technical-people/>

1. In order to <receive benefit> as a <role>, I can <goal/desire>.

INVEST – критерій гарної історії:

I – independent – незалежна від інших історій, тобто історії можуть бути реалізовані в будь-якому порядку;

N – negotiable – обговорювана, відображає суть, а не деталі; не містить конкретних кроків реалізації;

V – valuable – цінна для клієнтів, бізнесу та стейкхолдерів;

E – estimable – оцінюється за складністю і трудовитратами;

S – small – компактна, може бути зроблена командою за одну ітерацію;

T – testable – така, яку можна протестувати (перевірити), має критерії приймання.

Структура побудови юзер сторі

- **User Story Title** — назва юзер сторі має бути стислою та зрозумілою.
- **User Story Statement** — речення-шаблон As a _ I want _ so that _.
- **Acceptance Criteria** — критерії приймання.
- **Conversation** — обговорення юзер сторі з усіма стейкхолдерами задля пошуку ідей, рішень тощо.
- **Attachments** — додатки для кращого розуміння, що саме, для кого та для чого потрібно розробити.

Приклад хорошої user story

View List of Products

Attach

Add a child issue

Link issue

...

Description

User Story

As a shopper I want to view a list of products so I can choose some to purchase

Acceptance Criteria

1. By navigating to product section, shopper must view product list

2. All active products must be included in the list

3. Shopper must view a thumbnail picture for each product and product title

4. Shopper can view up to 20 products per page

5. Shopper can view product details of each product

6. Product list must be displayed in ascending order by product title

Product Details Shall consist of

Title

Category

Description

4. ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.3. Аналіз вимог: класифікація вимог до програмного забезпечення: функціональні та нефункціональні вимоги, обмеження

Вимоги до ПЗ складаються з 3-х рівнів:

- бізнес вимоги;
- вимоги користувачів;
- функціональні вимоги.

До того ж, кожна система має свої нефункціональні вимоги.

Бізнес вимоги (business requirements) містять високорівневі цілі організації або замовників системи. Вони можуть бути записані в документі про спосіб і межах проекту, в якому пояснюється, чому організації потрібна така система, тобто, описані цілі, які організація має намір досягти з її допомогою. Вимоги користувачів (user requirements) описують цілі і завдання, які користувачам дозволить вирішити система. Ці вимоги можуть бути записані в документ про варіанти використання, де вказано, що клієнти зможуть зробити за допомогою системи.

Функціональні вимоги (functional requirements) визначають функціональність системи, яку розробники повинні побудувати, щоб користувачі змогли виконати свої завдання в рамках бізнес вимог.

Системні вимоги (system requirements) – це високорівневі вимоги до продукту.

Бізнес правила включають корпоративні політики, урядова постанова, промислові стандарти і обчислювальний алгоритм.

Функціональні вимоги документуються в специфікації вимог до програмного забезпечення, де описується так повно, як необхідно очікуване поведінка системи. Специфікація вимог до ПЗ використовується при розробці, тестуванні, гарантії якості продукту, управлінні проектом і пов'язаним з проектом функцій.

На додаток до функціональним вимогам, специфікація містить **нефункціональні**, де описані цілі і атрибути якості.

Атрибути якості (quality attributes) – це додатковий опис функцій продукту, виражені через опис його характеристик, важливих для користувачів або розробників (легкість і простота використання, легкість переміщення, цілісність, ефективність і стійкість до збоїв).

Обмеження (constraints) стосуються вибору можливості розробки зовнішнього вигляду і структури проекту.

Функціональні вимоги зазвичай мають такі характеристики:

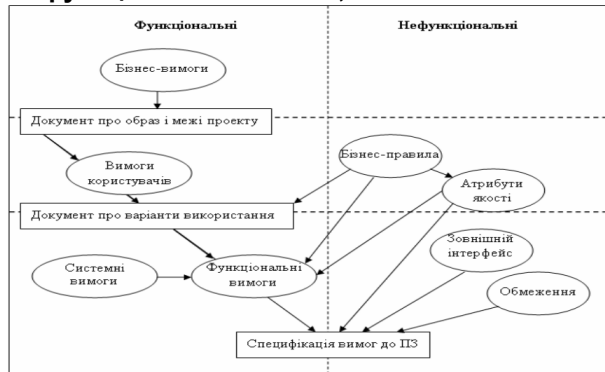
- Специфікація: вони детальні та конкретні, залишаючи мало місця для двозначності. Вони окреслюють точні функції, входи та виходи системи.
- Можливість перевірки: функціональні вимоги можна перевірити та підтвердити, щоб переконатися, що програмне забезпечення працює належним чином.
- Орієнтація на користувача: вони тісно узгоджені з потребами та очікуваннями користувача, гарантуючи, що програмне забезпечення виконує заплановану мету.
- Змінність: функціональні вимоги можуть змінюватися протягом проекту, оскільки відгуки користувачів і потреби бізнесу розвиваються.

Приклади функціональних вимог:

Автентифікація користувача: система повинна дозволяти користувачам створювати облікові записи, входити в систему та скидати свої паролі.

Кошик для покупок: система повинна дозволяти користувачам додавати товари до свого кошика для покупок, переглядати вміст кошика та переходити до оформлення замовлення.

Функція пошуку: система повинна забезпечувати функцію пошуку, яка дозволяє користувачам знаходити продукти за ключовими



Нефункціональні вимоги, часто скорочені як SUPL або NFR, доповнюють функціональні вимоги, вказуючи, як програмна система повинна виконувати певні функції. Вони визначають якість, характеристики та обмеження системи,

а не її специфічні особливості. По суті, нефункціональні вимоги встановлюють стандарти продуктивності, безпеки та зручності використання системи.

Характеристики нефункціональних вимог:

- Якісні: на відміну від функціональних вимог, які зазвичай є кількісними, нефункціональні вимоги зосереджені на якісних аспектах, таких як продуктивність, надійність і безпека.
- Глобально: нефункціональні вимоги застосовуються до всієї системи та впливають на її загальну поведінку.
- Стабільність: вони, як правило, більш стабільні протягом життєвого циклу проекту, зміни відбуваються рідше порівняно з функціональними вимогами.
- Вимірність: Хоча нефункціональні вимоги може бути важко точно визначити кількісно, їх все одно можна виміряти та протестувати.

Приклади нефункціональних вимог:

Продуктивність: система має завантажувати веб-сторінку менш ніж за 3 секунди навіть за 100 одночасних користувачів.

Безпека: система має відповідати галузевим стандартам безпеки та протоколам шифрування.

Масштабованість: програма повинна мати змогу обробляти 50% збільшення трафіку користувачів протягом шести місяців без погіршення продуктивності.

4. ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.3. Аналіз вимог: структуризація функціональних вимог

Вимогами необхідно керувати. Керування вимогами являє собою частину загального керування проектом. Воно пов'язане з трьома основними питаннями:

1. Ідентифікація, класифікація, організація й документування вимог.
2. Зміна вимог (за допомогою процесів, що встановлюють способи вивчення, узгодження, перевірки вірогідності та документування неминучих змін до вимог).
3. Простежуваність вимог (за допомогою процесів, що підтримують відносини взаємозалежності між вимогами й іншими системними артефактами, а також, власно, між вимогами).

Вимоги описуються природною мовою, наприклад: «Система повинна запланувати наступний телефонний дзвінок клієнтові по запити», «Система повинна автоматично набирати запланований телефонний номер» і т. д.

Типова система може складатися з сотень або тисяч формулювань вимог. Для належного керування такою величезною кількістю вимог їх необхідно пронумерувати за допомогою певної схеми ідентифікації. Схема може включати класифікацію вимог у вигляді груп, що легше піддаються керуванню.

Існує декілька методів ідентифікації й класифікації вимог:

- унікальний ідентифікатор – звичайно послідовний номер, привласнений вручну або згенерований з використанням бази даних case-засобу;
- послідовний номер усередині ієрархії документа – привласнюється з урахуванням положення вимог у межах документа опису вимог;
- послідовний номер у межах категорії вимог – привласнюється на додаток до мнемонічного імені, що позначає категорію вимог.

Вимоги можна впорядкувати у вигляді ієрархічно впорядкованої структури, наприклад відношення батько-нащадок. Відношення батько-нащадок подібно відношенню композиції Батьківська вимога складається з дочірніх вимог. Дочірня вимога – це фактично «під-вимога» батьківської вимоги.

Ієрархичні відносини дозволяють ввести додатковий рівень класифікації вимог. Це може безпосередньо позначатися в ідентифікаційному номері (вимога, пронумерована як 4.9, може бути дев'ятим нащадком «батька» з ідентифікаційним номером, рівним 4).

Простежуваність вимог (requirements traceability) – це всього лише частина керування змінами. Блок вимог технології керування змінами підтримує відносини простежуваності, щоб фіксувати зміни, що виходять від або внесені у вимоги протягом

Документ опису вимог

Документ, що описує вимоги, є відчутним результатом етапу встановлення вимог. Шаблони для документів опису вимог широко доступні. Згодом кожна організація розробляє свої власні стандарти, які відповідають прийнятій в організації практиці, корпоративній культурі й т. п. Документ опису вимог повинен створити прецедент для системи.

Приклад змісту документа:

1. Попередні зауваження до проекту a. Мета й рамки проекту b. Діловий контекст c. Учасники проекту d. Ідеї відносно рішень e. Огляд документа 2. Системні сервіси a. Рамки системи b. Функціональні вимоги c. Вимоги до даних 3. Системні обмеження a. Вимоги до інтерфейсу b. Вимоги до продуктивності c. Вимоги до безпеки d. Експлуатаційні вимоги e. Політичні і юридичні вимоги f. Інші обмеження	4. Проектні питання a. Відкриті питання b. Попередній план-графік c. Попередній бюджет 5. Додатки a. Глосарій b. Ділові документи та форми c. Посилання
---	---

Основна частина документа опису вимог присвячена визначенню системних сервісів. Ця частина може займати до половини всього обсягу документа. Ця частина документа може містити узагальнені моделі – моделі бізнес-вимог.

Рамки системи можна моделювати за допомогою діаграми контексту. Функціональні вимоги можна моделювати за допомогою діаграми бізнес-прецедентів. Однак діаграма охоплює перелік функціональних вимог тільки в найбільш загальному вигляді. Всі вимоги необхідно позначити, класифікувати й визначити.

Вимоги до даних можна моделювати за допомогою діаграми бізнес-класів.

Системні сервіси визначають, що повинна робити система. Системні обмеження визначають, наскільки система обмежена при виконанні обслуговування. Системні обмеження зв'язані з такими видами вимог:

- вимоги до інтерфейсу, що визначають як система взаємодіє з користувачами;
- вимоги до продуктивності, що у вузькому сенсі задають швидкість відгуку системи, з якої повинні виконуватися різні завдання;
- вимоги до безпеки, які описують права доступу користувача до інформації, контрольовані системою.

Експлуатаційні вимоги, які визначають програмно-технічне середовище, якщо воно відоме на етапі проектування, у якому повинна функціонувати система.

4.7 Забезпечення якості: спільне та відмінності процесів тестування, верифікації,

Верифікація (Verification)	Валідація (Validation)
це статична практика перевірки документів, дизайну, архітектури, коду, тощо. 1) включає перевірку планів, специфікацій вимог, специфікацій дизайну, коду, тест-кейсів, чек-листів тощо 2) завжди проходить без запуску коду. 3) використовує методи – reviews, walkthroughs, inspections тощо 4) відповідає на питання “Чи робимо ми продукт правильно?” 5) допоможе визначити, чи є програмне забезпечення високої якості, але воно не гарантує, що система корисна. Перевірка пов’язана з тим, що система добре спроектована і безпомилкова. 6) відбувається до Валідації На практиці, відмінності верифікації та валідації мають велике значення: замовника цікавить більше валідація (задоволення власних вимог); виконавця хвилює не тільки дотримання всіх норм якості (верифікація) при реалізації продукту, а й відповідність всіх особливостей продукту бажанням замовника Приклад: Перевірка вебформи. Верифікація: перевіряємо наявність полів. Всі поля повинні відповідати специфікації. Їх наявність визначено макетами. Необхідна інформація вноситься в ТЗ або до мокапів. Також перевіряється, що поля всі робочі, в них можливо ввести різні дані згідно найменувань. Валідація: перевіряємо інформацію, що вводиться в поля та її відповідність специфікації	це процес оцінки кінцевого продукту. Перевіряє, чи відповідає програмне забезпечення очікуванням і вимогам клієнта. Це динамічний механізм перевірки та тестування фактичного продукту 1) завжди включає в себе запуск коду програми. 2) використовує методи, такі як тестування Black Box, тестування White Box і нефункціональне тестування. 3) відповідає на питання “Чи робимо ми правильний продукт?” 4) перевіряє, чи відповідає програмне забезпечення вимогам і очікуванням клієнта. 5) може знайти помилки, які процес Верифікації не може зловити відбувається після Верифікації

4.7.1 Тестування методами білої та чорної скрині

Критерії	Black Box	White Box
Визначення	Тестування, як функціональне, так і нефункціональне, не передбачає знання внутрішнього складу компоненту або системи. Засноване на специфікації або тестуванні поведінки, зосереджується на роботі виключно з зовнішніми інтерфейсами	Тестування, засноване на аналізі внутрішньої структури компонента або системи, відомі всі деталі реалізації. Вибираються вхідні значення, ґрунтуючись на знанні коду, який буде їх обробляти, також відомий результат цієї обробки
Рівні, де застосовується техніка	В основному: – прийомочне тестування – системне тестування	В основному: – Юніт тестування – Інтеграційне тестування
Хто виконує	Як правило, тестувальник	Як правило, розробник
Знання програмування	Не потрібно	Необхідно
Знання реалізації	Не потрібно	Необхідно
Основа для тест-кейсів	Специфікація, вимоги	Проектна документація

4.7.2 Рівні тестування: модульний, інтеграційний, системний, валідаційний

Тестування на різних рівнях проводиться протягом усього життєвого циклу розробки і супроводу ПЗ. Рівень тестування визначає те, над чим виробляються тести: над окремим модулем, групою модулів або системою в цілому.

1) Компонентне або Модульне тестування (Component testing or Unit testing) перевіряє функціональність і шукає дефекти в частинах програми, які доступні і можуть бути протестовані окремо (модулі програми, об'єкти, функції тощо). Зазвичай проводиться викликаючи код, який необхідно перевірити чи за підтримки середовищ розробки, таких як фреймовки (каркаси) для модульного тестування або інструменти для дебагу. Всі знайдені дефекти, як правило виправляються в коді без формального їх опису в системі багів. Один з найбільш ефективних підходів до модульного тестування – це підготовка автоматизованих тестів до початку основного кодування ПЗ. Це називається розробка від тестування (test-driven development).

2) Інтеграційне тестування призначено для перевірки зв'язку між компонентами, а також взаємодії з різними частинами системи (операційної системи, обладнанням небудь зв'язку між різними системами).

Рівні інтеграційного тестування:

- **Компонентний інтеграційний рівень (Component Integration testing)** перевіряє взаємодія між різними системами після проведення компонентного тестування.
- **Системний інтеграційний рівень (System Integration testing)** перевіряє взаємодію між різними системами після проведення системного тестування.

3) Системне тестування – основним завданням є перевірка як функціональних так і не функціональних вимог у системі в цілому. При цьому виявляються дефекти, такі як невірне використання ресурсів системи, непередбачені комбінації даних рівня користувача, несумісність з оточенням, непередбачені сценарії використання, відсутня або невірна функціональність, незручність використання тощо. Для мінімізації ризиків, пов'язаних з особливостями поведінки системи в тому чи іншому середовищі, під час тестування рекомендується використовувати оточення максимально наближене до того, на яке буде встановлений продукт після релізу.

Існує два підходи до системного тестування:

- **На базі вимог (requirements based)** – Для кожної вимоги пишуться тестові випадки (test cases), перевіряючи виконання даної вимоги.
- **На базі випадків використання (use case based)** – На основі уявлення про способи використання продукту створюються випадки використання системи (Use Cases). По конкретному випадку використання можна визначити один або більше сценаріїв. На перевірку кожного сценарію пишуться тест кейси (test cases), які повинні бути протестовані.

4) Валідація (Validation) – це процес перевірки значень за певним раніше затвердженим стандартом, вимогою, правилом. Валідація забезпечує збір інформативної, потрібної, відсортованої інформації. Також валідація може слугувати як засіб безпеки, що запобігає передачі до БД некоректних даних. Наприклад: поле дня народження приймає тільки ціле позитивне число (включаючи нуль) і не може бути більше, ніж 31.

Є три типи валідації:

- **Миттєва** – застосовується в тому випадку, коли під час написання значення система визначає чи підходить воно вимогам чи ні.
- **Після втрати фокусу** – валідація відбувається після перемикання на інше поле або після натискання на іншу область.
- **Після відправки форми** – процес валідації починається тільки після того, як буде натиснута умовна кнопка «Надіслати», після цього система почне перевіряти чи заповнені поля згідно з вимогами.

4.7.3 Розробка через тестування (Test-driven development)

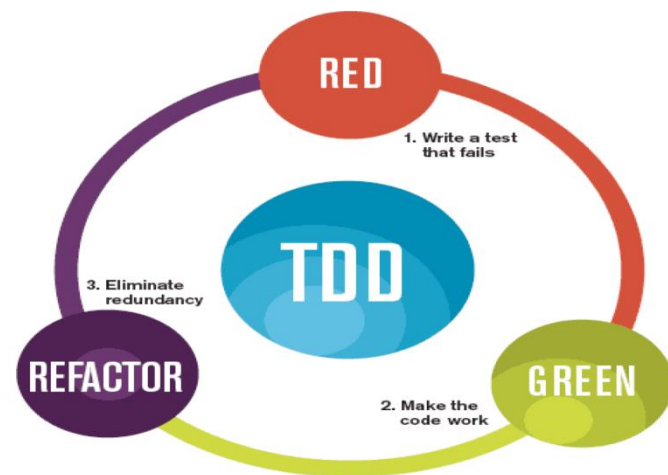
Розробка від тестування (test-driven development) або підхід тестування спочатку (test first approach) – створюються і інтегруються невеликі шматки коду, навпроти яких запускаються тести, написані до початку кодування. Розробка ведеться до тих пір поки всі тести не будуть успішними.

Розробка через тестування передбачає циклічне проходження трьох етапів знову і знову :

- Червона фаза: написання тесту;
- Зелена фаза: завершення тесту, написавши код, який він перевіряє;
- Синя фаза: рефакторинг.

TDD дозволяє:

- створювати завжди актуальну документацію;
- розробляти код, що буде легко тестуватись, це робить його чистим, нескладним, легким для розуміння та змін;
- створювати сітку страхування, щоб впевнено вносити зміни.
- надавати ранні повідомлення про помилку;
- робити просту діагностику помилок, оскільки тести точно визначають, що не так.



4.7.4 Додаткові техніки верифікації та валідації: інспекція коду, перевірка на відповідність стандартам і вимогам, оцінювання зручності використання та користувацького досвіду, перевірка продуктивності та масштабованості

Інспекція коду (code inspection) – систематична перевірка вихідного коду програми з метою виявлення та виправлення помилок, які залишилися непоміченими у початковій фазі розробки. Метою огляду є покращення якості програмного продукту та вдосконалення навичок розробника. У процесі інспекції коду можуть бути знайдені та усунені такі проблеми, як помилки у форматуванні рядків, витік пам'яті, переповнення буфера тощо, що покращує безпеку програмного продукту. Системи контролю версій надають можливість проведення спільної інспекції коду. Крім того, є спеціальні інструментальні засоби для спільної інспекції коду.

Перевірка на відповідність стандартам і вимогам - використовується для підтвердження того, що ПЗ відповідає стандартам і нормам, визначеним IEEE, W3C або ETSI. Це може стосуватися деяких технічних аспектів, але обов'язково включає: продуктивність, функціонал, надійність, поведінку системи.

Ефективне тестування на відповідність включає наступні кроки:

- Аналіз стандартів і специфікацій
- Вибір інструментів тестування та набору тестів
- Розробка процедур тестування
- Проведення необхідних перевірок
- Адаптація відповідної політики тестування та сертифікації



Оцінювання зручності використання та користувацького досвіду, (usability, usability engineering, user experience, UX) - метод тестування, спрямований на встановлення ступеня зручності використання та навчання, зрозумілості та привабливості для користувачів розроблювального продукту в контексті заданих умов. Дає оцінку рівня зручності використання програми за наступними пунктами:

- продуктивність, ефективність (efficiency) – скільки часу і кроків знадобиться користувачеві для завершення основних завдань програми;
- правильність (accuracy) – скільки помилок зробив користувач;
- активізація в пам'яті (recall) – як багато користувач пам'ятає про роботу програми після припинення роботи з нею на тривалий період;
- емоційна реакція (emotional response) – як користувач себе почуває після завершення завдання

Основні види юзабіліті-тестування:

- метод карткового сортування;
- метод зворотного карткового сортування;
- метод тестування сподівань;
- метод оцінки сприйняття дизайну;
- метод eye tracking;
- метод «думки вголос»;
- конструктивна взаємодія;
- метод фокус-груп;
- експертна оцінка;
- евристична оцінка;
- паперове прототипування.

Перевірка продуктивності та масштабованості – це процес тестування програмного забезпечення, який використовується для перевірки швидкості, часу відгуку, стабільності, надійності, масштабованості та використання ресурсів програмного продукту під певним навантаженням.

Основні параметри (метрики), які контролюються під час тестування продуктивності:

- Завантаження ЦП (CPU utilization) – відсоток потужності ЦП, який використовується для обробки запитів;
- Використання пам'яті (Memory utilization) – показник використання основної пам'яті ПК під час обробки будь-яких робочих запитів;
- Час відповіді (Response times) – загальний час між відправленням запиту та отриманням відповіді;
- Середній час завантаження (Average load time) – показник, який відображає час, який потрібний вебсторінці для завершення процесу завантаження та появи на екрані користувача;
- Пропускна здатність (Throughput) – вимірює кількість транзакцій, які програма може обробити за секунду;
- Середня затримка/час очікування (Average latency/Wait time) – час, проведений запитом у черзі перед обробкою;
- Пропускна здатність (Bandwidth) – вимірювання обсягу даних, що передаються за секунду;
- Запитів за секунду (Requests per second) – показник кількості запитів, оброблених програмою за секунду;
- Частота помилок (Error rate) – відсоток запитів, які призвели до помилок, порівняно із загальною кількістю;
- Транзакції пройдено/не виконано (Transactions Passed/Failed) – відсоток виконаних/невдалих транзакцій від загальної кількості транзакцій.

Типи тестування продуктивності

- Тестування навантаження (Load testing)
- Стрес-тестування (Stress testing)
- Тестування на довговічність (Endurance testing)
- Тестування стрибків (Spike testing)
- Об'ємне тестування (Volume testing)
- Тестування масштабованості (Scalability testing)

ІНЖЕНЕРІЯ СИСТЕМ І ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1. Складні системи: властивості; відкриті та закриті системи; класифікація за призначенням, походженням, видом елементів, способом організації; спільне та відмінності складних і великих систем

Складні системи: класифікація за призначенням, походженням, видом елементів, способом організації; спільне та відмінності складних і великих систем (Дивись таблицю)

В залежності від кількості елементів, рівнів, зв'язків, функцій, відношення до мети системи можна поділити на прості, складні, великі

Системи Параметри	Прості	Складні	Великі
Складність	Множина елементів	Множина простих систем	Множина складних систем
Кількість елементів на рівні	Небагато	Багато	Дуже багато
Кількість зв'язків між елементами	Мало	Багато	Дуже багато
Кількість рівнів	Один	Декілька	Багато
Кількість цілей	Одна	Одна	Декілька
Відношення до цілі	Цілеспрямована	Цілескерована	Цілескерована

Відкриті та закриті системи: У теорії систем, залежно від взаємодії системи із зовнішнім середовищем, розрізняють відкриті та закриті системи.

Відкриті системи: Це системи, які активно взаємодіють із навколишнім середовищем, обмінюючись енергією, речовиною чи інформацією. Вони здатні приймати зовнішні впливи та впливати на них у відповідь. Прикладами відкритих систем можуть бути живі організми, економічні системи чи екосистеми.

Закриті системи: Це системи, які відокремлені від зовнішнього середовища та не обмінюються з нею ні енергією, ні речовиною, ні інформацією. У закритих системах зовнішні дії ігноруються чи передбачаються мінімальними. Приклади закритих систем можуть включати ізольовані фізичні системи у лабораторії або математичні моделі, які вивчаються у відриві від зовнішніх впливів.

4.2. Моделі систем: структура системи; класифікація моделей; зв'язок між системою та моделлю; ізо- та гомоморфізм

Структура системи.

Система може бути показана простим перерахуванням елементів або "чорною шухлядою" (моделлю "вхід-вихід"). Однак частіше при дослідженні об'єкту такого відображення недостатньо, бо вимагається з'ясувати, що собою представляє об'єкт, що в ньому забезпечує виконання поставленої мети. В цих випадках систему зображують шляхом розчленування на підсистеми, компоненти, елементи зі взаємозв'язками, що можуть носити різноманітний характер, і вводять поняття структури.

Структурою системи називається її розчленування на групи елементів з вказівкою зв'язків між ними, яке дає уявлення про систему в цілому і є незмінним на весь час розгляду.

Одна і та ж система може бути подана різними типами структур в залежності від стадії пізнання об'єкту або процесу аспекту їхнього розгляду, мети створення. При цьому в процесі дослідження або проектування структура системи може змінюватися.

Наведемо приклади структур. Речовинна структура збірного мосту складається з його окремих секцій, що збираються на місці. Груба структурна схема такої системи вкаже тільки ці секції і порядок їхнього поєднання. Приклад функціональної структури - це поділ двигуна внутрішнього згоряння на системи живлення, мастила, де речовинні і функціональні структури злиті, - це відділи проектного інституту, що займаються різними сторонами однієї і тієї ж проблеми. Типовою алгоритмічною структурою буде алгоритм (схема) програмного засобу, що вказує послідовність дій, також алгоритмічною структурою буде інструкція, яка визначає дії при відшукуванні несправності технічного об'єкту.

Різнманітні види структур мають специфічні особливості і можуть розглядатися як самостійні поняття теорії систем і системного аналізу. Стисло охарактеризуємо основні з них (рис. 2.1.).

Мережева структура або мережа (рис. 2.1, а) являє собою декомпозицію системи в часу.

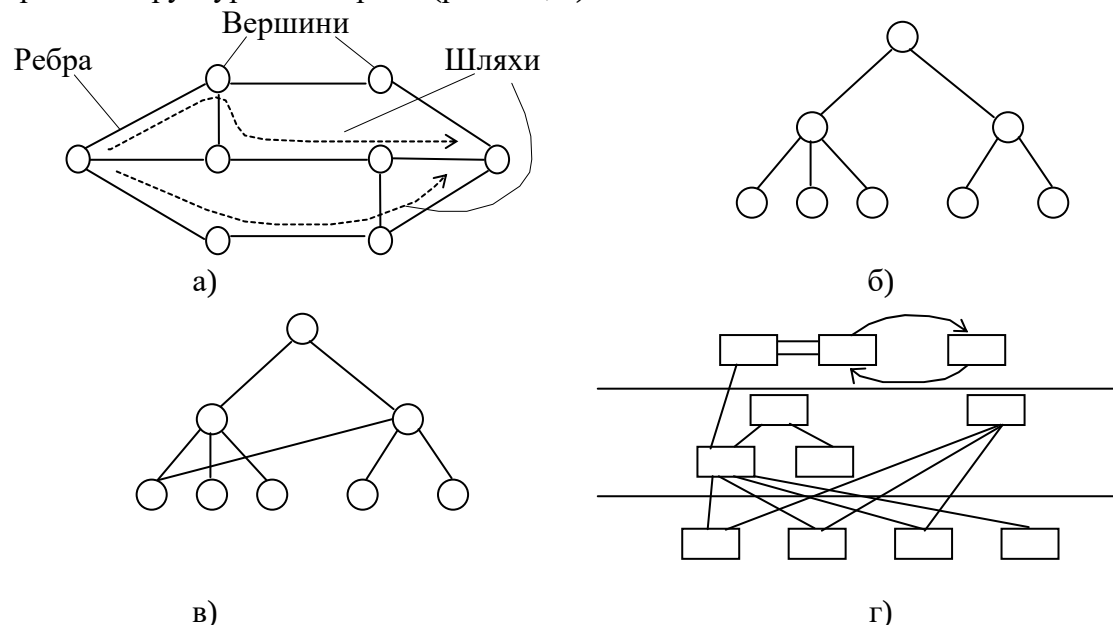


Рис. 2.1

Мережева структура може відображати порядок дії технічної системи (телефонна мережа, електрична мережа і т. і.), етапи діяльності людини (при виробництві продукції - мережевий графік, при проектуванні - мережева модель, при плануванні - мережевий план і т. і.).

Ієрархічні структури (рис. 2.1, б-г) являють собою декомпозицію системи в просторі. Всі вершини (вузли) і зв'язки (дуги, ребра) існують в цих структурах водночас (не рознесені в часі). Такі структури можуть мати не два (як для простоти показано на рис. 2.1, б і в), а більше число рівнів декомпозиції (структуризації).

Структури типу рис. 2.1 б, в який кожний елемент нижчого рівня підпорядкований одному вузлу (одній вершині) вищого (і це справедливо для всіх рівнів ієрархії), називають **ієрархічними структурами з "сильними" зв'язками, деревоподібними структурами, структурами типу дерева, структурами**, на яких виконується відношення деревного порядку.

Структура типу рис. 2.1, в, де елемент нижчого рівня (один або декілька) може бути підпорядкований двом і більш вузлам (вершинам) вищого, називають ієрархічними структурами зі **"слабкими" зв'язками**.

Розглянуті структури мають найбільше розповсюдження.

Класифікація моделей систем:

Фізичні моделі: Імітують структуру та поведінку реальних об'єктів чи систем.

Математичні моделі: Описують систему з використанням математичних рівнянь та формул.

Комп'ютерні моделі: Створюються за допомогою комп'ютерних програм і дозволяють аналізувати систему у віртуальному середовищі.

Аналітичні моделі: Використовуються для теоретичного аналізу системи на основі її характеристик та властивостей.

Зв'язок між системою та моделлю: Модель є абстракцією чи поданням реальної системи. Вона створюється у тому, щоб зрозуміти чи передбачити поведінку системи, зазвичай шляхом спрощення її структури та процесів. Модель може бути заснована на спостереженнях, експериментах чи теоретичних припущеннях про систему.

Ізоморфізм та гомоморфізм:

Ізоморфізм: Це властивість, у якому дві системи мають однакову структуру чи форму, але можуть відрізнятися за своїми елементами чи змістом.

Гомоморфізм: Це властивість, коли одна система може бути представлена у вигляді моделі іншої системи, зберігаючи її структуру і відносини між елементами. Гомоморфні моделі зберігають деякі основні характеристики та взаємодії системи.

4.4.1 Види проєктування: архітектурне, деталізоване, проєктування інтерфейсу користувача; класифікація моделей UML; стандартні архітектури: клієнт-серверна, багаторівнева (Layered Architecture), мікросервісна.

Види проєктування:

Архітектурне проєктування: Визначає загальну структуру та організацію системи, спираючись на її вимоги та цілі.

Деталізоване проєктування: Розробляє деталі кожного компонента системи, включаючи алгоритми, структури даних та інтерфейси.

Проектування інтерфейсу користувача: Фокусується на створенні зручного та інтуїтивно зрозумілого інтерфейсу для взаємодії з програмним продуктом.

Класифікація моделей UML:

1. Структурні моделі:

Діаграма класів (Class Diagram): Описує структуру системи шляхом показу класів, їх атрибутів, методів та відносин між ними.

Діаграма об'єктів (Object Diagram): Показує конкретні екземпляри класів та зв'язки між ними у певний момент часу.

Діаграма компонентів (Component Diagram): Описує фізичну структуру системи, показуючи компоненти, їх інтерфейси та залежність між ними.

Діаграма розгортання (Deployment Diagram): Показує розміщення компонентів системи на апаратних пристроях та зв'язок між ними.

2. Поведінкові моделі:

Діаграма варіантів використання (Use Case Diagram): Описує функціональні вимоги системи, показуючи варіанти використання та акторів (користувачів).

Діаграма активностей (Activity Diagram): Показує послідовність дій і переходів між ними у межах певного варіанта використання чи процесу.

Діаграма послідовності (Sequence Diagram): Показує послідовність повідомлень, що передаються між об'єктами системи у межах конкретного варіанта використання.

Діаграма станів (State Diagram): Описує різні стани об'єкта та переходи між ними залежно від зовнішніх подій.

3. Загальні моделі:

Діаграма пакетів (Package Diagram): Показує організацію елементів моделі UML у логічні групи (пакети).

Діаграма комунікації (Communication Diagram): Показує зв'язки та повідомлення між об'єктами системи у межах певного сценарію чи процесу.

Діаграма часових послідовностей (Timing Diagram): Показує зміну станів об'єктів та часові інтервали в рамках певного сценарію чи процесу.

Стандартні архітектури:

Клієнт-серверна архітектура: Розділяє систему на клієнтські та серверні компоненти, які взаємодіють через мережу.

Багаторівнева (Layered) архітектура: Розбиває систему на шари, кожен із яких відповідає за певний аспект функціональності.

Мікро-сервісна архітектура: Розділяє систему на незалежні мікросервіси, кожен з яких вирішує конкретне завдання та взаємодіє з іншими сервісами API.

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

Національний університет «Запорізька політехніка»

Кафедра програмних засобів

КОНСПЕКТ ЛЕКЦІЙ

**ДИСЦИПЛІНА: «БЕЗПЕКА ТА ЗАХИСТ ПРОГРАМ ТА
ДАНИХ»**

ЗМІСТ

1. Основи кібербезпеки

1.1. Поняття кіберпростору та інформаційного простору.

1.2. Інформаційна безпека як сфера національної безпеки України, безпеки підприємства/установи, особистої безпеки.

1.3. Поняття кібербезпеки, захисту інформації та кіберзахисту.

1.4. Види захисту інформації: технічний, інженерний, криптографічний, організаційний.

1.5. Поняття конфіденційності, цілісності, доступності.

1.6. Принципи кібербезпеки.

2. Кіберзахист та кібератаки.

2.1. Поняття загроз, атак, вразливостей.

2.2. Класифікація загроз, атак.

2.3. Кіберзлочини. Кібервійна. Кібероборона.

2.4. Кібертероризм. Кіберрозвідка.

2.5. Модель порушника.

2.6. Поняття, сутність та основні завдання комплексної системи захисту інформації.

3. Безпека мережі.

3.1. Поняття про шкідливе програмне забезпечення.

3.2. Шпигунські програми: вимагачі, фішинг, програми-вимагачі.

3.3. DDOS-атаки.

1.Основи кібербезпеки

1.1.Поняття кіберпростору та інформаційного простору.

Кіберпростір – це форма співіснування сукупності матеріальних та нематеріальних об'єктів і процесів, спрямованих на породження, сприйняття, запам'ятовування, переробку та обмін інформацією.

Кіберпростір – це віртуальний світ, який базується на реальному матеріальному фундаменті та з реальними наслідками свого “існування та розвитку”.

Кіберпростір є дуже складним явищем, що об'єднує в собі реальність і віртуальність, матеріальне і нематеріальне, абстрактність і дійсність.

Кіберпростір має наступні властивості:

- протяжність;
- єдність дискретності та неперервності;
- матеріальність та нематеріальність;
- абстрактність і дійсність;
- реальність загальнодіючого впливу.

Кіберпростір має свою розмірність/протяжність, яка визначається кількістю наявних матеріальних і нематеріальних об'єктів на даний період часу. Коли протяжність кіберпростору з точки зору наявності матеріальних об'єктів, обмежена поверхнею земної кулі, то з точки зору наявності нематеріальних об'єктів – протяжність кіберпростору практично необмежена.

Об'єктами кіберпростору є живі істоти та їх угруповання, які спроможні сприймати, запам'ятовувати та переробляти інформацію, а також обмінюватися нею, серед яких, в першу чергу, людина, визначені верстви суспільства та суспільство в цілому, держава, природні та штучні інформаційні відносини між ними та їх формування і використання, а також матеріальні та нематеріальні об'єкти і процеси, спрямовані на породження, сприйняття, запам'ятовування, збереження, переробку та обмін інформацією;

Суб'єктами кіберпростору є людина, суспільство, держава, а також жива істота, яка спроможна сприймати, запам'ятовувати та переробляти інформацію, а також обмінюватися нею.

Інформаційний простір – це форма співіснування сукупності матеріальних та нематеріальних об'єктів і процесів, спрямованих на задоволення інформаційних потреб всіх живих істот на Землі.

Дане визначення охоплює інформаційний зв'язок та інформаційні відносини між суб'єктами “живої” і “неживої” природи.

Виходячи з наведеного визначення можна сказати, що:

- об'єктами інформаційного простору є суб'єкти природного середовища, природні та штучні інформаційні відносини між ними та їх формування і використання, матеріальні та нематеріальні об'єкти і процеси, спрямовані на задоволення інформаційних потреб для забезпечення збереження життя та життєдіяльності живої істоти, угруповання живих істот;

- суб'єктами інформаційного простору є живі істоти та їх угруповання, які спроможні сприймати, запам'ятовувати та переробляти інформацію, а також обмінюватися нею.

Інформаційний простір має властивості:

- протяжність;
- єдність дискретності та неперервності;
- матеріальність та нематеріальність; - абстрактність і дійсність; - реальність загальнодіючого впливу.

Порівнюючи надані визначення понять “кіберпростір” та “інформаційний простір”, одразу бачимо, що це – два різних “простори”, і відмінність їх полягає у їх спрямованості. Кіберпростір, який охоплює лише живі істоти, які спроможні сприймати, запам'ятовувати та переробляти інформацію, а також обмінюватися нею, є невід'ємною складовою інформаційного простору, якій охоплює абсолютно всі джерела інформації без вимог щодо спроможності сприймати, запам'ятовувати та переробляти інформацію.

1.2. Інформаційна безпека як сфера національної безпеки України, безпеки підприємства/установи, особистої безпеки.

Інформаційна безпека — це стан захищеності систем обробки і зберігання даних, при якому забезпечено конфіденційність, доступність і цілісність інформації, використання й розвиток в інтересах громадян або комплекс заходів, спрямованих на забезпечення захищеності інформації особи, суспільства і держави від несанкціонованого доступу, використання, оприлюднення, руйнування, внесення змін, ознайомлення, перевірки запису чи знищення (у цьому значенні частіше використовують термін «захист інформації»).

Інформаційна безпека держави характеризується ступенем захищеності і, отже, стійкістю основних сфер життєдіяльності (економіки, науки, техносфери, сфери управління, військової справи, суспільної свідомості і т. д.) стосовно небезпечних (дестабілізаційних, деструктивних, суперечних інтересам країни тощо), інформаційним впливам, причому як до впровадження, так і до вилучення інформації.

Поняття **інформаційної безпеки** не обмежується безпекою технічних інформаційних систем чи безпекою інформації у чисельному чи електронному вигляді, а стосується усіх аспектів захисту даних чи інформації незалежно від форми, у якій вони перебувають.

Розуміється стан захищеності інформаційного простору України, при якому спеціальні інформаційні операції (ІПСО), акти зовнішньої інформаційної агресії, інформаційний тероризм, незаконне зняття інформації за допомогою спеціальних технічних засобів, комп'ютерні злочини та інший деструктивний інформаційний вплив не завдає суттєвої шкоди національним інтересам.

Відповідно до законодавства України поняття "інформаційна безпека" має таке визначення:

"стан захищеності життєво важливих інтересів людини, суспільства і держави, при якому запобігається нанесення шкоди через: неповноту, невчасність та невірогідність інформації, що використовується; негативний інформаційний вплив; негативні наслідки застосування інформаційних технологій; несанкціоноване поширення, використання, порушення цілісності, конфіденційності та доступності інформації."

"Інформаційна безпека України - складова частина національної безпеки України, стан захищеності державного суверенітету, територіальної цілісності, демократичного конституційного ладу, інших життєво важливих інтересів людини, суспільства і держави, за якого належним чином забезпечуються конституційні права і свободи людини на збирання, зберігання, використання та поширення інформації, доступ до достовірної та об'єктивної інформації, існує ефективна система захисту і протидії нанесенню шкоди через поширення негативних інформаційних впливів, у тому числі скоординоване поширення недостовірної інформації, деструктивної пропаганди, інших інформаційних операцій, несанкціоноване розповсюдження, використання й порушення цілісності інформації з обмеженим доступом".

Виокремлюють три рівня забезпечення інформаційної безпеки:

- **рівень особи** (формування раціонального, критичного мислення на основі принципів свободи вибору);

- **суспільний рівень** (формування якісного інформаційно-аналітичного простору, плюралізм, багатоканальність отримання інформації, незалежні потужні ЗМІ, які належать вітчизняним власникам);
- **державний рівень** (інформаційно-аналітичне забезпечення діяльності державних органів, інформаційне забезпечення внутрішньої і зовнішньої політики на міждержавному рівні, система захисту інформації з обмеженим доступом, протидія правопорушенням в інформаційній сфері, комп'ютерним злочинам)[5]

Концепція державної інформаційної політики

Протягом 2002-2010 рр. було три спроби ухвалити концепцію державної інформаційної політики - 2002, 2009 та 2010 року. 11 січня 2011 року черговий проект концепції прийняли у першому читанні за основу закону і направили на доопрацювання Комітету Верховної Ради України з питань свободи слова та інформації.

Забезпечення інформаційної безпеки в Україні наразі регламентовано законами України «Про інформацію», «Про національну безпеку України», «Про Національну програму інформатизації», а також «Стратегією інформаційної безпеки», затвердженої Указом Президента України від 28 грудня 2021 року № 685/202, та Стратегією кібербезпеки України, затвердженої Указом Президента України від 26 серпня 2021 року № 447/2021.

Загрози національній безпеці в інформаційній сфері

Однією з основних загроз інформаційній безпеці ЗУ "Про основи національної безпеки" називає "намагання маніпулювати суспільною свідомістю, зокрема, шляхом поширення недостовірної, неповної або упередженої інформації".

До інших загроз віднесено:

- прояви обмеження свободи слова та доступу громадян до інформації;
- поширення засобами масової інформації культу насильства, жорстокості, порнографії;
- комп'ютерна злочинність та комп'ютерний тероризм;
- розголошення інформації, яка становить державну таємницю, а також конфіденційної інформації, що є власністю держави або спрямована на забезпечення потреб та національних інтересів суспільства і держави.

У сучасному світі глобалізму серед інших цивілізаційних викликів великою небезпекою стають нові оберти реалізації різних доктрин керованого хаосу, пропаганди війни, екстремізму, сепаратизму, насильства, жорстокості, розпалювання національної, міжетнічної, релігійної ворожнечі і ненависті, маніпулювання громадською думкою, свідомістю. Нерідко інформація може

виступати способом прихованого впливу на психіку людини, маніпулювання свідомістю (навіювання), прямим вторгненням у психічне життя людей, використовуватись для здійснення комп'ютерної злочинності чи кібертероризму.

В Доктрині інформаційної безпеки України, підписаній Президентом в липні 2009 р., серед всього виділялись наступні загрози інформаційній безпеці країни:

- поширення у світовому інформаційному просторі викривленої, недостовірної та упередженої інформації, що завдає шкоди національним інтересам України;
- зовнішні деструктивні інформаційні впливи на суспільну свідомість через засоби масової інформації, а також мережу Інтернет;
- деструктивні інформаційні впливи, які спрямовані на підрив конституційного ладу, суверенітету, територіальної цілісності і недоторканності України;
- прояви сепаратизму в засобах масової інформації, а також у мережі Інтернет, за етнічною, мовною, релігійною та іншими ознаками.

Після 2014 року розпочалася розробка нової Доктрини інформаційної безпеки, яка б відповідала вимогам часу.

Новітнім нормативним актом - Стратегією інформаційної безпеки, затвердженої Указом Президента України від 28 грудня 2021 року № 685/2021, визначено: "Метою Стратегії є посилення спроможностей щодо забезпечення інформаційної безпеки держави, її інформаційного простору, підтримки інформаційними засобами та заходами соціальної та політичної стабільності, оборони держави, захисту державного суверенітету, територіальної цілісності України, демократичного конституційного ладу, забезпечення прав та свобод кожного громадянина. Досягнення мети здійснюватиметься шляхом ужиття заходів щодо стримування та протидії загрозам інформаційній безпеці України та нейтралізації інформаційної агресії, у тому числі спеціальних інформаційних операцій держави-агресора, спрямованих на підрив державного суверенітету, територіальної цілісності України, забезпечення інформаційної стійкості суспільства та держави, створення ефективної системи взаємодії між органами державної влади, органами місцевого самоврядування та суспільством, а також розвиток міжнародної співпраці у сфері інформаційної безпеки на засадах партнерства та взаємної підтримки."

1.3.Поняття кібербезпеки, захисту інформації та кіберзахисту.

Кібербезпека — застосування технологій і методів захисту комп'ютерних систем, мереж, програм і даних в кіберпросторі від несанкціонованого доступу, зміни, знищення та інших кіберзагроз.

Кібербезпека – це практична діяльність, спрямована на захист комп’ютерних систем і мереж від кібератак. Вона охоплює весь комплекс заходів – від керування паролями до інструментів комп’ютерної безпеки на основі технологій машинного навчання. Кібербезпека дає змогу надійно виконувати торговельні операції, спілкуватися та переглядати контент в Інтернеті.

Кібербезпека – це комплекс процесів, практичних порад і технологічних рішень, які допомагають захищати важливі системи й мережу від кібератак.

Захист інформації (англ. Data protection) — сукупність методів і засобів, що забезпечують цілісність, конфіденційність і доступність інформації за умов впливу на неї загроз природного або штучного характеру, реалізація яких може призвести до завдання шкоди власникам і користувачам інформації.

Кіберзахист (англ. Cyber Defence) — сукупність організаційних, правових, інженерно-технічних заходів, а також заходів криптографічного та технічного захисту інформації, спрямованих на запобігання кіберінцидентам, виявлення та захист від кібератак, ліквідацію їх наслідків, відновлення сталості і надійності функціонування комунікаційних, технологічних систем.

1.4. Види захисту інформації: технічний, інженерний, криптографічний, організаційний.

Технічний — забезпечує обмеження доступу до носія повідомлення апаратно-технічними засобами (антивіруси, фаєрволи, маршрутизатори, токени, смарткарти тощо):

- попередження витоку по технічним каналам;
- запобігання блокуванню ;

Інженерний — запобігає руйнуванню носія внаслідок навмисних дій або природного впливу інженерно-технічними засобами (сюди відносять обмежувальні конструкції, охоронно-пожежна сигналізація).

Криптографічний — попереджує доступ за допомогою математичних перетворень повідомлення (ІП):

- попередження несанкціонованої модифікації ;
- попередження НС розголошення.

Організаційний — попередження доступу на об'єкт інформаційної діяльності сторонніх осіб за допомогою організаційних заходів (правила розмежування доступу).

1.5.Поняття конфіденційності, цілісності, доступності.

Три принципи інформаційної безпеки: поняттєва тріада -

Конфіденційність, цілісність і доступність — це запорука ефективного захисту та безпеки інфраструктури організації.

модель CIA:

Конфіденційність (англ. confidentiality) — властивість інформації, яка полягає в тому, що інформація не може бути отримана неавторизованим користувачем

Цілісність (англ. integrity) — означає неможливість модифікації неавторизованим користувачем

Доступність (англ. availability) — властивість інформації бути отриманою авторизованим користувачем, за наявності у нього відповідних повноважень, в необхідний для нього час

1.6.Принципи кібербезпеки.

Забезпечення кібербезпеки в Україні ґрунтується на принципах:

- 1) верховенства права, законності, поваги до прав людини і основоположних свобод та їх захисту в порядку, визначеному законом;
- 2) забезпечення національних інтересів України;
- 3) відкритості, доступності, стабільності та захищеності кіберпростору, розвитку мережі Інтернет та відповідальних дій у кіберпросторі;
- 4) державно-приватної взаємодії, широкої співпраці з громадянським суспільством у сфері кібербезпеки та кіберзахисту, зокрема шляхом обміну інформацією про інциденти кібербезпеки, реалізації спільних наукових та дослідницьких проектів, навчання та підвищення кваліфікації кадрів у цій сфері;
- 5) пропорційності та адекватності заходів кіберзахисту реальним та потенційним ризикам, реалізації невід'ємного права держави на самозахист відповідно до норм міжнародного права у разі вчинення агресивних дій у кіберпросторі;
- 6) пріоритетності запобіжних заходів;

- 7) невідворотності покарання за вчинення кіберзлочинів;
- 8) пріоритетного розвитку та підтримки вітчизняного наукового, науково-технічного та виробничого потенціалу;
- 9) міжнародного співробітництва з метою зміцнення взаємної довіри у сфері кібербезпеки та вироблення спільних підходів у протидії кіберзагрозам, консолідації зусиль у розслідуванні та запобіганні кіберзлочинам, недопущення використання кіберпростору в терористичних, воєнних, інших протиправних цілях.

2. Кіберзахист та кібератаки.

Кіберзахист (англ. *Cyber Defence*) — сукупність організаційних, правових, інженерно-технічних заходів, а також заходів криптографічного та технічного захисту інформації, спрямованих на запобігання кіберінцидентам, виявлення та захист від кібератак, ліквідацію їх наслідків, відновлення сталості і надійності функціонування комунікаційних, технологічних систем.

Кіберзахист покликаний допомагати у розробці та керувати стратегіями, необхідними для протидії шкідливим атакам або загрозам.

Широкий спектр різних видів діяльності залучається до кібербезпеки для захисту визначеного суб'єкта, а також для швидкого реагування на кібератаки.

Це може включати зменшення привабливості визначеного суб'єкта для можливих зловмисників, розуміння критичних місць та конфіденційної інформації, запровадження запобіжних заходів для забезпечення протидії кібератакам, можливості виявлення кібератак та реагування.

Кіберзахист також передбачає проведення технічного аналізу для визначення шляхів і областей, які можуть атакувати зловмисники. [2]

Впровадження системи кіберзахисту передбачено законом «Про основні засади забезпечення кібербезпеки України».

Кібератака — спроба реалізації інформаційної загрози.

Тобто, це дії кібер-зловмисників або шкідливих програм, які спрямовані на захоплення інформаційних даних віддаленого комп'ютера, отримання повного контролю над ресурсами комп'ютера або на виведення системи з ладу.

Під **атакою** (англ. *attack*, англ. *intrusion*) на інформаційну систему розуміють дії (процеси) або послідовність зв'язаних між собою дій порушника,

які приводять до реалізації загроз інформаційним ресурсам шляхом використання уразливостей цієї інформаційної системи.

Типи кібератак

Основними типами атак на інформаційні системи є:

- Віддалене проникнення (remote penetration).
- Локальне проникнення (local penetration).
- Атака на відмову в обслуговуванні (denial of service).
- Мережні сканери (network scanners).
- Сканери уразливостей (vulnerability scanners).
- Зламувачі паролів (password crackers).
- Аналізатори протоколів (sniffers).
- Спам e-mail (Mailbombing).
- Перехоплення каналу зв'язку (Man-in-the-Middle).

2.1.Поняття загроз, атак, вразливостей.

Загроза інформаційної безпеки — сукупність умов і факторів, що створюють небезпеку порушення інформаційної безпеки.

Під загрозою (в загальному) розуміється потенційно можлива подія, дія (вплив), процес або явище, які можуть призвести до заподіяння шкоди чийм-небудь інтересам.

Під загрозою інтересів суб'єктів інформаційних відносин розуміють потенційно можливу подію, процес або явище, яке з допомогою впливу на інформацію або інші компоненти інформаційної системи, може прямо або опосередковано призвести до заподіяння шкоди даним того чи іншого суб'єкта.

Атака - це навмисне використання виявлених вразливостей комп'ютерних інформаційних систем як з конкретною метою, так і просто для розваги.

Під **атакою** (англ. attack, англ. intrusion) на інформаційну систему розуміють дії (процеси) або послідовність зв'язаних між собою дій порушника, які приводять до реалізації загроз інформаційним ресурсам шляхом використання уразливостей цієї інформаційної системи.

Вразливість (англ. system vulnerability) — нездатність системи протистояти реалізації певної загрози або сукупності загроз[1]. Тобто, це певні недоліки в комп'ютерній системі, завдяки яким можна навмисно порушити її цілісність і викликати неправильну роботу.

Вразливість може виникати в результаті допущених помилок програмування, недоліків, допущених при проектуванні системи, ненадійних паролів, вірусів та інших шкідливих програм, скриптових і SQL-

ін'єкцій. Деякі уразливості відомі тільки теоретично, інші ж активно використовуються і мають відомі експлойти.

Причини вразливостей

- *Складність*: великі, складні системи збільшують ймовірність дефектів і ненавмисних точок доступу.[3]
- *Відомість*: використання загального, відомого коду, програмного забезпечення, операційних систем та/або обладнання збільшує ймовірність того, що зловмисник може знайти інформацію та інструменти, щоб використати недолік[4].
- *Зв'язок*: фізичні з'єднання, привілеї, порти, протоколи, служби та час, який вони є доступними, збільшує вразливість.
- *Недостатній контроль паролів*: користувач використовує слабкі паролі, які можуть бути виявлені за допомогою грубої сили; користувач комп'ютера зберігає пароль на комп'ютері, на якому програма може отримати до нього доступ; користувач повторно використовує паролі між багатьма програмами і вебсайтами[3].
- *Фундаментальні недоліки дизайну операційної системи*: дизайнер операційної системи вибирає для забезпечення роботи неоптимальні політики управління «користувач — програма». Наприклад, операційні системи з політикою щодо дозволу на отримання повного доступу до всього комп'ютера для будь-якої програми чи користувача. Ці недоліки операційної системи дозволяють вірусам і шкідливим програмам виконувати команди від імені адміністратора[5].
- *Перегляд вебсайту*: Деякі сайти можуть містити шкідливі шпигунські або рекламні програми, які можуть бути автоматично встановлені на комп'ютерних системах. Після відвідування цих сайтів, комп'ютерні системи можуть заразитися і особиста інформація буде збиратися і передаватися до сторонніх осіб[6].
- *Помилки програмного забезпечення*: Програміст залишає помилку в програмі, яка може дозволити зловмиснику зловживати програмою[3].
- *Необмежений вхід користувача*: Програма припускає, що все, що вводить користувач безпечно. Програми, які не перевіряють введення користувача, можуть дозволити ненавмисне безпосереднє виконання команд або операторів SQL (відомих як переповнення буфера, ін'єкції SQL та інші неперевірені входи).

2.2.Класифікація загроз, атак.

Загрози інформаційної безпеки можуть бути класифіковані за різними ознаками:

За аспектом інформаційної безпеки, на який спрямовані загрози:

- **Загрози конфіденційності** (неправомірний доступ до інформації). Загроза порушення конфіденційності полягає в тому, що інформація стає відомою тому, хто не володіє повноваженнями доступу до неї. Вона стається, коли отримано доступ до деякої інформації обмеженого доступу, що зберігається в комп'ютерній системі або передається від однієї системи до іншої. У зв'язку з загрозою порушення конфіденційності, використовується термін «витік». Подібні загрози можуть виникати внаслідок «людського фактора» (наприклад, випадкове делегування тому або іншому користувачеві привілеїв іншого користувача), збоїв роботи програмних та апаратних засобів. До інформації обмеженого доступу належить державна таємниця (комерційна таємниця, персональні дані, професійні види таємниці: лікарська, адвокатська, банківська, службова, нотаріальна таємниця страхування, слідства й судочинства, листування, телефонних переговорів, поштових відправлень, телеграфних або інших повідомлень (таємниця), відомості про сутність винаходу, корисної моделі або промислового зразка до офіційної публікації (ноу-хау) та ін).
- **Загрози цілісності** (неправомірна зміна даних). Загрози порушення цілісності — це загрози, пов'язані з імовірністю модифікації тієї чи іншої інформації, що зберігається в інформаційній системі. Порушення цілісності може бути викликано різними чинниками — від умисних дій персоналу до виходу з ладу обладнання.
- **Загрози доступності** (здійснення дій, які унеможливають чи ускладнюють доступ до ресурсів інформаційної системи). Порушення доступності являє собою створення таких умов, при яких доступ до послуги або інформації або заблокований, або можливий за час, який не забезпечить виконання тих чи інших бізнес-цілей.

За розташуванням джерела загроз:

- **Внутрішні** (джерела загроз розташовуються всередині системи);
- **Зовнішні** (джерела загроз знаходяться поза системою).

За розмірами завданого збитку:

- **Загальні** (завдавання збитку об'єктові безпеки в цілому, заподіяння значної шкоди);
- **Локальні** (заподіяння шкоди окремими частинами об'єкта безпеки);
- **Приватні** (заподіяння шкоди окремим властивостям елементів об'єкта безпеки).

За ступенем впливу на інформаційну систему:

- **Пасивні** (структура і зміст системи не змінюються);
- **Активні** (структура і зміст системи піддається змінам).

За природою виникнення:

- **Природні (об'єктивні)** — викликані впливом на інформаційне середовище об'єктивних фізичних процесів або стихійних природних явищ, що не залежать від волі людини;
- **Штучні (суб'єктивні)** — викликані впливом на інформаційну сферу людини. Серед штучних загроз своєю чергою виділяють:
- **Ненавмисні (випадкові)** погрози, помилки програмного забезпечення, персоналу, збої в роботі систем, відмови обчислювальної та комунікаційної техніки;
- **Навмисні (умисні)** загрози — неправомірний доступ до інформації, розробка спеціального програмного забезпечення, використовуваного для здійснення неправомірного доступу, розробка та поширення вірусних програм і т. д. Навмисні загрози зумовлені діями людей. Основні проблеми інформаційної безпеки пов'язані насамперед з умисними погрозами, оскільки вони є головною причиною злочинів і правопорушень.

Класифікація джерел загроз інформаційної безпеки

Носіями загроз безпеки інформації є **джерела загроз**. Як джерела загроз можуть виступати як суб'єкти (особистість), так і об'єктивні прояви, наприклад, конкуренти, злочинці, корупціонери, адміністративно-управлінські органи. Джерела загроз переслідують при цьому наступні цілі: ознайомлення з охоронюваними відомостями, їх модифікація в корисливих цілях і знищення для нанесення прямого матеріального збитку.

Всі джерела загроз інформаційної безпеки можна поділити на три основні групи:

1. Обумовлені діями суб'єкта (антропогенні джерела) — суб'єкти, дії яких можуть призвести до порушення безпеки інформації, дані дії можуть бути кваліфіковані як навмисні або випадкові злочини. Джерела, дії яких можуть призвести до порушення безпеки інформації можуть бути як

зовнішніми, так і внутрішніми. Ці джерела можна спрогнозувати, і прийняти адекватні заходи.

2.Обумовлені технічними засобами (техногенні джерела) — ці джерела загроз менш прогнозовані, безпосередньо залежать від властивостей техніки і тому вимагають особливої уваги. Дані джерела загроз інформаційній безпеці, також можуть бути як внутрішніми, так і зовнішніми.

3.Стихійні джерела — дана група об'єднує обставини, що становлять непереборну силу (стихійні лиха або інші обставини, які неможливо передбачити або запобігти чи можливо передбачити, але неможливо запобігти), такі обставини, які мають об'єктивний і абсолютний характер, поширюється на всіх. Такі джерела загроз абсолютно не піддаються прогнозуванню і тому заходи проти них повинні застосовуватися завжди. Стихійні джерела, як правило, є зовнішніми щодо захищеного об'єкта і під ними, як правило, розуміються природні катаклізми.

Типи кібератак:

Відмова в обслуговуванні

Атака типу "Відмова в обслуговуванні" (DoS) - є різновидом мережної атаки. Результатом DoS-атаки є переривання доступу користувачів, пристроїв або застосунків до мережних сервісів.

Існує два основних типи DoS-атак:

1.Перевантаження великою кількістю трафіку - Нападник надсилає величезну кількість даних з такою швидкістю, що мережа, хост або застосунок не встигає їх обробляти. Це спричиняє уповільнення передачі або реагування, іноді призводить до аварійного завершення роботи пристрою чи сервісу.

2.Пакети неправильного формату (Maliciously Formatted Packets) - Нападник надсилає пакет даних неправильного формату хосту або застосунку і одержувач не може його обробити.

Аналіз трафіку (Sniffing)

Аналіз трафіку (Sniffing) є дуже схожим на підглядання за людиною в реальному світі. Хакери досліджують весь мережний трафік, який проходить через їх мережну інтерфейсну карту (NIC), незалежно від того, кому він адресований.

Sniffing може бути корисним. Мережні адміністратори можуть використовувати такі засоби для аналізу мережного трафіку, визначення проблем з пропускнуою здатністю та вирішення інших проблем в мережній інфраструктурі.

Підміна

Підміна (Spoofing) – це атака шляхом імперсоніфікації, яка відбувається за рахунок використання довірчих відносин між двома системами. Якщо дві системи підтримують єдину аутентифікацію, користувач, який увійшов до однієї системи, може не проходити повторно процес аутентифікації для доступу до іншої системи. Зловмисник може скористатися цими довірчими відносинами, відправивши пакет до однієї системи, який виглядає як такий, що надійшов з іншої довіреної системи. Оскільки між системами діють довірчі відносини, цільова система може виконати запит без аутентифікації.

Man-in-the-middle (людина посередині)

Злочинець, який виконує атаку Man-in-the-middle (MitM), перехоплює повідомлення, якими обмінюються комп'ютери, щоб викрасти інформацію, яка проходить мережею. Злочинець також може маніпулювати повідомленнями та передавати підставні дані між хостами, оскільки вузли не усвідомлюють, що відбулася модифікація повідомлень. MitM дозволяє злочинцю контролювати пристрій без відому користувача.

Атаки нульового дня

Атака нульового дня (zero-day attack), яку іноді називають загрозою нульового дня (zero-day threat), - це комп'ютерна атака, яка намагається використати вразливості ПЗ, що досі невідомі постачальнику програмного забезпечення, або які він приховує.

Клавіатурні шпигуни (кейлогери)

Клавіатурний шпигун (кеулогер) - це програма, яка записує або вносить в спеціальний журнал натискання клавіш користувачем системи. Злочинці можуть реалізовувати кейлогери за допомогою ПЗ, встановленого на комп'ютері або через обладнання, фізично приєднане до комп'ютера. Зловмисник налаштовує ПЗ клавіатурного шпигуна таким чином, щоб воно відправляло зібрану в журналі інформацію електронною поштою. Перехоплені та записані до лог-файлу натискання клавіш можуть розкрити імена користувачів, паролі, відвідані веб-сайти та іншу конфіденційну інформацію.

Атаки на бездротові системи та мобільні пристрої

- **Grayware** може відстежувати місцезнаходження користувача. Автори умовно шкідливого ПЗ зазвичай підтримують видимість легітимності, включивши опис функцій програми маленьким шрифтом до ліцензійної угоди. Користувачі встановлюють багато мобільних додатків, не ознайомлюючись з їх функціями.

- **Smishing (SMiShing)** - коротка форма терміну SMS фішинг. Він використовує службу SMS для надсилання підроблених текстових повідомлень. Злочинці обманним шляхом змушують користувача відвідати веб-сайт або зателефонувати за номером телефону. Довірливі жертви можуть надавати таку конфіденційну інформацію, як інформація про кредитні картки. Відвідування веб-сайту може призвести до несвідомого завантаження користувачем шкідливого ПЗ, яке заразить його пристрій.

Несанкціоновані точки доступу

Несанкціонована точка доступу (Rogue Access Points) - це точка доступу, встановлена в захищеній мережній інфраструктурі без дозволу.

Несанкціонована точка доступу також може означати точку доступу, яка використовується в злочинних цілях. У цьому випадку злочинці налаштовують точку доступу для атак через посередника (MitM), щоб перехоплювати реєстраційні дані користувачів.

Глушіння радіочастот (RF Jamming)

Бездротові сигнали чутливі до електромагнітних (EMI) та радіочастотних перешкод (RFI), а також до розряду блискавки або завад від флуоресцентних ламп. Бездротові сигнали також чутливі до навмисного глушіння. Радіочастотні перешкоди (Radio frequency (RF) jamming) блокують сигнал радіо або супутникової станції, щоб він не доходив до станції отримувача.

Bluejacking та Bluesnarfing

Bluetooth - малопотужний протокол ближньої дії. Він використовується для передачі даних в персональній мережі (PAN), яка може містити такі пристрої, як мобільні телефони, ноутбуки та принтери.

Bluejacking - це термін, який означає передачу несанкціонованих повідомлень на інший пристрій через Bluetooth. Як варіант цей спосіб використовують для розсилання шокуючих зображень.

Bluesnarfing (підключення до пристрою Bluetooth з метою крадіжки даних) – це термін, що описує ситуацію, коли зловмисник копіює інформацію

жертви з її пристрою. Ця інформація може містити електронні листи та списки контактів.

Атаки на застосунки

-Міжсайтовий скриптинг

Міжсайтовий скриптинг (Cross-Site Scripting, XSS) - вразливість, виявлена у веб-застосунках. XSS дозволяє злочинцям вбудовувати скрипти у веб-сторінки, які переглядають користувачі. Такий скрипт може містити зловмисний код.

Міжсайтовий скриптинг включає трьох учасників: злочинець, жертва та веб-сайт. Кібер-злочинець не націлений на жертву напряду. Він використовує вразливість веб-сайту або веб-застосунку.

-Ін'єкція коду

Після того, як користувач надає вхідні дані, система відправляє запит для отримання доступу до необхідної інформації в базі даних. Проблема виникає, коли система не належним чином перевіряє вхідний запит користувача. Злочинці можуть маніпулювати запитами, змінюючи їх відповідно до своїх потреб і отримати доступ до інформації в базі даних.

-Переповнення буфера

Переповнення буфера (buffer overflow) - відбувається, коли об'єм даних перевищує розмір виділеного буфера.

Буфери - це області пам'яті, виділені для застосунку. Змінюючи дані за межами буфера, програма звертається до пам'яті, яка була виділена для інших процесів.

Це може спричинити крах системи, призвести до несанкціонованого доступу до даних або ескалації прав користувача.

-Віддалений запуск програм

Вразливості дозволяють кібер-злочинцям виконати зловмисний код і отримати контроль над системою з привілеями користувача, який запустив застосунок.

Віддалене виконання коду дозволяє злочинцю виконати на цільовій машині будь-яку команду.

Вразливість (англ. system vulnerability) — нездатність системи протистояти реалізації певної загрози або сукупності загроз. Тобто, це певні недоліки в комп'ютерній системі, завдяки яким можна навмисно порушити її цілісність і викликати неправильну роботу.

Вразливість може виникати в результаті допущених помилок програмування, недоліків, допущених при проектуванні системи, ненадійних паролів, вірусів та інших шкідливих програм, скриптових і SQL-ін'єкцій. Деякі уразливості відомі тільки теоретично, інші ж активно використовуються і мають відомі експлойти.

Вразливості, класифікуються відповідно до класу активів, до яких вони відносяться:

Апаратне забезпечення:

- Сприйнятливості до вологості;
- Сприйнятливості до пилу;
- Сприйнятливості до забруднень;
- Сприйнятливості до незахищеної зберігання.

Програмне забезпечення:

- Недостатнє тестування;
- Відсутність аудиту.
- Комп'ютерна мережа:
- Незахищені лінії зв'язку;
- Незахищена мережева архітектура.

Персонал:

- Недоліки найму працівників;
- Недостатня поінформованість про небезпеку[en].

Сайт:

- Ненадійне джерело живлення.

Організація:

- Відсутність регулярних перевірок;
- Відсутність планів безперервності;
- Відсутність безпеки.
-

Поширені типи вразливостей включають в себе:

Порушення безпеки доступу до пам'яті, такі як:

- Переповнення буфера;
- Завислі вказівники.

Помилки перевірки введених даних, такі як:

- Помилки форматування рядка.
- Невірна підтримка інтерпретації метасимволів командної оболонки
- SQL ін'єкція;
- Ін'єкція коду;
- E-mail ін'єкція;
- Обхід каталогів;
- Міжсайтовий скриптинг у вебдодатках (Міжсайтовий скриптинг за наявності SQL-ін'єкції).
- Помилки часу перевірки до часу використання;
- Гонки символьних посилань.

Помилки плутанини привілеїв, такі як:

- Підробка міжсайтових запитів у вебдодатках.
- Підвищення привілеїв, наприклад:
- Shatter attack («Підбивна атака»).

2.3. Кіберзлочини. Кібервійна. Кібероборона.

Кіберзлочинність — це злочинна діяльність, яка передбачає використання комп'ютерів, комп'ютерних мереж або мережевих пристроїв. Кіберзлочини здійснюють кіберзлочинці — люди, які використовують інформаційні технології у протиправний спосіб.

Кіберзлочинність — незаконні дії спрямовані на розкрадання або руйнування інформації в інформаційних системах і мережах, які виходять з корисливих або хуліганських спонукань.

До основних видів кіберзлочинності можна віднести такі: розповсюдження шкідливого програмного забезпечення, крадіжка номерів кредитних карт і банківських рахунків, злом паролів, порушення авторських прав.

Класифікація кіберзлочинів:

З 1991 за класифікатором Інтерполу інформаційні злочини поділяються:

- QA — Несанкціонований доступ та перехоплення • QDT - Троянський кінь
- QAT - крадіжка часу (ухилення від плати за користування) • QDV - комп'ютерний вірус
- QD — Зміна комп'ютерних даних • QFC - шахрайство з банкоматами
- QF — Комп'ютерне шахрайство • QZ — Інші комп'ютерні злочини
- QR — Незаконне копіювання • QFT - телефонне шахрайство

- QS — Комп'ютерний саботаж • QFF- комп'ютерна підробка (та інші)

За конвенцією Ради Європи по боротьбі з кіберзлочинністю, яка була ратифікована Верховною Радою України виділено чотири основних типи кіберзлочинів:

- 1.Злочини проти конфіденційності, цілісності та доступності комп'ютерних даних і систем.
- 2.Шахрайство та підробка, що пов'язані з використанням комп'ютерів.
- 3.Злочини пов'язані з розміщенням у мережах протиправної інформації.
- 4.Злочини щодо авторських і суміжних прав.

Кібервійна (англ. cyberwarfare) — комп'ютерне протистояння у просторі Інтернету.

Спрямована передусім на дестабілізацію комп'ютерних систем і доступу до інтернету державних установ, фінансових та ділових центрів і створення безладу та хаосу в житті країн, які покладаються на інтернет у повсякденному житті.

Міждержавні стосунки і політичне протистояння часто знаходить продовження в Інтернеті у вигляді кібервійни: вандалізму, пропаганді, шпигунстві та безпосередніх атаках на комп'ютерні системи та сервери.

Види кібервійни

Основні види: наступальна та захисна.

Спеціалісти виділяють такі види атак в інтернеті:

Вандалізм — використання хакерами інтернету для паплюження інтернет-сторінок, заміни змісту образливими чи пропагандистськими зображеннями.

Пропаганда — розсилка звернень пропагандистського характеру, або вставка пропаганди в зміст інших інтернет-сторінок.

Збір інформації — зламування приватних сторінок чи серверів для збору секретної інформації чи її заміни на фальшиву, корисну іншій державі.

Відмова сервісу — атаки з різних комп'ютерів для унеможливлення функціонування сайтів чи комп'ютерних систем.

Втручання в роботу обладнання — атаки на комп'ютери, які займаються контролем над роботою цивільного чи військового обладнання, що призводить до його відключення чи поломки.

Атаки на об'єкти критичної інфраструктури — атаки на комп'ютери, які забезпечують життєдіяльність міст, їхньої інфраструктури, таких як телефонні системи, водопостачання, електроенергії, пожежної охорони, транспорту тощо.

Кібероборона - (англ. Cyber Defence) — сукупність організаційних, правових, інженерно-технічних заходів, а також заходів криптографічного та технічного захисту інформації, спрямованих на запобігання кіберінцидентам, виявлення та захист від кібератак, ліквідацію їх наслідків, відновлення сталості і надійності функціонування комунікаційних, технологічних систем.

Кібероборона покликана допомагати у розробці та керуванні стратегіями, необхідними для протидії шкідливим атакам або загрозам.

Широкий спектр різних видів діяльності залучається до кібербезпеки для захисту визначеного суб'єкта, а також для швидкого реагування на кібератаки. Це може включати зменшення привабливості визначеного суб'єкта для можливих зловмисників, розуміння критичних місць та конфіденційної інформації, запровадження запобіжних заходів для забезпечення протидії кібератакам, можливості виявлення кібератак та реагування. Кіберзахист також передбачає проведення технічного аналізу для визначення шляхів і областей, які можуть атакувати зловмисники.

Впровадження системи кібероборони передбачено законом «Про основні засади забезпечення кібербезпеки України».

Кібероборона визначена у Статті 5 оновленої Угоди НАТО в якості операційного домену, при цьому для розвитку спроможностей у сфері кібернетичної оборони створено відповідну систему.

2.4.Кібертероризм. Кіберрозвідка.

Комп'ютерний тероризм (кібертероризм) — використання комп'ютерних та телекомунікаційних технологій (насамперед, інтернету) в терористичних цілях.

Наприклад, акти, спрямовані на залякування з метою досягнення політичних результатів, або завдання шкоди комп'ютерним мережам, особливо персональним комп'ютерам, підключеним до Інтернету, за допомогою таких засобів, як комп'ютерні віруси.

Законодавство України визначає: «Кібертероризм - терористична діяльність, що здійснюється у кіберпросторі або з його використанням».[1]
Організація, створена для вироблення узгодженої політики з питань економіки

і внутрішньої безпеки (The National Conference of State Legislatures} визначає кібертероризм наступним чином:

Використання інформаційних технологій терористичними групами і терористами-одинаками для досягнення своїх цілей. Може включати використання інформаційних технологій для організації та виконання атак проти телекомунікаційних мереж, інформаційних систем і комунікаційної інфраструктури, або обмін інформацією, а також загрози з використанням засобів електрозв'язку. Прикладами можуть служити злом інформаційних систем, внесення вірусів у вразливі мережі, дефейс вебсайтів, DoS-атаки, терористичні загрози, спричинені електронними засобами зв'язку.

Кібершпигунство або комп'ютерний шпівонаж (вживається також термін «кіберрозвідка») — термін, який позначає несанкціоноване отримання інформації з метою отримання особистої, економічної, політичної чи військової переваги, здійснюваний з використанням обходу (злому) систем комп'ютерної безпеки, з застосуванням шкідливого програмного забезпечення.

Кібершпигунство може здійснюватися як дистанційно, за допомогою Інтернету, так і шляхом проникнення в комп'ютери і комп'ютерні мережі підприємств звичайними шпигунами ("кротами"), а також хакерами.

Кібершпигунство включає також спостереження за цифровим слідом поведінки користувачів соціальних мереж з метою виявлення екстремістської, терористичної чи антиурядової діяльності.

2.5. Модель порушника.

Неформальна модель порушника за суттю є описом його реальних або теоретичних можливостей, апріорних знань, технічної оснащеності, часу та місця дії тощо. Слід враховувати, що для досягнення своїх цілей порушник повинен прикласти деякі зусилля, затратити певні ресурси.

Детально дослідивши умови порушень, у деяких випадках можна або вплинути на причини їх виникнення з метою їх усунення, або точніше визначити вимоги до системи захисту від даного виду порушень або злочинів.

Під час розробки моделі порушника встановлюються:

- припущення щодо категорії осіб, до яких він належить;
- припущення щодо мотивів та мети його дій;
- припущення щодо його кваліфікації та технічної оснащеності, застосованих методів та засобів;
- обмеження й припущення про характер можливих дій.

По відношенню до інформаційних ресурсів розрізняють категорії **внутрішніх порушників** (персонал підприємства) або **зовнішніх порушників** (сторонні особи).

Внутрішнім порушником може бути особа з наступних категорій персоналу, що має доступ у будинки й приміщення, де розташовані компоненти автоматизованої системи:

- користувачі (оператори) автоматизованої системи;
- персонал, що обслуговує технічні засоби системи: інженери, техніки;
- співробітники відділів розробки й супроводження програмного забезпечення: системні та прикладні програмісти;
- технічний персонал, якій обслуговує будівлі: сантехніки, електрики, прибиральники тощо;
- співробітники служби охорони;
- керівники різних рівнів посадової ієрархії.

До **сторонніх осіб**, що можуть бути порушниками, слід віднести наступні категорії осіб:

- партнери підприємства або його клієнти - фізичні або юридичні особи;
- відвідувачі, що звернулися з власних справ або запрошені з будь-якого питання;
- співробітники підприємств постачальників товарів та послуг, включаючи забезпечення життєдіяльності об'єкту інформаційної діяльності (зв'язок, ліфтове господарство, утримання систем сигналізації та відео спостереження, енерго-, водо-, теплопостачання тощо);
- співробітники підприємств – конкурентів, іноземні делегації;
- особи, випадково або навмисне порушили режим доступу на об'єкт інформаційної діяльності;
- співробітники дипломатичний місій або спецслужб. Останні можуть видавати себе за будь-яку категорію з тих, що перелічені вище;
- будь-які особи за межами контрольованої території.
-

Можна виділити три основні мотиви порушень: недбалість (безвідповідальність), самоствердження (помста) і корисливий інтерес.

У випадку порушень, що обумовлені безвідповідальністю, користувач цілеспрямовано або випадково провадить які-небудь руйнуючі дії, не зв'язані проте зі злим наміром. У більшості випадків це слідство некомпетентності або недбалості.

Порушення безпеки інформаційної системи може бути викликане й корисливим інтересом користувача системи. У цьому випадку він буде

цілеспрямовано намагатися подолати систему захисту для доступу до збереженої, переданої й оброблюваної у системі інформації.

Порушників можна класифікувати наступним чином.

За рівнем знань про інформаційну систему:

- знає функціональні особливості системи, основні закономірності формування в ній масивів даних і потоків запитів до них, уміє користуватися штатними засобами;
- має високий рівень знань і досвід роботи з технічними засобами системи і їх обслуговування;
- має високий рівень знань в області програмування й обчислювальної техніки, проектування й експлуатації автоматизованих інформаційних систем;
- знає структуру, функції й механізм дії засобів захисту, їх сильні й слабкі сторони.

За рівнем можливостей та використовуваним методам і засобам:

- застосовує чисто агентурні методи здобування відомостей або втручання у роботу автоматизованої системи;
- застосовує нештатні пасивні методи та технічні засоби перехоплення інформації без модифікації компонентів системи;
- використовує тільки штатні засоби й недоліки системи захисту для її подолання (несанкціоновані дії з використанням дозволених засобів), а також компактні машинні носії інформації, які можуть бути приховано пронесені через пости охорони;
- застосовує методи й засоби активного впливу на інформаційні ресурси, їх модифікації шляхом застосування спеціальних апаратно-програмних засобів, технічних приладів, які підключені до каналів передачі даних, впровадження програмних закладок, використання спеціального програмного забезпечення.

За часом дії:

- у процесі функціонування автоматизованої системи або її окремих компонентів;
- у період не активності компонентів системи, поза робочим часом, під час планових перерв у її роботі для обслуговування та ремонту тощо;
- як у процесі функціонування системи, так і в період не активності її компонентів.

За місцем дії:

- без доступу на контрольовану територію;
- з контрольованої території без доступу в будівлі та споруди;
- усередині приміщень, але без доступу до технічних засобів системи;

- з робочих місць кінцевих користувачів або операторів;
- з доступом у зону сховищ даних (ресурси, що знаходяться у статусі хостінгу, бази даних, архіви тощо);
- з доступом у зону керування засобами забезпечення безпеки.

Доцільно враховувати наступні обмеження й припущення про характер дій можливих порушників:

- порушник, плануючи спроби несанкціонованих дій, може приховувати їх від інших співробітників;
- робота з добору кадрів і спеціальні заходи ускладнюють спроби створення коаліцій двох і більш порушників, тобто їх змови та узгоджених дій з метою подолання підсистеми захисту;
- несанкціоновані дії можуть бути наслідком прорахунків у системі навчання та підготовки персоналу, помилок користувачів та адміністраторів автоматизованої системи, а також недоліків прийнятої технології обробки інформації.

Зауважимо, що визначення конкретних значень характеристик потенційних порушників у значній мірі суб'єктивно, тому доцільно створювати модель порушника групою досвідчених експертів.

В цілому, модель порушника, що побудована з урахуванням особливостей конкретної предметної області й технології обробки інформації, може бути представлена перерахуванням декількох варіантів його виду. Кожний вид порушника повинен бути охарактеризований значеннями характеристик, наведених вище.

Зокрема, нормативні документи системи криптографічного захисту інформації (КЗІ), залежно від вірогідних розумів експлуатації засобів криптографічного захисту інформації та відповідно до цінності інформації, що захищається, ***визначаються чотири рівні можливостей порушника:***

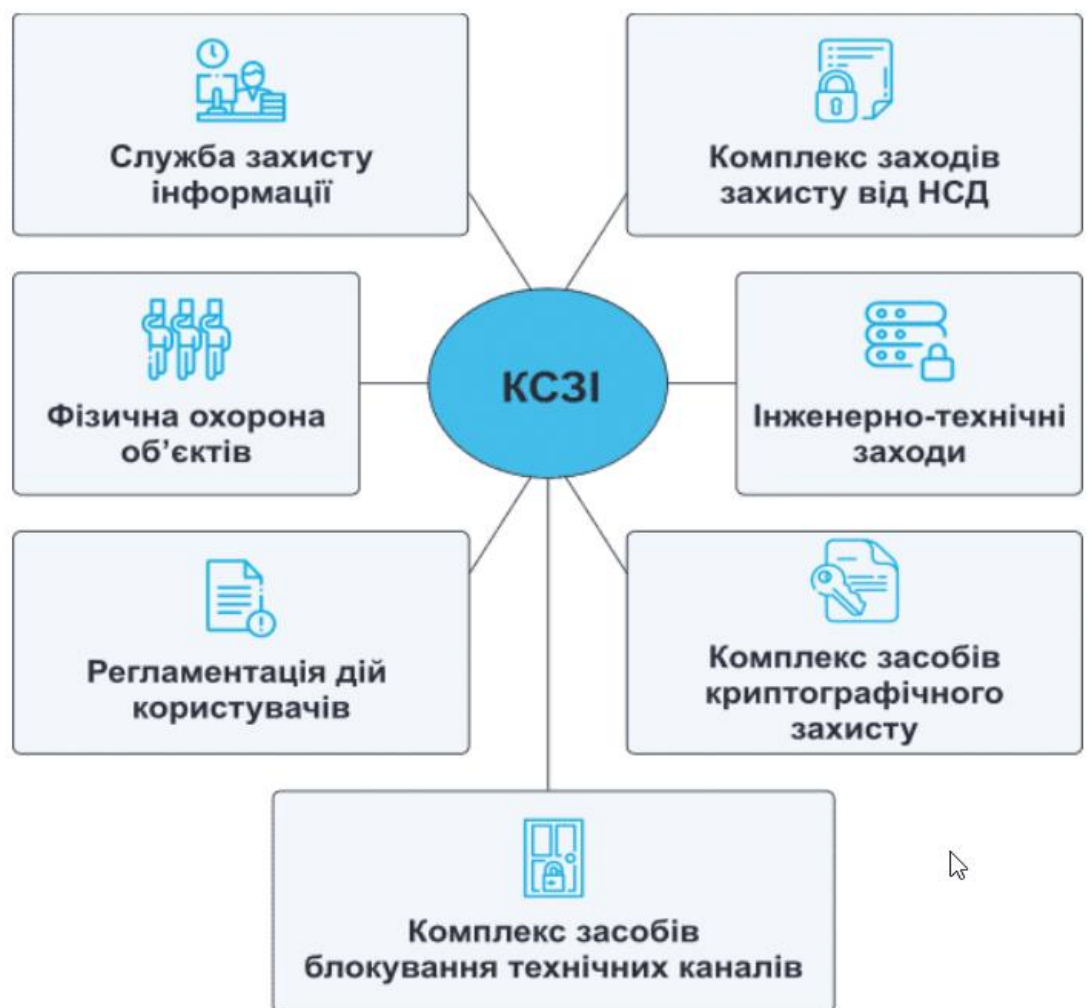
- нульовий рівень - ненавмисне порушення конфіденційності, цілісності та підтвердження авторства інформації;
- перший рівень - порушник має обмежені засоби та самостійно створює засоби, розробляє методи атак на засоби КЗІ, а також інформаційно-телекомунікаційні системи із застосуванням широко розповсюджених програмних засобів та електронно-обчислювальної техніки;
- другий рівень - порушник корпоративного типу має змогу створення спеціальних технічних засобів, вартість яких співвідноситься з можливими фінансовими збитками, що виникатимуть від порушення конфіденційності, цілісності та підтвердження авторства інформації, зокрема при втраті, спотворенні та знищенні інформації, що захищається. У цьому разі для розподілу обчислень при проведенні атак можуть застосовуватися локальні обчислювальні мережі;

- третій рівень - порушник має науково-технічний ресурс, який прирівнюється до науково-технічного ресурсу спеціальної служби економічно розвиненої держави.

Слід звернути увагу, у силу непередбачуваності усіх можливостей порушника, навіть якщо у автоматизованій системі створюється комплекс засобів захисту, що роблять таке проникнення надзвичайно складним, повністю захистити її від проникнення «на всі 100%» практично неможливо.

2.6. Поняття, сутність та основні завдання комплексної системи захисту інформації.

Комплексна система захисту інформації (КСЗІ) — взаємопов'язана сукупність організаційних та інженерно-технічних заходів, засобів і методів захисту інформації.



3.Безпека мережі.

Безпека мережі (Network security) — заходи, які захищають інформаційну мережу від несанкціонованого доступу, випадкового або навмисного втручання в роботу мережі або спроб руйнування її компонентів.

Безпека інформаційної мережі включає захист обладнання, програмного забезпечення, даних і персоналу.

Мережева безпека складається з положень і політики, прийнятої адміністратором мережі, щоб запобігти і контролювати несанкційований доступ, неправильне використання, зміни або відмови в комп'ютерній мережі та мережі доступних ресурсів.

Мережева безпека містить у собі дозвіл на доступ до даних в мережі, який надається адміністратором мережі. Користувачі вибирають або їм призначаються ID і пароль або інші перевірки автентичності інформації, що дозволяє їм здійснити доступ до інформації і програм у рамках своїх повноважень.

Мережева безпека охоплює різні комп'ютерні мережі, як державні, так і приватні, які використовуються в повсякденних робочих місцях для здійснення угод і зв'язків між підприємствами, державними установами та приватними особами.

Мережі можуть бути приватними, такими як всередині компанії або відкритими, для публічного доступу.

Мережева безпека бере участь в організаціях, підприємствах та інших типів закладів.

Найбільш поширений і простий спосіб захисту мережевих ресурсів є присвоєння їм унікального імені та відповідного паролю.

Концепції мережевої безпеки

Мережева безпека починається з аутентифікації, що зазвичай включає в себе ім'я користувача і пароль. Коли для цього потрібно тільки одна деталь аутентифікації (ім'я користувача), то це називають однофакторною аутентифікацією.

При двофакторній аутентифікації, користувач ще повинен використати маркер безпеки або 'ключ', кредитну картку або мобільний телефон, при трьохфакторній аутентифікації, користувач повинен застосувати відбитки пальців або пройти сканування сітківки ока.

Після перевірки дійсності, брандмауер забезпечує доступ до послуг користувачам мережі. Для виявлення і пригнічування дії шкідливих програм використовується антивірусне програмне забезпечення або системи запобігання вторгнень (IPS).

Зв'язок між двома комп'ютерами з використанням мережі може бути зашифрований, щоб зберегти конфіденційність.

Як працює система безпеки

Система безпеки мережі не ґрунтується на одному методі, а використовує комплекс засобів захисту. Навіть якщо частина обладнання виходить з ладу, решта продовжує захищати дані Вашої компанії від можливих атак.

Встановлення рівнів безпеки мережі надає Вам можливість доступу до цінної ділової інформації з будь-якого місця, де є доступ до мережі Інтернет, а також захищає її від загроз.

Система безпеки мережі:

- Захищає від внутрішніх та зовнішніх мережних атак. Небезпека, що загрожує підприємству, може мати як внутрішнє, так і зовнішнє походження. Ефективна система безпеки стежить за активністю в мережі, сигналізує про аномалії та реагує відповідним чином.

- Забезпечує конфіденційність обміну інформацією з будь-якого місця та в будь-який час. Працівники можуть увійти до мережі, працюючи вдома або в дорозі, та бути впевненими у захисті передачі інформації.

- Контролює доступ до інформації, ідентифікуючи користувачів та їхні системи. Ви маєте можливість встановлювати власні правила доступу до даних. Доступ може надаватися залежно від ідентифікаційної інформації користувача, робочих функцій, а також за іншими важливими критеріями.

- Забезпечує надійність системи. Технології безпеки дозволяють системі запобігти як вже відомим атакам, так і новим небезпечним вторгненням. Працівники, замовники та ділові партнери можуть бути впевненими у надійному захисті їхньої інформації.

Ключові елементи захищених мережних служб

Брандмауери. Централізовані брандмауери та брандмауери окремих комп'ютерів можуть запобігати проникненню зловмисного мережного трафіку до мережі, яка підтримує діяльність компанії.

Антивірусні засоби. Більш захищена мережа може виявляти загрози, що створюють віруси, хробаки та інше зловмисне програмне забезпечення, і боротися з ним попереджувальними методами, перш ніж вони зможуть заподіяти шкоду.

Знаряддя, які відстежують стан мережі, грають важливу роль під час визначення мережних загроз.

Захищений віддалений доступ і обмін даними. Безпечний доступ для всіх типів клієнтів із використанням різноманітних механізмів доступу грає важливу роль для забезпечення доступу користувачів до потрібних даних, незалежно від їх місцезнаходження та використовуваних пристроїв.

3.1.Поняття про шкідливе програмне забезпечення.

Шкідливе програмне забезпечення – це зловмисні програми, призначені для отримання несанкціонованого доступу до пристроїв і даних та нанесення їм шкоди.

Яка мета шкідливого програмного забезпечення?

- Фінансова вигода

Використовуючи шкідливі програми, кіберзлочинці намагаються отримати гроші шляхом шантажу, підроблених платежів або внаслідок викрадення даних кредитних карток та облікових записів інтернет-магазинів.

- Викрадення даних
- Шпигунство
- Бот-мережі

3.2.Шпигунські програми: вимагачі, фішинг, програми-вимагачі.

3.3.DDOS-атаки.

Атака типу "Відмова в обслуговуванні" (DoS) - є різновидом мережної атаки. Результатом DoS-атаки є переривання доступу користувачів, пристроїв або застосунків до мережних сервісів.

Існує два основних типи DoS-атак:

1.Перевантаження великою кількістю трафіку - Нападник надсилає величезну кількість даних з такою швидкістю, що мережа, хост або застосунок не встигає їх обробляти. Це спричиняє уповільнення передачі або реагування, іноді призводить до аварійного завершення роботи пристрою чи сервісу.

2.Пакети неправильного формату (Maliciously Formatted Packets) - Нападник надсилає пакет даних неправильного формату хосту або застосунку і одержувач не може його обробити.

Теорія границь займає центральне місце в математиці. Вона є фундаментом математичного аналізу, на ній побудоване диференціальне й інтегральне числення. Вже в елементарній математиці за допомогою граничних переходів визначається довжина кола, знаходяться об'єм конуса й проводиться ряд інших обчислень у геометрії. Ця теорія була також використана при визначенні суми нескінченно спадної геометричної прогресії.

Щоб поступово перейти до строгого визначення операції граничного переходу, розглянемо спочатку найпростішу форму такої операції, що ґрунтується на понятті границі числової послідовності.

Визначення 1. Якщо кожному натуральному числу $n \in N$ ($n=1,2,\dots$) за певним законом поставлене у відповідність дійсне число x_n , то нескінченна множина дійсних чисел

$$\{x_n\} = \{x_1, x_2, \dots\}$$

називається числовою послідовністю або просто послідовністю.

При цьому окремі числа $x_1, \dots, x_n, \dots$ називаються елементами або членами послідовності, символ x_n позначає так званий загальний елемент послідовності, а число n при цьому називається номером елемента послідовності й означає, на якому місці в послідовності знаходиться число x_n .

Зауваження. З визначення випливає, що множина значень послідовності може бути як скінченою так і нескінченною, у той час як множина її елементів завжди є нескінченною, як нескінченний натуральний ряд чисел.

Надалі ми будемо, як правило, користуватися двома основними способами завдання числових послідовностей.

1. При формульному способі повинен бути заданим аналітичним способом вираз (формула), що дозволяє обчислити кожний член послідовності за його номером.

2. При рекурентному способі повинен задаватися перший елемент x_1 (або декілька перших x_1, x_2, x_3), а також бути відомою формула, що пов'язує n -й член із сусідніми, наприклад, з $(n-1)$ -м і $(n+1)$ -м.

Визначення 2. Число a називається границею послідовності $\{x_n\}$, якщо для будь-якого малого додатного числа ε існує (знайдеться) залежне від ε натуральне число N_ε таке, що для всіх $n \geq N_\varepsilon$ виконується нерівність

$$|x_n - a| < \varepsilon$$

У цьому випадку пишуть $\lim_{n \rightarrow \infty} x_n = a$ або $x_n \rightarrow a$ при $n \rightarrow \infty$.

Використовуючи символи математичної логіки дане визначення можна записати у вигляді такого ланцюжка формул

$$\lim_{n \rightarrow \infty} x_n = a \Leftrightarrow \forall \varepsilon > 0 \exists N_\varepsilon : \forall n \geq N_\varepsilon \rightarrow |x_n - a| < \varepsilon \quad (1.1)$$

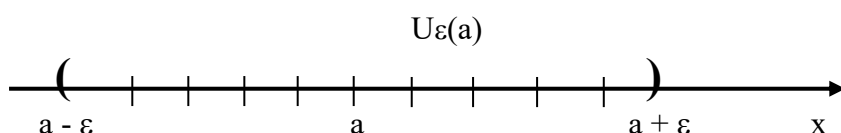
Визначення 3. Послідовність, у якої існує границя, надалі називається збіжною, якщо ж $\lim$ не існує, то така послідовність називається розбіжною.

Коротке визначення збіжної послідовності можна записати так

$$\exists a \in R : \forall \varepsilon > 0 \exists N_\varepsilon : \forall n \geq N_\varepsilon \rightarrow |x_n - a| < \varepsilon \quad (1.2)$$

Зауваження. З визначення границі випливає, що для $n \geq N_\varepsilon$ виконується подвійна нерівність $a - \varepsilon < x_n < a + \varepsilon$, тобто при $n \geq N_\varepsilon$ всі члени послідовності потрапляють в інтервал $(a - \varepsilon, a + \varepsilon)$. Цей інтервал називається ε -околом точки a й надалі позначається так

$$U_\varepsilon(a) = \{x : a - \varepsilon < x < a + \varepsilon\} = \{x : |x - a| < \varepsilon\} \quad (1.4)$$



Мал.1

Тоді “мовою околів” визначення границі можна записати так

$$\lim_{n \rightarrow \infty} x_n = a \Leftrightarrow \forall \varepsilon > 0 \exists N_\varepsilon : \forall n \geq N_\varepsilon \rightarrow x_n \in U_\varepsilon(a) \quad (1.5)$$

Зауваження. Відповідно до формули (1.5), якщо $x_n \rightarrow a$, то яким-би не був ε -окіл точки a , всі точки x_n послідовності, починаючи з деякого номера, повинні потрапляти в цей окіл, тобто за межами будь-якого вузького околу точки a може перебувати лише скінчене число елементів послідовності.

Теорема 1. (про єдиність границі). Числова послідовність може мати тільки одну границю.

Визначення 4. Послідовність $\{x_n\}$ називається обмеженою зверху, якщо виконується умова

$$\exists C_1 : \forall n \in N \rightarrow x_n \leq C_1 \quad (1.9)$$

Аналогічно визначається послідовність обмежена знизу

$$\exists C_2: \forall n \in N \rightarrow C_2 \leq x_n \quad (1.10)$$

Послідовність $\{x_n\}$ називається просто обмеженою, якщо вона обмежена й знизу, і зверху, тобто

$$\exists C_1 \exists C_2: \forall n \in N \rightarrow C_2 \leq x_n \leq C_1 \quad (1.11)$$

Зауваження. Найчастіше визначення обмеженої послідовності записують так

$$\exists C > 0: \forall n \in N \rightarrow |x_n| \leq C \quad (1.12)$$

Це визначення еквівалентне (1.11), якщо припустити, що

$$C = \max\{|C_1|, |C_2|\}$$

Геометрично факт обмеженості $\{x_n\}$ означає, що всі члени послідовності належать $U_C(0)$.

Теорема 2. (про обмеженість збіжної послідовності). Якщо послідовність має скінчену границю, то вона обмежена.

Зауваження. Відповідно до доведеної теореми всяка збіжна послідовність обмежена, але зворотне твердження невірне, тобто не всяка обмежена послідовність має скінчену границю, наприклад, послідовність $\{(-1)^n\}$ обмежена, але не є збіжною.

Теорема 3. (теорема про границю "затиснутої" послідовності). Якщо послідовності $\{x_n\}, \{y_n\}, \{z_n\}$ такі, що

$$1) \quad \forall n \geq N_0: x_n \leq y_n \leq z_n$$

$$2) \quad \lim_{n \rightarrow \infty} x_n = \lim_{n \rightarrow \infty} z_n = a$$

тоді послідовність $\{y_n\}$ сходиться й

$$\lim_{n \rightarrow \infty} y_n = a$$

Теорема 4. Якщо $\lim_{n \rightarrow \infty} x_n = a, \lim_{n \rightarrow \infty} y_n = b$, причому $a < b$, тоді

$$\exists N_0: \forall n \geq N_0 \rightarrow x_n < y_n$$

Наслідок 1. Якщо $\lim_{n \rightarrow \infty} x_n = a$ й $a < b$, то $\exists N_0: \forall n \geq N_0 \rightarrow x_n < b$

Наслідок 2. Якщо $\lim_{n \rightarrow \infty} x_n = a, \lim_{n \rightarrow \infty} y_n = b$ й $\forall n \geq N \rightarrow x_n \geq y_n$, то $a \geq b$.

Нескінченно малі й нескінченно великі послідовності. Арифметичні операції над збіжними послідовностями

Визначення 5. Послідовність $\{\alpha_n\}$ називається нескінченно малою (н.м.п.), якщо

$$\lim_{n \rightarrow \infty} \alpha_n = 0 \quad \left(\forall n \geq N_\varepsilon \rightarrow |\alpha_n| < \varepsilon \right)$$

Визначення 6. Сумою, різницею, добутком і часткою двох послідовностей $\{x_n\}, \{y_n\}$ називаються відповідно послідовності $\{x_n + y_n\}, \{x_n - y_n\}, \{x_n y_n\}, \left\{ \frac{x_n}{y_n} \right\}$, причому в останньому випадку $y_n \neq 0 \quad \forall n \in N$.

Теорема 5. Алгебраїчна сума кінцевого числа н.м.п. є н.м.п.

Теорема 6. Добуток н.м.п. на обмежену послідовність є н.м.п.

Визначення 7. Послідовність $\{\alpha_n\}$ називається нескінченно великою (н.в.п.), якщо виконується умова

$$\forall \delta > 0 \quad \exists N_\delta: \forall n \geq N_\delta \rightarrow |\alpha_n| > \delta$$

У цьому випадку пишуть $\lim_{n \rightarrow \infty} x_n = \infty$ і говорять, що послідовність має нескінчену границю. Тоді, якщо $\lim_{n \rightarrow \infty} x_n = \infty$, то в кожному δ -околі нескінченності лежать всі члени $\{x_n\}$ за винятком можливо скінченного числа членів.

Аналогічно визначаються нескінченні границі певного знака за формулами

$$\lim_{n \rightarrow \infty} x_n = -\infty \Leftrightarrow \forall \delta > 0 \quad \exists N_\delta: \forall n \geq N_\delta \rightarrow x_n < -\delta$$

$$\lim_{n \rightarrow \infty} x_n = +\infty \Leftrightarrow \forall \delta > 0 \quad \exists N_\delta: \forall n \geq N_\delta \rightarrow x_n > \delta$$

Теорема 7. (про арифметичні операції над збіжними послідовностями). Якщо $\lim_{n \rightarrow \infty} x_n = a, \lim_{n \rightarrow \infty} y_n = b$, то

$$a) \lim_{n \rightarrow \infty} (x_n \pm y_n) = a \pm b$$

$$б) \lim_{n \rightarrow \infty} (x_n y_n) = ab$$

$$в) \lim_{n \rightarrow \infty} \frac{x_n}{y_n} = \frac{a}{b} \quad \text{за умови, що } \forall n \in N: y_n \neq 0 \wedge b \neq 0.$$

Невизначені вирази

Якщо $\lim_{n \rightarrow \infty} y_n = 0$, при цьому $\lim_{n \rightarrow \infty} x_n \neq 0$, то використовуючи поняття н.в.п. і нескінченної границі можемо записати, що в цьому випадку $\lim_{n \rightarrow \infty} \frac{x_n}{y_n} = \infty$. Інакше виглядає ситуація у тому випадку, коли

$\lim_{n \rightarrow \infty} x_n = 0 \wedge \lim_{n \rightarrow \infty} y_n = 0$. Тоді про границю послідовності $\left\{ \frac{x_n}{y_n} \right\}$ заздалегідь нічого певного сказати не можна.

Для знаходження границі послідовності $\left\{ \frac{x_n}{y_n} \right\}$ мало знати, що $x_n \rightarrow 0, y_n \rightarrow 0$, потрібні ще додаткові відомості про характер зміни x_n й y_n , тобто про швидкість прагнення до нуля. Для знаходження границі в кожному конкретному випадку потрібні спеціальні прийоми. Говорять, що вирази $\frac{x_n}{y_n}$ при $x_n \rightarrow 0, y_n \rightarrow 0$ являє собою невизначеність типу $\left(\frac{0}{0} \right)$.

Аналогічні міркування приводять нас до інших типів невизначеностей. Якщо $x_n \rightarrow \infty, y_n \rightarrow \infty$, то вираз $\frac{x_n}{y_n}$ називається невизначеністю типу $\left(\frac{\infty}{\infty} \right)$. При $x_n \rightarrow 0, y_n \rightarrow \infty$ вираз $x_n y_n$ приводить до невизначеності типу $(0 \cdot \infty)$. Якщо $x_n \rightarrow +\infty, y_n \rightarrow -\infty$, то вираз $x_n + y_n$ називається невизначеністю типу $\infty - \infty$.

Зауваження. Також зустрічаються ще три типа невизначеностей $(1^\infty), (0^0), (\infty^0)$, тобто всього в математичному аналізі є сім типів невизначеностей.

Зауваження. Одна з важливих задач і в теорії числових послідовностей, і в теорії функцій полягає в розкритті невизначеностей. Розкрити відповідну невизначеність - це значить знайти границю (якщо вона існує), що, як ми вже відзначали вище, не завжди просто й немає загального прийому або правила для розв'язання цього питання. Ряд

часткових прийомів у теорії числових послідовностей ми розглянемо при розв'язанні прикладів. У теорії функцій є більше сильні правила (правило Лопіталя, метод виділення головної частини й ін.).

Границя монотонної послідовності

Визначення 8. Послідовність $\{x_n\}$ називається зростаючою (неспадною), якщо

$$\forall n \in N \rightarrow x_{n+1} \geq x_n$$

Аналогічно визначається спадна (незростаюча) послідовність

$$\forall n \in N \rightarrow x_{n+1} \leq x_n$$

Якщо в тому й іншому випадку має місце строга нерівність, то послідовність називається строго зростаючою або строго спадною. Всі ці послідовності разом узяті називаються монотонними (або строго монотонними).

Елементи монотонних послідовностей можна розташувати в ланцюжки $x_1 \leq x_2 \leq \dots \leq x_n \leq x_{n+1} \leq \dots$ або $x_1 \geq x_2 \geq \dots \geq x_n \geq x_{n+1} \geq \dots$, звідки видно, що неспадна послідовність обмежена знизу, а незростаюча - зверху.

Визначення 9. Число M називається точною верхньою межею числової множини X (або "супремумом") і позначається $M = \sup$, якщо виконуються дві умови

$$\begin{aligned} 1) & \forall x \in X \rightarrow x \leq M \\ 2) & \forall M' < M \exists x_{M'} \in X: x_{M'} > M' \end{aligned} \quad (1.15)$$

Аналогічно визначається точна нижня межа числової множини (або "інфімум") $m = \inf X$

$$\begin{aligned} 1) & \forall x \in X \rightarrow x \geq m \\ 2) & \forall m' > m \exists x_{m'} \in X: x_{m'} < m' \end{aligned} \quad (1.16)$$

Звернемо увагу на те, що самі точні межі можуть як належати множині X , так і не належати тій множині, для якої вони є точними межами. Тут очевидно, що для інтервалу (a, b) обидві межі X не належать, для відрізка $[a, b]$ - обидві належать, для напівінтервалу $[a, b)$ - нижня належить, а верхня - не належить. Якщо в якості X взяти нескінченний напівінтервал $[a, +\infty)$ або $(-\infty, a]$, то тут є тільки одна межа, якщо ж $X = (-\infty, +\infty)$, то точних меж така множина не має.

Визначення 10. Точною верхньою (нижньою) межею числової послідовності $\{x_n\}$ називається відповідна точна межа множини значень цієї послідовності

$$a = \sup\{x_n\} \Leftrightarrow \begin{aligned} &1) \forall n \in N \rightarrow x_n \leq a \\ &2) \forall \varepsilon > 0 \exists n_\varepsilon : x_{n_\varepsilon} > a - \varepsilon \end{aligned} \quad (1.19)$$

$$b = \inf\{x_n\} \Leftrightarrow \begin{aligned} &1) \forall n \in N \rightarrow x_n \geq b \\ &2) \forall \varepsilon > 0 \exists n_\varepsilon : x_{n_\varepsilon} < b + \varepsilon \end{aligned} \quad (1.20)$$

Теорема 8. (ознака збіжності монотонної послідовності). Якщо послідовність $\{x_n\}$ є зростаючою й обмежена зверху, то вона є збіжною, причому

$$\lim_{n \rightarrow \infty} x_n = \sup\{x_n\}$$

Якщо послідовність $\{x_n\}$ є спадною й обмежена знизу, то існує

$$\lim_{n \rightarrow \infty} x_n = \inf\{x_n\}$$

Число e (приклад використання ознаки збіжності монотонної послідовності)

Розглянемо числову послідовність $\{x_n\}$, де

$$x_n = \left(1 + \frac{1}{n}\right)^n \quad (1.22)$$

Якщо спробувати безпосередньою підстановкою замість n нескінченності визначити граничне значення, то прийдемо до вище вже відмічуваної невизначеності типу (1^∞) . Тому, що ця послідовність зростаюча й обмежена зверху, відповідно до ознаки збіжності монотонної послідовності вона має скінчену границю.

За пропозицією Ейлера ця межа позначається буквою e , тобто справедлива наступна формула

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e \quad (1.25)$$

Тут e - широко використовуване як в елементарній математиці (основа натурального логарифму), так і в математичному аналізі число, для якого

справедливий такий наближений вираз $e \cong 2,718281$. Число e - ірраціональне й, більш того, трансцендентне, тобто не є коренем ніякого алгебраїчного рівняння із цілими коефіцієнтами. Використовуючи формулу (1.25) можна знаходити границі великої кількості послідовностей, що є невизначеностями типу (1^∞) .

Визначення 14. Послідовність $\{x_n\}$ називається фундаментальною, якщо вона задовольняє умові Коші

$$\forall \varepsilon > 0 \exists N_\varepsilon: \forall n \geq N_\varepsilon \quad \forall m \geq N_\varepsilon \rightarrow |x_n - x_m| < \varepsilon \quad (1.29)$$

Зауваження. Іноді умову Коші зручніше записати в іншому еквівалентному (1.29) вигляді

$$\forall \varepsilon > 0 \exists N_\varepsilon: \forall n \geq N_\varepsilon \quad \forall p \in \mathbb{N} \rightarrow |x_{n+p} - x_n| < \varepsilon$$

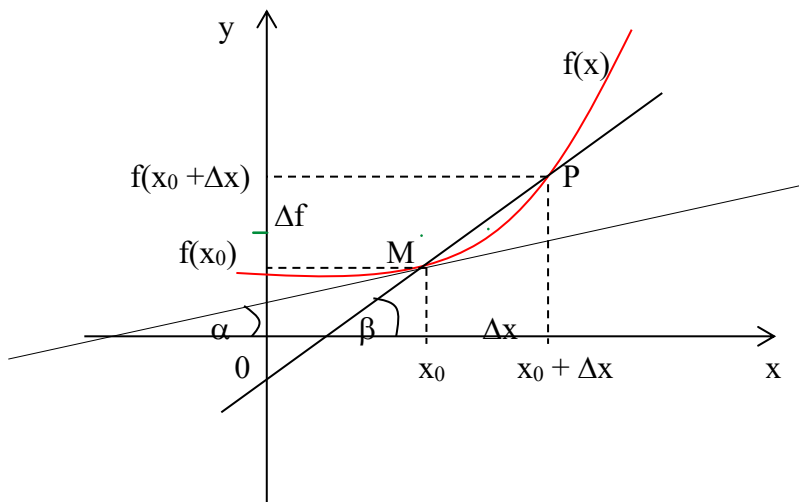
Теорема 11. (критерій Коші). Для того щоб послідовність мала скінчену межу необхідно й досить, щоб вона була фундаментальною.

Похідна функції в точці

Нехай функція $y = f(x)$ визначена в деякому околі точки x_0 . Для будь-якої точки x з цього околу різниця $x - x_0$ називається *приростом аргументу* і позначається Δx , а різниця значень функції $f(x) - f(x_0)$ називається *приростом функції* і позначається Δy . Оскільки $x = x_0 + \Delta x$, то $\Delta y = f(x_0 + \Delta x) - f(x_0)$.

Похідною функції $y = f(x)$ в точці x_0 називається границя відношення її приросту Δy в цій точці до приросту аргументу Δx при $\Delta x \rightarrow 0$ (якщо ця границя існує):

$$y'_x = f'(x) = \lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}.$$



Нехай $f(x)$ визначена на деякому проміжку $(a; b)$. Тоді $\operatorname{tg} \beta = \frac{\Delta f}{\Delta x}$ — тангенс кута нахилу січної MP до графіка функції.

$\lim_{\Delta x \rightarrow 0} \operatorname{tg} \beta = \lim_{\Delta x \rightarrow 0} \frac{\Delta f}{\Delta x} = f'(x_0) = \operatorname{tg} \alpha$, де α — кут нахилу дотичної до графіка функції $y = f(x)$ в точці $(x_0, f(x_0))$.

Рівняння дотичної має вигляд: $y = f(x_0) + f'(x_0)(x - x_0)$.

У фізичному сенсі відношення $\frac{\Delta y}{\Delta x}$ є середньою швидкістю зміни функції на відрізку $[x_0; x_0 + \Delta x]$, а $\lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}$ — миттєвою швидкістю зміни цієї функції в точці x_0 .

Операцію знаходження похідної функції $y = f(x)$ називають диференціюванням цієї функції. Функція називається *диференційованою* в точці x_0 , якщо вона має в цій точці похідну. Якщо функція має похідну в кожній точці деякого проміжку, то її називають диференційованою на цьому проміжку.

Теорема. Якщо функція диференційована в точці x_0 , то вона неперервна в цій точці.

Слідство. Неперервність функції – необхідна умова її диференційованості. Це означає, що в точках розриву функція не має похідних, тобто вона не диференційована. Ця умова не є достатньою, тобто функція, неперервна в точці x_0 , може бути не диференційованою в цій точці. Наприклад, $y = |x|$ неперервна в точці $x = 0$, але не має похідної в цій точці, тому що $\lim_{\Delta x \rightarrow 0^-} \frac{\Delta y}{\Delta x} = -1$, $\lim_{\Delta x \rightarrow 0^+} \frac{\Delta y}{\Delta x} = 1$, тобто границя залежить від способу прагнення $\Delta x \rightarrow 0$.

Правила диференціювання

Похідні основних елементарних функцій:

1. Стала величина: $C' = 0$;
2. Степенева функція: $(x^n)' = nx^{n-1}$;
3. Показникова функція: $(a^x)' = a^x \ln a$;
4. Показникова функція з основою e : $(e^x)' = e^x$;
5. Натуральний логарифм: $(\ln x)' = 1/x$;
6. Логарифмічна функція: $(\log_a x)' = 1/(x \ln a)$;
7. Тригонометричні функції: $(\sin x)' = \cos x$; $(\cos x)' = -\sin x$; $(\operatorname{tg} x)' = 1/\cos^2 x$; $(\operatorname{ctg} x)' = -(1/\sin^2 x)$.
8. Обернені тригонометричні функції: $(\arcsin x)' = 1/\sqrt{1-x^2}$; $(\arccos x)' = -(1/\sqrt{1-x^2})$; $(\operatorname{arctg} x)' = 1/(1+x^2)$; $(\operatorname{arcctg} x)' = -(1/(1+x^2))$.

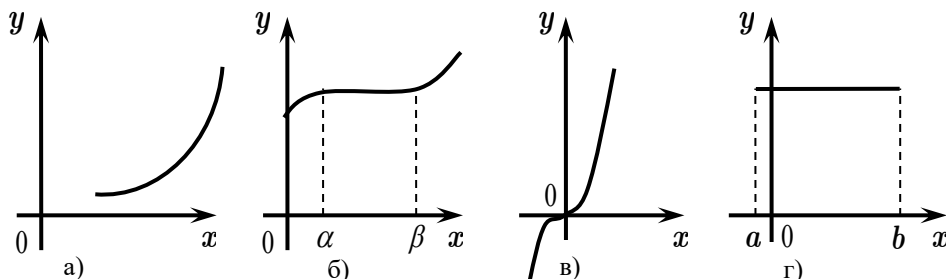
Структурні правила диференціювання:

1. Похідна суми: $(f(x) \pm g(x))' = f'(x) \pm g'(x)$;
2. Похідна добутку: $(f(x) \cdot g(x))' = f'(x)g(x) + f(x)g'(x)$;
3. Похідна частки: $(f(x)/g(x))' = (f'(x)g(x) - f(x)g'(x))/g^2(x)$;
4. Похідна складної функції: $(u(f(x)))' = u'(f(x)) \cdot f'(x)$
5. Похідна оберненої функції: $\frac{dy}{dx} = \frac{1}{\frac{dx}{dy}}$.
6. $(c(f(x)))' = c \cdot f'(x)$

Дослідження функцій за допомогою похідної

Теорема 1. (Ознака сталості функції). Нехай функція $f(x)$ неперервна на проміжку $[a; b]$ і диференційована на $]a; b[$. Для того, щоб $f(x)$ була постійною на проміжку $[a; b]$, необхідно і достатньо, щоб для будь-якого $x \in]a; b[$ виконувалась умова $f'(x) = 0$.

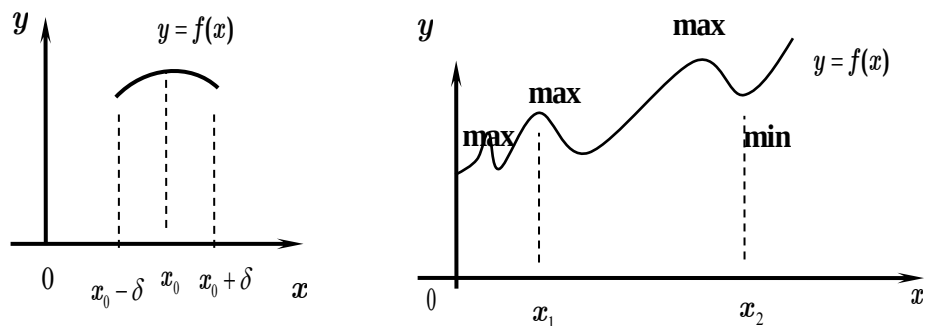
Теорема 2. (Достатня ознака зростання (спадання) функції). Якщо $f'(x) > 0$ ($f'(x) < 0$) всередині певного проміжку, то $f(x)$ зростає (спадає) на цьому проміжку.



На мал.1 г зображено графік функції $y = const$. На мал.1а зображена строго зростаюча функція. Дотична до графіка в будь-якій точці утворює гострий кут з віссю OX , отже, $\tan \alpha > 0$. Зауважимо, що у строго зростаючої функції не обов'язково $f'(x) > 0$; можуть знайтися точки, в яких буде $f'(x) = 0$ (мал. 1в для строго зростаючої функції $y = x^3$ в точці $x_0 = 0$ $y'(x_0) = 0$). На мал.1б зображено не спадна, але не строго зростаюча функція.

Екстремуми функції

Визначення 1. Функція $f(x)$, визначена в деякому околі точки x_0 , має в цій точці **максимум**, якщо для всіх x з цього околу (мал. 2), виконується нерівність $f(x_0) \geq f(x)$. При цьому пишуть: $\max f(x) = f(x_0)$.



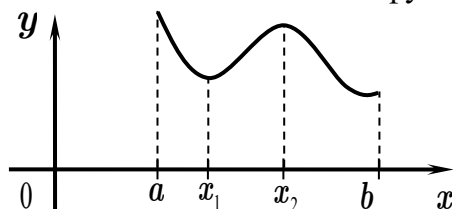
Визначення 2. Функція $f(x)$, визначена в деякому околі точки x_0 , має в цій точці **мінімум**, якщо для всіх x з цього околу (мал. 2), виконується нерівність $f(x_0) \leq f(x)$. При цьому пишуть: $\min f(x) = f(x_0)$.

Максимум або мінімум функції називається одним словом: **екстремум**. Очевидно, що

функція, визначена на відрізку може мати максимум і мінімум тільки в точках, що знаходяться всередині цього відрізка, причому функція може мати кілька максимумів або мінімумів. Не можна плутати максимум і мінімум функції з її найбільшим і найменшим значенням на відрізку.

Найбільше і найменше значення функції

Припустимо, що деяка функція $f(x)$ неперервна на проміжку $[a; b]$, тоді на цьому проміжку вона має найбільше і найменше значення. Щоб їх знайти, потрібно відшукати всі максимуми і мінімуми функції, обчислити її значення на кінцях проміжку, а потім порівняти їх між собою і вибрати найменше та найбільше. На мал. функція $f(x)$ має найбільше значення в точці $x = a$, яке більше $y_{\max} = f(x_2)$, а найменшим значенням є $f(b)$, яке менше мінімального значення функції $y_{\min} = f(x_1)$.



Теорема Ферма (необхідна умова існування екстремуму). Якщо функція $f(x)$ диференційована в деякій точці x_0 , що належить інтервалу $]x_0 - \delta, x_0 + \delta[$ і має в цій точці екстремум, то обов'язково $f'(x_0) = 0$.

Точки, в яких похідна функції дорівнює нулю, $\pm\infty$ або не існує, називаються критичними або стаціонарними. Критичні точки можуть бути чи не бути точками екстремуму. Наприклад, функція $y = x^3$ в точці $x_0 = 0$ має $f'(0) = 0$, але в точці $x_0 = 0$ функція екстремумів не має. Точки, підозрілі на екстремум, підлягають додатковому дослідженню з метою з'ясування, чи є в них максимум або мінімум.

Теорема. Перший достатній признак екстремуму. Якщо функція $f(x)$ диференційована в деякому околі точки $x_0:]x_0 - \delta, x_0 + \delta[$ і похідна $f'(x)$ обертається на нуль в цій точці, то, якщо при проходженні через точку x_0 похідна змінює знак з “+” на “-”, в точці x_0 функція має максимум; з “-” на “+” – мінімум.

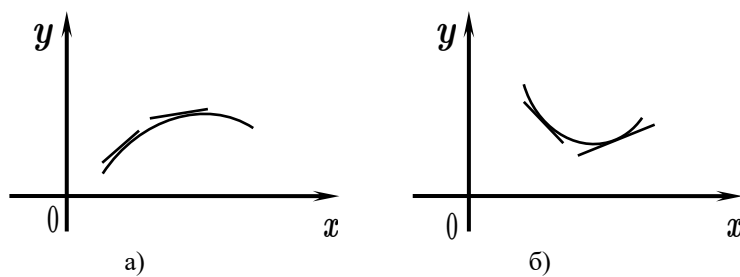
Теорема залишається в силі, якщо в критичних точках похідна не існує або обертається в нескінченність, аби тільки в самій критичній точці функція мала кінцеве значення. При розв'язуванні прикладів корисно робити схему, що дозволяє звести воедино одержувані результати і зробити відповідні висновки, а саме: на осі OX наносять критичні точки, вказують інтервали зростання і спадання функції, а також характер

критичних точок.

Теорема. Другий достатній признак екстремуму. Якщо в околі точки x_0 функція $f(x)$ неперервна і двічі диференційована, причому в цьому околі $f''(x)$ неперервна, а в точці x_0 перша похідна обертається в нуль, то якщо $f''(x_0) < 0$, в точці x_0 функція має максимум, а якщо $f''(x_0) > 0$, в точці x_0 функція має мінімум.

Опуклість і увігнутість кривих

Неперервна крива на проміжку $[a; b]$, опукла вгору, або просто *опукла*, якщо графік розташовується нижче дотичної до графіка функції, проведеної через будь-яку точку графіка (мал. а). Неперервна крива на проміжку $[a; b]$, опукла вниз, або *увігнута*, якщо її графік розташовується вище дотичної до графіка функції, проведеної через його будь-яку точку (мал. б).



Теорема. Якщо функція $f(x)$ двічі диференційована на проміжку $[a; b]$, причому $f''(x) < 0$ на цьому проміжку, то на проміжку $[a; b]$, графік функції опуклий, а якщо $f''(x) > 0$, то на проміжку $[a; b]$, графік функції увігнутий.

Точки перегину

Точка на графіку функції $f(x)$, що відокремлює опуклу частину графіка від увігнутої, називається *точкою перегину*. З визначення випливає, що при проходженні через точку перегину друга похідна змінює знак. Якщо x_0 – абсциса точки перегину, то $f''(x_0) = 0$, або $f''(x_0) = \infty$, або $f''(x_0)$ не існує.

Загальна схема дослідження функції

1. Визначити область існування функції.
2. З'ясувати, є дана функція парною або непарною.
3. Знайти точки, підозрілі на екстремум і з'ясувати характер екстремумів за допомогою першої або другої похідної, а також обчислити $y_{\min}$ і $y_{\max}$.
4. Визначити інтервали зростання і спадання функції.

5. Знайти інтервали опуклості й увігнутості функції.

6. Знайти точки перегину.

7. Знайти асимптоти графіка функції.

8. Обчислити значення функції в деяких контрольних точках (наприклад, значення функції в початку координат), точки перетину з координатними осями.

9. Намалювати графік функції.

Порядок проходження пунктів може бути змінений при розв'язанні кожної конкретної задачі.

Обчислення визначеного інтеграла

Якщо функція $f(x)$ неперервна на проміжку $[a; b]$ і функція $F(x)$ є її деякою первісною на цьому відрізку, то має місце формула Ньютона-Лейбніца:

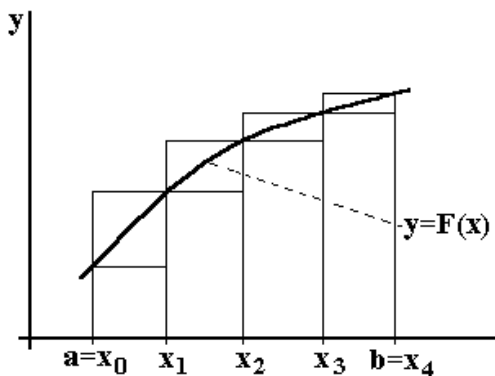
$$\int_a^b f(x)dx = F(x)\Big|_a^b = F(b) - F(a).$$
 Тобто обчислення визначених інтегралів зводиться до вже

розробленої техніки обчислення невизначених інтегралів.

Водночас існує величезна кількість функцій, інтеграл від яких не може бути виражений через елементарні функції. Для знаходження інтегралів від подібних функцій застосовують різноманітні наближені методи. Чисельні методи інтегрування використовують заміну площі криволінійної трапеції на скінчену суму площ простіших геометричних фігур, які можуть бути обчислені точно.

Наближене обчислення визначених інтегралів

Формула прямокутників



Нехай на відрізку $[a; b]$ задана неперервна функція

$y = f(x)$. Потрібно обчислити інтеграл $\int_a^b f(x)dx$, який

чисельно дорівнює площі криволінійної трапеції, що обмежена графіком функції $y = f(x)$, вертикальними

прямими $x = a$, $x = b$ і віссю OX . Якщо відомі

значення функції $f(x)$ в деяких точках $x_0, x_1, x_2, \dots, x_n$,

то криволінійну трапецію можна замінити сумою n більш "вузьких" трапецій. Площу кожної з "вузьких" криволінійних трапецій можна у свою чергу замінити площею прямокутника.

Якщо розбити відрізок інтегрування на n рівних частин, то довжина кожної з них

$\Delta x = \frac{b-a}{n}$. При цьому: $y_0 = f(x_0)$, $y_1 = f(x_1)$, ..., $y_n = f(x_n)$. Складемо суми:

$$y_0 \Delta x + y_1 \Delta x + \dots + y_{n-1} \Delta x = \Delta x (y_0 + y_1 + \dots + y_{n-1}) \quad \text{и}$$

$$y_1 \Delta x + y_2 \Delta x + \dots + y_n \Delta x = \Delta x (y_1 + y_2 + \dots + y_n).$$

Це відповідно нижня та верхня інтегральні суми. Перша відповідає вписаній ламаній, друга – описаній.

Тоді $\int_a^b f(x)dx \approx \frac{b-a}{n} (y_0 + y_1 + \dots + y_{n-1})$ або $\int_a^b f(x)dx \approx \frac{b-a}{n} (y_1 + y_2 + \dots + y_n)$. Кожна з цих

формулу може застосовуватися для наближеного обчислення визначеного інтеграла і називається загальною формулою прямокутників. Як видно, одна з формул дає наближення до інтегралу з нестачею, а друга – з надлишком. Можна запропонувати ще один спосіб, очевидно кращий, ніж обидві формули – використовувати для апроксимації значення функції в середині відрізка інтегрування: $\int_a^b f(x)dx \approx \frac{b-a}{n} \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right)$ – формула середніх прямокутників.

Доведено, що похибка застосування формули прямокутників не більше ніж

$$\Delta = \frac{(b-a)^2}{24n^2} M, \quad M = \max_{x \in [a,b]} |f''(x)|$$

В якості прикладів візьмемо інтеграли, які можна обчислити за формулою Ньютона-Лейбніца.

Приклад. Обчислити наближене значення визначеного інтеграла $\int_1^2 \frac{dx}{x}$ за допомогою формули прямокутників, розбивши відрізок інтегрування на 10 частин.

За формулою Ньютона-Лейбніца отримаємо $\int_1^2 \frac{dx}{x} = \ln x \Big|_1^2 = \ln 2$.

За формулою прямокутників $\Delta x = \frac{1-0}{10} = 0,1$. $x_0 = 1; x_1 = 1,1; x_2 = 1,2; \dots; x_{10} = 2$. Тоді

$$x_1' = \frac{1+1,1}{2} = 1,05; x_2' = \frac{1,1+1,2}{2} = 1,15; \dots; x_{10}' = \frac{1,9+2}{2} = 1,95.$$

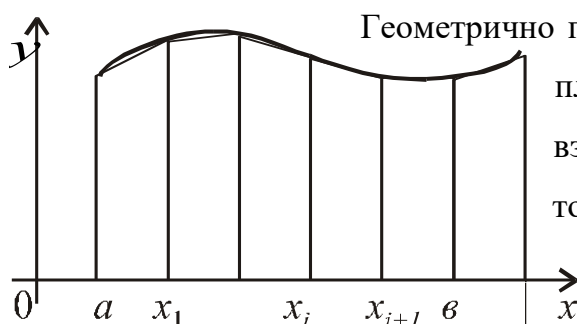
$$y(x_1') = 0,9524; y(x_2') = 0,8696; y(x_3') = 0,8; y(x_4') = 0,7407; y(x_5') = 0,6897$$

$$y(x_6') = 0,6452; y(x_7') = 0,6061; y(x_8') = 0,5714; y(x_9') = 0,5405; y(x_{10}') = 0,5128 \text{ Тоді}$$

$$\int_1^2 \frac{dx}{x} \approx 0,1 \cdot 6,9284 = 0,69284. \text{ Більш точне значення цього інтеграла } \ln 2 \approx 0,693147 \dots$$

Формула трапецій

Формула трапецій є точнішою у порівнянні з формулою прямокутників. Підінтегральна функція в цьому випадку замінюється на вписану ламану.



Геометрично площа криволінійної трапеції замінюється сумою площ вписаних трапецій. Очевидно, що чим більше взяти точок розбиття інтервалу, тим з більшою точністю буде обчислений інтеграл.

Площа кожної трапеції може бути знайдена як

добуток напівсуми основ на висоту: $\frac{y_0 + y_1}{2} \Delta x$; $\frac{y_1 + y_2}{2} \Delta x$; ... , $\frac{y_{n-1} + y_n}{2} \Delta x$. Сумуючи,

отримаємо $\int_a^b f(x) dx \approx \frac{y_0 + y_1}{2} \Delta x + \frac{y_1 + y_2}{2} \Delta x + \dots + \frac{y_{n-1} + y_n}{2} \Delta x$. У випадку n відрізків

інтегрування для всіх вузлів, за винятком крайніх точок відрізка, значення функції увійде в загальну суму двічі (оскільки сусідні трапеції мають одну спільну сторону). Остаточного

отримуємо формулу трапецій: $\int_a^b f(x) dx \approx \frac{b-a}{n} \left(\frac{y_0 + y_n}{2} + y_1 + y_2 + \dots + y_{n-1} \right)$.

Як правило, що менша довжина кожного інтервалу, тобто чим більше цих інтервалів, тим менше відрізняються наближене і точне значення інтеграла. Це справедливо для більшості функцій. У методі трапецій помилка обчислення інтегралу приблизно пропорційна квадрату кроку інтегрування.

Таким чином, для обчислення інтегралу деякі функції в межах a, b необхідно поділити відрізок $[a; b]$ на n інтервалів та знайти суму площ трапецій. Потім потрібно збільшити кількість інтервалів, знову обчислити суму трапецій та порівняти отримане значення з попереднім результатом. Це слід повторювати доти, доки не буде досягнуто заданої точності результату. Для методів прямокутників та трапецій зазвичай на кожному кроці ітерації кількість інтервалів збільшується у 2 рази.

Приклад. Користуючись правилом трапецій обчислити інтеграл $\int_0^1 x^2 dx$.

За формулою Ньютона-Лейбніца отримаємо $\int_0^1 x^2 dx = \frac{x^3}{3} \Big|_0^1 = \frac{1}{3} \approx 0,33333$.

За формулою трапецій: Для $n = 2$: $\Delta x = \frac{1-0}{2} = 0,5$; $x_0 = 0; x_1 = 0,5; x_2 = 1$

$$y(x_0) = 0; y(x_1) = 0,25; y(x_2) = 1; \int_0^1 x^2 dx \approx 0,5 \left(\frac{0+1}{2} + 0,25 \right) = 0,375$$

Для $n = 4$: $\Delta x = \frac{1-0}{4} = 0,25$; $x_0 = 0; x_1 = 0,25; x_2 = 0,5; x_3 = 0,75; x_4 = 1$

$$y(x_0) = 0; y(x_1) = 0,0625; y(x_2) = 0,25; y(x_3) = 0,5625; y(x_4) = 1;$$

$$\int_0^1 x^2 dx \approx 0,25 \left(\frac{0+1}{2} + 0,0625 + 0,25 + 0,5625 \right) = 0,34375.$$

Головна перевага правила трапецій – його простота. Однак якщо при обчисленні інтеграла потрібна висока точність, застосування цього методу може вимагати надто великої кількості ітерацій або машинного часу.

Функції декількох змінних

Методи аналізу функцій багатьох змінних використовуються для вивчення різних об'єктів і явищ матеріального світу. Зокрема, для визначення довжини кривої, площі поверхні, знаходження екстремумів, наближених значень функції.

Числення багатьох змінних можна застосувати для аналізу детермінованих систем, що мають багато ступенів свободи. Функції із незалежними змінними, що відповідають кожному із ступенів свободи часто використовують для моделювання цих систем, а числення багатьох змінних надає інструментарій для характеристики динаміки системи.

Числення багатьох змінних використовується в теорії оптимального управління динамічними системами неперервного часу. І застосовується в регресійному аналізі для отримання формул, що визначають зв'язок між різноманітними наборами емпіричних даних.

В економіці за допомогою функції кількох змінних описують виробничі функції, модель максимізації прибутку підприємства, модель цінової дискримінації, модель поведінки споживачів і т.і.

Отже,

Точкою X в n -вимірному просторі називається впорядкована множина з n чисел

$x_i, i = \overline{1, n}$, які називаються *координатами* точки X :

$X = (x_1, x_2, \dots, x_n)$. Областю в n -вимірному просторі називається

будь-яка множина точок з цього простору. Нехай в n -вимірному

просторі задана деяка область D . Правило, яке кожній точці

області D з координатами $X = (x_1, x_2, \dots, x_n)$ ставить у

відповідність деяке число z , називається *функцією* і

позначається символом $z = f(X)$ або $z = f(x_1, x_2, \dots, x_n)$.

Множина точок X , де це правило має сенс, називається

областю визначення функції. Множина чисел z називається *областю значень* функції.

Змінні $x_1, x_2, \dots, x_n$ є аргументами цієї функції, а змінна z – значенням функції від n змінних.

Далі будемо говорити лише про функції двох змінних, тому що всі отримані результати будуть справедливими для функцій довільного числа змінних. Якщо кожній парі незалежних одне від одного чисел (x, y) з деякої області D за яким небудь правилом ставиться у

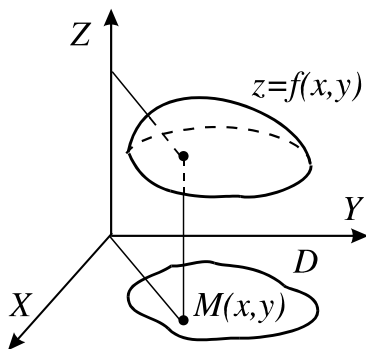


Рис. 1

відповідність одне або кілька значень змінної z , то змінна z називається *функцією двох змінних* $z = f(x, y)$. Якщо парі (x, y) відповідає єдине значення z , то функція називається *однозначною*, а якщо більше одного, то – *багатозначною*. Множина D називається *областю визначення* функції.

Графік функції двох змінних є деяка поверхня –множина точок $(x, y, f(x, y))$, де $(x, y) \in D$.

Частинні похідні функції двох змінних

Нехай в деякому околі точки (x_0, y_0) визначена функція $z = f(x, y)$. Визначимо приріст змінних x і y формулами: $\Delta x = x - x_0$, $\Delta y = y - y_0$. Тоді будь-яка точка (x, y) з околу точки (x_0, y_0) може бути представлена як $(x, y) = (x_0 + \Delta x, y_0 + \Delta y)$. При зміні x від x_0 до $x_0 + \Delta x$ (і сталому $y = y_0$) функція z змінюється на величину $\Delta_x z = f(x_0 + \Delta x, y_0) - f(x_0, y_0)$. Ця різниця називається частинним прирощенням функції z по x . Частинне прирощення по y визначається аналогічно: $\Delta_y z = f(x_0, y_0 + \Delta y) - f(x_0, y_0)$.

Частинною похідною функції декількох змінних по одній з цих змінних називається границя відношення частинного прирощення функції (якщо ця границя існує) до прирощення відповідної незалежної змінної, коли це прирощення прагне до нуля

$\lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x, y_0) - f(x_0, y_0)}{\Delta x}$. Позначається ця частинна похідна будь-яким з

наступних символів: $z'_x(x_0, y_0)$; $\frac{\partial z(x_0, y_0)}{\partial x}$; $f'_x(x_0, y_0)$; $\frac{\partial f(x_0, y_0)}{\partial x}$ (похідна по x);

$z'_y(x_0, y_0)$; $\frac{\partial z(x_0, y_0)}{\partial y}$; $f'_y(x_0, y_0)$; $\frac{\partial f(x_0, y_0)}{\partial y}$ (похідна по y). Якщо частинні похідні

функції $z = f(x, y)$ існують на деякій множині, а точка, в якій обчислюються частинні похідні, несуттєва, то користуються більш короткими позначеннями:

z'_x ; z'_y ; f'_x ; f'_y ; $\frac{\partial f}{\partial x}$; $\frac{\partial f}{\partial y}$. Частинна похідна по x є звичайна похідна від функції

$z = f(x, y)$, що розглядається як функція тільки від змінної x при фіксованому значенні змінної y . Тому відшукування частинних похідних здійснюється за тими самими правилами, за якими здійснюється відшукування похідних функції однієї змінної.

Приклад 1. $z = x^2 y + \sqrt{x}$; $\frac{\partial z}{\partial x} = 2xy + \frac{1}{2\sqrt{x}}$; $\frac{\partial z}{\partial y} = x^2$.

Приклад 2. $z(x, y) = e^{\frac{x}{y}}$; $\frac{\partial z}{\partial x} = \frac{1}{y} e^{\frac{x}{y}}$; $\frac{\partial z}{\partial y} = -\frac{x}{y^2} e^{\frac{x}{y}}$.

Самі частинні похідні можуть бути функціями від декількох змінних на деякій множині. У цих функцій теж можуть існувати частинні похідні за x і за y . Вони називаються *другими частинними похідними* або *частинними похідними другого порядку*

і позначаються $f''_{xx}, f''_{yy}, f''_{xy}, f''_{yx}$ або $\frac{\partial^2 f}{\partial x^2}, \frac{\partial^2 f}{\partial y^2}, \frac{\partial^2 f}{\partial x \partial y}, \frac{\partial^2 f}{\partial y \partial x}$. Згідно з

визначенням $z''_{xx} = \frac{\partial^2 z}{\partial x^2} = (z'_x)'_x$; $z''_{xy} = \frac{\partial^2 z}{\partial x \partial y} = (z'_x)'_y$. Остання частинна похідна другого

порядку називається *змішаною*. Мішана похідна другого порядку, взагалі кажучи, залежить від того, в якій послідовності беруться змінні, за якими обчислюється похідна.

Так, похідна $z''_{xy} = (z'_x)'_y$ може не бути рівною $z''_{yx} = (z'_y)'_x$. Однак існує теорема, яка стверджує, що *якщо змішані частинні похідні другого порядку неперервні, то вони не залежать від того, в якій послідовності обчислювалися частинні похідні за x і за y ,*

тобто вірним є співвідношення: $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$. Продовжуючи диференціювати отримані

рівності, отримаємо частинні похідні вищих порядків.

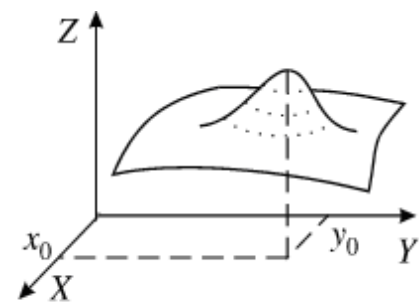
Якщо функція має в деякій точці неперервні частинні похідні, то вона називається *диференційованою* в цій точці.

Теорема. Якщо функція диференційована в деякій точці, то вона неперервна в цій точці.

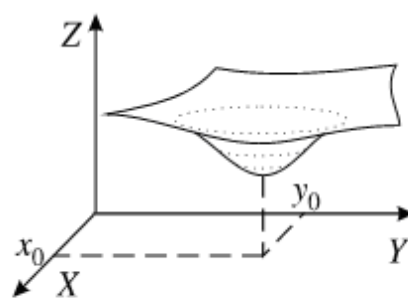
Протилежне твердження невірно. Функція, неперервна в точці, може бути не диференційована в ній.

Екстремум функції двох змінних

Точка $M_0(x_0, y_0)$ є точкою *максимуму* (*мінімуму*) функції $z = f(x, y)$, якщо знайдеться такий окіл точки M_0 , що для всіх точок $M(x, y)$ з цього околу виконується нерівність $f(x_0, y_0) > f(x, y)$ ($f(x_0, y_0) < f(x, y)$). Точки максимуму і мінімуму називаються *точками локального екстремуму*. Аналогічно для функції будь-якого числа змінних.



$M_0(x_0, y_0)$ - точка максимуму.



$M_0(x_0, y_0)$ - точка минимуму

Сформулюємо *необхідну умову екстремуму*. Якщо в точці екстремуму існує перша частинна похідна (з якого-небудь аргументу), то вона дорівнює нулю.

Точки екстремуму диференційованої функції (тобто функції, що має неперервні частинні похідні у всіх точках деякої області) треба шукати тільки серед тих точок, в яких всі перші частинні похідні дорівнюють нулю або хоча б одна з них не існує (критичних точок). Там, де виконується необхідна умова, екстремуму може і не бути (тут повна аналогія з функцією однієї змінної). Для відповіді на питання, чи є точка області визначення функції точкою екстремуму, потрібно використовувати *достатню умову екстремуму*. Її формулювання:

Нехай в околі критичної точки (x_0, y_0) функція $f(x, y)$ має неперервні частинні похідні до другого порядку включно. Введемо позначення: $f''_{xx}(x_0, y_0) = A$; $f''_{xy}(x_0, y_0) = B$;

$$f''_{yy}(x_0, y_0) = C; \quad D = AC - B^2 = \begin{vmatrix} A & B \\ B & C \end{vmatrix} = \begin{vmatrix} f''_{xx} & f''_{xy} \\ f''_{yx} & f''_{yy} \end{vmatrix}.$$

Тоді, якщо $D < 0$, то в точці (x_0, y_0) екстремуму немає. Якщо $D > 0$, то в точці (x_0, y_0) екстремум функції z , причому якщо $A > 0$, то мінімум, а якщо $A < 0$, то максимум. Якщо $D = 0$, то екстремум може бути, а може і не бути. В даному випадку потрібні додаткові дослідження.

Дослідження функції двох змінних на екстремум зводиться до наступного: спочатку виписують необхідні умови екстремуму: $f'_x(x, y) = 0$, $f'_y(x, y) = 0$, які розглядаються як система рівнянь. Її рішенням є деяка множина точок. У кожній з цих точок обчислюються значення D , і перевіряється виконання достатніх умов екстремуму.

Математичне програмування займається дослідженням властивостей і розробкою методів розв'язання задачі знаходження оптимального (максимального чи мінімального) значення цільової функції, причому значення змінних повинні належати деякій області припустимих значень.

У найзагальнішому вигляді задача математично записується так: $F = f(X) \rightarrow \max$; $X \in D$, де $X = (x_1, x_2, \dots, x_n)$; D – область допустимих значень змінних $x_1, x_2, \dots, x_n$; $f(X)$ – цільова функція.

Щоб розв'язати задачу оптимізації, досить знайти її оптимальне рішення, тобто вказати $X_0 \in D$ таке, що $f(X_0) \geq f(X)$ при будь-якому $X \in D$, або для випадку мінімізації – $f(X_0) \leq f(X)$ при будь-якому $X \in D$.

Задача оптимізації є нерозв'язною, якщо вона не має оптимального розв'язку. Зокрема, задача максимізації буде нерозв'язна, якщо цільова функція $f(X)$ не обмежена зверху на допустимій множині D .

Задачею лінійного програмування ($f(X)$ і D лінійні) називається задача дослідження операцій, математична модель якої має вигляд: $f(X) = \sum_{j=1}^n c_j x_j \rightarrow \max(\min)$;

(1) на деякій множині $D \subset R^n$, де $x \in D$ задовольняють системі:

$$\sum_{j=1}^n a_{ij} x_j = b_i, \quad i \in I, \quad I \subseteq M = \{\overline{1, m}\}; \quad (2)$$

$$\sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i \in M; \quad (3)$$

$$\text{і, можливо, обмеженням } x_j \geq 0, \quad j \in J, \quad J \subseteq N = \{\overline{1, n}\}. \quad (4)$$

При цьому система лінійних рівнянь (2) і нерівностей (3), (4) називається *системою обмежень* задачі лінійного програмування, а лінійна функція $f(X)$ називається *цільовою функцією* або *критерієм оптимальності*. В окремому випадку, якщо $I = \emptyset$, то система обмежень складається тільки з лінійних нерівностей, а якщо $I = M$, то з лінійних рівнянь.

Щодо знаків нерівностей будемо припускати, що ліва частина менше або дорівнює правій. Домогтися цього можна, помноживши на (-1) обидві частини тих нерівностей, які мають протилежний знак. Обмеження (4), взагалі кажучи, можуть бути розглянуті як окремий випадок обмежень у формі нерівностей, але в силу особливої структури їх зазвичай виділяють окремо і називають умовами невід'ємності (або тривіальними обмеженнями). Вибір типу шуканого екстремуму (максимуму або мінімуму) також носить відносний характер. Так, задача пошуку максимуму функції $f(X) = \sum_{j=1}^n c_j x_j$

еквівалентна задачі пошуку мінімуму функції $-f(X) = \sum_{j=1}^n (-c_j) x_j$.

Планом задачі лінійного програмування називається всякий вектор з простору R^n . *Допустимим планом* називається такий план ЗЛП, який задовольняє обмеженням (2) - (4), тобто міститься в області D . Сама область D називається при цьому *областю допустимих планів* (*областю допустимих розв'язків*). План, відповідний вершині допустимої області, називається *опорним планом*. Оптимальним планом X^* називається такий допустимий план, при якому цільова функція досягає оптимального (мінімального або максимального) значення, тобто план, що задовольняє умові

$\min f(X) = f(X^*)$ або $\max f(X) = f(X^*)$. Величина $f^* = f(X^*)$ називається *оптимальним значенням* цільової функції.

Розв'язком ЗЛП називається пара (X^*, f^*) , що складається з оптимального плану і оптимального значення цільової функції, а процес вирішення полягає в знаходженні безлічі всіх рішень ЗЛП.

Найбільш часто зустрічаються два різновиди завдання лінійного програмування.

1. Якщо математична модель задачі лінійного програмування має вигляд:

$$f(X) = \sum_{j=1}^n c_j x_j \rightarrow \min;$$

$$\sum_{j=1}^n a_{ij} x_j = b_i, \quad b_i \geq 0, \quad i = \overline{1, m}; \quad x_j \geq 0, \quad j = \overline{1, n},$$

тобто, крім тривіальних обмежень система включає в себе тільки рівняння, то задача представлена в *канонічній формі*. Прикладом може служити транспортна задача.

2. Якщо система обмежень складається тільки з нерівностей, кажуть, що задана *стандартна задача лінійного програмування*. Прикладом можуть служити задачі про банк, дієти, використанні ресурсів.

Будь-яку ЗЛП можна звести до ЗЛП в канонічній формі.

1. Якщо у вихідній задачі потрібно визначити максимум лінійної функції, то слід змінити знак і шукати мінімум цієї функції;

2. Якщо в обмеженнях права частина від'ємна, то слід помножити це обмеження на (-1);

3. Якщо серед обмежень є нерівності, то шляхом введення додаткових невід'ємних змінних вони перетворюються в рівності;

4. Якщо деяка змінна x_k не має обмежень по знаку, то вона замінюється (в цільовій функції і у всіх обмеженнях) різницею між двома новими невід'ємними змінними: $x_k = x'_k - x_l$, де l – вільний індекс, $x'_k \geq 0$, $x_l \geq 0$.

Задачею нелінійного програмування (ЗНП) називається задача математичного програмування, в якій нелінійні або цільова функція $F(X) = f(x_1, x_2, \dots, x_n) \rightarrow \max(\min)$ або функції, що задають обмеження $g_i(x_1, x_2, \dots, x_n) \{ \leq, =, \geq \} b_i, \quad i = \overline{1, m}$. Нелінійність функцій $F(X)$ і $g_i(x_1, x_2, \dots, x_n)$ викликає суттєві відмінності ЗНП від ЗЛП. Так, якщо в ЗЛП екстремальна точка є крайньою точкою багатогранника допустимих розв'язків, то в ЗНП вона може перебувати як на границі ОДР, так і всередині неї. Нелінійна цільова функція може мати кілька екстремальних точок. Багатоекстремальність цільової функції створює значні обчислювальні труднощі, оскільки в практичних задачах зазвичай потрібно знайти точку глобального екстремуму.

Нелінійність обмежень також накладає свої особливості. Якщо в ЗЛП оптимальний розв'язок відшукується на скінченій множині крайніх точок опуклої області, то для ЗНП ці умови в загальному випадку не зберігаються. Нелінійність обмежень може призвести до неопуклості або незв'язаності ОДР, яка до того ж може мати нескінчену множину крайніх точок.

Ці особливості, що відрізняють нелінійні задачі від лінійних, істотно ускладнюють їх розв'язання. Зокрема, якщо для ЗЛП є ознака оптимальності, то для загальної ЗНП такої ознаки немає. У ЗЛП завжди може бути встановлено, чи є довільне припустиме рішення оптимальним. Якщо деякий план не оптимальний, то можна перейти до нового плану, на якому значення цільової функції ближче до оптимального, ніж на

попередньому. У загальній ЗНП такої можливості немає. Якщо навіть на якомусь етапі рішення отримано план, який є оптимальним, встановити це можна шляхом обчислення цільової функції у всіх інших, підозрілих на екстремум точках і порівняння їх між собою. Труднощі розв'язання ЗНП загального вигляду змушують поділяти їх на окремі класи зі своїми методами розв'язання, які базуються, в основному, на теорії диференціального обчислення.

Задачі НЛП можна класифікувати відповідно до виду цільової функції, обмежень, і розмірності і змістом вектора змінних.

Методи розв'язання ЗНП бувають прямими і непрямими. *Прямими* методами оптимальні розв'язки шукають в напрямку якнайшвидшого зростання або зменшення цільової функції – градієнтні методи. *Непрямі* методи полягають у зведенні задачі до такої, знаходження оптимуму якої вдається спростити. До них відносять, наприклад, методи *квадратичного* і *сепарабельного* програмування. Оптимізаційні задачі, на змінні яких не накладаються обмеження, розв'язують методами класичної математики. Оптимізацію з обмеженнями-рівностями виконують методами *Якобі*, *множників Лагранжа* і ін. В задачах оптимізації з обмеженнями-нерівностями досліджують необхідні і достатні умови існування екстремуму *Куна-Таккера*. Деякі ЗНП з двома змінними можуть бути розв'язані графічно.

Метод градієнтного спуску

Чільне місце серед прямих методів розв'язання екстремальних завдань займають градієнтні методи пошуку стаціонарних точок диференційованої функції. Ці методи ґрунтуються на використанні градієнта цільової функції. Градієнт будь-якої функції кількох змінних $F(X) = f(x_1, x_2, \dots, x_n)$ – це вектор, координати якого є частинними

похідними цієї функції $\overrightarrow{grad} f = \nabla f = \frac{\partial f}{\partial x_1} \cdot \vec{e}_1 + \frac{\partial f}{\partial x_2} \cdot \vec{e}_2 + \dots + \frac{\partial f}{\partial x_n} \cdot \vec{e}_n$, де $\vec{e}_1, \vec{e}_2, \dots, \vec{e}_n$ –

одичні вектори у напрямку координатних осей. Вектор градієнт показує напрям, за яким функція зростає найшвидше. Отже, напрям, протилежний градієнтному, вкаже напрям найбільшого спадання функції а методи, засновані на виборі шляху оптимізації за допомогою градієнта, називаються градієнтними.

Застосовуючи градієнтні методи знаходять множину точок локальних максимумів (або мінімумів), серед яких визначається максимум (або мінімум) глобальний.

Принцип роботи всіх градієнтних методів полягає в покроковому переході від деякого початкового рішення до нових рішень у напрямку градієнта (для завдань, в яких потрібна максимізація цільової функції) або протилежному (для завдань мінімізації). У більшості випадків розв'язання задачі на основі градієнтних методів включає наступні основні етапи.

1. Визначається деяке початкове припустиме розв'язання задачі.
2. Виконується перехід від поточного рішення до нового рішення у напрямку градієнта (або протилежному). Величина цього переходу визначається по-різному в залежності від умов завдання та методу, що використовується.
3. Визначається різниця значень цільової функції для нового та попереднього рішень. Якщо ця різниця мала (не перевищує певної заданої точності), то знайдене рішення приймається як оптимальне. В іншому випадку повертається до кроку 2.

Розрізняють різні варіанти градієнтного методу. Зупинимось одному з них.

Процес відшукування точки мінімуму функції $F(X) = f(x_1, x_2, \dots, x_n)$ за методом **градієнтного спуску** полягає в наступному: на початку вибираємо деяку початкову

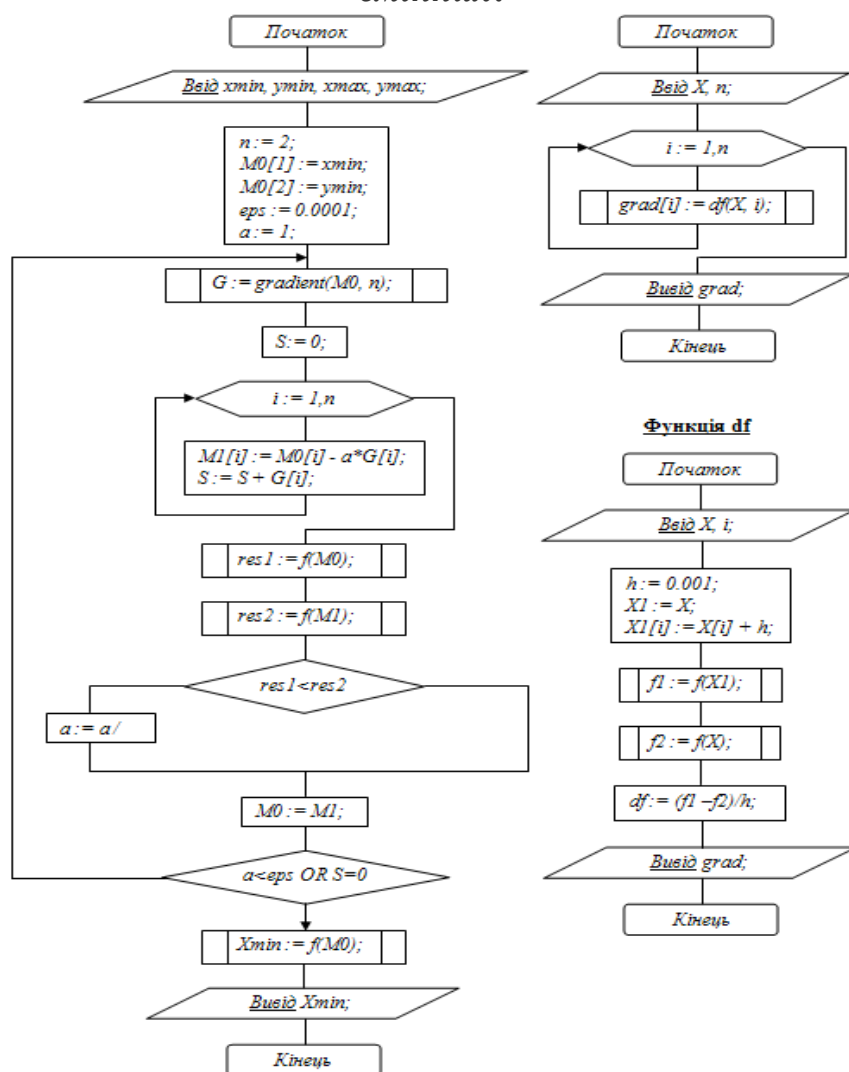
точку $M_0(X) = M_0(x^0_1, x^0_2, \dots, x^0_n)$ і обчислюємо в ній градієнт функції. Далі, робимо крок у антиградієнтному напрямку $x^1 = x^0 - a^1 \text{grad } f(M_0)$ (де $a^1 > 0$). У результаті отримуємо нову точку $M_1(x^1_1, x^1_2, \dots, x^1_n)$, значення функції в якій зазвичай менше за значення функції в точці M_0 . Якщо ця умова не виконується, тобто значення функції не змінилося або навіть зросло, то потрібно зменшити крок $a^1 \left(a^2 = \frac{a^1}{2} \right)$, після чого, у новій точці обчислюємо градієнт і знову робимо крок у зворотному до нього напрямку $x^2 = x^1 - a^2 \text{grad } f(M_1)$.

Процес продовжується до отримання найменшого значення цільової функції. Строго кажучи, момент закінчення пошуку настане тоді, коли рух з отриманої точки, при виборі будь-якого кроку, призводить до зростання значення цільової функції. Якщо мінімум функції досягається всередині розглядуваної області, то в цій точці градієнт дорівнює нулю, що також може служити сигналом про закінчення процесу оптимізації.

Формули для частинних похідних можна отримати в явному вигляді лише в тому випадку, коли цільова функція задана аналітично. В іншому випадку ці похідні обчислюються за допомогою чисельного диференціювання:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x_1, \dots, x_i + \Delta x_i, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{\Delta x_i}, \quad i = \overline{1, n}$$

Блок-схема програмної реалізації методу градієнтного спуску для функції двох змінних:



При практичній обробці результатів експериментів часто буває потрібно подати спостережувані (виміряні) дані у вигляді функціональної залежності. Процес одержання на основі результатів експериментальних досліджень математичної залежності $y = f(x)$, яка з достатньою точністю відтворює досліджувану закономірність, називається *апроксимацією*. Функціональні залежності, одержані способом апроксимації експериментальних даних, називаються *емпіричними*.

Емпірична залежність $y = f(x)$ по суті є математичною моделлю процесу дослідження, результати якої дійсні тільки в межах зміни аргументу, тобто в інтервалі варіації факторів $x_1, x_2, \dots, x_n$.

Необхідність в емпіричних залежностях виникає тоді, коли аналітичні залежності вважаються складними і вимагають громіздких обчислень для практичного використання або ж тоді, коли аналітичні залежності взагалі відсутні. Можна вважати, що емпіричні залежності - це наближене виявлення аналітичних, а процес апроксимації - спосіб заміни складного або неможливого процесу одержання точних аналітичних виразів

Задача апроксимації складається з двох частин:

1. встановлення виду залежності $y = f(x)$ і, відповідно, виду емпіричної формули (лінійна, квадратична, логарифмічна тощо);
2. визначення чисельних значень невідомих параметрів обраної емпіричної формули, для яких наближення до заданої функції виявляється найкращим.

Метод найменших квадратів

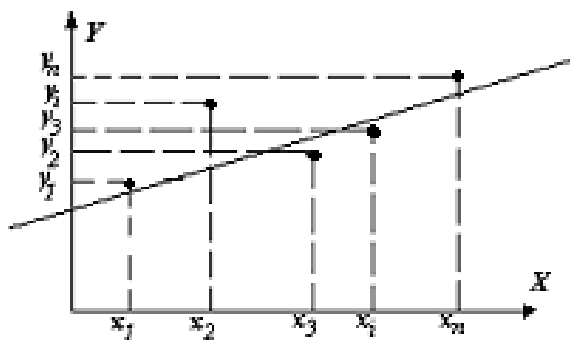
Нехай передбачається, що вид функціональної залежності (лінійна, логарифмічна, експоненціальна тощо) відомий, і потрібно визначити лише параметри цієї залежності. Одним із найпростіших методів знаходження параметрів функціональної залежності є метод найменших квадратів (МНК). Суть його полягає в тому, що параметри визначаються за умови мінімізації суми квадратів відхилень експериментальних даних від теоретичної кривої з

невідомими параметрами.

Нехай, наприклад, в результаті вимірювань отримано таблицю залежності однієї величини y від іншої величини x :

x	x_1	x_2	$\dots$	x_i	$\dots$	x_n
y	y_1	y_2	$\dots$	y_i	$\dots$	y_n

За цією таблицею будують графік функції $y = f(x)$. Якщо, наприклад, точки графіка розташувалися як на малюнку, можна припустити, маючи на увазі можливі помилки виміру величин x і y , що залежність між x і y лінійна,



тобто що $y = ax + b$ (1).

Для вибору a і b застосовується МНК наступним чином.

Нехай числа a і b якимось чином вибрані. Підставимо x_i в формулу (1), отримаємо $ax_i + b$, а в результаті експерименту отримали y_i . Отже, формула дає розбіжність із експериментом $y_i - ax_i - b$, яка отримана за рахунок похибок вимірів, неточної лінійної залежності y від x та інших причин. Така різниця між лівою та правою частинами рівності називається відхиленням.

Метод найменших квадратів полягає в тому, що числа a і b вибираються з умови, щоб сума квадратів відхилень була мінімальною, тобто

$U(a, b) = \sum_{i=1}^n (y_i - ax_i - b)^2 \rightarrow \min$. Приходимо до задачі пошуку мінімуму функції двох змінних $U(a, b)$.

Використовуємо необхідні умови $U'_a = -\sum_{i=1}^n 2(y_i - ax_i - b)x_i = 0$,

$U'_b = -\sum_{i=1}^n 2(y_i - ax_i - b) = 0$. Отримаємо систему двох лінійних рівнянь із двома

невідомими a і b .

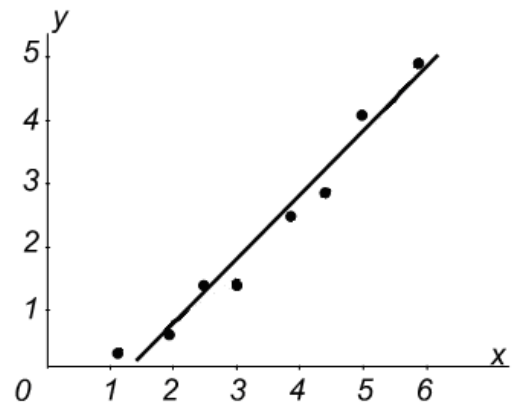
$$\begin{cases} a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i = \sum_{i=1}^n x_i y_i \\ a \sum_{i=1}^n x_i + b \cdot n = \sum_{i=1}^n y_i \end{cases} \quad (2)$$

Аналогічним чином отримують формули для визначення коефіцієнтів інших видів залежностей.

Приклад. Методом найменших квадратів побудувати функціональну залежність для таких даних:

x	1,1	1,7	2,4	3,0	3,7	4,5	5,1	5,8
y	0,3	0,6	1,1	1,7	2,3	3,0	3,8	4,6

За виглядом розташування точок, які отримані в результаті експерименту, можна припустити, що величини x і y знаходяться в лінійній залежності $y = ax + b$. Складемо і розв'яжемо систему рівнянь (2). Для цього складемо допоміжну таблицю:



	x	y	$x \cdot y$	x^2
1	1,1	0,3	0,33	1,21
2	1,7	0,6	1,02	2,89
3	2,4	1,1	2,64	5,76
4	3,0	1,7	5,1	9
5	3,7	2,3	8,51	13,69
6	4,5	3,0	13,5	20,25
7	5,1	3,8	19,38	26,01
8	5,8	4,6	26,68	33,64
Разом	27,3	17,4	77,16	112,45

Система (2) приймає вигляд: $\begin{cases} 112,45a + 27,3b = 77,16 \\ 27,3a + 8b = 17,4 \end{cases}$. Розв'язавши її,

знайдемо $a = 0,922$; $b = -0,971$. Таким чином, лінійна залежність між величинами x і y має вигляд $y = 0,922x - 0,971$.

Числові ряди

Нескінченна сума чисел $a_1 + a_2 + a_3 + \dots + a_n + \dots$ (1) носить назву *числового ряду*. Частіше його записують за допомогою знаку сігма: $\sum_{n=1}^{\infty} a_n$. Числа

$a_1, a_2, a_3, \dots, a_n, \dots$ є членами ряду, a_n – загальним членом ряду.

Наприклад, числовий ряд $\frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots = \sum_{n=1}^{\infty} \frac{1}{n(n+1)}$ має загальний член

$$a_n = \frac{1}{n(n+1)}.$$

З кожним числовим рядом зв'язується послідовність часткових сум: $S_1 = a_1$, $S_2 = a_1 + a_2, \dots, S_n = a_1 + a_2 + a_3 + \dots + a_n$.

Числовий ряд $\sum_{n=1}^{\infty} a_n$ називається *збіжним*, якщо послідовність $S_1, S_2, \dots, S_n, \dots$ має скінчену границю, причому число $S = \lim_{n \rightarrow \infty} S_n$ називається сумою ряду і

записується це так: $\sum_{n=1}^{\infty} a_n = S$. Якщо ряд не збігається, то він називається *розбіжним*. Розбіжний ряд суми не має. (Казати, що він має нескінченну суму неправильно).

Нехай ряд (1) збігається. Тоді його часткова сума S_n є наближеним значенням для суми S . Похибка цього наближення $r_n = S - S_n$ зветься залишком ряду. Цей

залишок є сумою ряду: $r_n = \sum_{k=n+1}^{\infty} a_k = a_{n+1} + a_{n+2} + \dots$

В теорії числових рядів вирізняються дві задачі: дослідження рядів на збіжність і знаходження сум рядів.

Знайти точне значення суми ряду вдається далеко не завжди. В цьому випадку використовуються методи наближеного обчислення суми ряду.

Існує досить багато прийомів, які дозволяють встановлювати збіжність або розбіжність рядів. Такі прийоми називаються ознаками збіжності.

Теорема про необхідну умову збіжності числового ряду. Якщо ряд $\sum_{n=1}^{\infty} a_n$

збігається, то $\lim_{n \rightarrow \infty} a_n = 0$.

Ця умова є лише необхідною, але не достатньою. Це означає, що існують розбіжні ряди, у яких $\lim_{n \rightarrow \infty} a_n = 0$. Для збіжності ряду загальний член ряду повинен не просто прагнути до нуля, але робити це досить швидко.

За допомогою необхідної ознаки збіжності не можна довести збіжність ряду, але іноді вдається довести розбіжність, застосовуючи наслідок з необхідної ознаки:

(Достатня ознака розбіжності). Якщо $\lim_{n \rightarrow \infty} a_n \neq 0$, то ряд $\sum_{n=1}^{\infty} a_n$ розбігається.

Числовий ряд зветься рядом з додатними членами або просто додатним рядом, якщо все числа $a_1, a_2, a_3, \dots, a_n, \dots \geq 0$.

Розглянемо деякі достатні ознаки збіжності для додатних рядів. Ознаки, що дозволяють з'ясувати питання про збіжність деякого ряду за допомогою іншого ряду, поведінка якого в сенсі збіжності відома, називаються ознаками порівняння.

Перша ознака порівняння. Нехай дано два ряди $\sum_{n=1}^{\infty} a_n$ (1), $\sum_{n=1}^{\infty} b_n$ (2) і $a_n \leq b_n$ для будь-якого n . Тоді зі збіжності ряду (2) впливає збіжність (1), а з розбіжності ряду (1) впливає розбіжність ряду (2). Зворотні твердження невірні. Ця ознака складна для застосування, однак існують ряди, які досліджуються на збіжність тільки з її допомогою.

Ознака Даламбера. Нехай існує границя $\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = d$, де d – число Даламбера.

Тоді, якщо $d < 1$, ряд $\sum_{n=1}^{\infty} a_n$ збігається. Якщо $d > 1$, то ряд $\sum_{n=1}^{\infty} a_n$ розбігається. Якщо $d = 1$, то про збіжність або розбіжність ряду нічого сказати не можна.

Ознака Коші (радикальна ознака). Якщо існує границя $\lim_{n \rightarrow \infty} \sqrt[n]{a_n} = K$, то ряд $\sum_{n=1}^{\infty} a_n$ збігається при $K < 1$ і розбігається при $K > 1$. Якщо $K = 1$, то про збіжність або розбіжність ряду нічого сказати не можна.

Інтегральна ознака Маклорена-Коші. Нехай $f(x)$ неперервна на $[1; \infty)$, спадає

на цьому проміжку, $\lim_{x \rightarrow \infty} f(x) = 0$ і ряд $\sum_{n=1}^{\infty} a_n$ пов'язаний з $f(x)$ співвідношенням $a_n = f(n)$. Тоді $\sum_{n=1}^{\infty} a_n$ збігається в тому і тільки тому випадку, коли $\int_1^{\infty} f(x) dx$ скінчений (збіжний).

Дуже важливий приклад. Дослідимо на збіжність ряд $\sum_{n=1}^{\infty} \frac{1}{n}$, який називається гармонійним рядом. Для цього досліджуємо функцію $f(x) = \frac{1}{x}$.

$$\int_1^{\infty} \frac{dx}{x} = \lim_{b \rightarrow \infty} \int_1^b \frac{dx}{x} = \ln |x| \Big|_1^{\infty} = \infty. \text{ Отже, ряд } \sum_{n=1}^{\infty} \frac{1}{n} \text{ розбігається.}$$

Ряд виду $\sum_{n=1}^{\infty} \frac{1}{n^p}$ називається загальногармонійним з показником p або рядом Дирихле. Якщо $p \leq 1$, то ряд розбігається, якщо $p > 1$, то ряд збігається.

Перевага інтегральної ознаки полягає в її високій чутливості: ця ознака чітко розмежовує збіжні та розбіжні ряди, навіть якщо члени одного з них незначно відрізняються від членів іншого. Крім того, якщо додатний ряд можна досліджувати на збіжність за інтегральною ознакою, то його залишок оцінюється за формулою: $r_n \leq \int_{n+1}^{\infty} f(x) dx$. Ця оцінка використовується для наближеного обчислення суми збіжних рядів.

Гранична ознака порівняння. Нехай дано два ряди $\sum_{n=1}^{\infty} a_n$, $\sum_{n=1}^{\infty} b_n$ і нехай існує скінчена і відмінна від 0 $\lim_{n \rightarrow \infty} \frac{a_n}{b_n}$. Тоді ряди $\sum_{n=1}^{\infty} a_n$ і $\sum_{n=1}^{\infty} b_n$ поводять себе однаково в сенсі збіжності.

При використанні ознак порівняння досліджуваний ряд найчастіше порівнюється або з нескінченною геометричною прогресією, або з узагальненими гармонійними рядами.

Функціональні ряди.

Функціональним рядом називається ряд, членами якого є функції від аргументу

$$x: u_1(x) + u_2(x) + u_3(x) + \dots + u_n(x) + \dots = \sum_{n=1}^{\infty} u_n(x) \quad (1).$$

Дослідження на збіжність функціональних рядів складніше дослідження числових рядів. Один і той же функціональний ряд може при одних значеннях змінної x збігаться, а при інших - розбігаться. Тому питання збіжності функціональних рядів зводиться до визначення тих значень змінної x , при яких ряд збігаться.

Якщо в членах ряду (1) зафіксувати значення аргументу $x = x_0$, то отримаємо числовий ряд $u_1(x_0) + u_2(x_0) + u_3(x_0) + \dots + u_n(x_0) + \dots = \sum_{n=1}^{\infty} u_n(x_0)$ (2). Якщо при

$x = x_0$ числовий ряд (2) збігаться, то x_0 називається точкою збіжності ряду (1).

Областю збіжності функціонального ряду називається множина всіх точок збіжності цього ряду.

На основі того, що сума ряду є деякою функцією від змінної x , можна деяку функцію надати в вигляді ряду (розкладання функції в ряд), що має широке застосування при інтегруванні, диференціюванні та інших діях з функціями. На практиці часто застосовується розкладання функцій в степеневий ряд.

Степеневі ряди.

Степеневим рядом за степенями x називається функціональний ряд виду

$$C_0 + C_1x + C_2x^2 + \dots + C_nx^n + \dots = \sum_{n=1}^{\infty} C_nx^n \quad (3), \text{ де } C_0, C_1, \dots, C_n, \dots \text{ не залежать від змінної}$$

x і називаються коефіцієнтами цього ряду.

Степеневий ряд завжди збігаться, принаймні в точці $x = 0$. За будь-яких конкретних $x = x_0$ ряд (3) перетворюється в числовий ряд

$$C_0 + C_1x_0 + C_2x_0^2 + \dots + C_nx_0^n + \dots \quad (4).$$

Степеневий ряд (3) збігаться в точці абсолютно, якщо збігаться ряд, складений з модулів членів числового ряду (4):

$$|C_0| + |C_1x_0| + |C_2x_0^2| + \dots + |C_nx_0^n| + \dots$$

За ознакою Даламбера ряд (3) явно збігаться при $|x| < \frac{1}{\lim_{n \rightarrow \infty} \left| \frac{C_{n+1}}{C_n} \right|} = \lim_{n \rightarrow \infty} \left| \frac{C_n}{C_{n+1}} \right|$ і

розбігається при $|x| > \frac{1}{\lim_{n \rightarrow \infty} \left| \frac{C_{n+1}}{C_n} \right|} = \lim_{n \rightarrow \infty} \left| \frac{C_n}{C_{n+1}} \right|$. Величина $R = \lim_{n \rightarrow \infty} \left| \frac{C_n}{C_{n+1}} \right|$ (5) або $R = \lim_{n \rightarrow \infty} \frac{1}{\sqrt[n]{C_n}}$ (6)

називається *радіусом збіжності* ряду (3). Формули (5) і (6) називається *формулою Коші-Адамара*. Ряд явно збігається в інтервалі $|x| < R$ або $-R < x < R$, який називається *інтервалом збіжності*. Ознака Даламбера нічого не говорить про збіжність ряду в точках $x = \pm R$. В цих точках збіжність ряду потрібно досліджувати окремо. Таким чином, дослідити степеневий ряд на збіжність означає знайти його інтервал збіжності і встановити збіжність або розбіжність в граничних точках інтервалу, тобто при $x = R$ і $x = -R$.

Разом зі степеневими рядами відносно змінної x часто розглядають степеневі ряди за змінною $x - a$, тобто ряди виду $C_0 + C_1(x - a) + C_2(x - a)^2 + \dots + C_n(x - a)^n + \dots$

Вочевидь, що підстановкою $y = x - a$ цей ряд перетворюється в ряд типу (3). Тому, якщо степеневий ряд (3) має інтервал збіжності $-R < x < R$, то відповідний ряд виду (6) має інтервал збіжності $a - R < x < a + R$, центр якого розташований у точці $x = a$.

Розкладання функцій в степеневі ряди.

Розкладання функцій в степеневий ряд має велике значення для розв'язання різних задач дослідження функцій, диференціювання, інтегрування, розв'язання диференціальних рівнянь, обчислення границь, обчислення наближених значень функції.

Якщо функція $f(x)$ має похідні будь-якого порядку в околі точки $x = a$, то для неї можна отримати нескінченний ряд, який називається рядом Тейлора:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \dots = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n \quad (7)$$

Якщо $a = 0$, отримаємо окремий випадок ряду Тейлора, який називають рядом

Маклорена:
$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \dots + \frac{f^{(n)}(0)}{n!}x^n + \dots \quad (8)$$

Крім рядів Тейлора і Маклорена існують також способи розкладання в степеневий ряд за допомогою *алгебраїчного ділення* або за допомогою *інтегрування*.

Звичайні диференціальні рівняння

Диференціальним рівнянням називається рівняння, що зв'язує незалежні змінні, їх функції та похідні (або диференціали) цих функцій. Якщо похідні, що входять в рівняння, беруться тільки по одній змінній, то диференціальне рівняння називається *звичайним*. Якщо в рівнянні зустрічаються похідні по декільком змінним, то рівняння називається рівнянням *в частинних похідних*. Найвищий порядок похідних, що входять в рівняння, називається *порядком диференціального рівняння*. Наприклад, звичайним диференціальним рівнянням n -ого порядку називається співвідношення виду: $F(x, y, y', y'', \dots, y^{(n)}) = 0$ (1), де x – незалежна змінна; y – шукана функція змінної; $y', y'', \dots, y^{(n)}$ – похідні шуканої функції; F – відома функція своїх аргументів.

Вважається, що похідна $y^{(n)}$ завжди входить у вираз (1), а величини $x, y, y', y'', \dots, y^{(n-1)}$ можуть і не входити в нього.

Приклад. 1) $y' + 2x^3 y = 0$ – звичайне диференціальне рівняння першого порядку;

2) $y'' + y = 0$ – звичайне диференціальне рівняння другого порядку;

3) $y^{(5)} + 2x^2 y''' + e^x y + 1 = 0$ – звичайне диференціальне рівняння п'ятого порядку;

4) $y^2 \frac{\partial z}{\partial x} + xy \frac{\partial z}{\partial y} = 0$ – диференціальне рівняння в частинних похідних першого порядку.

Загальним розв'язком диференціального рівняння називається така диференційована функція $y = \varphi(x, C)$, яка при підстановці в вихідне рівняння замість невідомої функції обертає рівняння в тотожність. Тут C – довільна постійна. А це значить, що рівняння має нескінчену кількість рішень, які залежать від одного параметра (C).

Можна показати, що рівняння n -ого порядку має сімейство рішень, що залежать від довільних незалежних одна від одної постійних. Наприклад, рівняння $y^{(n)} = 0$ має розв'язок: $y(x) = C_1 x^{n-1} + C_2 x^{n-2} + \dots + C_{n-1} x + C_n$.

Процес пошуку розв'язку диференціального рівняння називається *інтегруванням* диференціального рівняння. Диференціальне рівняння може мати не один, а кілька загальних розв'язків. Наприклад, для рівняння $y' = y$ функції $y = C e^x$ і $y = e^{x+C}$ є загальними розв'язками, причому різними, оскільки перший з них обертається на нуль (при $C = 0$), а другий – ніколи в нуль не обертається.

Співвідношення $\Phi(x, y, C_1, C_2, \dots, C_n) = 0$, яке зв'язує між собою незалежну змінну, шукану функцію і n довільних постійних, називається *загальним інтегралом* рівняння (1). В загальному інтегралі розв'язок заданий в неявному вигляді. Наприклад, для рівняння $y y' = x$ вираз $y^2 + x^2 = C \geq 0$ – загальний інтеграл; $y = \pm \sqrt{C - x^2}$ – загальний розв'язок.

При будь-яких початкових умовах $x = x_0$, $y(x_0) = y_0$ існує таке значення $C = C_0$, при якому розв'язком диференціального рівняння є функція $y = \varphi(x, C_0)$.

Розв'язок виду $y = \varphi(x, C_0)$ називається *частинним розв'язком* диференціального рівняння.

Наприклад, рівняння $y' = y$ має загальний розв'язок $y = C e^x$. Нехай $C = 2$, тоді $y = 2e^x$ – частинний розв'язок.

Задачею Коші називається знаходження будь-якого частинного розв'язку диференціального рівняння виду $y = \varphi(x, C_0)$, що задовольняє початковій умові $y(x_0) = y_0$.

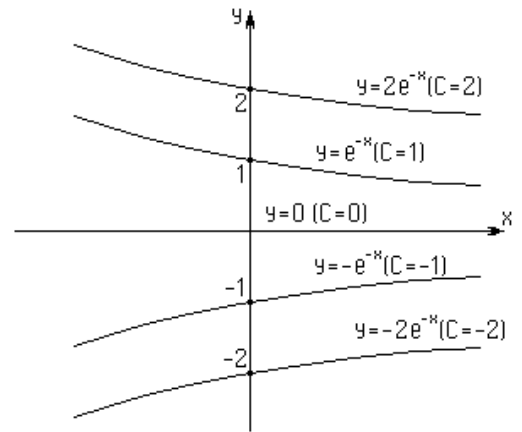
Особливим розв'язком по відношенню до даного загального розв'язку називається такий розв'язок, який не може бути отриманий ні при яких значеннях довільних постійних, що входять в загальний розв'язок.

Наприклад, рівняння $y' = y$ має два загальних розв'язки:

1) $y = C e^x$ 2) $y = e^{x+C}$. Розв'язок $y = 0$ є частинним по відношенню до першого і особливим по відношенню до другого загального розв'язку.

Не кожне диференціальне рівняння має особливі розв'язки.

Графік частинного розв'язку називається інтегральною кривою розглядуваного диференціального рівняння. Рівняння цієї лінії можна отримати із загального розв'язку при фіксованих $C_1, C_2, \dots, C_n$. Таким чином, загальний



розв'язок (або загальний інтеграл) визначає сімейство інтегральних кривих, кожна з яких відповідає певному набору значень $C_1, C_2, \dots, C_n$ довільних постійних.

Приклад. На малюнку зображено сімейство інтегральних кривих $y' = -y$. Загальний розв'язок $y = C e^{-x}$.

Диференціальні рівняння першого порядку

Диференціальне рівняння *першого порядку* може бути записано у вигляді

$$F(x, y, y') = 0 \quad (2). \text{ Тут } x \text{ — незалежна змінна, } y \text{ — її невідома функція, } y' = \frac{dy}{dx} \text{ —}$$

похідна функції y , F — задана функція трьох змінних. У деяких випадках (2) визначає змінну y' як функцію незалежних змінних x і y : $y' = f(x, y)$ (3). Тоді диференціальне рівняння (3) рівносильне диференціальному рівнянню (2) і називається вирішеним відносно похідної. Якщо $f(x, y)$ записати як

$$f(x, y) = -\frac{P(x, y)}{Q(x, y)}, \quad Q(x, y) \neq 0, \quad \text{а} \quad y' = \frac{dy}{dx}, \quad \text{то} \quad P(x, y)dx + Q(x, y)dy = 0 \quad \text{—}$$

диференціальна форма рівняння першого порядку.

Наведемо приклади диференціальних рівнянь першого порядку:

$$x^3 y' + 8y - x + 5 = 0; \quad y' = x + y; \quad y dy = \frac{-2x}{\cos y} dx.$$

Диференціальні рівняння вищих порядків

Загальний вигляд звичайного диференціального рівняння n -ого порядку:

$$F(x, y, y', y'', \dots, y^{(n)}) = 0. \text{ В деяких випадках це рівняння можна розв'язати відносно}$$

похідної вищого порядку: $y^{(n)} = f(x, y, y', \dots, y^{(n-1)})$.

Так само як і рівняння першого порядку, рівняння вищих порядків мають нескінченну кількість розв'язків.

Диференціальні рівняння вищих порядків, рішення яких може бути знайдено аналітично, можна розділити на кілька основних типів. Зниження порядку диференціального рівняння дає можливість порівняно легко знаходити рішення, однак, цей метод може бути застосований далеко не до всіх рівнянь.

Чисельне розв'язання звичайних диференціальних рівнянь

Нехай потрібно чисельно розв'язати задачу Коші для звичайного диференціального рівняння першого порядку, тобто знайти наближений розв'язок диференціального рівняння $y' = F(x, y)$, що задовольняє початкову умову $y(x_0) = y_0$. Чисельне розв'язання задачі полягає в побудові таблиці наближених значень $y_1, y_2, y_3, \dots, y_n$ - розв'язку рівняння $y = \varphi(x)$ у точках $x_1, x_2, x_3, \dots, x_n$ - вузлах сітки.

На рисунку 1 * позначені точки, що відповідають наближеному розв'язку задачі Коші. Треба зазначити, що частіше використовують систему рівновіддалених вузлів $x_i = x_0 + ih$ ($i=1, 2, \dots, n$), де h - крок сітки ($h > 0$).

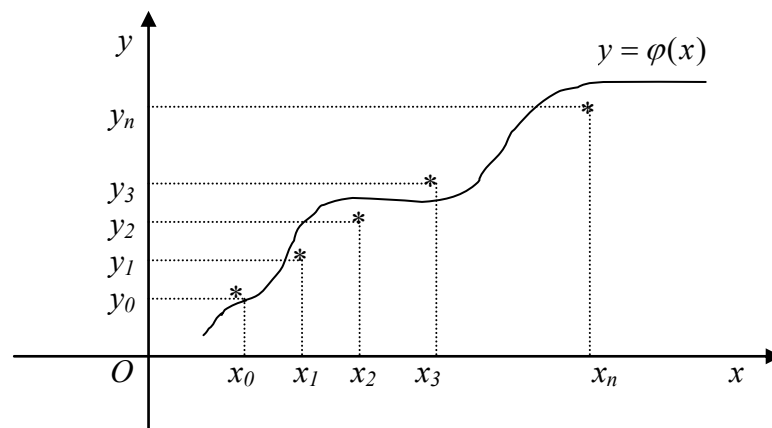


Рисунок 1 – Розв'язання задачі Коші

Методи Рунге-Кутти

Різні представники цієї категорії методів потребують більшого чи меншого об'єму обчислень і відповідно забезпечують більшу чи меншу точність. Ці методи мають ряд важливих переваг є явними, одноступінчастими, тобто значення y_{i+1} обчислюється за раніше знайденими значеннями y_i .

Допускають використання змінного кроку, що дає можливість зменшити його там, де функція швидко змінюється, і збільшити в протилежному випадку.

Є легко застосовними, оскільки що для початку розрахунку досить вибрати сітку x_i і задати значення $y_0 = f(x_0)$.

Узгоджуються з рядом Тейлора включно до членів порядку h^p , де степінь p не однаковий для різних методів і називається порядком методу.

Не потребують обчислення похідних від $f(x, y)$, а вимагають тільки обчислення самої функції.

При розв'язанні конкретної задачі виникають питання, якою із формул Рунге-Кутта доцільно скористатися і як вибрати крок сітки.

Якщо $f(x, y)$ неперервна й обмежена разом із своїми четвертими похідними, то добрі результати дає метод четвертого порядку. Він описується системою таких п'яти співвідношень: $y_{i+1} = y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$; $k_1 = f(x_i, y_i)$; $k_2 = f(x_i + \frac{h}{2}, y_i + \frac{h}{2} \cdot k_1)$.
($0 \leq i \leq n-1$); $k_3 = f(x_i + \frac{h}{2}, y_i + \frac{h}{2} \cdot k_2)$; $k_4 = f(x_i + h, y_i + h \cdot k_3)$.

Якщо функція не має зазначених похідних, порядок точності вищенаведеного методу не може бути реалізований. Тоді необхідно користуватися методами меншого порядку точності, що відповідає порядку наявних похідних.

Одним із найбільш простих і досить ефективних методів оцінки похибки й уточнення отриманих результатів є правило Рунге. Для оцінки похибки за правилом Рунге порівнюють наближені розв'язки, отримані при різних кроках сітки. При цьому використовується таке припущення: глобальна похибка методу порядку p у точці x_i подається у вигляді $y_i - y(x_i) = w(x_i) \cdot h^p + o(h^{p+1})$; $x_i = a + ih$.

За формулою Рунге $y_{2i} - y(x_{2i}) = \frac{y_i - y_{2i}}{2^p - 1} + o(h^{p+1})$.

Таким чином, із точністю до $o(h^{p+1})$ (величина більш високого порядку малості) при $h \rightarrow 0$ похибка методу має вигляд $y_{2i} - y(x_{2i}) = \frac{y_i - y_{2i}}{2^p - 1}$, де y_i – наближене значення, отримане в точці x_{2i} із кроком h ; y_{2i} – із кроком $h/2$; p - порядок методу; $y(x_{2i})$ - точний розв'язок задачі.

Формула Рунге для методу четвертого порядку $|y_{2i} - y(x_{2i})| \approx \frac{1}{15} |y_i - y_{2i}|$.

Обчислювальна схема (алгоритм) методу Рунге-Кутта

Вибираємо початковий крок h на відрізку $[a, b]$, задаємо точність ε .

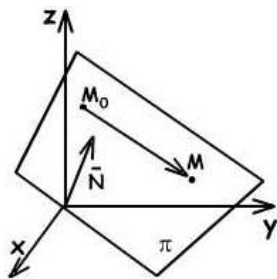
Створюємо множину рівновіддалених точок (вузлів) $x_i = a + ih$, $0 \leq i \leq n$, $n = \frac{b-a}{h}$.

Знаходимо розв'язок y_{i+1} за формулами при кроку h і при кроку $h/2$, $0 \leq i \leq n-1$.

Перевіряємо нерівність $\frac{|y_h - y_{h/2}|}{15} < \varepsilon$.

Якщо ця нерівність виконується, то беремо $y(x_i) = y_{h/2}(x_i)$ і продовжуємо обчислення y_{i+2} з тим самим кроком, якщо ні, то зменшуємо початковий крок h у два рази і переходимо до пункту 3.

Площина в просторі



Нехай площину π задано точкою $M_0(x_0, y_0, z_0)$ та вектором $\bar{N} = \{A, B, C\} \neq \bar{0}$, $\perp$ до π . Візьмемо на π довільну точку $M(x, y, z)$. Вектор $\overline{M_0M} = \{x - x_0; y - y_0; z - z_0\}$ при будь-якому положенні точки M на площині $\pi \perp$ до вектору $\bar{N}$. $(\bar{N} \cdot \overline{M_0M}) = 0$ або $A(x - x_0) + B(y - y_0) + C(z - z_0) = 0$ (1)

Це рівняння площини, що проходить через точку $M_0 \perp$ до вектору $\bar{N}$. Розкриємо дужки в рівнянні (1) і позначимо $D = -Ax_0 - By_0 - Cz_0$, отримаємо: $Ax + By + Cz + D = 0$ (2) – загальне рівняння площини. Рівняння (2) є рівнянням першого ступеня (або лінійним) щодо x, y, z . Вектор $\bar{N}$ називають нормальним вектором площини.

Особливості розташування площини

1) Якщо в (2) вільний член $D = 0$, тоді площина $Ax + By + Cz = 0$ проходить через початок координат.

2) $A = 0$; площина $By + Cz + D = 0 \parallel$ осі OX . Аналогічно, якщо $B = 0$, то площина $\parallel OY$; $C = 0$, то площина $\parallel OZ$. $Ax + By + D = 0$

3) Якщо $A = B = 0$, то площина $Cz + D = 0 \parallel$ координатної площини XOY . Аналогічно, якщо $A = C = 0$, це $By + D = 0 \parallel XOZ$; якщо $B = C = 0$, це $Ax + D = 0 \parallel YOZ$.

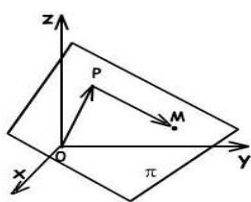
4) Якщо $A = D = 0$, то площина $By + Cz = 0$, проходить через вісь OX . Аналогічно, $B = D = 0$, то π проходить через OY ; $C = D = 0$, то π проходить через OZ .

5) $A = B = D = 0$; то $CZ = 0$; або $z = 0$, співпадає з площиною XOY ; $B = C = D$, то $x = 0$ співпадає з YOZ ;

$A = C = D$, то $y = 0$ співпадає з XOZ .

Нормальне рівняння площини

Нехай OP – нормаль до площини π . $P \in \pi$. Довжина $|OP| = p$; α, β, γ – кути, які утворює нормаль з осями OX, OY, OZ .



$$\overline{OP} = \{p \cos \alpha; p \cos \beta; p \cos \gamma\};$$

точка

$$P(p \cos \alpha; p \cos \beta; p \cos \gamma).$$

Рівняння площини у вигляді (1) буде:

$$p \cos \alpha (x - p \cos \alpha) + p \cos \beta (y - p \cos \beta) + p \cos \gamma (z - p \cos \gamma) = 0 \quad (4)$$

Скоротивши (4) на p і розкривши дужки, отримуємо:

$x \cos \alpha + y \cos \beta + z \cos \gamma - p = 0$ (5), де α, β, γ – направляючі кути нормального вектору площини, p – довжина нормального вектору. (5) називають *нормальним* рівнянням площини.

Воно має такі властивості:

1⁰. Вільний член нормального рівняння площини завжди від'ємний ($p > 0$; $-p < 0$).

2⁰. Сума квадратів коефіцієнтів при x, y, z дорівнює одиниці (або $|\bar{N}| = \sqrt{\cos^2 \alpha + \cos^2 \beta + \cos^2 \gamma} = 1$).

Щоб привести загальне рівняння площини (2) до нормального вигляду, помножимо обидві його частини на число $\mu \neq 0$ (нормуючий множник): $A\mu x + B\mu y + C\mu z + D\mu = 0$

Знайдемо нормуючий множник з умов:

$$1^0. D\mu < 0; \quad 2^0. (A\mu)^2 + (B\mu)^2 + (C\mu)^2 = 1.$$

З останнього рівняння $\mu = \pm \frac{1}{\sqrt{A^2 + B^2 + C^2}}$. Знак перед дробом беремо таким, щоб $D\mu < 0$,

тобто знак μ протилежний D .

Відхилення і відстань точки від площини

Відстань від точки $M_0(x_0, y_0, z_0)$, яка не належить площині π , має вигляд:

$$d = \frac{|Ax_0 + By_0 + Cz_0 + D|}{\sqrt{A^2 + B^2 + C^2}} \quad (6); \text{ відстань } d \text{ завжди додатна, а}$$

результат підстановки координат т. M_0 в ліву частину рівняння може бути як додатним, так і від'ємним числом. Це число називають

відхиленням точки M_0 від площини π і позначають δ , а відстань $d = |\delta|$.

Рівняння площини, що проходить через три точки.

Нехай на площині π задано три точки $M_1(x_1, y_1, z_1)$; $M_2(x_2, y_2, z_2)$ і $M_3(x_3, y_3, z_3)$, що не лежать на одній прямій. Візьмемо на π довільну (поточну) точку $M(x, y, z)$. Вектори

$$\overline{M_1M} = \{x - x_1; y - y_1; z - z_1\}; \quad \overline{M_1M_2} = \{x_2 - x_1; y_2 - y_1; z_2 - z_1\};$$

$$\overline{M_1M_3} = \{x_3 - x_1; y_3 - y_1; z_3 - z_1\} \text{ компланарні, тому їх змішаний добуток}$$

$$(\overline{M_1M} \times \overline{M_1M_2}) \cdot \overline{M_1M_3} = 0, \text{ або } \begin{vmatrix} x - x_1 & y - y_1 & z - z_1 \\ x_2 - x_1 & y_2 - y_1 & z_2 - z_1 \\ x_3 - x_1 & y_3 - y_1 & z_3 - z_1 \end{vmatrix} = 0 \quad (7) - \text{рівняння площини, що}$$

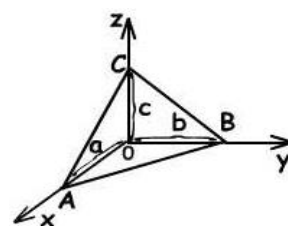
проходить через три точки.

Рівняння площини у відрізках на осях

Нехай задані відрізки a, b, c , які площина відсікає відповідно на осях $OX; OY; OZ$. Отже, відомі три точки площини π : $A(a, 0, 0)$; $B(0, b, 0)$; $C(0, 0, c)$. Рівняння

$$(7) \text{ буде: } \begin{vmatrix} x - a & y - 0 & z - 0 \\ 0 - a & b - 0 & 0 - 0 \\ 0 - a & 0 - 0 & c - 0 \end{vmatrix} = 0; \quad \frac{x}{a} + \frac{y}{b} + \frac{z}{c} = 1 - \text{рівняння площини}$$

в відрізках на осях.



Кут між двома площинами

Дві площини π_1 і π_2 задані загальними рівняннями:

$$A_1x + B_1y + C_1z + D_1 = 0 \text{ и } A_2x + B_2y + C_2z + D_2 = 0$$

Двогранний кут між двома площинами вимірюється лінійним кутом між нормальними векторами $\overline{N_1} = \{A_1, B_1, C_1\}$ и $\overline{N_2} = \{A_2, B_2, C_2\}$ цих площин,

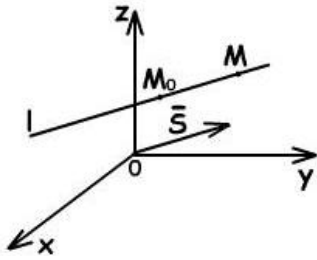
$$\cos \theta = \pm \frac{|\overline{N_1} \cdot \overline{N_2}|}{|\overline{N_1}| \cdot |\overline{N_2}|} = \pm \frac{A_1A_2 + B_1B_2 + C_1C_2}{\sqrt{A_1^2 + B_1^2 + C_1^2} \cdot \sqrt{A_2^2 + B_2^2 + C_2^2}} \quad (\text{знаки "}\pm\text{" в цій формулі}$$

відповідають косинусам двох суміжних двогранних кутів).

$$\text{Якщо } \pi_1 \parallel \pi_2, \text{ то } \overline{N_1} \parallel \overline{N_2} \text{ і } \frac{A_1}{A_2} = \frac{B_1}{B_2} = \frac{C_1}{C_2}.$$

$$\text{Якщо } \pi_1 \perp \pi_2, \text{ то } \overline{N_1} \perp \overline{N_2} \text{ і } A_1A_2 + B_1B_2 + C_1C_2 = 0.$$

Пряма лінія в просторі



Нехай в просторі задана пряма l точкою $M_0(x_0, y_0, z_0)$ і вектором $\bar{S} = \{m, n, p\}$, паралельним l ($\bar{S}$ – направляючий вектор l).
 $M(x, y, z)$ – поточна точка прямої l .
 $\overrightarrow{M_0M} = \{x - x_0; y - y_0; z - z_0\}$ завжди $\parallel \bar{S}$, а якщо два вектори колінеарні, то їх координати пропорційні

$$\frac{x - x_0}{m} = \frac{y - y_0}{n} = \frac{z - z_0}{p} \quad (8)$$
 – канонічне рівняння прямої. Якщо

кожне з відношень (8) прирівняти до параметра t , тобто: $\frac{x - x_0}{m} = t$; $\frac{y - y_0}{n} = t$; $\frac{z - z_0}{p} = t$, і

виразити x, y, z через t , то отримаємо:
$$\begin{cases} x = mt + x_0 \\ y = nt + y_0 \\ z = pt + z_0 \end{cases} \quad (9)$$
 рівняння, які називають

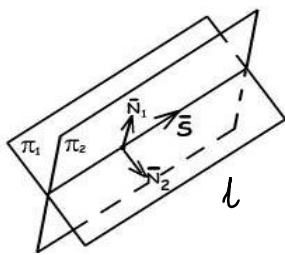
параметричними рівняннями прямої; t – змінний параметр прямої. Знак t залежить від того, однаково або протилежно спрямовані вектори $\overrightarrow{M_0M}$ і $\bar{S}$. Абсолютна величина t дорівнює відношенню модулів $\overrightarrow{M_0M}$ и $\bar{S}$.

$$M_1(x_1, y_1, z_1) \in l, M_2(x_2, y_2, z_2) \in l, \bar{s} = \overrightarrow{M_1M_2} = \{x_2 - x_1; y_2 - y_1; z_2 - z_1\},$$

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1} = \frac{z - z_1}{z_2 - z_1} \quad (10)$$

Пряма лінія як перетин двох площин

Пряму, як перетин двох площин, можна задати системою двох лінійних рівнянь
$$\begin{cases} A_1x + B_1y + C_1z + D_1 = 0; \\ A_2x + B_2y + C_2z + D_2 = 0; \end{cases} \quad (11)$$



Необхідно, щоб вони не були паралельні, тобто: $\bar{N}_1 = \{A_1; B_1; C_1\}$ і $\bar{N}_2 = \{A_2; B_2; C_2\}$ не були колінеарними, а це означає, що їх координати не повинні бути пропорційні.

Маючи рівняння прямої в формі (11), знайдемо канонічні рівняння цієї прямої. Для цього визначимо одну точку прямої M_0 і вектор $\bar{S}$, $\parallel$ прямій.

За т. M_0 можна прийняти точку перетину прямої (10) з однією з координатних осей.

Наприклад, припустивши $z = 0$, x і y визначимо із системи
$$\begin{cases} A_1x + B_1y + D_1 = 0 \\ A_2x + B_2y + D_2 = 0 \end{cases} \quad (\text{якщо ця}$$

система не сумісна, припускаємо $x = 0$, або $y = 0$). Пряма (10) $\perp$ до нормальних векторів площин $\bar{N}_1$ і $\bar{N}_2$, і $\parallel$ вектору їх векторного добутку. Таким образом, за вектор $\bar{S}$ можна взяти вектор $[\bar{N}_1, \bar{N}_2]$.

$$\bar{S} = [\bar{N}_1, \bar{N}_2] = \begin{vmatrix} \bar{i} & \bar{j} & \bar{k} \\ A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \end{vmatrix}. \quad \text{Тоді рівняння прямої буде мати вигляд:}$$

$$\frac{x - x_0}{\begin{vmatrix} B_1 & C_1 \\ B_2 & C_2 \end{vmatrix}} = \frac{y - y_0}{\begin{vmatrix} C_1 & A_1 \\ C_2 & A_2 \end{vmatrix}} = \frac{z - z_0}{\begin{vmatrix} A_1 & B_1 \\ A_2 & B_2 \end{vmatrix}}.$$

Кут між двома прямими. Умови перетину двох прямих.

Дві прямі в просторі можуть бути паралельними, перетинатися та бути мимобіжними.

Кут θ між двома прямими l_1 і l_2 в просторі вимірюється кутом між їх напрямними векторами

$$(l_1) \frac{x - x_1}{m_1} = \frac{y - y_1}{n_1} = \frac{z - z_1}{p_1}; \quad \bar{S}_1 = \{m_1, n_1, p_1\};$$

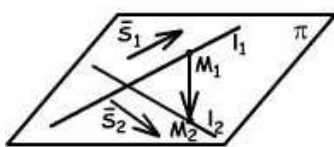
$$(l_2) \frac{x - x_2}{m_2} = \frac{y - y_2}{n_2} = \frac{z - z_2}{p_2}; \quad \bar{S}_2 = \{m_2, n_2, p_2\}.$$

Тоді $(l_1, l_2) = (\bar{S}_1, \bar{S}_2)$;

$$\cos \theta = \pm \frac{\bar{S}_1 \cdot \bar{S}_2}{|\bar{S}_1| \cdot |\bar{S}_2|} = \pm \frac{m_1 m_2 + n_1 n_2 + p_1 p_2}{\sqrt{m_1^2 + n_1^2 + p_1^2} \cdot \sqrt{m_2^2 + n_2^2 + p_2^2}}.$$

$$\text{Умова } l_1 \parallel l_2 \Rightarrow \bar{S}_1 \parallel \bar{S}_2 \Rightarrow \frac{m_1}{m_2} = \frac{n_1}{n_2} = \frac{p_1}{p_2}$$

$$\text{Умова } l_1 \perp l_2 \Rightarrow \bar{S}_1 \perp \bar{S}_2 \Rightarrow m_1 m_2 + n_1 n_2 + p_1 p_2 = 0$$



Якщо дві прямі l_1 і l_2 перетинаються, то вони лежать в одній площині π . У цій площині лежить вектор $\overline{M_1 M_2} = \{x_2 - x_1; y_2 - y_1; z_2 - z_1\}$, що з'єднує задані точки M_1 і M_2 прямих. Тоді вектори $\overline{M_1 M_2}$; $\bar{S}_1$ і $\bar{S}_2$ –

компланарні, а їх змішаний добуток дорівнює нулю: $(\overline{M_1 M_2} \times \bar{S}_1) \cdot \bar{S}_2 = 0$; або

$$\begin{vmatrix} x_2 - x_1 & y_2 - y_1 & z_2 - z_1 \\ m_1 & n_1 & p_1 \\ m_2 & n_2 & p_2 \end{vmatrix} = 0 \quad (12) \text{ – умова перетину двох прямих.}$$

Перетин прямої з площиною.

Задана пряма l : $\frac{x - x_0}{m} = \frac{y - y_0}{n} = \frac{z - z_0}{p}$ і площина π : $Ax + By + Cz + D = 0$.

Параметричні рівняння l : $x = x_0 + mt$; $y = y_0 + nt$; $z = z_0 + pt$. Підставивши значення x, y, z в рівняння площини π , отримаємо:

$$A(x_0 + mt) + B(y_0 + nt) + C(z_0 + pt) + D = 0;$$

$$(A \cdot m + B \cdot n + C \cdot p) \cdot t = -(A \cdot x_0 + B \cdot y_0 + C \cdot z_0 + D) \quad (11)$$

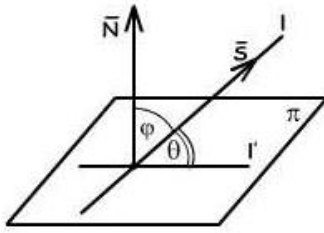
При розв'язанні (11) можливі такі три випадки:

$$\text{а) } Am + Bn + Cp \neq 0, \text{ тоді } t = -\frac{Ax_0 + By_0 + Cz_0 + D}{Am + Bn + Cp} = t_0. \text{ Координати точки}$$

перетину знайдемо, підставивши значення $t = t_0$ в параметричні рівняння l : $x = x_0 + mt_0$; $y = y_0 + nt_0$; $z = z_0 + pt_0$.

б) $Am + Bn + Cp = 0$, а $Ax_0 + By_0 + Cz_0 + D \neq 0$. Тоді (11) розв'язання не має, пряма l не перетинає π .

в) $Am + Bn + Cp = 0$ і $Ax_0 + By_0 + Cz_0 + D = 0$. Рівняння (11) має нескінченну множину розв'язків, пряма лежить l в площині π .



Кут між прямою і площиною.

Кут θ між прямою і площиною вимірюється кутом між прямою і її проекцією на площину:

$$l: \frac{x-x_0}{m} = \frac{y-y_0}{n} = \frac{z-z_0}{p}; \quad \bar{S} = \{m; n; p\};$$

$$\pi: Ax + By + Cz + D = 0; \quad \bar{N} = \{A; B; C\};$$

$$l' - \text{проекція } l \text{ на } \pi. \quad \varphi + \theta = \frac{\pi}{2}; \quad t = t_0$$

$$\cos \varphi = \cos\left(\frac{\pi}{2} - \theta\right) = \sin \theta = \frac{|\bar{S} \cdot \bar{N}|}{|\bar{S}| \cdot |\bar{N}|} \quad \text{або} \quad \sin \theta = \frac{Am + Bn + Cp}{\sqrt{A^2 + B^2 + C^2} \cdot \sqrt{m^2 + n^2 + p^2}}$$

$$\text{Умова } l \parallel \pi \Rightarrow \bar{S} \perp \bar{N} \Rightarrow Am + Bn + Cp = 0; \quad \text{Умова } l \perp \pi \Rightarrow \bar{S} \parallel \bar{N} \Rightarrow \frac{m}{A} = \frac{n}{B} = \frac{p}{C}.$$

Узагальненням поняття прямої на площині та площини в тривимірному просторі є поняття *гіперплощини*.

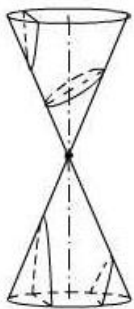
Гіперплощиною в n -вимірному просторі називають геометричне місце точок $(x_1, x_2, \dots, x_n)$, координати яких задовольняють рівняння

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = c$$

Вважатимемо, що ця гіперплощина нормальна до вектора $\bar{N} = (a_1, a_2, \dots, a_n)$ і

що рівнянню $\frac{x_1}{a_1} + \frac{x_2}{a_2} + \dots + \frac{x_n}{a_n} = 1$ відповідає гіперплощина, яка відтинає на координатних осях відрізки $a_1, a_2, \dots, a_n$.

Криві другого порядку, їх форма і канонічні рівняння.



До кривих другого порядку належать такі лінії: окружність, еліпс, гіпербола і парабола, рівняння яких є рівняннями другого ступеня відносно x і y . Їх ще називають конічними перерізами, оскільки їх можна отримати як лінії перетину прямого кругового конуса площинами.

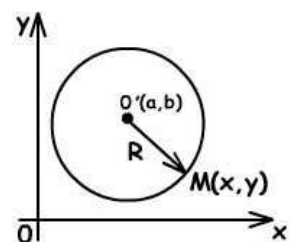
Окружність

Окружністю називають множину всіх точок площини, відстані яких від заданої точки (центра кола) дорівнюють постійному числу (радіусу).

$O'(a, b)$ – центр кола; R – радіус; $M(x, y)$ – поточна точка

$$R = O'M = \sqrt{(x-a)^2 + (y-b)^2}, \quad \text{або} \quad (x-a)^2 + (y-b)^2 = R^2 \quad (1)$$

канонічне рівняння кола.



Якщо центр кола міститься на початку координат, то $a = 0$; $b = 0$ і рівняння окружності має вигляд: $x^2 + y^2 = R^2$.

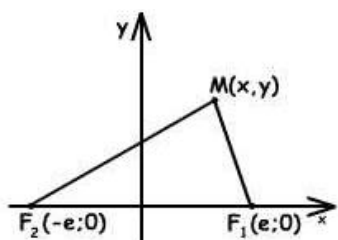
Якщо в рівнянні (1) розкрити дужки і привести подібні члени, отримаємо загальне рівняння окружності: $x^2 + y^2 + Ax + By + C = 0$, де $A = -2a$; $B = -2b$; $C = a^2 + b^2 - R^2$.

Властивості рівняння окружності:

- 1°. це рівняння другого ступеня щодо x і y ;
- 2°. коефіцієнти при квадратах x і y однакові;
- 3°. відсутній член с добутком xy .

Еліпс

Еліпсом називають множину точок площини, сума відстаней яких від двох фіксованих точок площини, які називаються фокусами, є величина постійна.



Нехай F_1 і F_2 – фокуси, M – довільна точка. $F_1M = r_1$ і

$F_2M = r_2$ називають фокальними радіусами.

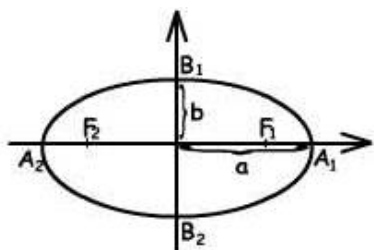
$$F_1M + F_2M = 2a \quad (2)$$

$F_1F_2 = 2c$. Очевидно: $2a > 2c$; $a > c$. $F_1M = \sqrt{(x-c)^2 + y^2}$; $F_2M = \sqrt{(x+c)^2 + y^2}$.

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1 \quad (3) \text{ – канонічне рівняння еліпса.}$$

Дослідимо форму еліпса:

а) оскільки змінні x і y входять в рівняння (3) в квадратах, то еліпс симетричний



відносно осей OY і OX ; б) із (3) будемо мати $\frac{x^2}{a^2} \leq 1$;

$$\frac{y^2}{b^2} \leq 1, \quad |x| \leq a; \quad |y| \leq b; \quad a \text{ і } b \text{ – параметри еліпса}$$

називають малою і великою напівосями відповідно.

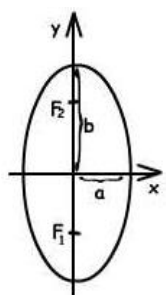
$A_1; A_2; B_1; B_2$ – вершини еліпса; O – центр. Якщо $a = b$, то $x^2 + y^2 = a^2$, тобто,

окружність є окремий випадок еліпса. Форма еліпса залежить від співвідношення $\frac{b}{a}$. Для

характеристики форми еліпса використовують величину $\varepsilon = \frac{c}{a} = \sqrt{\frac{a^2 - b^2}{a^2}} = \sqrt{1 - \frac{b^2}{a^2}}$

($0 < \varepsilon < 1$) яку називають *ексцентриситетом*. При $\varepsilon = 0$ маємо окружність ($b^2 = a^2 - c^2$; $c = 0$; $a = b$).

в) згідно (2) $a^2 = b^2 + c^2$, звідси $F_1 B_1 = F_2 B_1 = a$



г) якщо в рівнянні $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$; $b > a$, тоді

$$F_1(0; -c); F_2(0; c); c^2 = b^2 - a^2; \varepsilon = \frac{c}{b};$$

д) *директрисами* еліпса називають дві прямі, перпендикулярні до фокальної осі еліпса і розміщені симетрично щодо центру еліпса на відстані $\frac{a}{\varepsilon}$ від нього,

або $\frac{b}{\varepsilon}$ від нього. Директриси мають рівняння $x = \pm \frac{a}{\varepsilon}$ (або $y = \pm \frac{b}{\varepsilon}$), вони еліпс не перетинають.

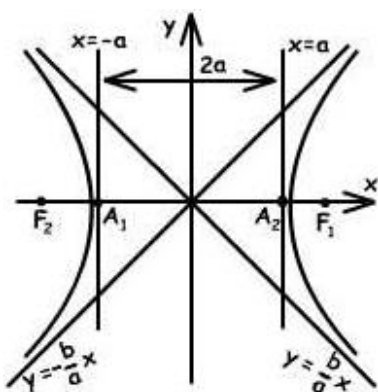
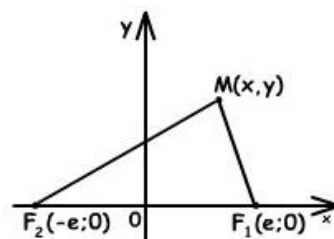
Гіпербола

Гіперболою називають множину точок площині, модуль різниці відстаней яких від двох фіксованих точок площини, які називаються фокусами, є величина постійна.

Нехай $M(x, y)$ – довільна точка гіперболи, F_1, F_2 – її фокуси. $F_1 M = r_1$ і $F_2 M = r_2$ називають фокальними радіусами. Відстань $F_1 F_2 = 2c$.

Позначимо $F_1 M - F_2 M = \pm 2a$, ($+2a$, якщо $F_1 M > F_2 M$; $-2a$, якщо $F_1 M < F_2 M$). Очевидно, що $2a < 2c$, тобто $a < c$,

$$F_1 M = \sqrt{(x - c)^2 + y^2}; \quad F_2 M = \sqrt{(x + c)^2 + y^2}.$$



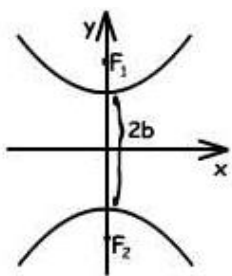
$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1 \quad (4) \quad \text{— канонічне рівняння гіперболи.}$$

Дослідимо форму гіперболи:

а) оскільки x і y входять в рівняння (4) в квадратах, то гіпербола симетрична відносно осей координат. Вісь Ox вона перетинає в двох точках $A_1(a, 0)$ і $A_2(-a, 0)$ — вершинах; a, b — відповідно дійсна і уявна півосі гіперболи.

б) із рівняння (4) будемо мати: $\frac{x^2}{a^2} \geq 1$, $x^2 \geq a^2$, або $|x| \geq a$. Всередині смуги, що

обмежена двома паралельними прямими $x = \pm a$, немає точок гіперболи.



в) гіперболу $\frac{x^2}{a^2} - \frac{y^2}{b^2} = -1$, або $\frac{y^2}{b^2} - \frac{x^2}{a^2} = 1$ називають

спряженою. $F_1(0, c)$; фокуси F_1 і F_2 лежать на осі Oy .

$F_2(0, -c)$; вісь Oy буде дійсною, а Ox – уявною.

г) *асимптотою* кривої називають таку пряму, що точка, яка віддаляється по кривій в нескінченність, необмежено наближається до цієї прямої. Прямі

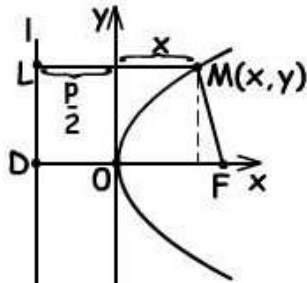
$y = \pm \frac{b}{a} x$ є асимптотами гіперболи. Асимптоти гіперболи значною мірою характеризують її форму.

д) *ексцентриситетом* гіперболи називають відношення відстані між її фокусами до

довжини її дійсної осі: $\varepsilon = \frac{2c}{2a} = \frac{c}{a} > 1$, де $c > a$. Оскільки $c^2 = a^2 + b^2$, то $\varepsilon = \frac{c}{a} = \sqrt{1 + \frac{b^2}{a^2}}$.

Парабола

Параболою називають множину точок площини, відстань яких від фіксованої точки площини, яку називають *фокусом*, дорівнює відстані від фіксованої прямої, яка називається *директрисою*.



$DO = OF$. Позначим $DF = p$ (p – параметр). Тоді

$F\left(\frac{p}{2}; 0\right)$ – фокус параболи, $x = -\frac{p}{2}$ – директриса. За

означенням $FM = ML$.

$y^2 = 2px$ – канонічне рівняння параболи. O – вершина

параболи; оскільки y^2 , парабола симетрична відносно осі Ox . Парабола $x^2 = 2py$ симетрична відносно осі Oy .

Поверхні другого порядку

Рівняння $F(x, y, z) = 0$ визначає в просторі $OXYZ$ деяку поверхню, тобто геометричне місце точок, координати яких x, y, z задовольняють цьому рівнянню. Це рівняння називається рівнянням поверхні, x, y, z – поточними координатами.

Поверхні другого порядку – це поверхні, рівняння яких у прямокутній декартовій системі координат є рівняннями другого порядку.

Існує три типи поверхонь другого порядку:

1. Центральні поверхні другого порядку.
2. Параболоїди.

3. Конічні та циліндричні поверхні другого порядку.

Крім того, окремим випадком кожного із зазначених типів поверхонь може бути *Поверхня обертання*.

Центральні поверхні другого порядку у свою чергу діляться на три типи:

1. еліпсоїд (дійсний або уявний);
2. однопорожнинний гіперболоїд;
3. двопорожнинний гіперболоїд.

Існує два види *параболоїдів*: еліптичний та гіперболічний. Параболоїди не мають центрів симетрії.

Конічною поверхнею називають поверхню, утворену рухом прямої (твірної), яка проходить через задану точку (вершину конуса) і перетинає задану криву.

Циліндричні поверхні – це поверхні другого порядку, утворені рухом прямої (твірної), яка перетинає задану криву (напрямну), залишаючись паралельною постійному вектору.

Матриці

Матрицею розміру $m \times n$ називається прямокутна таблиця, що складається з m рядків і n стовпців:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{pmatrix}$$

Числа a_{ij} називаються *елементами матриці*. Перший індекс фіксує номер рядка, а другий - номер стовпця, в яких розташований даний елемент. Якщо $m = n$, тобто число стовпців матриці дорівнює числу рядків, то матриця називається *квадратною*. Елементи a_{ii} утворюють *головну діагональ* матриці. Можна користуватися скороченою формою запису:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} = \|a_{ij}\|_{mn}$$

Сума елементів, що стоять на головній діагоналі квадратної матриці, називається її *слідом*.

Матриця, що складається з одного рядка чи одного стовбця, називається відповідно *вектор-рядком* або *вектор-стовпцем*. Вектор-стовпці і вектор-рядки називають просто *векторами*. Матриця, що складається з одного числа, ототожнюється з цим числом.

Приклади матриць: $C = \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix}; m = 3, n = 1,$ матриця-стовбець;

$A = (a), m = 1, n = 1,$ матриця з одного елемента. $B = (b_1 \quad b_2 \quad b_3); m = 1, n = 3,$ матриця-рядок (вектор).

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}; \quad m = n = 3, \text{ квадратна матриця.}$$

Квадратна матриця називається *трикутною*, якщо всі елементи, які стоять вище

(або нижче) головної діагоналі, дорівнюють нулю $\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{pmatrix}$. Квадратні

матриці, які мають відмінні від нуля лише елементи головної діагоналі, називаються *діагональними матрицями* і записуються так: $A = \begin{pmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{pmatrix}$.

Якщо ці елементи однакові $a_{11} = a_{22} = \dots = a_{nn} = \lambda$, то матриця називається *скалярною*. Якщо відмінні від нуля елементи діагональної матриці дорівнюють 1, то матриця називається *одиничною* і позначається буквою E : $E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Матриця, всі елементи якої дорівнюють 0, називається *нульовою*: $O = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$.

Транспонуванням називається таке перетворення матриці, при якому рядки і стовпці міняються місцями зі збереженням їх номерів. Позначається транспонування значком T нагорі. Для матриці $A = (a_{ij})$, $i = \overline{1, m}$; $j = \overline{1, n}$ транспонованою є матриця $A^T = (a_{ji})$.

Зокрема, при транспонуванні вектор-стовпця отримаємо вектор-рядок і навпаки. Матриця називається *симетричною*, якщо вона не змінюється при транспонуванні ($A = A^T$). В симетричній матриці елементи, симетричні відносно головної діагоналі, рівні. Дві матриці однакової розмірності $m \times n$ називаються рівними, якщо в них однакові місця зайняті рівними числами (на перетині i -го рядка і j -го стовпця в одній і в іншій матриці знаходиться одне і те ж число).

Дії над матрицями та їх властивості.

1) Матриці однакового розміру $m \times n$ можна додавати і віднімати за правилом $(c_{ij})_{mn} = (a_{ij})_{mn} \pm (b_{ij})_{mn}$, де a_{ij} і b_{ij} – елементи матриць-доданків, $c_{ij} = a_{ij} + b_{ij}$. Матриці різного розміру додавати не можна.

2) Матрицю довільного розміру можна помножити на число за правилом : $\lambda A = (\lambda \cdot a_{ij})_{mn}$.

Властивості додавання і множення на число

1°. $A + B = B + A$; (комутативна властивість)

2°. $(A + B) + C = A + (B + C)$; (асоціативна властивість)

3°. $\alpha(A + B) = \alpha A + \alpha B$; (дистрибутивна)

4°. $(\alpha + \beta) \cdot A = \alpha A + \beta A$; (дистрибутивна)

5°. $\alpha(\beta \cdot A) = (\alpha \cdot \beta) \cdot A$; α, β – числа.

3) Добуток матриць.

Добуток $C = A \cdot B$ можливий тільки в тому випадку, якщо число стовпців матриці A дорівнює числу рядків матриці B . Якщо A – матриця порядку $m \times n$, B – порядку $n \times k$, то матриця–добуток $C = (c_{ij})$ буде порядку $m \times k$, елементи її знаходяться за формулою: $c_{ij} = a_{i1} \cdot b_{1j} + a_{i2} \cdot b_{2j} + \dots + a_{in} \cdot b_{nj}$; $(i = \overline{1, m})$ або скорочено: $c_{ij} = \sum_{\alpha=1}^n a_{i\alpha} \cdot b_{\alpha j}$; $(i = \overline{1, m}; j = \overline{1, k})$. З наведеного визначення видно, що операція множення матриць визначена тільки для матриць, число стовпців першої з яких дорівнює числу рядків другої.

Взагалі $AB \neq BA$. Якщо в деяких випадках для квадратних матриць виконується $AB = BA$, то їх називають *комутуючими* або *переставними*. Найхарактернішим прикладом може служити одинична матриця, яка є переставною з будь-якою іншою матрицею того ж розміру.

Переставними можуть бути тільки квадратні матриці одного і того ж порядку. $AE = EA = A$.

Для будь-яких матриць відповідної розмірності виконується наступна властивість: $AO = OA = O$, O – нульова матриця.

Властивості добутку матриць:

1° $A(BC) = (A \cdot B) \cdot C = ABC$;

2°
$$\begin{cases} A \cdot (B + C) = A \cdot B + A \cdot C \\ (A + B) \cdot C = A \cdot C + B \cdot C \end{cases}$$

3°. $(AB)^T = B^T A^T$

Визначники другого та третього порядку.

Визначником або *детермінантом* квадратної матриці називається число або вираз, яке ставиться у відповідність матриці і може бути складено з її елементів.

Визначник матриці другого порядку $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ (за числом рядків і стовпців)

записується: $\Delta_2 = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$ і обчислюється за формулою $\Delta = \det A = a_{11}a_{22} - a_{21}a_{12}$.

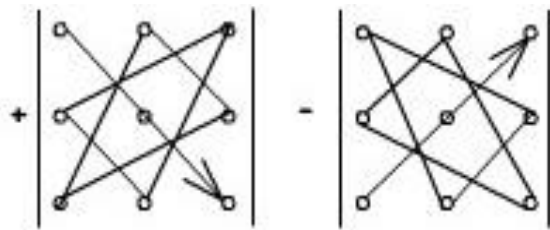
Елементи a_{11}, a_{22} вважаються елементами *головної діагоналі*, елементи a_{21}, a_{12} – елементами *побічної діагоналі*. Поняття головна і побічна діагональ мають місце і

для визначників вищих порядків. $\Delta = \begin{vmatrix} 5 & 7 \\ 3 & 11 \end{vmatrix} = 5 \cdot 11 - 3 \cdot 7 = 55 - 21 = 34$

Поняттям визначник можна користуватися і без зв'язку його з відповідною матрицею. Визначником (детермінантом – $\det$) третього порядку називають число, яке зображують у вигляді таблиці

$$\Delta = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

і обчислюють за допомогою правила трикутника (правила Саррюса) або за допомогою правила таблиці.



Правило таблиці описує ту ж саму формулу: добутки елементів, розташованих на головній діагоналі і на діагоналях, їй паралельних, беруться зі знаком «+», а добутки елементів, розташованих на побічній діагоналі і на діагоналях, паралельних їй, беруться зі знаком «-».

Властивості визначників.

Властивості визначників доводяться розкриттям визначників і порівнянням результатів.

1. Величина визначника не зміниться від заміни всіх його рядків відповідними стовпцями і навпаки. Таке перетворення називають транспонуванням визначника.

2. Якщо у визначнику поміняти місцями два рядки (або два стовпці), то визначник змінить знак на протилежний.

3. Якщо елементи двох рядків (або стовпчиків) однакові, то визначник дорівнює нулю.

4. Якщо помножити всі елементи рядка (або стовпця) на одне й те саме число k , то значення визначника помножиться на це ж число k .

Звідси випливає – загальний множник елементів одного рядка (стовпця), можна винести за знак визначника.

5. Якщо кожен елемент якого-небудь рядка (стовпчика) можна представити у вигляді суми двох доданків, то цей визначник можна представити у вигляді суми двох визначників, у першому з яких відповідний рядок (стовпець) складається з перших доданків, а в другому – з других доданків:

$$\begin{vmatrix} a_{11}^1 + a_{11}^2 & a_{12} & a_{13} \\ a_{21}^1 + a_{21}^2 & a_{22} & a_{23} \\ a_{31}^1 + a_{31}^2 & a_{32} & a_{33} \end{vmatrix} = \begin{vmatrix} a_{11}^1 & a_{12} & a_{13} \\ a_{21}^1 & a_{22} & a_{23} \\ a_{31}^1 & a_{32} & a_{33} \end{vmatrix} + \begin{vmatrix} a_{11}^2 & a_{12} & a_{13} \\ a_{21}^2 & a_{22} & a_{23} \\ a_{31}^2 & a_{32} & a_{33} \end{vmatrix}$$

6. Якщо всі елементи рядка (стовпчика) визначника дорівнюють нулю, то визначник також дорівнює нулю.

7. Якщо визначник має пропорційні рядки (стовпці), то він дорівнює нулю.

8. Величина визначника не зміниться, якщо до всіх елементів рядка (або стовпця) додати елементи іншого рядка (або стовпця), помножені на одне і те ж число.

9. Для того щоб визначник дорівнював нулю необхідно і достатньо, щоб один його рядок (стовпець) був лінійною комбінацією інших рядків (стовпців).

Визначники вищих порядків. Мінори та алгебраїчні доповнення.

Визначники вищих порядків відповідають квадратним матрицями n -го порядку.

$$\Delta = \det = \begin{vmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{vmatrix}$$

Визначення: Мінором M_{ij} деякого елемента a_{ij} називають визначник $(n-1)$ -го порядку, який може бути отриманий з визначника n -го порядку в результаті викреслювання i -го рядка і j -го стовпця, що містять цей елемент.

Наприклад, мінором M_{32} визначника третього порядку є визначник

$$M_{32} = \begin{vmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{vmatrix}.$$

Визначення: Алгебраїчним доповненням елемента a_{ij} визначника n -го порядку називається його мінор, помножений на $(-1)^{i+j}$, де i – номер рядка, а j – номер стовпця, на перетині яких знаходиться a_{ij} .

Алгебраїчним доповненням до елемента a_{32} буде $A_{32} = (-1)^{3+2} M_{32} = - \begin{vmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{vmatrix}$.

Таким чином M_{ij} і A_{ij} можуть відрізнятися один від іншого лише знаком.

Продовжимо викладення властивостей визначників.

10. *Теорема Лапласа.* Сума добутків елементів будь-якого рядка (або стовпця) на їх алгебраїчні доповнення не залежить від номера ряду і дорівнює цьому визначнику. Ця властивість може бути використана у якості визначення поняття

визначник: $\det A = \sum_{k=1}^n (-1)^{k+1} a_{1k} M_{1k}$.

11. Сума добутків елементів будь-якого рядка (або стовпця) визначника на алгебраїчні доповнення елементів іншого рядка (або стовпця) дорівнює нулю.

Методи обчислення визначників вищих порядків.

1. Метод ефективного зниження порядку.

У відповідності з властивостями визначників обчислення визначника n -го порядку зводиться до обчислення n визначників $(n-1)$ -го порядку. Однак, якщо всі елементи рядка (стовпця), що розкладаємо, крім одного, дорівнюють нулю, то обчислювати доведеться тільки один визначник $(n-1)$ -го порядку. Хоча в заданому визначнику може не виявитися рядка з потрібною кількістю нулів, проте, завжди можна, не змінюючи величини визначника, перетворити його так, щоб в вибраному рядку (стовпці) всі елементи, крім одного, виявилися нулями.

2. Приведення визначника до трикутного вигляду.

Визначник, у якого всі елементи, що знаходяться нижче або вище головної діагоналі, дорівнюють нулю, називається визначником трикутного вигляду. Вочевидь, що в цьому випадку визначник дорівнює добутку елементів його головної діагоналі. Приведення будь-якого визначника до трикутного вигляду завжди можливе.

Обернена матриця та умови її існування.

Визначення 1. Квадратна матриця A називається *невиродженою*, якщо $\det A \neq 0$.

Визначення 2. Матриця A^{-1} називається *оберненою* до невивродженої матриці A , якщо $A \cdot A^{-1} = A^{-1} \cdot A = E$, де E — одинична матриця.

Зрозуміло, що A^{-1} — квадратна матриця того ж розміру, що і матриця A . Будь-яка невивроджена матриця A має обернену, яку можна знайти за формулою

$$A^{-1} = \frac{1}{\det A} \cdot A^*.$$

Матриця $A^* = \begin{pmatrix} A_{11} & A_{21} & \dots & A_{n1} \\ A_{12} & A_{22} & \dots & A_{n2} \\ \dots & \dots & \dots & \dots \\ A_{1n} & A_{2n} & \dots & A_{nn} \end{pmatrix} = (A_{ij}), \quad i, j = \overline{1, n}$ є транспонованою матрицею

алгебраїчних доповнень A_{ij} до елементів a_{ij} матриці A . Вона називається *приєднаною* або *союзною*. Для знаходження оберненої матриці використовується алгоритм:

1. Переконаємося, що матриця квадратна.
2. Обчислюємо визначник $\det A$, якщо він $\neq 0$, то
3. Замість кожного елемента ставимо його алгебраїчне доповнення (п.3 і 4 можна поміняти місцями).
4. Транспонуємо отриману матрицю.
5. Кожен елемент отриманої матриці ділимо на визначник вихідної матриці і отримуємо обернену до неї.

Властивості оберненої матриці:

1°. Якщо для матриці A існує обернена A^{-1} , то вона буде єдиною.

2°. $\det A^{-1} = \frac{1}{\det A}.$

3°. $(A \cdot B)^{-1} = B^{-1} \cdot A^{-1}.$

Обернену матрицю можна знаходити іншим способом. Якщо до вихідної матриці приписати справа одиничну і за допомогою елементарних перетворень над рядками зведеної матриці привести A («ліву половину») до одиничної матриці, то на місці спочатку приписаної матриці E (праворуч) виявиться матриця A^{-1} . В процесі перетворень слід використовувати тільки рядки. Якщо матрицю A не можна перетворити до одиничної матриці, то A^{-1} не існує.

Розглянемо квадратну матрицю A порядку n . Нехай X, Y – вектори-стовпці розміру n . Розглянемо лінійне відображення R^n у R^n : $Y = AX$ (1)

У загальному випадку вектор Y відмінний від вектора X як за довжиною, так і за напрямом.

Приклад. Розглянемо відображення R^2 у R^2 :

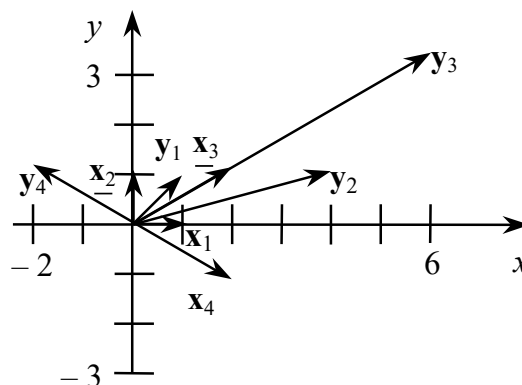
$$Y = AX, A = \begin{pmatrix} 1 & 4 \\ 1 & 1 \end{pmatrix}, X = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, Y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}.$$

Узявши $X_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, X_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, X_3 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, X_4 = \begin{pmatrix} 2 \\ -1 \end{pmatrix},$

знайдемо відповідні вектори

$$Y_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, Y_2 = \begin{pmatrix} 4 \\ 1 \end{pmatrix}, Y_3 = \begin{pmatrix} 6 \\ 3 \end{pmatrix}, Y_4 = \begin{pmatrix} -2 \\ 1 \end{pmatrix}.$$

Зобразимо ці вектори на координатній площині:



Вектори X_1 і X_2 відображаються у вектори Y_1 і Y_2 , які відрізняються від векторів X_1 і X_2 довжиною та напрямом.

Вектори X_3 і X_4 відображаються у вектори Y_3 і Y_4 , які збігаються з векторами X_3 і X_4 за напрямом. Такі вектори відіграють важливу роль у теорії матриць.

Означення. Ненульовий вектор X називається **власним вектором** матриці A , якщо існує число λ , таке що $AX = \lambda X$ (2)

Число λ називається **власним числом** матриці A , що відповідає власному вектору X .

Отже, множення матриці на власний вектор рівносильне множенню числа λ на цей вектор.

Систему рівнянь (2) можна записати у вигляді

$$AX = \lambda EX, \quad (\lambda E - A)X = 0 \quad (3)$$

Ця система рівнянь визначає власний вектор з точністю до довільного ненульового множника. Якщо X – власний вектор, то cX , $c = \text{const} \neq 0$ – також власний вектор.

Власний вектор X є ненульовим розв'язком системи лінійних алгебраїчних рівнянь (3) з визначником $\det(\lambda E - A) \equiv f(\lambda) = \lambda^n + b_1\lambda^{n-1} + \dots + b_n$ (4)

Для того щоб однорідна система (3) мала ненульовий розв'язок, необхідно і достатньо, щоб її визначник дорівнював нулю, тобто:

$$f(\lambda) = \lambda^n + b_1\lambda^{n-1} + \dots + b_n = 0. \quad (5)$$

Це рівняння називається **характеристичним рівнянням** матриці A , а його корені – **власними числами** матриці A . Якщо відоме власне число λ , то із системи (3) можна знайти відповідний власний вектор X . З рівняння (4) дістанемо:

$$\det A = (-1)^n b_n = \lambda_1 \lambda_2 \dots \lambda_n,$$

тобто визначник квадратної матриці дорівнює добутку її власних чисел. Отже, матриця є особливою, якщо вона має нульове власне число.

Приклад. Знайти власні числа і власні вектори матриці $A = \begin{pmatrix} 1 & 4 \\ 1 & 1 \end{pmatrix}$.

Запишемо характеристичне рівняння

$$\det(\lambda E - A) = \begin{vmatrix} \lambda - 1 & -4 \\ -1 & \lambda - 1 \end{vmatrix} = \lambda^2 - 2\lambda - 3 = 0,$$

яке має корені $\lambda_1 = -1$, $\lambda_2 = 3$. При $\lambda = -1$ система (3) набуває вигляду

$$\begin{cases} -2x_1 - 4x_2 = 0, \\ -x_1 - 2x_2 = 0, \end{cases}$$

а при $\lambda = 3$ маємо систему рівнянь

$$\begin{cases} 2x_1 - 4x_2 = 0, \\ -x_1 - 2x_2 = 0. \end{cases}$$

Ці системи рівнянь визначають власні вектори матриці A :

$$X_1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad X_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

Справді,

$$AX_1 = \begin{pmatrix} 1 & 4 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} -2 \\ 1 \end{pmatrix} = -1X_1, \quad AX_2 = \begin{pmatrix} 1 & 4 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 3 \end{pmatrix} = 3X_2,$$

звідки випливає, що X_1, X_2 – власні вектори матриці A .

Наведемо деякі важливі властивості власних векторів і чисел.

Теорема 1. Якщо всі власні числа матриці різні, то всі її власні вектори лінійно незалежні.

Наслідок. Якщо всі власні числа матриці A порядку n різні, то матриця

$$T = (X_1, X_2, \dots, X_n),$$

стовпцями якої є власні вектори $X_1, X_2, \dots, X_n$ матриці A , неособлива і має

обернену матрицю $T^{-1} = \begin{pmatrix} Y_1 \\ Y_2 \\ \dots \\ Y_n \end{pmatrix}$

Теорема 2. Якщо матриця A порядку n має власні числа $\lambda_1, \lambda_2, \dots, \lambda_n$ і відповідні власні вектори $X_1, X_2, \dots, X_n$, то матрицю A можна подати у вигляді

$$A = \lambda_1 X_1 Y_1 + \lambda_2 X_2 Y_2 + \dots + \lambda_n X_n Y_n.$$

Системи лінійних алгебраїчних рівнянь (СЛАР)

Систему m лінійних рівнянь з n невідомими будемо записувати в наступному вигляді:

[illegible]

Тут $x_1, x_2, \dots, x_n$ – невідомі величини, a_{ij} ($i=1,2,\dots,m$; $j=1,2,\dots,n$) – числа, які називаються коефіцієнтами системи (перший індекс фіксує номер рівняння, другий – номер невідомої), $b_1, b_2, \dots, b_m$ – числа, які називаються *вільними членами*.

Розв'язком системи називається упорядкований набір чисел $x_1, x_2, \dots, x_n$, який обертає кожне рівняння системи в правильну рівність. *Розв'язати систему* - значить знайти всі її розв'язки або довести, що жодного розв'язку немає. Система, що має розв'язки, називається *сумісною*. Якщо система не має розв'язків, то вона називається *несумісною*. Якщо система має тільки один розв'язок, вона називається *визначеною*. Система, яка має більше одного розв'язку, називається *невизначеною* (сумісною і невизначеною). Система, у якій всі вільні члени дорівнюють нулю ($b_1 = b_2 = \dots = b_m = 0$), називається *однорідною*. Однорідна система завжди сумісна, так як набір з n нулів задовольняє будь-якому рівнянню такої системи. Якщо число рівнянь системи збігається з числом невідомих ($m = n$), то система називається *квадратною*. Дві системи, множини розв'язків яких збігаються, називаються *еквівалентними* або *рівносильними*. Дві несумісні системи вважаються еквівалентними.

Перетворення, застосування якого перетворює систему в нову систему, еквівалентну початковій, називається *еквівалентним* або *рівносильним перетворенням*. Прикладами еквівалентних перетворень можуть бути такі перетворення: перестановка місцями двох рівнянь системи, перестановка місцями двох невідомих разом з коефіцієнтами у всіх рівняннях, множення обох частин будь-якого рівняння системи на відмінне від нуля число.

Використовуючи поняття добутку матриць, можна переписати систему (1) у

вигляді: $AX = B$ (2), де $A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} = \|a_{ij}\|_{mn}$ – матриця, що

складається з коефіцієнтів при невідомих системи (1), яка називається *матрицею*

системи; $X = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{pmatrix} = (x_1, x_2, \dots, x_n)^T$; $B = \begin{pmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{pmatrix} = (b_1, b_2, \dots, b_m)^T$ – вектори-стовпці, що

складаються відповідно з невідомих x_j та із вільних членів b_i .

Матриця $A_p = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \dots & \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} & b_m \end{pmatrix}$, яка утворена шляхом приписування

справа до матриці A стовпця вільних членів, називається *розширеною матрицею системи*.

Питання про сумісність системи (1) вирішується наступною теоремою.

Теорема Кронекера-Капеллі. Система лінійних рівнянь сумісна тоді і тільки тоді, коли ранги матриць A і A_p збігаються, тобто $r(A) = r(A_p) = r$.

Чисельні методи розв'язання СЛАР

Методи розв'язання систем лінійних рівнянь діляться на дві групи – прямі і ітераційні. Прямі методи використовують кінцеві співвідношення (формули) для обчислення невідомих. Вони дають рішення після виконання заздалегідь відомого числа операцій. Ці методи порівняно прості і найбільш універсальні, тобто придатні для розв'язання широкого класу лінійних систем. До прямих методів відносять формули Крамера, метод Жордана-Гаусса та розв'язання СЛАР за допомогою зворотної матриці.

Ітераційні методи – це методи послідовних наближень. У них необхідно задати деяке наближене рішення – початкове наближення. Після цього за допомогою деякого алгоритму проводиться один цикл обчислень, що називається ітерацією. В результаті ітерації знаходять нове наближення. Ітерації проводяться до отримання

рішення з необхідною точністю. Алгоритми рішення лінійних систем з використанням ітераційних методів зазвичай складніші в порівнянні з прямими методами. Об'єм обчислень заздалегідь визначити важко.

Проте ітераційні методи у ряді випадків прийнятніше. Вони вимагають зберігання в пам'яті машини не усієї матриці системи, а лише декількох векторів з n компонентами. Іноді елементи матриці можна зовсім не зберігати, а обчислювати їх в міру необхідності. Погрішності остаточних результатів при використанні ітераційних методів не накопичуються, оскільки точність обчислень в кожній ітерації визначається лише результатами попередньої ітерації і практично не залежить від раніше виконаних обчислень. Ці достоїнства ітераційних методів роблять їх особливо корисними у разі великого числа рівнянь, а також погано обумовлених систем. Слід зазначити, що при цьому збіжність ітерацій може бути дуже повільною; тому шукаються ефективні шляхи її прискорення.

Метод простої ітерації

Цей метод широко використовується для чисельного розв'язання рівнянь і їх систем різних видів. Розглянемо застосування методу простої ітерації до розв'язання систем лінійних алгебраїчних рівнянь.

Запишемо початкову систему рівнянь у векторно-матричному виді (2) і виконаємо ряд тотожних перетворень:

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b}; \mathbf{0} = \mathbf{b} - \mathbf{Ax}; \mathbf{x} = \mathbf{b} - \mathbf{Ax} + \mathbf{x}; \\ \mathbf{x} &= (\mathbf{b} - \mathbf{Ax})\tau + \mathbf{x}; \mathbf{x} = (\mathbf{E} - \tau\mathbf{A})\mathbf{x} + \tau\mathbf{b}; \\ \mathbf{x} &= \boldsymbol{\alpha}\mathbf{x} + \boldsymbol{\beta}, \end{aligned} \tag{3}$$

де $\tau \neq 0$ – деяке число, $\mathbf{E}$ — одинична матриця, $\boldsymbol{\alpha} = \mathbf{E} - \tau\mathbf{A}$, $\boldsymbol{\beta} = \tau\mathbf{b}$. Отримана система (3) еквівалентна початковій системі і служить основою для побудови методу простої ітерації.

Виберемо деяке початкове наближення $\mathbf{x}^{(0)}$ і підставимо його в праву частину системи (3): $\mathbf{x}^{(1)} = \boldsymbol{\alpha}\mathbf{x}^{(0)} + \boldsymbol{\beta}$.

Оскільки $\mathbf{x}^{(0)}$ не є рішенням системи, в лівій частині (3) знайдеться деякий стовпець $\mathbf{x}^{(1)}$, в загальному випадку відмінний від $\mathbf{x}^{(0)}$. Отриманий стовпець $\mathbf{x}^{(1)}$ розглядатимемо в якості наступного (першого) наближення до розв'язку. Аналогічно, по відомому k -му наближенню можна знайти $(k + 1)$ -е наближення:

$$\mathbf{x}^{(k+1)} = \boldsymbol{\alpha}\mathbf{x}^{(k)} + \boldsymbol{\beta}. \quad k = 0, 1, 2, \dots \tag{4}$$

Формула (4) і виражає собою метод простої ітерації. Для її застосування треба задати невизначений доки параметр τ . Від значення τ залежить, чи сходиться метод, а якщо

Припустимо, що діагональні елементи a_{11} , a_{22} , a_{33} відмінні від нуля (інакше можна переставити рівняння). Виразимо невідомі x_1 , x_2 і x_3 відповідно з першого, другого і третього рівнянь системи (5):

$$x_1 = \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3), \quad (6)$$

$$x_2 = \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3), \quad (7)$$

$$x_3 = \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2). \quad (8)$$

Задамо деякі початкові (нульові) наближення значень невідомих: $x_1 = x_1^{(0)}$, $x_2 = x_2^{(0)}$, $x_3 = x_3^{(0)}$. Підставляючи ці значення в праву частину виразу (34), отримуємо нове (перше) наближення для x_1 :

$$x_1^{(1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(0)} - a_{13}x_3^{(0)}).$$

Використовуючи це значення для x_1 і наближення $x_3^{(0)}$ для x_3 , знаходимо з (6) перше наближення для x_2 :

$$x_2^{(1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(1)} - a_{23}x_3^{(0)}).$$

І нарешті, використовуючи вичислені значення $x_1 = x_1^{(1)}$, $x_2 = x_2^{(1)}$ знаходимо з допомогою виразу (36) перше наближення для x_3 :

$$x_3^{(1)} = \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(1)} - a_{32}x_2^{(1)}).$$

На цьому закінчується перша ітерація розв'язання системи (6) - (8). Використовуючи тепер значення $x_1^{(1)}$, $x_2^{(1)}$, $x_3^{(1)}$ можна таким же способом провести другу ітерацію, в результаті якої будуть знайдені другі наближення до розв'язку: $x_1^{(2)}$, $x_2^{(2)}$, $x_3^{(2)}$ і т. д.

Наближення з номером k можна вичислити, знаючи наближення з номером $k - 1$, як:

$$\begin{aligned} x_1^{(k)} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k-1)} - a_{13}x_3^{(k-1)}), \\ x_2^{(k)} &= \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k-1)}), \\ x_3^{(k)} &= \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)}). \end{aligned}$$

Ітераційний процес триває до тих пір, поки значення $x_1^{(k)}$, $x_2^{(k)}$, $x_3^{(k)}$ не стануть близькими із заданою похибкою до значень $x_1^{(k-1)}$, $x_2^{(k-1)}$, $x_3^{(k-1)}$.

Розглянемо тепер систему n лінійних рівнянь з n невідомими. Запишемо її у вигляді:

$$\begin{aligned} a_{i1}x_1 + \dots + a_{i,i-1}x_{i-1} + a_{ii}x_i + a_{i,i+1}x_{i+1} + \dots + a_{in}x_n &= b_i, \\ i &= 1, 2, \dots, n. \end{aligned}$$

Тут також будемо, припускати, що усі діагональні елементи відмінні від нуля. Тоді відповідно до методу Гауса-Зейделя k -е наближення до рішення можна представити у виді

$$\begin{aligned} x_i^{(k)} &= \frac{1}{a_{ii}} \left(b_i - a_{i1}x_1^{(k)} - \dots - a_{i,i-1}x_{i-1}^{(k)} - \right. \\ &\quad \left. - a_{i,i+1}x_{i+1}^{(k-1)} - \dots - a_{in}x_n^{(k-1)} \right), \quad i = 1, 2, \dots, n. \end{aligned} \quad (9)$$

Ітераційний процес триває до тих пір, поки усі значення $x_i^{(k)}$ не стануть близькими $x_i^{(k-1)}$.

Для збіжності ітераційного процесу (9) достатньо, щоб модулі діагональних коефіцієнтів для кожного рівняння системи були не менше сум модулів усіх інших коефіцієнтів (переважання діагональних елементів):

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|, \quad i = 1, 2, \dots, n. \quad (10)$$

При цьому хоч би для одного рівняння нерівність повинна виконуватися строго. Ці умови є достатніми для збіжності методу, але вони не є необхідними, тобто для деяких систем ітерації сходяться і при порушенні умов (10).

Лінійний векторний простір та його властивості

Відволікаючись від геометричної інтерпретації вектора і розглядаючи тільки його координатну форму, можна ввести *визначення*: арифметичним n -вимірним *вектором* називається будь-яка послідовність з n дійсних чисел $a_1, a_2, \dots, a_n$. Позначається $\bar{a} = \{a_1, a_2, \dots, a_n\}$. Числа $a_1, a_2, \dots, a_n$, що задають вектор $\bar{a}$, називаються *координатами* цього вектора або його *компонентами*. Кількість компонент вектора називається його *розмірністю*. Компоненти вектора не можна міняти місцями. Множина всіх n -вимірних арифметичних векторів, в якій введено операції додавання векторів і множення вектора на число, називається *арифметичним n -мірним векторним простором* і позначається R^n .

Для R^n можна узагальнити поняття лінійної залежності векторів:

Теорема 1. Будь-яка система векторів має хоч би один базис. Кількість елементів в будь-якому базисі даної системи одна і та сама. Координати будь-якого вектора в базисі визначаються однозначно.

Теорема 2. В просторі R^n будь-яка система векторів в кількості більше за n є лінійно залежною.

Теорема 3. Лінійно незалежна система векторів в просторі R^n є базисом тоді і тільки тоді, коли кількість цих векторів дорівнює n .

Приклад. Базис в просторі R^n утворюють вектори $\bar{e}_1 = \{1; 0; \dots; 0\}$, $\bar{e}_2 = \{0; 1; \dots; 0\}$, $\dots$, $\bar{e}_n = \{0; 0; \dots; 1\}$.

Взагалі, якщо залишити осторонь природу конкретних об'єктів і ввести аксіоматично визначені дві операції, які мають певні властивості, то можна побудувати загальну теорію, що узагальнює конкретні випадки. Таким чином і виникло поняття лінійного простору як безпосереднє узагальнення дво- та тривимірних векторних просторів.

Лінійним (векторним) простором L над полем K називається множина деяких елементів, на якій введено операції додавання і множення на скаляр (елемент поля K), які не виводять за межі L :

$x, y \in L, \alpha \in K \Rightarrow x + y \in L, \alpha x \in L$. Для будь-яких елементів $x, y, z \in L$ і будь-яких чисел $\alpha, \beta \in K$ виконуються такі аксіоми:

- 1) $x + y = y + x$ – комутативність додавання;
- 2) $(x + y) + z = x + (y + z)$ – асоціативність додавання;
- 3) $(\alpha \beta)x = \alpha(\beta x)$ – асоціативність множення на скаляр;
- 4) $(\alpha + \beta)x = \alpha x + \beta x$ – дистрибутивність відносно скалярів;
- 5) $\alpha(x + y) = \alpha x + \alpha y$ – дистрибутивність відносно векторів;
- 6) існує нульовий елемент $\bar{0} \in L$: $x + \bar{0} = x$;
- 7) $\forall x \in L \quad \exists x'$ – протилежний елемент, такий що $x + x' = \bar{0}$;
- 8) $1 \cdot x = x$.

Елементи лінійного простору називають **векторами**, сам простір – **векторним простором**.

Зауваження. Символом $\bar{0}$ будемо позначати нульовий вектор лінійного простору, а символом 0 – дійсне число нуль.

Прикладами лінійного простору може слугувати множина дійсних чисел $\mathbb{R}$, множина комплексних чисел $\mathbb{C}$, множина векторів на площині E^2 , множина векторів у просторі E^3 . Множина $C_{[a,b]}$ всіх неперервних функцій $f(x)$, які визначені на відрізку $[a, b]$, з поточковим додаванням і множенням на константу, а також множина многочленів P_n степені $\leq n$ теж є лінійними просторами.

Властивості лінійного простору

1. В довільному лінійному просторі існує єдиний нульовий елемент.
2. В довільному лінійному просторі для будь-якого елемента існує єдиний обернений.
3. В довільному лінійному просторі $0 \cdot x = \bar{0}$, а протилежний елемент $x' = (-1) \cdot x$.

Базисом лінійного простору L називається максимальна лінійно незалежна система векторів (приєднання будь-якого вектора з L перетворює цю систему на лінійно залежну). Якщо базис містить n векторів, то лінійний простір L називається **n -вимірним**, а число n – **розмірністю** лінійного простору і позначається $\dim L = n$.

Зауваження. Якщо L містить лише $\bar{0}$, то $\dim L = 0$.

Алгоритм побудови базису формулюється таким чином.

1) Вибираємо довільний вектор $e_1 \in L$, $e_1 \neq \bar{0}$.

2) Вибираємо довільний вектор $e_2 \in L$, $e_2 \neq \bar{0}$ таким чином, щоб вектори e_1 та e_2 були лінійно незалежними.

3) Припустимо, що вказана процедура дозволила побудувати лінійно незалежні вектори $e_1, e_2, \dots, e_n$. Якщо приєднання до $e_1, e_2, \dots, e_n$ довільного вектору $b \in L$ перетворює цю систему векторів на лінійно залежну, то вектори $e_1, e_2, \dots, e_n$ утворюють базис лінійного простору L . Тоді $b = \sum_{i=1}^n \alpha_i e_i$.

Система векторів простору L називається **повною**, якщо кожний вектор простору L лінійно виражається через вектори цієї системи.

Базисом лінійного простору L називається повна лінійно незалежна система векторів.

Теорема. Довільний вектор $x \in L$ єдиним чином розкладається за базисом $e_1, e_2, \dots, e_n$.

Розмірність лінійного простору не залежить від вибору базису, тобто кількість векторів в різних базисах – однакова.

У просторі L існує нескінченна кількість лінійно незалежних систем векторів.

Будь-яка система векторів, що складається з більшої кількості векторів, ніж розмірність простору L , буде лінійно залежною.

Лінійно незалежна система n -вимірних векторів $\vec{a}_1, \vec{a}_2, \vec{a}_3, \dots, \vec{a}_s$ називається **максимальною**, або **повною**, лінійно незалежною системою, якщо в результаті додавання до неї будь-якого відмінного від $\bar{0}$ n -вимірного вектора $\vec{b}$ вона стає лінійно залежною.

Звідси випливає, що в n -вимірному просторі кожна лінійно незалежна система, яка складається з n векторів, буде максимальною, а також будь-яка максимальна, лінійно незалежна система векторів у цьому просторі складається з n векторів.

Поняття множини належить до числа фундаментальних понять математики, яке вводиться аксіоматично.

Визначення. *Множина* – це об'єднання в одне ціле об'єктів, що добре розрізняються нашою інтуїцією або нашою думкою (1872 р. Георг Кантор).

Математичне поняття поступово виділилося зі звичних представлень про сукупність.

Визначення. *Множина* – це сукупність деяких об'єктів (елементів множини), виділених за певною ознакою з інших об'єктів. При цьому повинно бути дано повний опис класу всіх об'єктів, які розглядаються (універсальна множина U).

Множини позначаються великими літерами латинського алфавіту, а їх елементи – малими. Факт належності елемента a множині A позначається $a \in A$. Якщо для всіх елементів множини A і тільки для них виконується властивість P , то це позначають $A = \{a \mid P(a)\}$. Інколи вдається перелічити всі елементи множини A , тоді: $A = \{a_1, a_2, \dots, a_n\}$. Множина, яка не має жодного елемента, називається *порожньою* і позначається $\emptyset$.

Розрізняють поняття чітких і нечітких множин. Найбільш вражаючою властивістю людського інтелекту є здатність приймати правильні рішення в умовах неповної і нечіткої інформації. Зміст терміна нечіткість багатозначний та включає такі основні компоненти: недетермінованість виведень, багатозначність, ненадійність, неповнота, неточність.

Нехай E – універсальна множина, x – елемент E , а R – певна властивість. Звичайна (чітка) підмножина A універсальної множини E , елементи якої задовольняють властивості R , визначається як множина впорядкованої пари $A = \{P(x)/x\}$, де $P(x)$ – характеристична функція, що приймає значення 1, якщо x задовольняє властивості R , і 0 – в іншому випадку. Нечітка підмножина відрізняється від звичайної тим, що для елементів $x \in E$ немає однозначної відповіді відносно властивості R . У зв'язку з цим, нечітка підмножина A універсальної множини E визначається як множина впорядкованої пари $A = \{P(x)/x\}$, де $P(x)$ – характеристична функція приналежності, що приймає значення з $M = [0,1]$. Функція приналежності вказує рівень приналежності елемента x до підмножини A .

Якщо множина містить скінченну кількість елементів, вона називається *скінченною*, у протилежному випадку множина називається *нескінченною*. Особливістю множини є те, що всі її елементи різні.

Визначення. Якщо кожен елемент множини A є елементом множини B , то A називається *підмножиною* множини B , що позначається $A \subset B$.

Визначення. Дві множини A і B рівні між собою, якщо $A \subset B$ і $B \subset A$.

Визначення. Множина всіх підмножин множини A називається *булеаном* і позначається $P(A)$, або 2^A . *Потужність* скінченної множини дорівнює кількості її елементів і позначається $|A|$. Потужність порожньої множини дорівнює 0, тобто $|\emptyset| = 0$, але $|\{\emptyset\}| = 1$.

Твердження. Якщо $|A| = n$, тоді $|P(A)| = 2^n$, або $|2^A| = 2^{|A|}$.

Операції над множинами

Над множинами можна виконувати три булеві операції: об'єднання, перетин, доповнення; а також для зручності запису формул, вводяться ще дві не булеві операції: різниця та симетрична різниця.

Визначення. *Об'єднанням* множин A і B називається множина, кожен елемент якої належить принаймні одній з множин A або B . Якщо елемент належить обом множинам, то він включається в об'єднання один раз. Позначається об'єднання множин символом $\cup$:

$$A \cup B = \{ x \mid x \in A \text{ або } x \in B \}.$$

Операцію об'єднання можна поширити на будь-яке число множин (скінченне або нескінченне), що є підмножинами універсуму U : скінченне об'єднання множин з $S = \{A_1, A_2, \dots, A_n\}$ є $\bigcup_{A \in S} A = \bigcup_{i=1}^n A_i$; нескінченне об'єднання множин $\bigcup_{i=1}^{\infty} A_i$.

Визначення. *Перетином* множин A і B називається множина, кожен елемент якої належить одночасно обом множинам. Позначається перетин множин символом $\cap$:

$$A \cap B = \{ x \mid x \in A \text{ і } x \in B \}.$$

Аналогічно об'єднанню розглядається перетинання скінченного або нескінченного набору множин: $\bigcap_{i=1}^n A_i$; $\bigcap_{i=1}^{\infty} A_i$.

Визначення. Якщо $A \cap B = \emptyset$, то A і B не перетинаються.

Визначення. *Доповненням* множини A до універсальної множини U називається множина, елементами якої є ті елементи U , що не належать A : $\bar{A} = \{x \mid x \in U \text{ і } x \notin A\}$.

Визначення. *Різницею* двох множин A і B називається множина елементів з A , що не належать множині B :

$$A \setminus B = \{ x \mid x \in A \text{ і } x \notin B \}.$$

$$\text{Таким чином, } \bar{A} = U \setminus A.$$

Різницю множин можна виразити за допомогою булевих операцій за формулою: $A \setminus B = A \cap \bar{B}$.

Визначення. *Симетричною різницею* двох множин A і B називається множина, що містить всі елементи з A , які не належать B , і всі елементи з B , які не належать A :

$$A \Delta B = \{ x \mid (x \in A \text{ і } x \notin B \text{ або } x \in B \text{ і } x \notin A) \}.$$

Інший спосіб позначення: $A \oplus B$.

Симетричну різницю множин можна виразити за допомогою булевих операцій формулою:

$$A \Delta B = (A \setminus B) \cup (B \setminus A) = (A \cap \bar{B}) \cup (B \cap \bar{A}).$$

Пріоритет виконання операцій у спадному порядку: доповнення, перетин, об'єднання, різниця, симетрична різниця. Змінити пріоритет можливо дужками.

Для ілюстрації операцій часто використовують *діаграми Ейлера-Венна*, де квадрат означає універсальну множину U , заштриховані області – результат застосування операції до множин A і B (див. рис.1-5).

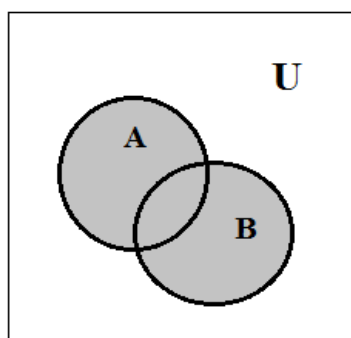


Рис.1 $A \cup B$

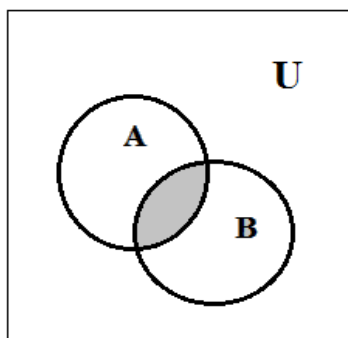


Рис.2 $A \cap B$

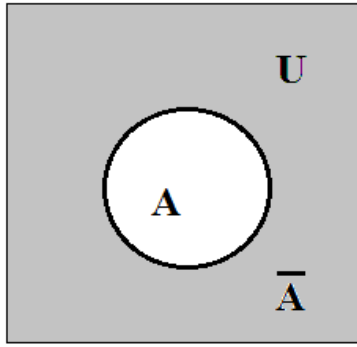


Рис.3 $\bar{A}$

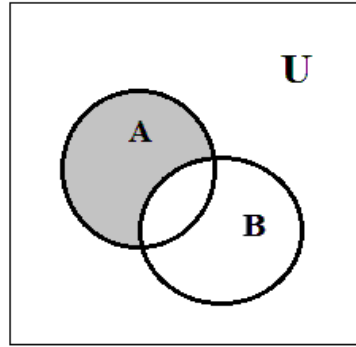


Рис.4 $A \setminus B$

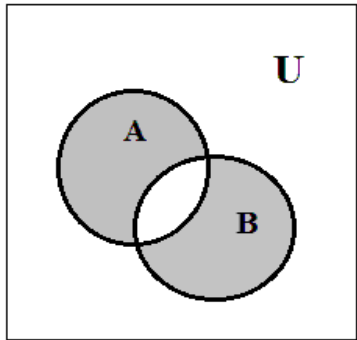


Рис.5 $A \Delta B$

Властивості булевих операцій

1. Комутативність: $\begin{cases} A \cup B = B \cup A, \\ A \cap B = B \cap A \end{cases}$
2. Асоціативність: $(A \cup B) \cup C = A \cup (B \cup C),$
 $(A \cap B) \cap C = A \cap (B \cap C).$
 Ця властивість дозволяє опускати дужки.
3. Дистрибутивність перетину щодо об'єднання:
 $A \cap (B \cup C) = (A \cap B) \cup (A \cap C).$
4. Дистрибутивність об'єднання щодо перетину:
 $A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$
5. Ідемпотентність: $\begin{cases} A \cup A = A, \\ A \cap A = A. \end{cases}$
6. Правила поглинання: $\begin{cases} A \cup (A \cap B) = A, \\ A \cap (A \cup B) = A. \end{cases}$
7. Інволюція: $\overline{\overline{A}} = A.$
8. Правила де Моргана: $\begin{cases} \overline{A \cup B} = \bar{A} \cap \bar{B}, \\ \overline{A \cap B} = \bar{A} \cup \bar{B}. \end{cases}$
9. Властивості універсальної і порожньої множин:
 $A \cup \bar{A} = U, \quad \bar{U} = \emptyset, \quad A \cup \emptyset = A, \quad A \cup U = U,$
 $A \cap \bar{A} = \emptyset, \quad \bar{\emptyset} = U, \quad A \cap U = A, \quad A \cap \emptyset = \emptyset.$

Принцип двоїстості. Якщо має місце якась тотожність алгебри множин, то має місце і двоїста до неї тотожність. Пари властивостей двоїсті друг до друга, якщо в одній з них замінити знак об'єднання $\cup$ знаком перетину $\cap$, універсальну множину U – порожньою множиною $\emptyset$, і

навіпаки: знак перетину $\cap$ замінити знаком об'єднання $\cup$, а $\emptyset$ – множиною U , тоді з однієї рівності виходить друга рівність.

Прямий добуток

Визначення. *Вектор* – це упорядкований набір елементів.

Елементи, що утворюють вектор, називаються його компонентами або координатами, причому координати нумеруються зліва направо і, на відміну від множини, серед компонентів вектора можуть бути однакові та важливим є порядок розташування елементів. Число компонентів вектора називається його довжиною або розмірністю. Вектори довжини 2 називаються *упорядкованими парами*.

Визначення. Два вектори називаються *рівними*, якщо вони мають однакову довжину і їх відповідні координати збігаються, тобто $(a_1, a_2, \dots, a_n) = (b_1, b_2, \dots, b_n)$, якщо $a_1 = b_1, a_2 = b_2, \dots, a_n = b_n$.

Визначення. *Прямим (декартовим) добутком* $A \times B$ множин A і B називається множина всіх пар (a, b) таких, що $a \in A, b \in B$:

$$A \times B = \{(a, b) \mid a \in A, b \in B\}.$$

При цьому вважається, що $(a_1, b_1) = (a_2, b_2)$ тоді і тільки тоді, коли $a_1 = a_2, b_1 = b_2$.

Визначення. *Прямим (декартовим) добутком* $A_1 \times A_2 \times \dots \times A_n$ множин $A_1, A_2, \dots, A_n$ називається множина усіх векторів $(a_1, a_2, \dots, a_n)$ довжини n таких, що $a_1 \in A_1, a_2 \in A_2, \dots, a_n \in A_n$:

$$A_1 \times A_2 \times \dots \times A_n = \{(a_1, a_2, \dots, a_n) \mid a_1 \in A_1, a_2 \in A_2, \dots, a_n \in A_n\}.$$

Якщо всі A_i дорівнюють між собою, тоді говорять про *n-ту степінь множини A*:

$$A^n = \underbrace{A \times A \times \dots \times A}_{n \text{ разів}}.$$

Теорема (про кількість елементів прямого добутку):

а) потужність декартового добутку двох скінченних множин дорівнює:

$$|A \times B| = |A| \cdot |B|.$$

б) нехай $A_1, A_2, \dots, A_n$ - скінченні множини, а їх потужності відповідно дорівнюють $|A_1|=m_1, |A_2|=m_2, \dots, |A_n|=m_n$. Тоді потужність множини $A_1 \times A_2 \times \dots \times A_n$ дорівнює:

$$|A_1 \times A_2 \times \dots \times A_n| = |A_1| \times |A_2| \times \dots \times |A_n| = m_1 \times m_2 \times \dots \times m_n.$$

Наслідок: $|A^n| = |A|^n$.

Тому що прямий добуток це множина (елементи якої є вектори), тому для нього визначені всі операції над множинами: перетин, об'єднання, доповнення, різниця і симетрична різниця.

Розглянемо множини $A_1, A_2, \dots, A_n$ (не обов'язково різні).

Визначення. *n*-місцеве (або *n*-арне) відношення ρ на множинах $A_1, A_2, \dots, A_n$ – це закон (або характеристична властивість), що виділяє в декартовому добутку $A_1 \times A_2 \times \dots \times A_n$ деяку підмножину, тобто: $\rho_{A_1, \dots, A_n} \subset A_1 \times A_2 \times \dots \times A_n$.

Говорять, що $a_1, a_2, \dots, a_n$ перебувають у відношенні ρ , якщо $(a_1, a_2, \dots, a_n) \in \rho$. Якщо $A_1 = A_2 = \dots = A_n = A$, тоді говорять про *n*-місцеве відношення ρ на множині A : $\rho_A \subset A^n$. Позначають відношення грецькими буквами $\alpha, \beta, \gamma, \dots$, літерою R , а також спеціальними символами $=, <, >, \leq, \geq$ і іншими.

Відношення ρ при $n=1$ на множині A називається унарним (одномісним). Це відношення складається з векторів, довжиною 1. Унарне відношення ρ на множині A являє собою характеристичну властивість деякої підмножини $\rho_A \subset A$. Таким чином, множина всіх можливих унарних відношень на A збігається з множиною $P(A)$ усіх підмножин множини A .

Найбільш вивчені і широко застосовуються на практиці двомісні відношення, які називаються бінарними.

Визначення. Бінарним відношенням ρ називається підмножина декартового добутку $A \times B$ (тобто $\rho \subset A \times B$). Тобто ρ – деяка множина упорядкованих пар (a, b) . Кажуть, що a і b знаходяться у відношенні ρ , якщо $(a, b) \in \rho$, допускається також запис $a \rho b$.

Областю визначення бінарного відношення $R \subset X \times Y$ називається множина $\delta_R = \{x \mid \exists y (x, y) \in R\}$, а областю значень – множина $\rho_R = \{y \mid \exists x (x, y) \in R\}$ ($\exists$ – існує).

Способи завдання бінарних відношень

1. У виді матриці з двох стовпців, у кожному рядку якої стоять пари елементів (a, b) , $a \in A$, $b \in B$, що задовольняють даному відношенню.
2. Переліком пар множини, що задовольняють відношенню.
3. За допомогою матриці відношення $R_{m \times n} = (r_{ij})$, де $m = |A|$, $n = |B|$, в якій рядки відповідають елементам множини A , стовпці – елементам множини B , причому на перетині i -го рядка і j -го стовпця стоїть 1, якщо $a_i \rho b_j$, і 0 у протилежному випадку. Елементами матриці є значення

$$r_{ij} = \begin{cases} 1, & \text{якщо } (a_i, b_j) \in R, \\ 0, & \text{якщо } (a_i, b_j) \notin R. \end{cases}$$

4. За допомогою стрілок. При цьому способі завдання відношення ρ на множинах A і B елементи множин зображуються у виді точок площини; потім точки a_i і b_j з'єднуються стрілкою, спрямованою від a_i до b_j , тоді і тільки тоді, коли $a_i \rho b_j$.

Для нескінченних множин бінарне відношення зображають у вигляді точок площини, які знаходяться у відношенні. Будь-яка підмножина $\rho \subset R^2$, де R – множина дійсних чисел, є графіком деякого бінарного відношення ρ на множині R .

Оскільки відношення на A задаються підмножинами A^2 , для них можна визначити ті ж операції, що і над множинами: об'єднання, перетин, доповнення, різниця, симетрична різниця.

Визначення. Якщо для будь-якої множини A відношення ρ , задане матрицею, у якій на головній діагоналі стоять тільки одиниці, а в інших місцях – нулі, тоді відношення ρ називається відношення рівності на A або тотожне відношення: $I := \{(a, a) \mid a \in A\}$.

Визначення. Відношення ρ^{-1} називається *оберненим* до відношення ρ , якщо $a_i \rho^{-1} a_j$ тоді і тільки тоді, коли $a_j \rho a_i$:

$$\rho^{-1} = \{(x, y) | (y, x) \in \rho\}.$$

Визначення. Відношення $\bar{\rho}$ називається *доповненням* до відношення ρ , якщо $a_i \bar{\rho} a_j$ тоді і тільки тоді, коли $(a_i, a_j) \notin \rho$: $\bar{\rho} := \{(a, b) | (a, b) \notin \rho\}$.

Визначення. Відношення називається *універсальним*, якщо воно містить усі елементи декартового добутку A^2 :

$$U := \{(a, b) | a \in A, b \in A\}$$

Визначення. *Композицією (або добутком)* відношень $\rho_1 \subset X \times Y$ і $\rho_2 \subset Y \times Z$ називається відношення $\rho_1 \circ \rho_2 \subset X \times Z$ таке, що $\rho_1 \circ \rho_2 = \{(x, z) | x \in X, z \in Z \text{ і існує } y \in Y \text{ таке, що } (x, y) \in \rho_1 \text{ і } (y, z) \in \rho_2\}$.

Визначення. *Степом* відношення ρ на множині A називають його композицію з самим собою:

$$\rho^n = \underbrace{\rho \circ \rho \circ \dots \circ \rho}_n.$$

Властивості бінарних відношень

Нехай задане бінарне відношення $\rho_A \subset A^2$ на множині A .

Визначення. Відношення ρ називається *рефлексивним*, якщо $\forall a \in A$ має місце $a \rho a$, тобто кожен елемент множини знаходиться у відношенні ρ до самого себе. Головна діагональ матриці рефлексивного відношення містить тільки одиниці.

Визначення. Відношення ρ називається *антирефлексивним*, якщо ні для якого $a \in A$ не виконується $a \rho a$, тобто $\forall a \in A (a, a) \notin \rho_A$. Головна діагональ матриці відношення містить тільки нулі.

Визначення. Відношення ρ називається *симетричним*, якщо для будь-яких $a, b \in A$ з того що $a \rho b$, випливає $b \rho a$.

$$\forall a, b \in A: a \rho b \rightarrow b \rho a.$$

Матриця такого відношення симетрична відносно головної діагоналі, тобто $\forall i, j: r_{ij} = r_{ji}$.

Визначення. Відношення називається *антисиметричним*, якщо для будь-яких $a, b \in A$ з того що $a \rho b$ і $b \rho a$ випливає $a = b$. Тобто $\forall a, b \in A: a \rho b, b \rho a \rightarrow a = b$.

Визначення. Відношення ρ називається *транзитивним*, якщо для будь-яких $a, b, c \in A$ з того що $a \rho b$ і $b \rho c$, випливає $a \rho c$. Тобто $\forall a, b, c \in A: (a \rho b \text{ і } b \rho c) \rightarrow a \rho c$.

Визначення. Відношення ρ називається *повним або лінійним*, якщо для будь-яких $a, b \in A$ $a \neq b$ випливає $a \rho b$ або $b \rho a$.

Визначення. Відношення ρ називається відношенням *еквівалентності*, якщо воно рефлексивне, симетричне і транзитивне, тобто:

$$\forall a \in A: a \rho a;$$

$$\forall a, b \in A: a \rho b \rightarrow b \rho a;$$

$$\forall a, b, c \in A: (a \rho b \text{ і } b \rho c) \rightarrow a \rho c.$$

Визначення. *Класом еквівалентності*, утвореним (породженим) елементом x , називається підмножина множини X , що складається з елемента x і всіх елементів $y \in X$, для яких $x \rho y$.

Позначення класу еквівалентності, породженого елементом x : $[X] = \{y | y \in X \text{ і } x \rho y\}$.

Визначення. Відношення ρ називається *відношенням порядку*, якщо воно антисиметричне і транзитивне. Будемо позначати його знаком $\prec$.

Відношення ρ називається відношенням *нестрогого порядку*, якщо воно рефлексивне, антисиметричне і транзитивне. Будемо позначати його знаком $\leq$.

Відношення ρ називається відношенням *строогого порядку*, якщо воно антирефлексивне, антисиметричне і транзитивне. Будемо позначати його знаком $<$.

Відношення ρ називається відношенням *повного або лінійного порядку* на множині, якщо воно є відношенням порядку і повне, тобто якщо будь-які два елементи множини порівнянні; *часткового порядку*, якщо воно не володіє властивістю повноти.

Визначення. Два елементи $a, b \in A$ називаються *порівнянними* по відношенню порядку ρ на множині A , якщо $a\rho b$ або $b\rho a$, і називаються *непорівнянними*, якщо не виконується жодне з цих відносин, тобто $(a,b) \notin \rho$ і $(b,a) \notin \rho$.

Множина, на якій визначено відношення часткового порядку називається *частково впорядкованою*, а на якій визначено відношення повного порядку - *цілком упорядкованою*.

Визначення. Елемент x множини A з відношенням порядку $\prec$ називається *мінімальним*, якщо не має менших елементів: $y \in A: y \prec x$ і $y \neq x$.

У багатьох практичних випадках виникає необхідність вибору та розміщення елементів скінченної множини згідно із заданими правилами. Цим займається комбінаторика - один з розділів дискретної математики. Якщо треба підрахувати кількість можливих комбінацій об'єктів, які відповідають певним умовам, то це так звана перша задача комбінаторики - *задача перерахунку*. Якщо треба не просто підрахувати кількість наборів, а знайти ці комбінації, то така задача називається *задачею перелічення* і є другою задачею комбінаторики.

Нехай X – скінченна множина, яка містить n елементів, тоді говорять, що "об'єкт x може бути обраний n способами" і пишуть $|X|=n$ (фактично це потужність множини).

Для комбінаторних задач використовують правила суми та добутку, які можна сформулювати наступним чином.

Правило суми. Якщо $X_1, \dots, X_k$ – множини, які попарно не перетинаються, тобто $\forall i \neq j \quad X_i \cap X_j = \emptyset$, тоді:

$$\left| \bigcup_{i=1}^k X_i \right| = \sum_{i=1}^k |X_i|.$$

В комбінаториці під $\left| \bigcup_{i=1}^k X_i \right|$ розуміють число способів вибору елемента хоча б з однієї з множин $X_1, \dots, X_k$.

Як окремий випадок, для $k=2$ правило суми можна сформулювати так: якщо x може бути обраний m способами, а y - іншими n способами, тоді вибір «або x , або y » може бути здійснений $m+n$ способами.

А правило добутку для $k=2$ так: якщо x може бути обраний m способами та після кожного з них y може бути обраний n способами, то вибір впорядкованої пари (x, y) може бути здійснений $m \cdot n$ способами.

Узагальнення правила добутку: якщо об'єкт x_1 може бути обраний n_1 способами, після чого об'єкт x_2 - n_2 способами, та для будь-якого i після вибору об'єктів $x_1, \dots, x_i$ об'єкт x_{i+1} може бути обраний n_{i+1} способами, тоді вибір впорядкованої послідовності $(x_1, \dots, x_m)$ може бути здійснений $n_1 \cdot n_2 \cdot \dots \cdot n_m$ способами.

Визначення. Розглянемо скінченну множину $X = \{x_1, \dots, x_n\}$, яка містить n елементів. Набір елементів $(x_{i1}, \dots, x_{ir})$ з множини X називається вибіркою r об'єктів з n елементів, або (n, r) - вибіркою.

Вибірка називається *впорядкованою*, якщо суттєвим є порядок розміщення в ній елементів. Дві впорядковані вибірки, що розрізняються лише порядком розміщення елементів, вважаються різними. Якщо ж порядок розміщення елементів у вибірці не суттєвий, тоді вона *невпорядкована*.

Визначення. Впорядкована (n, r) - вибірка, в якій елементи можуть повторюватися, називається (n, r) - розміщенням з повтореннями. Якщо ж у впорядкованій (n, r) - вибірці всі елементи різні, то вона називається просто (n, r) - розміщенням, або розміщенням без повторень. (n, n) - розміщення без повторень є *перестановками*. Невпорядкована (n, r) - вибірка, в якій елементи можуть повторюватися, називається (n, r) - комбінацією з повтореннями, якщо ж всі елементи різні – (n, r) - комбінацією без повторень, або просто *комбінацією*.

Число (n, r) - розміщень з повтореннями позначається через $\overline{A}_n^r$, а без повторень – A_n^r ; число перестановок n - елементної множини – P_n , число комбінацій з повтореннями – $\overline{C}_n^r$, без повторень – C_n^r .

Твердження 1.

$$\overline{A}_n^r = n^r$$

Твердження 2.

$$A_n^r = n \cdot (n-1) \cdot \dots \cdot (n-r+1) = \frac{n!}{(n-r)!}, \quad r \leq n.$$

Наслідок 1.

$$P_n = A_n^n = n \cdot (n-1) \cdot \dots \cdot 1 = n!$$

Наслідок 2.

Якщо маємо n елементів, з яких n_1 елементів першого типу, n_2 – другого типу, ..., n_k – k -го типу ($\sum_{i=1}^k n_i = n$), тоді перестановки цих елементів називаються *перестановки з повтореннями* і їх кількість знаходиться за формулою:

$$P(n_1, n_2, \dots, n_k) = \frac{n!}{n_1! n_2! \dots n_k!}.$$

Твердження 3.

$$C_n^r = \frac{n!}{(n-r)! r!}, \quad r \leq n.$$

Твердження 4.

$$\overline{C}_n^r = C_{n+r-1}^r.$$

Формула включень і виключень

Нехай X_1, X_2 – дві скінченні множини.

Тоді, якщо $X_1 \cap X_2 = \emptyset$, то: $|X_1 \cup X_2| = |X_1| + |X_2|$. Нехай тепер $X_1 \cap X_2 \neq \emptyset$, тоді в $|X_1| + |X_2|$ кожен елемент з $X_1 \cap X_2$ буде врахований двічі, тому: $|X_1 \cup X_2| = |X_1| + |X_2| - |X_1 \cap X_2|$.

Отримаємо тепер відповідну формулу для довільного числа множин, яка називається *формулою включень і виключень*.

Твердження 7.

Нехай X_i – скінченна множина, $i=1, 2, \dots, n$, $n \geq 2$, тоді:

$$\begin{aligned} |X_1 \cup \dots \cup X_n| = & (|X_1| + \dots + |X_n|) - (|X_1 \cap X_2| + |X_1 \cap X_3| + \dots \\ & + |X_{n-1} \cap X_n|) + (|X_1 \cap X_2 \cap X_3| + \dots + |X_{n-2} \cap X_{n-1} \cap X_n|) - \\ & - \dots + (-1)^{n+1} |X_1 \cap \dots \cap X_n| \end{aligned}$$

Наслідок.

Нехай X – скінченна множина, $X_1, \dots, X_n$ – підмножини X , тоді:

$$\begin{aligned} |X \setminus (X_1 \cup \dots \cup X_n)| = & |X| - (|X_1| + \dots + |X_n|) + \\ & + (|X_1 \cap X_2| + \dots + |X_{n-1} \cap X_n|) - \dots + (-1)^n |X_1 \cap \dots \cap X_n| \end{aligned}$$

Наведемо ще одну форму запису формули включень і виключень. Нехай X - скінченна множина, що складається з N елементів, $\alpha_1, \dots, \alpha_n$ - деякі властивості, якими можуть володіти або ні елементи з X . Позначимо через $X_i = \{x \in X \mid \alpha_i(x)\}$ - множину елементів в X , які володіють властивістю α_i , а $N(\alpha_i)$ - кількість цих елементів, $N(\alpha_{i_1}, \dots, \alpha_{i_k})$ - кількість елементів $x \in X$, які володіють одночасно властивостями $\alpha_{i_1}, \dots, \alpha_{i_k}$:

$$N(\alpha_{i_1}, \dots, \alpha_{i_k}) = |X_{i_1} \cap \dots \cap X_{i_k}| = |\{x \in X \mid \alpha_{i_1}(x) \wedge \dots \wedge \alpha_{i_k}(x)\}|,$$

$N_0 = |X \setminus (X_1 \cup \dots \cup X_n)|$ - кількість елементів, які не володіють жодною з властивостей $\alpha_{i_1}, \dots, \alpha_{i_k}$.

Тоді за наслідком формули включень і виключень отримуємо:

$$N_0 = N - S_1 + S_2 - \dots + (-1)^n S_n,$$

де для всіх $k=1, 2, \dots, n$: $S_k = \sum_{1 \leq i_1 \leq \dots \leq i_k \leq n} N(\alpha_{i_1}, \dots, \alpha_{i_k})$.

Якщо треба знайти кількість елементів, які володіють рівно m властивостями, тоді

використовують наступну формулу: $\hat{N}_m = \sum_{k=0}^{n-m} (-1)^k C_{m+k}^m S_{m+k}.$

Математична логіка - це аналіз методів міркувань. При цьому насамперед досліджуються форми міркувань, а не їх зміст. Систематична формалізація правильних способів міркувань – одне із завдань логіки. Якщо при цьому логік застосовує математичний апарат, і його дослідження присвячені аналізу математичних міркувань, то предмет його занять може бути названий математичною логікою. Є дві конкретні системи логіки: обчислення висловлювань (ОВ) та обчислення предикатів (ОП).

Визначення. Висловлюванням називається оповідальне речення, про яке можна сказати в даний момент часу, що воно істинно чи хибно, але не те й інше одночасно.

Кожному висловлюванню ставиться у відповідність змінна, що має значення істина - 1, якщо воно істинно, і має значення брехня - 0, якщо висловлювання хибне. З висловлювань шляхом з'єднання їх різними способами можна скласти нові, складніші, висловлювання, звані складними. Значення істинності складного висловлювання залежить від значень істинності складових його висловлювань. Для складання складних висловлювань застосовуються логічні зв'язки-операції, найважливішими з яких є булеві:

P	Q	$P \vee Q$	P	Q	$P \wedge Q$	P	$\bar{P}$
0	0	0	0	0	0	0	1
0	1	1	0	1	0	1	0
1	0	1	1	0	0	1	0
1	1	1	1	1	1		

Рис. 1

$\vee$ – логічне додавання (або), диз'юнкція;

$\wedge$ – логічне множення (і), кон'юнкція;

$\neg$ – логічне заперечення (ні).

Якщо P і Q – деякі висловлювання, то можна скласти нові висловлювання $P \vee Q$, $P \wedge Q$, $\bar{P}$, $\bar{Q}$, причому істинність складових висловлювань визначається з допомогою таблиць істинності, що представлені на Рис. 1.

Властивості булевих операцій

- Ідемпотентність: $A \vee A = A$, $A \wedge A = A$.
- Комутативність: $A \vee B = B \vee A$, $A \wedge B = B \wedge A$.
- Асоціативність: $A \vee (B \vee C) = (A \vee B) \vee C$, $A \wedge (B \wedge C) = (A \wedge B) \wedge C$.
- Дистрибутивність кон'юнкції відносно диз'юнкції:
 $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$.
- Дистрибутивність диз'юнкції відносно кон'юнкції:
 $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$.
- Інволюція: $\bar{\bar{A}} = A$ (подвійне заперечення).
- Правила де Моргана: $\begin{cases} \overline{A \vee B} = \bar{A} \wedge \bar{B}, \\ \overline{A \wedge B} = \bar{A} \vee \bar{B}. \end{cases}$
- Склеювання: а) $(A \wedge B) \vee (A \wedge \bar{B}) = A$,
 б) $(A \vee B) \wedge (A \vee \bar{B}) = A$.
- Поглинання: $A \vee (A \wedge B) = A$, $A \wedge (A \vee B) = A$.
- Закон протиріччя: $A \wedge \bar{A} = 0$.

11. Закон виключеного третього: $A \vee \bar{A} = 1$.

12. Дії з нулем та одиницею: $\bar{0} = 1$; $A \vee 0 = A$; $A \vee 1 = 1$;
 $\bar{1} = 0$; $A \wedge 0 = 0$; $A \wedge 1 = A$.

Часто в логіці крім трьох основних булевих операцій для утворення складних висловлювань вводять ще дві: імплікація (наслідування)– $A \rightarrow B$ або $A \supset B$ («якщо A , то B »); еквівалентність – $A \sim B$ або $A \equiv B$ або $A \leftrightarrow B$ («якщо і тільки якщо»), які мають таблиці істинності, представлені на Рис. 2.

A	B	$A \rightarrow B$	A	B	$A \sim B$
0	0	1	0	0	1
0	1	1	0	1	0
1	0	0	1	0	0
1	1	1	1	1	1

Рис.2

Будь-яке висловлювання, побудоване за допомогою зазначених вище операцій $\wedge$, $\vee$, $\bar{}$, $\rightarrow$, $\sim$, має деяке істинне значення (1 або 0), що залежить від значень вихідних висловлювань.

Логічні функції двох змінних

До кожного складеного (складного) висловлювання можна побудувати таблицю істинності. Якщо складове висловлювання залежить від n складових, то у таблиці істинності такого висловлювання буде 2^n рядків. З іншого боку, кожному висловлюванню можна поставити у відповідність деяку функцію $f(x_1, x_2, \dots, x_n)$ від n змінних, де кожна змінна відповідає простому висловлюванню. Така функція називається логічною або булевою, причому за своїм змістом вона може приймати тільки два значення: 0 або 1, тоді як аргументи набувають тих же значень.

Твердження. Існує взаємно-однозначна відповідність між таблицею істинності деякого складеного висловлювання та логічною функцією цього висловлювання.

Ми розглядали досі 4 логічні операції над двома змінними (висловлюваннями). Усього таких операцій 16. Представимо їх у вигляді таблиці.

x_1	x_2	ψ_0	ψ_1	ψ_2	ψ_3	ψ_4	ψ_5	ψ_6	ψ_7	ψ_8	ψ_9	ψ_{10}	ψ_{11}	ψ_{12}	ψ_{13}	ψ_{14}	ψ_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Нижче наведені формули, що описують усі 16 функцій:

$$\psi_0(x_1, x_2) = 0, \psi_{15}(x_1, x_2) = 1;$$

$$\psi_1(x_1, x_2) = x_1 \wedge x_2 = x_2 \wedge x_1 = x_1 x_2 = x_1 \& x_2 \text{ (кон'юнкція);}$$

$$\psi_7(x_1, x_2) = x_1 \vee x_2 = x_2 + x_1 \text{ (диз'юнкція);}$$

$$\psi_6(x_1, x_2) = x_1 \oplus x_2 = x_1 \Delta x_2 = x_1 \neq x_2 \pmod{2} \text{ – додавання за модулем 2) – операція протилежна } \sim;$$

$$\psi_8(x_1, x_2) = x_1 \downarrow x_2 \text{ (стрілка Пірса);}$$

$$\psi_9(x_1, x_2) = x_1 \sim x_2 = x_1 \equiv x_2 = x_1 \leftrightarrow x_2 \text{ (еквівалентність);}$$

$$\psi_{13}(x_1, x_2) = x_1 \rightarrow x_2 = x_1 \supset x_2 \text{ (імплікація, слідування);}$$

$$\psi_{14}(x_1, x_2) = x_1 / x_2 = x_1 \mid x_2 \text{ (штрих Шиффера);}$$

$$\psi_3(x_1, x_2) = x_1 \text{ (функція з фіктивною змінною } x_2);$$

$$\psi_{12}(x_1, x_2) = \bar{x}_1;$$

$$\psi_5(x_1, x_2) = x_2 \text{ (функція з фіктивною змінною } x_1);$$

$$\psi_{10}(x_1, x_2) = \bar{x}_2;$$

$$\psi_{11}(x_1, x_2) = x_1 \leftarrow x_2 \text{ (обернена імплікація);}$$

$$\psi_2(x_1, x_2) = x_1 \rightarrow x_2 \text{ (пряма антиімплікація);}$$

$$\psi_4(x_1, x_2) = x_1 \nleftarrow x_2 \text{ (обернена антиімплікація).}$$

Тавтології та протиріччя

Визначення. Складове висловлювання, яке істинно незалежно від значень висловлювань, що входять до нього, називається тавтологією або тотожно істинним висловлюванням.

Таблиця істинності складного висловлювання містить лише значення “істинно”, а його функція тотожно дорівнює одиниці. Щоб перевірити, чи є тавтологією та чи інша функція, треба скласти для неї таблицю істинності з 2^n рядків на всіх наборах значень змінних, або ж привести цю форму за допомогою еквівалентних перетворень до відомої тавтології.

Визначення. Складове висловлювання, яке хибно незалежно від значень висловлювань, що входять до нього, називається протиріччям або тотожно хибним висловлюванням.

Для зручності прийнято угоду про більш економічне використання дужок у складових висловлюваннях:

- 1) завжди можна опускати пару зовнішніх дужок;
- 2) логічні операції розташовуються в порядку зменшення пріоритетів таким чином: $-, \wedge, \vee, \rightarrow, \sim$;
- 3) якщо складове висловлення містить входження лише однієї логічної операції, то для кожного входження цієї операції опускаються зовнішні дужки у того з двох висловлювань, що стоїть ліворуч.

Суперпозиції та формули

Нехай задана система логічних функцій $f_1, f_2, \dots, f_n$.

Визначення. Функція f називається суперпозицією функцій $f_1, f_2, \dots, f_n$, якщо вона отримана з цих функцій шляхом підстановки їх одну в одну і перейменування змінних, а вираз, що описує цю суперпозицію, називається формулою функції f .

В обчисленні висловлювань (ОВ) формули будуються за такими правилами:

- 1) змінна є формула;
- 2) якщо F і P – формули, то формулами будуть також $\bar{F}$, $\bar{P}$, $F \wedge P$, $F \vee P$, $F \rightarrow P$, $F \sim P$;
- 3) інших формул немає.

Формула алгебри висловлювань приймає одне із двох значень (1 чи 0) залежно від значень елементарних висловлювань, які її утворюють. Якщо розглядати останні як незалежні змінні, то з формулою відповідно до вищесказаного зіставляється деяка функція. Її області визначення та значень збігаються - це є множина $B = \{0, 1\}$. На відміну від табличного способу уявлення функції, уявлення її за допомогою формули не є єдиним, тобто ту саму функцію можна записати різними формулами.

Визначення. Формули, що представляють ту саму функцію, називаються еквівалентними або рівносильними. Еквівалентність записується за допомогою знака « $\equiv$ ».

Виникає питання: як на практиці визначити еквівалентність двох формул? Існує універсальний спосіб, що у всіх випадках приводить до відповіді: за кожною формулою відновити таблицю істинності; якщо дві таблиці збігаються, формули еквівалентні.

Існує інший спосіб - за допомогою еквівалентних перетворень.

Булева алгебра та еквівалентні перетворення

Нехай функція f_1 задана формулою F_1 , а функція f_2 – формулою F_2 . Підставимо формули F_1 і F_2 в диз'юнкцію $x_1 \vee x_2$, отримаємо деяку формулу $F = F_1 \vee F_2$. Розглянемо формули F_1' та F_2' , еквівалентні відповідно F_1 і F_2 , тобто $F_1' \sim F_1$, $F_2' \sim F_2$. Тоді отримаємо формулу $F' = F_1' \vee F_2'$, причому $F' \sim F$ (еквівалентність цих двох формул можна довести за допомогою таблиці істинності). Таким чином, диз'юнкцію можна розглядати як бінарну операцію над безліччю логічних функцій P_2 , яка кожній парі функцій f_1 і f_2 незалежно від формул, що їх представляють, однозначно ставить у відповідність функцію $f_1 \vee f_2$.

Аналогічно можна показати, що кон'юнкція є бінарною операцією на множині P_2 , а заперечення - унарною операцією на тій же множині логічних функцій.

Алгебра $A = \{P_2, \vee, \wedge, \neg\}$ називається булевою алгеброю логічних функцій. Сигнатура цієї алгебри містить 3 логічні операції.

При еквівалентних перетвореннях формул будемо користуватися основними властивостями булевих операцій. Причому ясно, що результат обчислень не залежить від того, чи є змінні в цих виразах незалежними або самі отримані перед цим за допомогою яких-небудь обчислень. Отже, залишаються справедливими й у тому випадку, коли замість змінних підставити якісь логічні функції чи формули, які їх представляють. Важливо лише дотримуватись наступного **правила підстановки**: формули замість змінної: при такій підстановці необхідно замінити формулою одночасно всі входження змінної, замість якої ця формула підставляється.

Правило заміни: якщо будь-яка формула F містить формулу F_1 як підформулу, і якщо формула F_2 еквівалентна F_1 , то заміна у формулі F підформули F_1 на F_2 приведе до формули F_3 , яка еквівалентна F ($F_3 \sim F$).

Визначення. Перетворення формул, що використовують еквівалентні співвідношення, число яких можна поповнити за допомогою правил підстановки та заміни, називаються еквівалентними перетвореннями.

Еквівалентні перетворення є потужним засобом доказу еквівалентності формул, як правило, більш ефективним, ніж доказ за допомогою таблиць істинності. Одним із додатків еквівалентних перетворень є спрощення формул.

Для спрощення формул використовують властивості булевих операцій, а також наступні три тотожності, що дозволяють замінити імплікацію ($\rightarrow$) та еквівалентність ($\sim$) булевими операціями заперечення ($\neg$), кон'юнкція ($\wedge$) та диз'юнкція ($\vee$):

$$13) x \rightarrow y = \bar{x} \vee y;$$

$$14) x \sim y = (\bar{x} \vee y) \wedge (x \vee \bar{y});$$

$$15) x \sim y = (x \wedge y) \vee (\bar{x} \wedge \bar{y}).$$

Досконала диз'юнктивна нормальна форма - ДДНФ

Теорема 1. Будь-яку логічну функцію n змінних $f(x_1, x_2, \dots, x_n)$ можна представити у вигляді:

$$f(x_1, \dots, x_m, x_{m+1}, \dots, x_n) = \bigvee_{\alpha_1, \dots, \alpha_m} x_1^{\alpha_1} \wedge \dots \wedge x_m^{\alpha_m} f(\alpha_1, \dots, \alpha_m, x_{m+1}, \dots, x_n), \quad (1)$$

де диз'юнкція береться за всіма 2^m наборами параметрів $\alpha_1, \dots, \alpha_m$, $m \leq n$.

Якщо $m=n$, то розклад функції за усіма змінними:

$$f(x_1, \dots, x_n) = \bigvee_{\substack{x_1, \dots, x_n \\ f(x_1, \dots, x_n)=1}} x_1^{\sigma_1} \wedge \dots \wedge x_n^{\sigma_n}, \quad (2)$$

де диз'юнкція береться за всіма тими наборами змінних, при яких $f=1$.

Розклад (2) функції за n змінними називається досконалою диз'юнктивною нормальною формою (ДДНФ) функції f . ДДНФ функції f містить рівно стільки кон'юнкцій, скільки одиниць у таблиці істинності f . Кожному відповідному набору $\sigma_1, \dots, \sigma_n$ таблиці відповідає кон'юнкція всіх змінних, у яких x_i взято з запереченням,

якщо $\sigma_i = 0$, x_i взято без заперечення, якщо $\sigma_i = 1$, тобто $x_i = \begin{cases} \bar{x}_i, & \sigma_i = 0; \\ x_i, & \sigma_i = 1. \end{cases}$

Висновок: існує взаємно-однозначна відповідність між ДДНФ та таблицею істинності функції f . Отже, на відміну від формули, ДДНФ кожної функції єдина.

Теорема 2. Будь-яку логічну функцію можливо представити булевою формулою, тобто суперпозицією $\neg, \wedge, \vee$.

Алгоритм побудови ДДНФ

Кожна функція $f(x_1, \dots, x_n) \in \mathbf{P}_2$ (множина логічних функцій n змінних), причому така, що $f \neq 0$, має ДДНФ, яка будується так:

1. У таблиці істинності виділяються ті рядки, тобто ті набори змінних $\sigma_1, \dots, \sigma_n$, для яких $f(\sigma_1, \dots, \sigma_n) = 1$.
2. Для кожного зазначеного рядка утворюємо кон'юнкцію $x_1^{\sigma_1} \wedge \dots \wedge x_n^{\sigma_n}$.
3. Складаємо диз'юнкцію всіх одержаних у другому пункті кон'юнкцій.

Визначення. Елементарною кон'юнкцією називається кон'юнкція змінних чи його заперечень, у якому кожна змінна входить не більше одного разу. Диз'юнкція елементарних кон'юнкцій називається диз'юнктивною нормальною формою (ДНФ).

Різниця між ДНФ і ДДНФ полягає в тому, що в ДДНФ кожна елементарна кон'юнкція містить усі n змінних або їх заперечень, тоді як у ДНФ деякі змінні можуть бути відсутніми. Тоді як для кожної функції ДДНФ існує єдина, ДНФ тієї ж функції існує безліч.

Диз'юнктивну нормальну форму, у тому числі і досконалу, можна побудувати за допомогою еквівалентних перетворень за таким алгоритмом:

- а) виключити $\rightarrow$ і $\sim$ за допомогою формул 13-15;
- б) за допомогою правил подвійного заперечення і де Моргана виключити заперечення над операціями;
- в) розкрити дужки та застосувати властивість ідемпотентності, закони виключеного третього та протиріччя;
- г) виключити константи за властивостями 12.

Будь-яку ДНФ можна привести до ДДНФ розщепленням кон'юнкцій, в які співмножниками входять не всі змінні, для цього множать на одиниці, представлені у вигляді диз'юнкцій кожної недостатньої змінної та її заперечення, і розкривають дужки згідно із законом дистрибутивності, нарешті, виключають повторення доданків.

Досконала кон'юнктивна нормальна форма

Визначення. Елементарною диз'юнкцією називається диз'юнкція змінних або їх заперечень, до якої кожна змінна входить не більше одного разу.

Визначення. Кон'юнктивною нормальною формою (КНФ) називається формула, представлена у вигляді кон'юнкції елементарних диз'юнкцій.

Приклад. $(x \vee \bar{z}) \wedge (y \vee z); (\bar{x} \vee y \vee \bar{z})$.

За допомогою еквівалентних перетворень будь-яка формула приводиться до КНФ аналогічно її приведенню до ДНФ.

Правило побудови КНФ. Якщо є формула F , для якої потрібно побудувати КНФ, то спочатку потрібно отримати формулу $\bar{F}$. Потім для формули $\bar{F}$ побудувати ДНФ, яка у загальному випадку має вид: $k_1 \vee k_2 \vee \dots \vee k_n$, де $k_1, k_2, \dots, k_n$ – елементарні кон'юнкції. Так як $\bar{\bar{F}} = F$, то КНФ отримаємо шляхом заперечення побудованої ДНФ та застосування узагальненого закону де Моргана:

$$\overline{k_1 \vee k_2 \vee \dots \vee k_n} = \bar{k}_1 \wedge \bar{k}_2 \wedge \dots \wedge \bar{k}_n, \text{ де } \bar{k}_1, \bar{k}_2, \dots, \bar{k}_n \text{ – елементарні диз'юнкції.}$$

Кожна формула алгебри висловлювань має багато різних КНФ.

Визначення. Досконала кон'юнктивна нормальна форма (ДКНФ) формули алгебри висловлювань називається КНФ, в якій:

- 1) кожен співмножник містить доданками всі змінні, причому кожен тільки один раз із запереченням або без;
- 2) відсутні повторення співмножників.

Алгоритми побудови ДКНФ

За допомогою еквівалентних перетворень. Формулу привести до КНФ. В отриманій КНФ до елементарних диз'юнкцій, в які доданками входять не всі змінні, додати нулі, представлені у вигляді кон'юнкцій кожної недостатньої змінної та її заперечення; за допомогою закону дистрибутивності привести ці співмножники до сум першого ступеня, що не містять добутку. Нарешті, виключити повторення співмножників.

Приклад. $(x \vee \bar{y}) \wedge (x \vee z) = (x \vee \bar{y} \vee \underbrace{z \wedge \bar{z}}_{=0}) \wedge (x \vee \underbrace{y \wedge \bar{y}}_{=0} \vee z) = (x \vee \bar{y} \vee z) \wedge (x \vee \bar{y} \vee \bar{z}) \wedge$
 $\wedge (x \vee y \vee z) \wedge (x \vee \bar{y} \vee z) = (x \vee \bar{y} \vee z) \wedge (x \vee \bar{y} \vee \bar{z}) \wedge (x \vee y \vee z) \text{ – ДКНФ.}$

Табличний метод. Розглядаються ті рядки таблиці істинності, де функція приймає значення 0. Кожному такому рядку відповідає диз'юнкція всіх змінних (без повторень), причому аргумент який приймає значення 0, береться без заперечення, а який приймає значення 1 – з запереченням. Нарешті утворюють кон'юнкцію отриманих диз'юнкцій.

Функція, що тотожно дорівнює одиниці, не має ДКНФ: $1 = x \vee \bar{x}$. Для нуля ДКНФ має вид: $0 = x \wedge \bar{x}$.

Теорема 3. Якщо логічна формула має ДДНФ, вона єдина.

Теорема 4. Якщо логічна формула має ДКНФ, вона єдина.

Принцип двоїстості в булевій алгебрі

Визначення. Функція $f_1(x_1, \dots, x_n)$ називається двоїстою для функції $f_2(x_1, \dots, x_n)$, якщо виконується співвідношення:

$$f_1(x_1, \dots, x_n) = \overline{f_2(\overline{x_1}, \dots, \overline{x_n})}. \quad (3)$$

Застосовуючи до обох частин рівності (*) заперечення і підставляючи замість усіх змінних їх заперечення, матимемо:

$$\overline{f_1(\overline{x_1}, \dots, \overline{x_n})} = f_2(x_1, \dots, x_n). \quad (4)$$

Тут функція f_2 двоїста до f_1 , тобто відношення двоїстості між функціями симетрично. З визначення двоїстості випливає, що для кожної функції двоїста функція визначається однозначно, зокрема, може виявитися, що функція двоїстна сама собі. У цьому випадку вона називається самодвоїстною, такою є операція заперечення.

У силу законів де Моргана диз'юнкція та кон'юнкція двоїстні одна одній:

$$\overline{a \vee b} = \overline{a} \wedge \overline{b}, \quad \overline{a \wedge b} = \overline{a} \vee \overline{b}.$$

Функцію, двоїстну до f , будемо позначати так:

$$[f(x_1, \dots, x_n)]^* = \overline{f(\overline{x_1}, \dots, \overline{x_n})} = f^* \quad (f^* \div f).$$

З визначення двоїстості очевидний вираз: $f^{**} = f$.

За допомогою таблиць істинності двоїста до f функція будується у два етапи:

1. стовпець значень функції інвертується (1 замінюється на 0 і навпаки);
2. отриманий стовпець перевертається.

Принцип двоїстості. Якщо у формулі F , що представляє функцію f , всі знаки функцій замінити відповідно на знаки двоїстих функцій, то отримана формула F^* представлятиме функцію f^* , двоїстну до f .

У булевій алгебрі принцип двоїстості має більш конкретний вигляд: якщо у формулі F , що представляє функцію f , всі кон'юнкції замінити на диз'юнкції, диз'юнкції замінити на кон'юнкції, 1 на 0, 0 на 1, то отримаємо формулу F^* , що представляє функцію f^* , двоїстну до f .

Якщо функції рівні, то й двоїсті їм функції також рівні, що дозволяє за допомогою принципу двоїстості отримувати нові еквівалентні співвідношення, переходячи від рівності $f_1 = f_2$ за допомогою зазначених заміни до рівності $f_1^* = f_2^*$.

Неформально граф можна визначити, як геометричну структуру, що складається з розкиданих в просторі або на площині точок (**вершин** графа), серед яких деякі пари точок з'єднані відрізками або кривими (**ребрами**). При цьому не має принципового значення, як вершини розташовані в просторі або на площині, і яку конфігурацію мають ребра.

Оскільки конфігурація графа не важлива, то його можна уявити безліччю способів. Наприклад, фігури: а), б), в) на рис. 25 являють собою один і той самий граф.

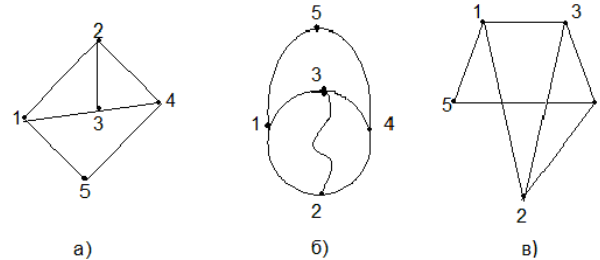


Рис. 25

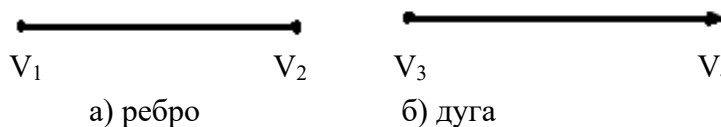
Іншими словами, графи а), б), в) *ізоморфні* між собою, тобто існує взаємно однозначна відповідність одночасно як між вершинами графів, так і між ребрами, що з'єднують ці вершини.

Зауваження. При зображенні графа на площині забороняється допускати самоперетин ребер, на рис. 26 не ребро графа.



Рис. 26

Визначення. Графом G називається пара множин $(V; E)$, де V – множина *вершин*, яка занумерована числами $1, 2, \dots, n$; $V = \{v\}$, E – множина впорядкованих або неупорядкованих пар $E = \{e\}$, $e = (V_1, V_2)$, які називаються відповідно *дугами* або *ребрами*.



Якщо у ребра або дуги початок і кінець збігаються, то їх називають *петлями*:

Вершини v_1, v_2 називаються *суміжними*, якщо їх пара утворює ребро або дугу, тобто якщо в графі $G = (V, E)$ існує ребро $e = (v_1, v_2) \in E$. У цьому випадку кожна з вершин v_1, v_2 та ребро e називаються *інцидентними*. Два ребра *суміжні*, якщо вони мають спільну вершину.

Всякому n -вершинному графу однозначно відповідає *матриця суміжності* $C = (c_{ij})$ розмірністю $n \times n$, де $c_{ij} = 1$, якщо існує ребро (або дуга) $e = (v_i, v_j) \in E$, $c_{ij} = 0$ у протилежному випадку. Наприклад, для графа на рис. 24 матриця суміжності буде виглядати наступним чином:

$$C = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

У множині E допускається більш ніж одне ребро з однаковими кінцевими вершинами. Такі ребра називають *паралельними* (*кратними*). Граф, в якому присутні паралельні ребра називають *мультиграфом*.

Визначення. Граф називається *простим*, якщо він не містить петель і паралельних ребер. Граф, що не містить ребер, називається *порожнім*, граф, який не має вершин - *нуль-графом*.

Визначення. Якщо елементом E може бути пара однакових елементів, тобто існують петлі, то граф називається *псевдографом*.

Визначення. Число інцидентних вершині V_i ребер називається *степенем вершини* і позначають $d(V_i)$. Вершину степеня 1 називають *вісячою* вершиною, її ребро теж називається *вісяче*. Вершину степеня 0 називають *ізолюваною*. За визначенням петля при вершині V_i додає число 2 в ступінь вершини.

Теорема 1. Сума степенів вершини графа G дорівнює $2m$, де m - число ребер графа G .

Теорема 2. Число вершин непарного степеня в будь-якому графі є парним числом.

Визначення. Орієнтований граф (орграф) – дугам присвоюється орієнтація, що позначається стрілками, спрямованими від початкової вершини до кінцевої.

Напівстепенем входу $d^+(v_i)$ вершини v_i називається число дуг, які заходять в вершину, а *напівстепенем виходу* $d^-(v_i)$ – число дуг, які виходять з вершини v_i .

Визначення. Граф $G'=(V',E')$ називається *підграфом* графа $G=(V, E)$, якщо мають місце включення: $V' \subset V, E' \subset E$ (рис.29 а), б)).

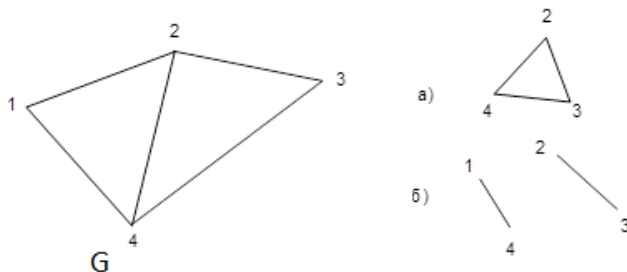


Рис. 29

Визначення. Частина $G'=(V',E')$ називається *суграфом* або *кістяковим підграфом* графа G , якщо виконані умови: $V'=V, E' \subset E$, але $E' \neq E$ (у нашому прикладі на рис.29 це б)).

Види графів

Тривіальний - складається з однієї вершини.

Повний - граф, в якому кожна пара вершин суміжна. K_p - повний граф з p вершинами; число ребер в такому графі дорівнює $m(K_p) = C_p^2 = (p \cdot (p-1))/2$.

Дводольний граф $G(V,E)$: $V=V_1 \cup V_2, V_1 \cap V_2 = \emptyset$, причому кожне ребро $e \in E$ є інцидентним вершинам з множин V_1 і V_2 . V_1, V_2 – частки дводольного графа. Якщо дводольний граф містить всі ребра, що з'єднують всі вершини множини V_1 з усіма вершинами з V_2 , то він називається *повним дводольним* графом і позначають $K_{m,n}$, де $|V_1|=m$ і $|V_2|=n$.

Зважені графи - граф $G(V,E)$ називається зваженим по вершинах, якщо на множині його вершин визначена вагова функція $U(v), v \in V$. Граф $G(V,E)$ називається зваженим по ребрах, якщо на множині його ребер визначена вагова функція $C(e), e \in E$.

Операції над графами

1. Об'єднання графів: $G_1(V_1,E_1) \cup G_2(V_2,E_2)$ - це граф $G(V,E)$, де $V=V_1 \cup V_2; E=E_1 \cup E_2$ (рис. 35).

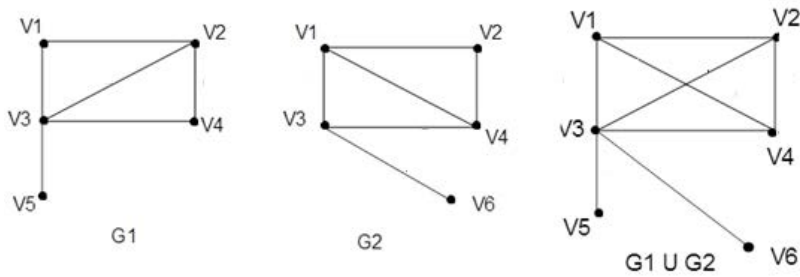


Рис. 35

2. З'єднання графів (пряма сума): $G_1(V_1, E_1) + G_2(V_2, E_2)$, за умови що $V_1 \cap V_2 = \emptyset$, $E_1 \cap E_2 = \emptyset$ – це граф $G(V, E)$, де $V = V_1 \cup V_2$ і $E = E_1 \cup E_2 \cup \{e = (v_1, v_2) : v_1 \in V_1 \text{ і } v_2 \in V_2\}$ (рис. 36).

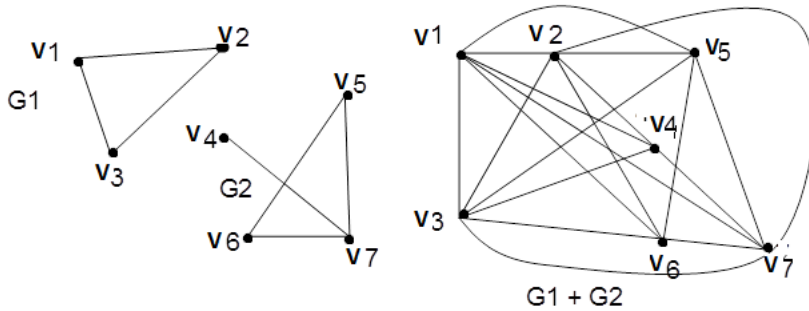


Рис. 36

3. Доповнення. Граф $G' = (V, E')$ називається доповненням простого графа $G = (V, E)$, якщо ребро (V_i, V_j) належить E' в тому і тільки в тому випадку, якщо воно не належить E , тобто V_i і V_j суміжні в G' тоді і тільки тоді, коли вони не суміжні в G .

4. Видалення вершини u з графа: $G_1(V_1, E_1) - u$ за умови що $u \in V_1$ – це граф $G_2(V_2, E_2)$, де $V_2 = V_1 \setminus \{u\}$ і $E_2 = E_1 \setminus \{e = (u, v_i) \text{ для кожного } i\}$.

5. Видалення ребра e з графа: $G_1(V_1, E_1) - e$ – це граф $G_2(V_2, E_2)$, де $V_2 = V_1$, $E_2 = E_1 \setminus \{e\}$ (рис.37).

6. Додавання вершини u в граф: $G_1(V_1, E_1) + u$ – граф $G_2(V_2, E_2)$, де $V_2 = V_1 \cup \{u\}$ і $E_2 = E_1$ (рис.37).

7. Додавання ребра e в граф: $G_1(V_1, E_1) + e$ дає граф $G_2(V_2, E_2)$, де $V_2 = V_1$ і $E_2 = E_1 \cup \{e\}$ (рис.37).

Зауваження. Всі наведені вище і нижче терміни сформульовані для графів неорієнтованих, проте вони вірні і для орієнтованих графів (орграфів), з тією лише відмінністю, що терміни ребро, ланцюг, цикл замінюються на терміни дуга, шлях, контур.

Нехай дано неорієнтований граф $G(V,E)$.

Визначення. *Маршрутом довжини $L-1$* з вершини V_1 до вершини V_L називається послідовність $M=\{(V_1,V_2), (V_2,V_3), \dots, (V_i,V_{i+1}), \dots, (V_{L-1},V_L)\}$ яка складається з ребер $e=(V_s,V_{s+1}) \in E$, при цьому кожні два сусідніх ребра мають спільну кінцеву вершину. *Довжиною маршруту M* називається кількість ребер в ньому. Наприклад, $M=\{(1,3),(3,4),(4,2)\}$ маршрут графа з рис. 30 довжиною 3. Якщо $V_1=V_L$, то маршрут замкнений, інакше - відкритий. Маршрут називається *ланцюгом*, якщо всі його ребра різні. Відкритий ланцюг (шлях) називається *простим ланцюгом (шляхом)*, якщо всі його вершини різні. Замкнений ланцюг називається *простим циклом*, якщо різні всі його вершини, за винятком рівних кінцевих вершин: $V_1=V_L$.

Визначення. *Ейлеревим ланцюгом* в графі G називається замкнутий ланцюг, що містить всі ребра графа G . До відкритого ейлерового ланцюга відноситься відкритий ланцюг, що містить всі ребра G . Граф, що містить ейлерів ланцюг називається *ейлерів граф*.

Теорема 1. Наступні твердження еквівалентні для зв'язного графа G :

- 1) G – ейлерів граф;
- 2) степені кожної вершини в графі G парний;
- 3) G є об'єднанням декількох реберно-непересічних циклів.

Природно виникає питання: як знайти хоча б один ейлерів цикл в ейлеревому графі G , тобто як занумерувати ребра графа числами 1, 2, ..., таким чином, щоб число, присвоєне ребру, вказувало порядковий номер цього ребра в ейлеревому циклі? Це можна зробити, якщо нумерувати ребра, дотримуючись наступних двох правил:

1). Починаючи з довільної вершини U , присвоюємо довільному ребру (U,V) номер 1. Потім викреслюємо це ребро (U, V) і переходимо у вершину V .

2). Нехай W - вершина, в яку ми прийшли в результаті виконання попереднього кроку і k - номер, присвоєний деякому ребру на цьому кроці. Вибираємо будь-яке ребро, яке інцидентне вершині W , причому міст вибираємо тільки в тому випадку, коли немає інших можливостей; присвоюємо обраному ребру номер $k+1$ і викреслюємо його.

Це є **алгоритм Флері**, який закінчується коли всі ребра викреслені, тобто занумеровані.

Визначення. *Гамільтоновим циклом* в графі G називається простий цикл, що містить всі вершини графа G . *Гамільтонів шлях* в G - це простий шлях, що містить всі вершини графа G . Граф G називається *гамільтоновим*, якщо він має гамільтонів цикл.

Задача Комівояжера

Постановка: комівояжеру необхідно відвідати кілька міст. Який маршрут він повинен обрати, щоб, почавши рухатися зі свого рідного міста, по одному разу побувати в кожному місті і потім повернутися додому по найкоротшому шляху? При цьому відомо відстань між кожною парою міст.

Уявимо міста у вигляді вершин графа, а дороги – у вигляді його ребер. Довжина дороги – вага відповідного ребра. Якщо між кожною парою міст існує дорога, що їх з'єднує, то математична постановка задачі буде записана наступним чином: у повному зваженому графі потрібно знайти гамільтонів цикл мінімальної ваги. Під вагою циклу розуміється сума ваг ребер, що належать цьому циклу.

Для задачі комівояжера найбільш популярним алгоритмом є метод гілок і границь. Він досить громіздкий. Ми ж розглянемо алгоритм знаходження гамільтонова циклу мінімальної ваги який має назву «йди в найближчий» і, взагалі кажучи, дає наближений розв'язок.

Алгоритм виконується по кроках $S=1,2,\dots,n$, де n - число вершин графа. На кроці $S=1$ фіксується будь-яка вершина $v_1 \in V$ і розглядається, множина $E(v_1) \subset E$ всіх ребер, які інцидентні v_1 . Серед них вибирається найкоротше, тобто таке ребро $e_1=(v_1, v_2)$, для якого вага $w(e_1)=\min_{e \in E(v_1)} w(e)$. Після цього вважаємо, що на першому кроці побудовано ланцюг $C_1=[v_1, v_2]$.

Нехай здійснено $S \leq n-2$ кроків, і в результаті S -го кроку побудовано простий ланцюг $C_S=[v_1, v_2, \dots, v_{s+1}]$. Позначимо через $E(v_{s+1})$ множину всіх ребер $e=(v_{s+1}, v) \in E$, які інцидентні вершині v_{s+1} , але не інцидентні ніякої вершині v з ланцюга C_S .

Крок $S+1$. Серед ребер множини $E(v_{s+1})$ вибираємо найкоротше e_{s+1} , тобто те, яке задовольняє умові $w(e_{s+1})=\min_{e \in E(v_{s+1})} w(e)$. Після чого ребро e_{s+1} приєднуємо до C_S і отримуємо простий ланцюг $C_{S+1}=[v_1, v_2, \dots, v_{s+1}, v_{s+2}]$.

Алгоритм закінчує роботу на кроці $S=n$, перед початком якого в результаті кроку $S=n-1$ виділено гамільтонів ланцюг $C_{n-1}=[v_1, v_2, \dots, v_n]$. Тоді на кроці $S=n$ гамільтонів ланцюг C_{n-1} замикається в гамільтонів цикл приєднанням до цього ланцюга ребра $e=(v_n, v_1) \in E$. Цей процес виявиться безрезультативним, якщо в графі не виявиться ребра $e=(v_n, v_1)$. На повному графі алгоритм «йди в найближчий» завжди знаходить деякий допустимий розв'язок задачі комівояжера.

Задача про найкоротший ланцюг

Постановка задачі: дано n -вершинний граф $G=(V, E)$, в якому виділено пара вершин $v_0, v' \in V$, і кожне ребро зважене числом $w(e) \geq 0$. Нехай $X=\{x\}$ - множина всіх простих ланцюгів, що з'єднують v_0 з v' , $x=(V_x, E_x)$. Цільова функція $F(x)=\sum w(e) \rightarrow \min$. Тобто необхідно знайти простий ланцюг мінімальної ваги, що з'єднує дві задані вершини.

Алгоритм Дейкстри

Введемо поняття "*К-ої найближчої вершини*", яке визначається індуктивно:

1-й крок індукції. Нехай зафіксовано вершину v_0 , E_1 - множина всіх ребер $e \in E$, які інцидентні v_0 . Серед ребер $e \in E_1$ обираємо ребро $e^{(1)}=(v_0, v_1)$, яке має найменшу вагу, тобто $w(e^{(1)})=\min(w(e))$. Тоді v_1 називається *першою найближчою вершиною*, число $w(e^{(1)})$ позначаємо $I^{(1)}=I(v_1)$ і називаємо *відстанню* до цієї найближчої вершини, прийmemo $I(v_0)=0$ і позначимо $V_1=\{v_0, v_1\}$ - множина найближчих вершин.

2-й крок індукції. Позначаємо E_2 - множину всіх ребер $e=(v', v'')$, $e \in E$, таких що $v' \in V_1$, $v'' \in (V \setminus V_1)$. Найближчим вершинам $v \in V_1$ приписані відстані $I(v)$ до кореня v_0 . Введемо позначення: U_1 - множина вершин $v \in (V \setminus V_1)$, для яких існують ребра виду $e=(v', v)$, де $v' \in V_1$. З усіх ребер $e \in E_2$ знаходимо таке ребро $e_2=(v', v_2)$, що величина $I(v')+w(e_2)$ є найменшою. Тоді v_2 називається *другою найближчою вершиною*, а ребра e_1, e_2 утворюють зростаюче дерево для виділених найближчих вершин $D_2=\{e_1, e_2\}$, $V_2=\{v_0, v_1, v_2\}$.

($S+1$)-й крок індукції. Нехай в результаті s кроків виділено множину найближчих вершин $V_s=(v_0, v_1, \dots, v_s)$ і відповідне їм зростаюче дерево $D_s=\{e_1, e_2, \dots, e_s\}$. Для кожної вершини $v \in V_s$ обчислено відстань $I(v)$ від кореня v_0 до v ; U_s - множина вершин $v \in (V \setminus V_s)$, для яких існують ребра виду $e=(v', v)$, де $v' \in V_s$. На кроці $S+1$ для кожної вершини $v_t \in V_s$ обчислимо наступну відстань:

$$L^{(s+1)}(v_r) = I(v_r) + \min(w(v_r, v)),$$

де $\min$ береться за всіма ребрами $e=(v_r, v)$, після чого знаходимо $\min$ серед $L^{(s+1)}(v_r)$. Нехай цей $\min$ досягнутий для вершин v_{r0} і відповідної їй $v^* \in (V \setminus V_s)$, тоді v^* назвемо v_{s+1} - це і є $(S+1)$ -а найближча вершина. Отримаємо множину $V_{s+1} = V_s \cup \{v_{s+1}\}$ і зростаюче дерево $D_{s+1} = D_s \cup (v_{r0}, v_{s+1})$. $(S+1)$ -й крок завершується перевіркою чи є чергова найближча вершина v_{s+1} зазначеною вершиною v' , яка повинна бути за умовою задачі пов'язана найкоротшим ланцюгом з вершиною v_0 . Якщо так, тоді довжина шуканого ланцюга дорівнює $I(v_{s+1}) = I(v_{r0}) + w(v_{r0}, v_{s+1})$, при цьому шуканий ланцюг відновлюється однозначно з ребер зростаючого дерева D_{s+1} . В іншому випадку слідє переходити до кроку $S+2$.

Визначення. Дві вершини V_i і V_j називаються *зв'язаними* в графі G , якщо в ньому існує шлях $V_i - V_j$. Граф, в якому всі вершини зв'язані, називається *зв'язним*.

Множину вершин V довільного графа можна розбити на такі підмножини $V_1, V_2, \dots, V_p$, що підграфи, побудовані на відповідних множинах V_i ($i=1, \dots, p$) є зв'язними, і ніяка вершина підмножини V_i не зв'язана ні з якою вершиною підмножини V_j , $V_i \cap V_j = \emptyset$, $i \neq j$. Підграфи $G_i = (V_i, E_i)$ ($i=1, \dots, p$) називаються *компонентами зв'язності* графа G . Ізольована вершина також є окремою компонентою зв'язності, вона вважається зв'язаною сама з собою.

Твердження. Якщо G - зв'язний граф, то у нього одна компонента зв'язності - G .

Визначення. *Мостом* графа $G = (V, E)$ називається таке ребро $e \in E$, що граф $G - G = (V, E \setminus \{e\})$ (тобто без цього ребра) має більше компонент зв'язності (рис. 31, e - міст).

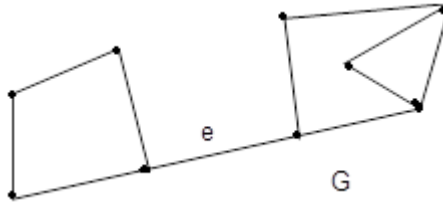


Рис. 31

Визначення. *Відстанню* $d(U, V)$ між вершинами U і V - називається довжина найкоротшого ланцюга, що з'єднує вершини U і V , а відповідний ланцюг - *геодезичним*.

Визначення. *Діаметром* графа $D(G)$ - називається довжина найдовшого геодезичного ланцюга.

Визначення. Граф без циклів називається *ациклічним або лісом*. Зв'язний ациклічний граф називається *деревом*. Таким чином, компонентами зв'язності лісу є дерева. *Деревом на графі G* називається зв'язний ациклічний підграф графа G. *Остов T графа G* – це дерево на графі G, що містить всі вершини графа G. Зв'язний підграф дерева T називається *піддеревом T*. *Кодерево T^** остова T графа G є підграф графа G, що містить всі вершини G і тільки ті ребра G, які не містяться в T. Ребра остова T називаються *гілками T*, а ребра відповідного кодера T^* - *хордами або зв'язками*. Кодерево може бути як зв'язним, так і незв'язним графом.

Теорема. Для графа G, що має n вершин і m ребер, наступні твердження еквівалентні:

- (1) G є деревом;
- (2) існує тільки один шлях між будь-якими двома вершинами в G;
- (3) G є зв'язним і $m=n-1$;
- (4) G-ациклічний граф і $m=n-1$;
- (5) G-ациклічний граф, і при з'єднанні ребром довільних двох несуміжних вершин буде отримано граф, що має рівно один цикл.

Визначення. Орієнтоване дерево — ациклічний оргграф (орієнтований граф, що не містить циклів), в якому тільки одна вершина має нульову напівстепінь входу, а всі інші вершини мають напівстепінь входу 1. Вершина з нульовим степенем входу називається *коренем* дерева, вершини з нульовим напівстепенем виходу (з яких не виходить жодне ребро) називаються *кінцевими вершинами або листям*.

Взагалі, якщо в оргграфі існує вершина, від якої можна дістатися орієнтованим шляхом до всіх інших вершин графа, то вона називається коренем. При цьому оргграф, в якому є корінь, називається *квазісильно зв'язним*. Квазісильно зв'язний граф не завжди є орієнтованим деревом.

Підграф оргграфа G називається *орієнтованим кістяком графа G*, якщо він є орієнтованим деревом і містить всі вершини графа G.

Задача про кістякове дерево мінімальної ваги.

Якщо задати довжини ребер, то можна поставити задачу знаходження найкоротшого кістяка. Ця задача має безліч практичних інтерпретацій. Наприклад, нехай маємо множину аеродромів і потрібно визначити мінімальний (за сумою відстаней) набір авіарейсів, який дозволив би перелетіти з одного аеродрому на будь-який інший. Розв'язком цієї задачі буде найкоротший кістяк повного графа відстаней між аеродромами. Розглянемо два алгоритми розв'язання цієї задачі.

Алгоритм Прима

Алгоритм Прима для даного n-вершинного графа $G=(V,E)$ складається з кроків $S=1,2, \dots, (n-1)$, де $|V|=n$ і на кожному кроці будується зростаюче дерево $D_S=(V,S,E_S)$ $V_S \subset V$, $E_S \subset E$.

$S=1$. Фіксуємо довільну вершину V_0 , та серед усіх ребер, інцидентних вершині V_0 знаходимо найкоротше ребро $e_1=(v_0,v_1)$, тобто мінімальної ваги. Покладаємо, що побудовано дерево $D_1=(V_1, E_1)$, $V_1=\{v_0,v_1\}$, $E_1=\{e_1\}$ і переходимо до кроку $S=2$.

Нехай виконано $S < n-1$ кроків, у результаті чого в графі G виділено поточне дерево $D_S=(V_S, E_S)$. Тоді на кроці $(s+1)$ серед усіх ребер $e=(v',v'')$, таких що $v' \in V_S$, $v'' \in (V \setminus V_S)$, знаходимо найкоротше ребро $e_{s+1}=(v_s, v_{s+1})$ і приєднуємо його до дерева D_S . Отримуємо дерево $D_{S+1}=(V_{S+1}, E_{S+1})$, $V_{S+1}=V_S \cup \{v_{s+1}\}$, $E_{S+1}=E_S \cup \{e_{s+1}\}$. Алгоритм закінчує свою роботу в двох випадках:

- 1) результативно на кроці $S = n-1$, у разі якщо граф G зв'язний;
- 2) безрезультативно, якщо G – незв'язний граф.

Алгоритм Краскала

Складається з двох етапів:

1. Підготовчий. Для даного графа G всі ребра впорядковуються в послідовність в порядку неспадання ваг цих ребер:

$$e_1, e_2, \dots, e_m, \quad m=|E|$$

$$w(e_1) \leq w(e_2) \leq \dots \leq w(e_3) \leq \dots \leq w(e_m)$$

2. Виконується за кроками $S=1, \dots, m_0 < m$ наступним чином: на кроках $S=1, 2$ ребра e_1, e_2 з послідовності фарбуються (включаються в розв'язок), на кожному наступному кроці $S \geq 3$, розглядається ребро e_s з послідовності і воно забарвлюється тоді і тільки тоді, коли не утворює циклу з ребрами, які вже є пофарбованими на попередніх кроках. В іншому випадку ребро e_s умовно викреслюється з графа $G=(V, E)$. Алгоритм закінчує роботу на кроці $S=m_0$, коли пофарбованими виявляться $(n-1)$ ребер, де $n=|V|$. Пофарбовані ребра будуть утворювати кістякове дерево n -вершинного графа мінімальної ваги.

Випадкові події

У теорії ймовірностей *подією* називають все те, що може статися при здійсненні деякого комплексу умов. Подія настає в результаті реалізації різних процесів, які будемо називати *дослідами (експериментами)*. Приклади подій:

A_1 – поява цифри при киданні монети;

A_2 – випадання парного числа очок при грі в кістки;

A_3 – замерзання води при сильному морозі;

A_4 – в переліку місяців року після січня безпосередньо йде квітень.

Якщо при реалізації деяких умов подія A завжди відбувається, то вона називається *достовірною*. Якщо ж ця подія при заданих умовах ніколи не настає, то її називають *неможливою*. Події називають *рівноможливими*, якщо є підстави вважати, що жодна з цих подій не є більш можливою, ніж інші.

Якщо в результаті експерименту при реалізації певного комплексу умов дана подія може наступити або не наступити, то вона називається *випадковою*. Умови проведення такого досліду часто називають *випадковим дослідом (експериментом)*. Зрозуміло, що після кидання гральної кістки число три може випасти, але воно може і не випасти. Через п'ять годин після включення комп'ютер може бути справним, але може і вийти з ладу. Стріляючи в мішень, можна влучити з першого пострілу, можна з третього, але можна не влучити і ніколи.

Нехай за даних умов проводиться випадковий дослід, в результаті якого обов'язково настає одна і тільки одна з можливих подій, нерозкладних на більш прості. Такі події називаються *елементарними* подіями. Множина Ω всіх елементарних подій w_i утворює простір елементарних подій. Наприклад, при киданні гральної кістки множина елементарних подій – випадків, утворює простір з шести елементарних результатів w_i : випало одне очко, випало два очки і т. д., випало шість очок: $\Omega = \{w_1 = 1; w_2 = 2; w_3 = 3; w_4 = 4; w_5 = 5; w_6 = 6\}$. Подія B називається *складеною*, якщо можна вказати, щонайменше, дві таких елементарних події, що з існування кожної з них окремо випливає існування події B . Цей факт записується у вигляді: $B = \{w_1; w_2\}$. Наприклад, при киданні гральної кістки складена подія $A = \{\text{число очок парне}\}$ можна записати так: $A = \{2; 4; 6\}$, маючи на

увазі при цьому: якщо випаде число 2 або 4 або 6, то настане подія A .

Події $A_1, A_2, \dots, A_n$ називаються *несумісними*, якщо в результаті одного експерименту ніякі дві з них не можуть відбутися одночасно. Це означає, що серед подій $A_1, A_2, \dots, A_n$ неможливо знайти таку пару подій A_i і A_j , в якій виявилось б хоча б по одній спільній елементарній події. Наприклад, при одноразовому киданні гральної кістки випадання парного і непарного числа – несумісні події. Несумісними є також промах і влучання при стрільбі по мішені.

Кілька подій утворюють *повну групу*, якщо вони є єдино можливими і несумісними наслідками випробування. Це означає, що в результаті випробування обов'язково має відбутися одна і тільки одна з цих подій. Наприклад, при киданні гральної кістки повну групу утворює множина з шести елементарних результатів. Іншим характерним прикладом є пара двох протилежних подій $(A, \bar{A})$. Наприклад, випадання герба і цифри в результаті підкидання монети, працездатність комп'ютера і його несправність в даний момент часу, влучання і невлучання в мішень при одному пострілі і т. і.

Правила дій над подіями

Сумою (об'єднанням) подій $A_1, A_2, \dots, A_n$ називається подія A , яка полягає в настанні хоча б однієї з цих подій. Позначається одним із таких способів:

$$A = A_1 + A_2 + \dots + A_n \quad \text{або} \quad A = A_1 \cup A_2 \cup \dots \cup A_n.$$

Приклад. В урні 6 куль, які відрізняються лише номером. Подія A_i – навмання витягнути кулю під номером i . Подія $A = A_1 + A_3 + A_5$ полягає в тому, що буде витягнута куля з номером 1 або 3, або 5, тобто з непарним номером.

Добутком (перетином) подій $A_1, A_2, \dots, A_n$ називається подія B , що полягає в обов'язковому настанні всіх цих подій. Позначається перетин подій так:

$$B = A_1 \cdot A_2 \cdot \dots \cdot A_n \quad \text{або} \quad B = A_1 \cap A_2 \cap \dots \cap A_n.$$

Приклад. В урні 12 куль, серед яких половина білих з номерами від 1 до 6, та половина чорних – з такими ж номерами. Нехай A = «витягнути білу кулю», B = «витягнути кулю з непарним номером». Тоді подія $C = A \cdot B$ означає «витягнути білу кулю з непарним номером». Як впливає з означення, перетин подій не зміниться, якщо поміняти місцями співмножники: $C = B \cdot A$.

Різницею подій A і B (позначається $A - B$ або $A \setminus B$) називається подія, яка полягає в настанні події A і одночасному ненастанні події B . Для попереднього прикладу подія $D = A - B$ означає «витягнути білу кулю з парним номером».

Доповнення (протилежна до A) – це подія $\bar{A}$, яка полягає в ненастанні події A . Так, якщо A = «витягнути білу кулю», то $\bar{A}$ = «витягнути не білу кулю».

Події A і B називаються *незалежними*, якщо поява однієї з них не змінює шанси появи іншої. Наприклад, одночасно підкидають дві гральні кістки. Поява на одній з них трьох очок жодним чином не залежить від того, яка кількість очок з'явилася на верхній грані іншої кістки. Якщо поява однієї події впливає на появу іншої, то такі події називаються *залежними*.

Приклад. В урні 4 пронумерованих підряд і однакових на дотик кулі. Виймається одна куля, а потім друга. Подія A – перша витягнута куля має парний номер. Подія B – витягнута другою куля має парний номер. Зрозуміло, що ці події залежні, оскільки від того, яку кулю було вийнято в першому досліді, залежать шанси настання другої події. Дійсно, якщо першою витягнули парну кулю, то шанси вийняти парну кулю і в другому досліді (подія $A \cdot B$) будуть удвічі менше, ніж якби першою було вийнято непарну кулю (подія $\bar{A} \cdot B$).

Поняття ймовірності

Класичний підхід. Розглянемо повну групу несумісних рівноможливих подій. Прикладами таких груп є число очок при киданні гральних кісток, число влучань в мішень при пострілах, що проводяться в однакових умовах, поява кулі з заданим номером при наявності в урні декількох нерозрізнених на дотик куль. Події, що утворюють таку групу, називаються *випадками*, а відповідна схема логічних побудов – системою випадків або *схемою урн*. Нехай серед усіх можливих у даній схемі випадків n нас цікавить тільки цілком певна їх підмножина m ($m \leq n$). Випадки, що входять в підмножину m будемо називати *сприятливими*. Наприклад, в урні дві білих, три чорних і п'ять червоних однакових на дотик куль. Будемо вважати сприятливим вибір білої кулі. Таких сприятливих випадків два. Поява ж чорної або червоної кулі – випадок несприятливий. Вочевидь, що їх вісім. Позначимо через A будь-який з сприятливих випадків. Імовірність події A обчислюється як відношення числа сприятливих випадків до загальної кількості

випадків: $p(A) = \frac{m}{n}$. Ця формула називається *класичною формулою* для обчислення ймовірностей, коли дослід зводиться до системи випадків.

Іноді в задачах кількість елементарних результатів буває настільки велика, що виписати їх усі не виявляється можливим. При визначенні значень m і n можуть виявитися корисними формули комбінаторики.

Якщо з даної різноелементної множини потужності n вибирається довільна впорядкована підмножина потужності m ($m < n$), то така вибірка називається *розміщеннями з n елементів по m без повторів*. Число розміщень з n по m позначається A_n^m : $A_n^m = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-m+1)$ або $A_n^m = \frac{n!}{(n-m)!}$. Комбінації, які

складаються з одних і тих самих n різних елементів і відрізняються тільки порядком їх розташування, називають *перестановками* і позначають P_n : $P_n = n!$

Комбінації, що складаються з n різних елементів по m елементів, які відрізняються хоча б одним елементом, називають *сполученнями* і позначають

$$C_n^m = \frac{n!}{m!(n-m)!}.$$

Нехай дана вибірка $\underbrace{aaa\dots a}_{m_1} \underbrace{bbb\dots b}_{m_2} \dots \underbrace{ccc\dots c}_{m_k}$, що складається з n елементів, причому, елемент a повторюється m_1 разів, елемент b – m_2 разів, і т. д., елемент c – m_k разів і $m_1 + m_2 + \dots + m_k = n$. Перестановки в такій вибірці, де є однакові елементи, називаються *перестановками з повтореннями* і число перестановок з повтореннями позначається $P_n(m_1, m_2, \dots, m_k)$: $P_n(m_1, m_2, \dots, m_k) = \frac{n!}{m_1! \cdot m_2! \cdot \dots \cdot m_k!}$.

Розміщеннями з повтореннями з n по m називаються впорядковані m -елементні вибірки, в яких елементи можуть повторюватися. Число розміщень з повтореннями з n по m позначається $\tilde{A}_n^m$: $\tilde{A}_n^m = n^m$.

Сполученнями з повтореннями з n по m називаються неупорядковані m -елементні вибірки, в яких елементи можуть повторюватися. Число сполучень з повтореннями з n по m позначається $\tilde{C}_n^m$. $\tilde{C}_n^m = \frac{(n+m-1)!}{(n-1)! \cdot m!}$. Для практичного

розв'язання використовують формулу $\tilde{C}_n^m = C_{n+m-1}^m$.

Геометричне визначення ймовірності було введено для подолання незастосовності класичного визначення ймовірності для нескінченного числа випадків досліду.

Визначення. Ймовірністю називається відношення довжини (площі, об'єму) області, що сприяє появі події A , до довжини (площі, об'єму) області всіх можливих значень досліду.

Зауваження. Недоліком є наступний факт. Для класичного визначення справедливими є зворотні твердження (наприклад, якщо $P = 0$, то подія A є неможливою), а для геометричного вони не виконуються (наприклад, ймовірність попадання точки в точку дорівнює нулю, при цьому дана подія не є неможливою).

Основні теореми теорії ймовірностей

Раніше випадкова подія визначалося як подія, яка при здійсненні сукупності умов (досліду) може відбутися або не відбутися. Якщо при обчисленні ймовірності події ніяких інших обмежень, крім цих умов, не накладається, то таку ймовірність називають *безумовною*. Якщо ж накладаються ще й інші додаткові умови, то ймовірність події називається *умовною*. Проводиться дослід з випадковим результатом, в результаті якого можливі дві події A і B . *Умовною ймовірністю* $p(B|A)$ називається ймовірність події B , яка обчислена в припущенні, що подія A відбулася.

Прямі методи обчислення ймовірностей не завжди ефективні, а іноді і просто неможливі. На практиці шукану ймовірність часто знаходять непрямыми методами, що дозволяють за відомими ймовірностями одних подій знаходити ймовірності інших подій, з ними пов'язаних. В основі цих непрямих методів лежать основні теореми теорії ймовірностей.

Теорема множення ймовірностей. Нехай розглядаються дві події A і B , для яких відомі ймовірності їх появи $p(A)$ і $p(B)$. Ймовірність добутку двох подій $p(A \cdot B)$ дорівнює добутку ймовірності однієї з них на умовну ймовірність іншої $p(B|A)$, тобто обчислену за умови, що перша подія відбулася: $p(A \cdot B) = p(A) \cdot p(B|A)$ (1)

Цю теорему можна поширити на n подій (2):
 $p(A_1 \cdot A_2 \cdot \dots \cdot A_n) = p(A_1) \cdot p(A_2 | A_1) \cdot p(A_3 | A_1 \cdot A_2) \cdot \dots \cdot p(A_n | A_1 \cdot A_2 \cdot \dots \cdot A_{n-1})$ Для незалежних

подій (коли $p(B | A) = p(B)$) формула (1) має спрощений вигляд: $p(A \cdot B) = p(A) \cdot p(B)$, тобто ймовірність добутку двох незалежних подій дорівнює добутку ймовірностей цих подій. Для ймовірності добутку n незалежних подій формула (2) набуває вигляду: $p(A_1 \cdot A_2 \cdot \dots \cdot A_n) = p(A_1) \cdot p(A_2) \cdot p(A_3) \cdot \dots \cdot p(A_n)$

Теорема додавання ймовірностей для несумісних подій: $p(A + B) = p(A) + p(B)$ – ймовірність настання в результаті експерименту хоча б однієї з двох несумісних подій дорівнює сумі ймовірностей цих подій.

Наслідок 1. Сума ймовірностей подій, що утворюють повну групу дорівнює одиниці: $p(A_1) + p(A_2) + \dots + p(A_n) = 1$.

Наслідок 2. Сума ймовірностей протилежних подій дорівнює одиниці: $p(A) + p(\bar{A}) = 1$.

У разі підрахунку ймовірності події C , яка настає або в разі настання події A , або при настанні події B , якщо A і B не є несумісними, користуються наступною теоремою:

Загальна теорема додавання ймовірностей: $p(A + B) = p(A) + p(B) - p(A \cdot B)$ (3) – ймовірність суми випадкових подій A і B дорівнює сумі ймовірностей цих подій мінус ймовірність їх спільної появи.

З формули (3) можна отримати формулу для ймовірності суми трьох подій:

$$p(A + B + C) = p(A) + p(B) + p(C) - p(AB) - p(AC) - p(BC) + p(ABC)$$

При збільшенні числа доданків формула ймовірності суми все більш ускладнюється. На практиці при розв'язанні деяких завдань є доцільним перехід до протилежної події. При цьому використовують правило: ймовірність суми протилежних подій завжди дорівнює одиниці, тобто.: $p(A + \bar{A}) = p(A) + p(\bar{A}) = 1$. В загальному випадку: $p(A_1 + A_2 + \dots + A_n) = 1 - p(\bar{A}_1 \cdot \bar{A}_2 \cdot \dots \cdot \bar{A}_n)$ (4).

Формула повної ймовірності

Наслідком обох теорем ймовірності: теореми додавання і теореми множення є формула повної ймовірності.

Нехай деяка подія A може відбутися з якоюсь імовірністю тільки як наслідок кожної з подій $H_1, H_2, \dots, H_n$, які утворюють повну групу несумісних подій. Такі події H_i ($i=1, n$) називаються *гіпотезами*. Задані ймовірності гіпотез $p(H_i)$ і умовні ймовірності настання події A з кожною з гіпотез $p(A|H_i)$. Імовірність настання події

A дається формулою повної ймовірності:
$$p(A) = \sum_{i=1}^n p(H_i) \cdot p(A|H_i)$$

$$p(A) = p(H_1)p(A|H_1) + p(H_2)p(A|H_2) + \dots + p(H_n)p(A|H_n) \quad (1).$$

Ця формула застосовується у випадках, коли випробування з випадковим результатом розпадається на два етапи: на першому немов би "розігруються" умови дослідів, а на другому – його результат.

Формула апостеріорної ймовірності

Базується на формулі повної ймовірності та теоремі множення ймовірностей. Розглянемо ситуацію в деякому сенсі протилежну попередньої. Нехай до проведення деякого дослідів про його умови можна зробити n припущень (гіпотез), що виключають одна одну і утворюють повну групу: $H_1, H_2, \dots, H_n$. Ймовірності гіпотез до дослідів (апостеріорні ймовірності) відомі: $p(H_i)$, $i=1, n$. Випробування проведене і відбулася деяка подія A . Потрібно визначити ймовірність того, що подія відбулася спільно саме з гіпотезою H_i . Відповідь на таке запитання дає формула апостеріорної ймовірності або формула Байеса:

$$p(H_i | A) = \frac{p(H_i) \cdot p(A|H_i)}{\sum_{i=1}^n p(H_i) \cdot p(A|H_i)} = \frac{p(H_i) \cdot p(A|H_i)}{p(A)} \quad (2).$$

Імовірність гіпотези після випробування дорівнює добутку ймовірності гіпотези до випробування на відповідну їй умовну ймовірність події, яка сталася під час випробування, поділений на повну ймовірність цієї події.

Схема дослідів Бернуллі

Нехай проводиться n однакових і незалежних випробувань, в кожному з яких подія A може з'явитися або не з'явитися. При цьому говорять, що реалізується схема

дослідів Бернуллі. Наступ події A зазвичай називають *успіхом*, а ненастання - *невдачею*. Будемо вважати, що ймовірність події A в кожному випробуванні однакова, а саме дорівнює p . Отже, ймовірність ненастання події A в кожному випробуванні також є сталою і дорівнює $q = 1 - p$. Поставимо своїм завданням обчислити ймовірність того, що при n випробуваннях подія A відбудеться рівно k разів і, отже, не відбудеться $n - k$ разів. Важливо підкреслити, що не потрібно, щоб подія A повторювалась рівно k разів в певній послідовності. Шукану ймовірність позначимо $P_n(k)$. Наприклад, символ $P_5(3)$ означає ймовірність того, що в п'яти випробуваннях подія з'явиться рівно 3 рази і, отже, не наступить 2 рази. Ймовірність однієї складної події, яка полягає в тому, що в випробуваннях подія A відбудеться k разів і не відбудеться $n - k$ разів, за теоремою множення ймовірностей незалежних подій, дорівнює $p^k \cdot q^{n-k}$. Оскільки в задачі не потрібно враховувати порядок послідовності подій A і $\bar{A}$, то необхідно врахувати всі складні події. Число складних подій, що відрізняються лише порядком настання подій A і $\bar{A}$, дорівнює числу сполучень з n по k . Отже, шукана формула має вигляд:

$$P_n(k) = C_n^k \cdot p^k \cdot q^{n-k} \quad \text{або} \quad P_n(k) = \frac{n!}{k!(n-k)!} \cdot p^k \cdot q^{n-k} \quad (3).$$

Отриману формулу називають *формулою Бернуллі*.

Формула Бернуллі важлива тим, що справедлива для будь-якої кількості незалежних випробувань.

Властивості формули Бернуллі:

1. Права частина формули (3) є загальним членом розкладання бінома Ньютона:

$$(q + p)^n = \sum_{k=0}^n P_n(k) = \sum_{k=0}^n C_n^k \cdot p^k \cdot q^{n-k} = 1, \quad \text{тому ймовірності } P_n(k) \text{ називають}$$

біноміальними ймовірностями.

2. Число k_0 , якому відповідає максимальна ймовірність $P_n(k_0)$, називається *найімовірнішим числом* появи події A і визначається нерівностями

$$np - q \leq k_0 \leq np + p \quad (4),$$

тобто ймовірність появи події k разів при n випробуваннях перевищує (або, принаймні, не менше) ймовірності інших можливих результатів випробувань. Якщо число $np - q$ — ціле число, то

найімовірніших чисел два: $np - q$ і $np + p$. Якщо $np - q < 0$, то найімовірніше число виграшів дорівнює нулю.

3. Імовірність $P_n(k_1 \leq k \leq k_2)$ того, що в n дослідах схеми Бернуллі подія A з'явиться від k_1 до k_2 разів ($0 \leq k_1 \leq k_2 \leq n$), дорівнює:

$$P_n(k_1 \leq k \leq k_2) = \sum_{k=k_1}^{k_2} P_n(k) = \sum_{k=k_1}^{k_2} C_n^k \cdot p^k \cdot q^{n-k} \quad (5).$$

Легко бачити, що користуватися формулою Бернуллі при великих значеннях n досить важко, оскільки формула вимагає виконання дій над величезними числами.

Тому виникає необхідність в знаходженні наближених формул для ймовірностей $P_n(k)$, а також для сум виду $\sum_{k=k_1}^{k_2} P_n(k)$ (1) при великих значеннях n .

Розглянемо чотири формули такого роду. Перші дві з них засновані на так званих граничних теоремах Лапласа. Дві інші наближені формули засновані на граничній теоремі Пуассона. Назва «гранична» в обох випадках пов'язана з тим, що теореми встановлюють поведінку ймовірностей $P_n(k)$ (або сум виду (1)) за певних умов, до числа яких обов'язково входить умова $n \rightarrow \infty$.

Локальна теорема Лапласа. Якщо ймовірність p появи події A в кожному випробуванні постійна і відмінна від нуля і одиниці, то ймовірність $P_n(k)$ того, що подія A з'явиться в n випробуваннях рівно k раз, наближено дорівнює (тим точніше, чим більше n) значенню функції $y = \frac{1}{\sqrt{npq}} \cdot \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$, де $x = \frac{k - np}{\sqrt{npq}}$.

$$\text{Формула Лапласа: } P_n(k) \approx \frac{1}{\sqrt{npq}} \cdot \varphi(x) \quad (2). \text{ Для функції } \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

складена таблиця її значень. Ця функція є парною, тобто $\varphi(-x) = \varphi(x)$, і спадною, $\varphi(x) \rightarrow 0$ при $x \rightarrow \infty$. На практиці можна вважати, що $\varphi(x) \approx 0$ вже при $x > 4$.

Для окремого випадку, а саме для $p = 1/2$, асимптотична формула була знайдена в 1730 р. Муавром; в 1783 р. Лаплас узагальнив формулу Муавра для довільного p , що відрізняється від 0 і 1. Тому теорему іноді називають теоремою Муавра-Лапласа.

Інтегральна теорема Лапласа. Якщо ймовірність p появи події A в кожному випробуванні постійна і відмінна від нуля і одиниці, то ймовірність $P_n(k_1 \leq k \leq k_2)$ того, що подія A з'явиться в n випробуваннях від k_1 до k_2 разів, наближено

дорівнює $P_n(k_1 \leq k \leq k_2) \approx \Phi\left(\frac{k_2 - np}{\sqrt{npq}}\right) - \Phi\left(\frac{k_1 - np}{\sqrt{npq}}\right)$ (3), де $\Phi(x)$ позначає функцію

$\Phi(x) = \int_0^x \varphi(t) dt = \frac{1}{\sqrt{2\pi}} \int_0^x e^{-\frac{t^2}{2}} dt$. Для функції $\Phi(x)$ також складена таблиця її значень,

оскільки невизначений інтеграл $\int e^{-\frac{t^2}{2}} dt$ не можна виразити через елементарні функції. Функція $\Phi(x)$ непарна, тому для від'ємних значень $\Phi(-x) = -\Phi(x)$. Ця функція монотонно зростаюча, причому $\Phi(x) \rightarrow 0,5$ при $x \rightarrow \infty$. На практиці $\Phi(x) \approx 0,5$ приймається для $x > 5$.

Зауваження: точність формул (2) і (3) поліпшується із зростанням добутку npq . Зазвичай цими формулами користуються, коли $npq \geq 20$. Звідси видно, що, чим ближче одне з чисел, p або q , до нуля (інше число в цьому випадку близько до одиниці), тим більшим слід брати n . Тому в разі близькості однієї з величин p або q до нуля значно більш точними є наближені формули Пуассона.

Будемо вважати для визначеності, що мале p (у випадку малого q потрібно було б просто змінити позначення – замість A розглянути подію $\bar{A}$, при цьому числа p і q поміняються ролями). На практиці формули Пуассона зазвичай використовуються при $p < 0,1$. Отже, нас цікавить оцінка для ймовірності того чи іншого числа успіхів, коли сам по собі успіх є *рідкісною подією*. Приклади таких подій: народження близнюків, досягнення столітнього віку і т. П. Шукану оцінку для $P_n(k)$ дає так звана *гранична теорема Пуассона*. Вираз для $P_n(k)$ є формально функцію трьох змінних: n , p і k . Припустимо, що k фіксовано, а n і p змінюються. Більш точно: нехай $n \rightarrow \infty$, а $p \rightarrow 0$, притому так, що величина $\lambda = np$ залишається постійною ($\lambda = \text{const}$).

Гранична теорема Пуассона. При зазначених вище умовах справедлива рівність $\lim_{n \rightarrow \infty} P_n(k) = \frac{\lambda^k}{k!} \cdot e^{-\lambda}$.

Із граничної теореми Пуассона випливають такі наближені *формули Пуассона*:

При великих n і малих p (таких, що $\lambda = np \leq 10$) справедливі наближені рівності:

$$P_n(k) \approx \frac{\lambda^k}{k!} \cdot e^{-\lambda} \quad (7); \quad \sum_{k=k_1}^{k_2} P_n(k) \approx \sum_{k=k_1}^{k_2} \frac{\lambda^k}{k!} \cdot e^{-\lambda} \quad (8).$$

Для виразу $\frac{\lambda^k}{k!} \cdot e^{-\lambda}$, що розглядається як функція двох змінних k і λ , складена таблиця значень. Звернемо увагу: щоб знайти ймовірність того чи іншого числа успіхів, зовсім не потрібно знати n , p . Все визначається числом $\lambda = np$, яке є не чим іншим, як *середнім числом успіхів*.

Під **законом великих чисел** у широкому сенсі розуміється принцип, за яким сукупна дія величезної кількості випадкових чинників дія призводить до результату, який майже не залежить від випадку. У вузькому значенні під законом великих чисел розуміється ряд математичних теорем, у кожній з яких для тих чи інших умов встановлюється факт наближення середніх характеристик великої кількості випробувань до певних постійних.

До них, зокрема, відносяться граничні теореми формули Бернуллі.

Числові характеристики дискретних випадкових величин

Закон розподілу повністю характеризує випадкову величину. Однак, коли неможливо знайти закон розподілу, або цього не потрібно, можна обмежитися знаходженням значень, які називаються числовими характеристиками випадкової величини. Ці величини визначають деяке середнє значення, навколо якого групуються значення випадкової величини, і ступінь їх розкиданості навколо цього середнього значення.

Мододою M_0 дискретної випадкової величини називається її найбільш ймовірне значення.

Математичним очікуванням дискретної випадкової величини називається сума добутків всіх можливих значень випадкової величини на їх ймовірності:

$$m_x = M(X) = x_1 p_1 + x_2 p_2 + \dots + x_n p_n = \sum_{i=1}^n x_i p_i$$

З точки зору ймовірності можна сказати, що математичне очікування приблизно дорівнює середньому арифметичному спостережуваних значень випадкової величини.

Нехай проводиться n незалежних випробувань, ймовірність появи події A в яких дорівнює p .

Теорема. Математичне очікування $M(X)$ числа появи події A в n незалежних випробуваннях дорівнює добутку числа випробувань на ймовірність появи події в кожному випробуванні: $M(X) = np$.

Крім математичного очікування вводиться величина, яка характеризує відхилення значень випадкової величини від математичного очікування. Це відхилення дорівнює різниці між випадковою величиною і її математичним очікуванням. При цьому математичне очікування відхилення дорівнює нулю. Це пояснюється тим, що одні можливі відхилення додатні, інші від'ємні, і в результаті їх взаємного погашення виходить нуль.

Дисперсією (розсіюванням) випадкової величини називається математичне очікування квадрата відхилення випадкової величини від її математичного очікування: $D(X) = M[X - M(X)]^2$.

Частіше застосовується інший спосіб.

Теорема. Дисперсія дорівнює різниці між математичним очікуванням квадрата випадкової величини X і квадратом її математичного очікування:
$$D(X) = M(X^2) - [M(X)]^2.$$

Теорема. Дисперсія кількості появ події A в n незалежних випробуваннях, в кожному з яких ймовірність p появи події постійна, дорівнює добутку числа випробувань на ймовірності появи і не появи події в кожному випробуванні:
$$D(X) = npq.$$

Середнім квадратичним відхиленням випадкової величини X називається квадратний корінь з дисперсії: $\sigma(X) = \sqrt{D(X)}.$

Числові характеристики неперервних випадкових величин

Нехай неперервна випадкова величина X задана функцією щільності розподілу $f(x)$. Припустимо, що всі можливі значення випадкової величини належать відрізку $[a; b]$.

Математичним очікуванням неперервної випадкової величини X , можливі значення якої належать відрізку $[a; b]$, називається визначений інтеграл

$$M(X) = \int_a^b xf(x)dx.$$

Якщо можливі значення випадкової величини розглядаються на всій числовій осі, то математичне очікування знаходять за формулою: $M(X) = \int_{-\infty}^{\infty} xf(x)dx$. При цьому вважається, що невластний інтеграл збігається.

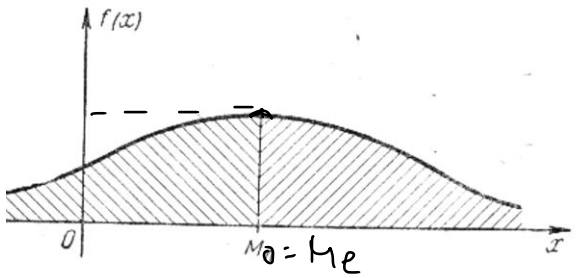
Дисперсією неперервної випадкової величини називається математичне очікування квадрата її відхилення.

$$D(X) = \int_{-\infty}^{\infty} [x - M(X)]^2 f(x)dx$$

За аналогією з дисперсією випадкової дискретної величини, для практичного обчислення дисперсії використовується формула: $D(X) = \int_{-\infty}^{\infty} x^2 f(x)dx - [M(X)]^2.$

Середнім квадратичним відхиленням називається квадратний корінь з

дисперсії: $\sigma(X) = \sqrt{D(X)}$.



Модою M_0 неперервної випадкової величини називається таке її значення, при якому щільність розподілу має максимум:

$$f(M_0) = \max.$$

Медіаною M_E випадкової величини X називається таке її значення, щодо якого рівноймовірно отримання більшого або меншого значення випадкової величини: $P(X < M_E) = P(X > M_E)$. Геометрично медіана – абсциса точки, в якій площа, що обмежена кривою розподілу, ділиться навпіл. Якщо мода і медіана збігаються з математичним очікуванням, то розподіл називається *симетричним*.

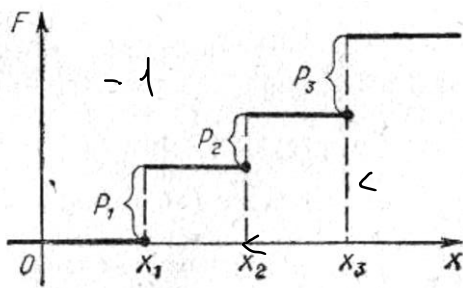
Закон розподілу дискретної випадкової величини

Співвідношення між можливими значеннями випадкової величини і їх ймовірностями називається *законом розподілу* дискретної випадкової величини. Закон розподілу може бути заданий аналітично, у вигляді таблиці або графічно. Таблиця відповідності значень випадкової величини x_i і їх ймовірностей p_i називається *рядом розподілу*. При цьому повинні виконуватися дві умови:

- 1) об'єднання всіх x_i має дати групу подій
- 2) сума всіх p_i повинна дорівнювати 1.

Функція розподілу

Найбільш загальною формою закону розподілу, яка придатна для всіх випадкових величин (як дискретних, так і неперервних), є функція розподілу $F(x)$. Функцією розподілу $F(x)$ випадкової величини X називається ймовірність того, що X прийме значення, яке менше, ніж задане x : $F(x) = P(X < x)$. Функція розподілу



дискретної випадкової величини є розривна ступінчаста функція, скачки якої відбуваються в точках, що відповідають можливим значенням випадкової величини, та дорівнюють ймовірностям цих значень. $F(x) = \sum_{x_i < x} p_i$, де підсумовування

поширюється на всі значення x_i , які менше за x .

Властивості функції розподілу:

1. Значення функції розподілу належать відрізку $[0;1]$: $0 \leq F(x) \leq 1$.
2. $F(-\infty) = 0$; $F(+\infty) = 1$.
3. $F(x_1) \leq F(x_2)$ при $x_1 < x_2$, тобто $F(x)$ – не спадна функція
4. $P(a \leq X < b) = F(b) - F(a)$. Ймовірність того, що випадкова величина прийме значення, що належить інтервалу $[a; b)$, дорівнює приросту функції розподілу на цьому інтервалі.

Функція розподілу неперервної випадкової величини (НВВ)

Для НВВ до перерахованих властивостей функції розподілу слід додати: 5) Ймовірність того, що неперервна випадкова величина прийме одне певне значення, дорівнює нулю. Таким чином, не має сенсу говорити про конкретне значення

випадкової величини. Інтерес представляє тільки ймовірність попадання випадкової величини в будь-якої інтервал, що відповідає більшості практичних задач. Функцію розподілу випадкової величини також називають інтегральною функцією.

Щільність розподілу

Функція розподілу повністю характеризує випадкову величину, проте, має один недолік. За функцією розподілу важко судити про характер розподілу випадкової величини в невеликому околі тієї чи іншої точки числової осі. *Щільністю розподілу* ймовірностей неперервної випадкової величини називається функція $f(x)$ – перша похідна від функції розподілу $F(x)$: $f(x) = F'(x)$. Щільність розподілу також називають диференціальною функцією. Графік функції щільності розподілу називається *кривою розподілу*. Для опису випадкової дискретної величини щільність розподілу є неприйнятною.

Сенс щільності розподілу полягає в тому, що вона показує, як часто з'являється випадкова величина X в деякому околі точки x при повторенні дослідів.

Таким чином, випадкова величина X називається *неперервною*, якщо її функція розподілу $F(x)$ неперервна на всій осі OX , а щільність розподілу $f(x)$ існує всюди, за винятком можливо, скінченного числа точок.

Теорема. Ймовірність того, що неперервна випадкова величина X прийме значення, що належить інтервалу $[a; b]$, дорівнює визначеному інтегралу від щільності розподілу, взятому в границях від a до b : $P(a \leq X \leq b) = \int_a^b f(x)dx = F(b) - F(a)$.

Геометрично це означає, що ймовірність того, що неперервна випадкова величина прийме значення, що належить інтервалу $[a; b]$, дорівнює площі криволінійної трапеції, яка обмежена віссю OX , кривою розподілу $f(x)$ і прямими $x = a$ і $x = b$. Функція розподілу може бути знайдена, якщо відома щільність розподілу, за формулою: $F(x) = \int_{-\infty}^x f(x)dx$.

Властивості щільності розподілу

1) Щільність розподілу – невід'ємна функція: $f(x) \geq 0$

2) Невласний інтеграл від щільності розподілу в межах от $-\infty$ до ∞ дорівнює

одиниці: $\int_{-\infty}^{\infty} f(x)dx = 1.$

Геометрично це означає, що площа криволінійної трапеції дорівнює одиниці.

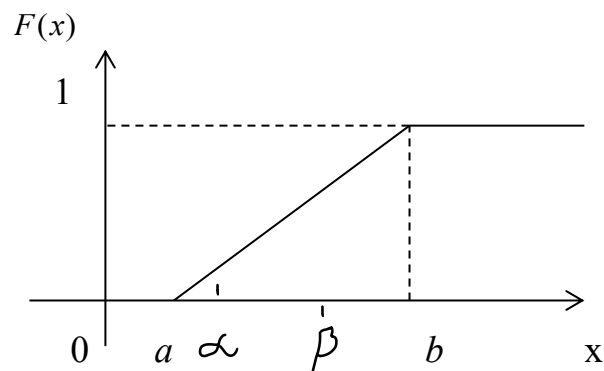
Розподіли НВВ

Неперервна випадкова величина має **рівномірний розподіл** на відрізку $[a; b]$, якщо на цьому відрізку щільність розподілу випадкової величини постійна, а поза

ним дорівнює нулю:
$$f(x) = \begin{cases} 0, & x < a \\ C, & a \leq x \leq b \\ 0, & x > b \end{cases}$$

Постійна величина C може бути визначена за умови рівності одиниці площі, обмеженої кривою розподілу.

Отримуємо $C = \frac{1}{b-a}$. Функція розподілу:
$$F(x) = \begin{cases} 0, & \text{при } x < a \\ \frac{x-a}{b-a}, & \text{при } a \leq x \leq b \\ 1, & \text{при } x > b \end{cases}$$



Для того щоб випадкова величина підпорядковувалася закону рівномірного розподілу необхідно, щоб її значення лежали всередині деякого певного інтервалу, і всередині цього інтервалу значення цієї випадкової величини були б рівноймовірні. Числові характеристики випадкової величини, підпорядкованої

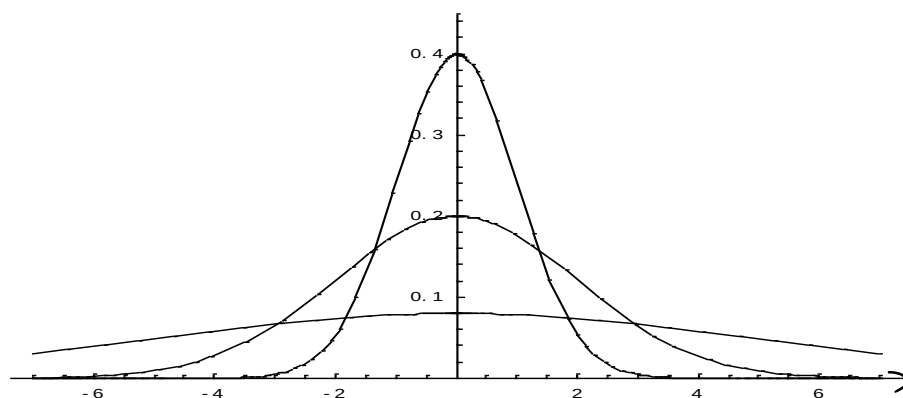
рівномірному закону розподілу: $m_x = \frac{a+b}{2}$; $D_x = \frac{(b-a)^2}{12}$; $\sigma_x = \sqrt{D_x} = \frac{b-a}{2\sqrt{3}}$.

Ймовірність потрапляння випадкової величини в заданий інтервал:

$$P(\alpha \leq X \leq \beta) = \int_{\alpha}^{\beta} \frac{dx}{b-a} = \frac{\beta - \alpha}{b - a}.$$

Нормальним називається розподіл ймовірностей неперервної випадкової

величини, який описується щільністю ймовірності $f(x) = \frac{1}{\sigma_x \sqrt{2\pi}} e^{-\frac{(x-m_x)^2}{2\sigma_x^2}}$, де m_x і σ_x є відповідно математичним очікуванням і середнім квадратичним відхиленням випадкової величини X . Нормальний закон розподілу також називається законом Гауса. Нормальний закон розподілу займає центральне місце в теорії ймовірностей. Це обумовлено тим, що цей закон проявляється у всіх випадках, коли випадкова величина є результатом дії великої кількості різних факторів. До нормального закону наближаються всі інші закони розподілу.



Функція розподілу: $F(x) = \frac{1}{\sigma_x \sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{(x-m_x)^2}{2\sigma_x^2}} dx$. Графік щільності

нормального розподілу називається нормальною кривою або кривою Гауса. При $a=0$ і $\sigma=1$ крива називається *нормованою*. Рівняння нормованої кривої:

$$\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}.$$

Властивість 1. Ймовірність потрапляння випадкової величини, що розподілена за нормальним законом, у заданий інтервал:

$$P(a \leq X \leq b) = \int_a^b f(x) dx = \frac{1}{\sigma \sqrt{2\pi}} \int_a^b e^{-\frac{(x-m)^2}{2\sigma^2}} dx. \text{ Позначимо } \frac{x-m}{\sigma\sqrt{2}} = t; \frac{a-m}{\sigma\sqrt{2}} = \alpha; \frac{b-m}{\sigma\sqrt{2}} = \beta.$$

Тоді
$$P(a \leq X \leq b) = \frac{1}{\sigma \sqrt{2\pi}} \int_{\alpha}^{\beta} e^{-t^2} \sigma \sqrt{2} dt = \frac{1}{\sqrt{\pi}} \int_{\alpha}^{\beta} e^{-t^2} dt = \frac{1}{2} [\bar{\Phi}(\beta) - \bar{\Phi}(\alpha)],$$
 де

$\bar{\Phi}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$ — функція Лапласа або *інтеграл ймовірностей*. Функцію

Лапласа також називають *функцією помилок* і позначають $\text{erf } x$. Частіше

використовується *нормована* функція Лапласа, яка пов'язана з функцією Лапласа співвідношенням: $\Phi(x) = \frac{1}{2} \overline{\Phi}\left(\frac{x}{\sqrt{2}}\right) = \frac{1}{\sqrt{2\pi}} \int_0^x e^{-\frac{t^2}{2}} dt$. Для неї складена таблиця значень.

Властивість 2. Ймовірність того, що відхилення випадкової величини X , яка розподілена за нормальним законом, від її математичного очікування не перевищує величину $\Delta > 0$ (за модулем), дорівнює $P(|X - a| \leq \Delta) = 2\Phi(t)$, де $t = \frac{\Delta}{\sigma}$.

Розглядаючи цю властивість, можна виділити важливий особливий випадок.

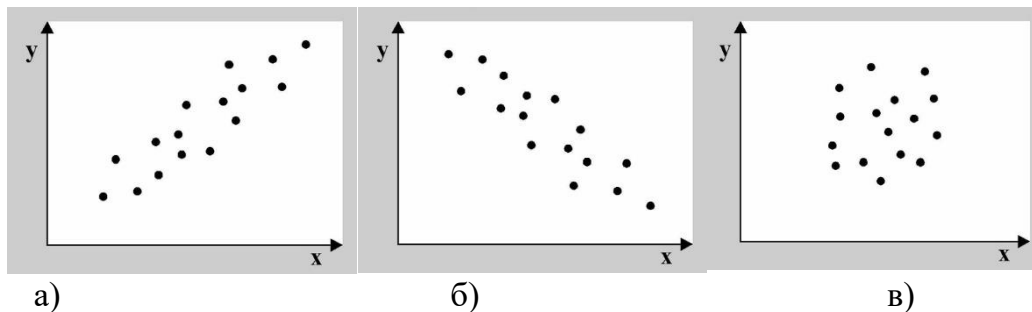
Нехай $\Delta = \sigma$, тоді $P(|X - a| \leq \sigma) = 2\Phi(1) = 0,6827$; якщо $\Delta = 2\sigma$, то $P(|X - a| \leq 2\sigma) = 2\Phi(2) = 0,9545$. Якщо прийняти $\Delta = 3\sigma$, то $P(|X - m| \leq 3\sigma) = 2\Phi(3) = 2 \cdot 0,49865 = 0,9973$. $\Rightarrow$
 $P(|X - m| > 3\sigma) = 1 - P(|X - m| \leq 3\sigma) = 1 - 0,9973 = 0,0027$, тобто ймовірність того, що випадкова величина відхилиться від свого математичного очікування на значення, яке більше, ніж потроєне середнє квадратичне відхилення, майже дорівнює нулю.

Це правило називається ***правилом трьох сигм***. На практиці прийнято вважати, що якщо для будь-якої випадкової величини виконується правило трьох сигм, то ця випадкова величина має нормальний розподіл.

Методи аналізу статистичних залежностей

Зв'язок, який існує між випадковими величинами різної природи, наприклад, між величиною X і величиною Y , не обов'язково є наслідком прямої залежності однієї величини від іншої (функціональний зв'язок). У деяких випадках обидві величини залежать від цілої сукупності різних факторів, загальних для обох величин, в результаті чого і формується пов'язані одна з одною закономірності. Такий зв'язок між величинами називається статистичним.

Якщо між двома випадковими величинами є зв'язок, кажуть, що між ними є *кореляція*. Якщо збільшення однієї випадкової величини пов'язане зі збільшенням іншої випадкової величини, кореляція називається *прямою*. Якщо зростання однієї величини пов'язане зі зменшенням іншої, говорять про *зворотну* кореляцію.



Якщо графік має вигляд а), то це говорить про наявність прямої кореляції, в разі, якщо він має вигляд б) – кореляція зворотна. Відсутність кореляції теж можна приблизно визначити за видом графіка – це випадок в).

Кореляційний аналіз дозволяє визначити наявність зв'язку між двома випадковими величинами, оцінити, наскільки тісний і істотний цей зв'язок. Все це виражається кількісно.

Парна лінійна кореляційна залежність

Ознаки X і Y знаходяться в *кореляційній залежності*, якщо кожному значенню x_i однієї ознаки відповідає певна умовна середня $\bar{y}_{x_i}$ іншої ознаки. *Парна* кореляційна залежність буде *лінійною*, якщо вона наближено виражається лінійною функцією. Вид залежності можна визначити графічно. З цією метою будуються точки з координатами $(x_i, \bar{y}_{x_i})$. За розташуванням побудованих точок підбирається лінія. Якщо це буде пряма, то зв'язок лінійний.

Метою кореляційного аналізу є оцінка тісноти зв'язку між ознаками. Для цього знаходиться *вибірковий лінійний коефіцієнт кореляції* за формулою

$r_b = \frac{\overline{xy} - \bar{x} \cdot \bar{y}}{\sigma_x \cdot \sigma_y}$, де $\bar{x}$, $\bar{y}$, $\overline{xy}$ – вибіркові середні; σ_x , σ_y – вибіркові середні квадратичні відхилення.

Оскільки коефіцієнт кореляції r_b розраховується за вибірковими даними і є оцінкою генерального коефіцієнта кореляції $r_{\text{ген}}$, то необхідно перевірити значущість r_b . З цією метою висуваються нульова і конкуруюча гіпотези: $H_0: r_{\text{ген}}=0$, $H_1: r_{\text{ген}} \neq 0$.

Нульова гіпотеза перевіряється при заданому рівні значущості α за допомогою випадкової величини T , яка має розподіл Стюдента з $k = n - 2$ ступенями свободи:

$$T = \frac{r_b \cdot \sqrt{n-2}}{\sqrt{1-r_b^2}}.$$

За вибірковими даними розраховують $T_{\text{спост}}$, а за таблицею критичних точок розподілу Стюдента знаходять $t_{\text{кр}}(\alpha, k)$ з урахуванням двосторонньої критичної області. Порівнюють $T_{\text{спост}}$ і $t_{\text{кр}}(\alpha, k)$. Якщо $|T_{\text{спост}}| < t_{\text{кр}}(\alpha, k)$, тобто спостережуване значення критерію потрапило в область прийняття гіпотези, то немає підстав відкидати нульову гіпотезу, за даними спостереження $r_{\text{ген}}=0$, r_b *незначущий*, ознаки X і Y *некорельовані*. А якщо $T_{\text{спост}}$ потрапило в критичну область, тобто $|T_{\text{спост}}| > t_{\text{кр}}(\alpha, k)$, то нульову гіпотезу відкидають, справедлива конкуруюча, тобто $r_{\text{ген}} \neq 0$, r_b *значущий*, ознаки X і Y *корельовані*.

За допомогою r_b аналізують тісноту взаємозв'язку між ознаками X і Y . Чим ближче $|r_b|$ до одиниці, тим тісніше зв'язок між ознаками, чим ближче $|r_b|$ до нуля, тим зв'язок слабкіше.

Далі знаходять *коефіцієнт детермінації* $D = r_b^2 \cdot 100\%$. Він показує, на скільки відсотків в середньому варіація результативної ознаки Y пояснюється за рахунок варіації факторної ознаки X .

Зауваження. Оскільки мова йде про випадкові величини, завжди існує ймовірність, що помічений зв'язок – випадкова обставина. Причому ймовірність знайти зв'язок там, де його немає, особливо велика тоді, коли точок у вибірці мало. Так, при наявності всього двох різних точок в будь-якій довільній вибірці, коефіцієнт кореляції буде дорівнює або +1 або -1. Вважається, що число спостережень має в 7-8 разів перевищувати число параметрів, які розраховуються при змінній x . Це означає, що шукати лінійну регресію, маючи менше 7 спостережень, взагалі не має сенсу. Якщо вид функції ускладнюється, то потрібно збільшити обсяг спостережень, тому що кожен параметр при x повинен розраховуватися хоча б за 7 спостереженнями. Таким чином, якщо ми вибираємо

параболу другого ступеня, то потрібний обсяг інформації вже не менше 14 спостережень.

Для оцінки статистичної достовірності факту виявленого зв'язку корисно використовувати так звану кореляційну поправку: $S_r = \frac{1-r^2}{\sqrt{n-1}}$. Зв'язок не можна вважати

випадковим, якщо: $\left| \frac{r_s}{S_r} \right| \geq 3$.

Основи регресійного аналізу

Наступним етапом є *регресійний аналіз*. У той час як задача кореляційного аналізу – встановити, чи є дані випадкові величини взаємопов'язаними, мета регресійного аналізу – описати цей зв'язок аналітичною залежністю, тобто за допомогою рівняння. Кореляційна залежність між ознаками наближено може бути виражена у вигляді *лінійного рівняння регресії* виду $y_x \approx a_0 + a_1 x$. Невідомі параметри a_0 і a_1 знаходяться методом найменших квадратів. Застосовуючи цей метод, отримуємо наступну систему

нормальних рівнянь: $\begin{cases} a_0 + a_1 \bar{x} = \bar{y} \\ a_0 \bar{x} + a_1 \bar{x}^2 = \overline{xy} \end{cases}$, розв'язавши яку, знаходимо оцінки параметрів

a_0 і a_1 . Рівняння регресії можна записати в такому вигляді: $\hat{y}_x - \bar{y} = a_1(x - \bar{x})$, де $a_1 = \frac{\overline{xy} - \bar{x} \cdot \bar{y}}{\sigma_x^2}$. Параметр a_1 – *коефіцієнт регресії* – показує, як зміниться в середньому

результативна ознака Y , якщо факторна ознака X збільшиться на одиницю свого виміру.

Моделі регресії, які отримані шляхом обробки вибіркового даних, можуть бути використані для прогнозування значень залежного параметра Y . Але в регресійному аналізі не рекомендується використовувати модель регресії для тих значень незалежного параметра, які не належать інтервалу, що заданий у вихідних даних.

Оцінка значущості рівняння регресії

Перевірити значущість рівняння регресії – значить встановити, чи відповідає математична модель, яка виражає залежність між змінними, експериментальним даним і чи достатньо пояснювальних змінних (однієї або декількох), що включені в рівняння, для опису залежної змінної.

Щоб зробити загальний висновок про якість моделі з відносних відхилень по кожному спостереженню, визначають середню помилку апроксимації: $\bar{A} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - y_x}{y_i} \right| \cdot 100\%$.

Середня помилка апроксимації не повинна перевищувати 8-10%.

Оцінка значущості рівняння регресії в цілому проводиться на основі критерію Фішера, якому передуює дисперсійний аналіз. Фактичне значення критерію Фішера $F = \frac{r_B^2}{1-r_B^2} \cdot (n-2)$ порівнюється з табличним значенням $F_{\text{табл}}(\alpha; k_1; k_2)$ при рівні значущості α і ступенях свободи $k_1 = 1$ (для парної лінійної регресії) і $k_2 = n - 2$. При цьому, якщо фактичне значення F -критерія більше за табличне, то визнається статистична значимість рівняння в цілому. Якщо статистична значимість рівняння в цілому не визнається, то для побудови значимого рівняння регресії потрібно взяти більшу кількість точок.

Логістична регресія – це різновид множинної регресії, загальне призначення якої полягає в аналізі зв'язку між декількома незалежними змінними (так званими регресорами або предикторами) і залежною змінною. Бінарна логістична регресія застосовується в тому разі, коли залежна змінна є бінарною (тобто може приймати тільки два значення). Логістична регресія, подібно до лінійної, також є функцією залежності цільової змінної від незалежних, які визначаються спостереженнями. При побудові множинної регресії часто виникає проблема передбачення значень, більших, ніж допустиме. Тому використовуються так звані логіт-перетворення, коли значення теоретично прогнозованої ймовірності вираховується за формулою

$$P = \frac{1}{1 + e^{-y}},$$

де P – це ймовірність настання прогнозованої події, $y = b_1x + b_0$ – значення регресії, e – основа натурального логарифму.

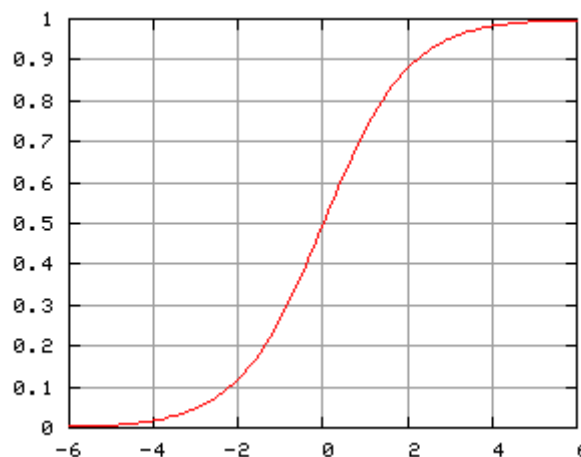
На даний момент маємо можливість використовувати декілька варіантів знаходження коефіцієнтів рівняння логістичної регресії. Часто вживаним вважається метод максимальної правдоподібності, який базується на оцінюванні параметрів генеральної сукупності. Це відбувається шляхом використання щільності сумісної ймовірності.

Логістичну регресію добре використовувати для завдань бінарної класифікації, тобто коли на виході потрібно отримати відповідь, до якого з двох класів належить об'єкт. Логістична регресія схожа на лінійну тим, що в ній теж потрібно знайти значення коефіцієнтів для вхідних змінних, але різниця в тому, що вихідне значення перетвориться за допомогою нелінійної або логістичної функції. Логістична функція перетворює будь-яке значення в число в межах від 0 до 1. Так і передбачається ймовірність належності до

одного чи другого класу. Тобто, на відміну від логістичної регресії, тут не проводять передбачення значення числової змінної, виходячи з вибірки вихідних значень, а замість цього значенням функції є ймовірність того, що це початкове значення належить до певного класу. До недоліків логістичної регресії можна віднести залежність від набору даних та низьку стійкість до помилок.

Прикладами проблем, які можна вирішити логістичною регресією, є: електронний лист є спамом чи ні; покупка в Інтернеті є шахрайською чи ні, оцінюючи умови покупки; у пацієнта перелом, оцінюючи його радіуси. За допомогою логістичної регресії ми можемо робити прогнозний аналіз, вимірюючи взаємозв'язок між тим, що ми хочемо передбачити (залежною змінною), і однією або декількома незалежними змінними, тобто характеристиками. Оцінка ймовірності здійснюється за допомогою логістичної функції. В подальшому ймовірності перетворюються на двійкові значення, і для того, щоб зробити прогноз реальним, цей результат присвоюється класу, якому він належить, виходячи з того, близький він чи ні до самого класу. Наприклад, якщо застосування логістичної функції повертає 0,85, то це означає, що вхідні дані генерують позитивний клас, присвоюючи його класу 1. І навпаки, якщо він отримав таке значення, як 0,4 або більш загально $< 0,5$.

Логістична функція, яка також називається сигмоїдною, – це крива, здатна приймати будь-яке число реальної величини і відображати її до значення між 0 і 1, виключаючи крайності.



На практиці часто зустрічаються із задачами, в яких результат досліду описується не однією випадковою величиною, а двома або більшою кількістю випадкових величин, що утворюють *систему* випадкових величин. Будемо позначати через (X,Y) двовимірну випадкову величину, кожен з величин X і Y будемо називати *складовою*. Систему двох випадкових величин можна зображати *випадковою точкою* на площині з координатами (X,Y) або ж *випадковим вектором*. Доцільно розрізняти *дискретні* (коли складові X і Y дискретні) і *неперервні* двовимірні випадкові величини.

Визначення. *Законом розподілу* дискретної двовимірної випадкової величини називають перелік всіх можливих значень цієї величини (x_i, y_j) і їх ймовірностей - $p(x_i, y_j)$, де $i = 1, \dots, n, j = 1, \dots, m$.

Зазвичай закон розподілу задають у вигляді таблиці:

$\begin{matrix} X \\ Y \end{matrix}$	x_1	x_2	$\dots$	x_n
y_1	$p(x_1, y_1)$	$p(x_2, y_1)$	$\dots$	$p(x_n, y_1)$
y_2	$p(x_1, y_2)$	$p(x_2, y_2)$	$\dots$	$p(x_n, y_2)$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
y_m	$p(x_1, y_m)$	$p(x_2, y_m)$	$\dots$	$p(x_n, y_m)$

Таблиця. Закон розподілу дискретної двовимірної випадкової величини

Оскільки події $(X=x_i, Y=y_j)$ утворюють повну групу подій, то сума всіх клітин дорівнює одиниці. Знаючи закон розподілу двовимірної випадкової величини, можна знайти закони розподілу кожної складової.

Визначення. *Функцією розподілу* двовимірної випадкової величини називають функцію $F(x,y)$, що визначає для кожної пари (x,y) ймовірність того, що $X \leq x$ і при цьому $Y \leq y$, тобто $F(x,y) = P(X \leq x, Y \leq y)$.

Геометричний зміст: $F(x,y)$ є ймовірність того, що випадкова точка (X,Y) потрапить у нескінченний квадрант лівіше і нижче точки (x,y) .

Властивості функції розподілу:

- 1). $0 \leq F(x,y) \leq 1$. З визначення ймовірності.
- 2). $F(x,y)$ - неспадна по кожному аргументу, тобто

$$F(x_2, y) \geq F(x_1, y) \quad \text{при } x_2 > x_1;$$

$$F(x, y_2) \geq F(x, y_1) \quad \text{при } y_2 > y_1.$$
- 3). $F(-\infty, y) = F(x, -\infty) = F(-\infty, -\infty) = 0, \quad F(\infty, \infty) = 1$.
- 4). $F(x, \infty) = F_1(x), \quad F(\infty, y) = F_2(y)$.

Ймовірність попадання випадкової точки в прямокутник буде обчислюватися за формулою:

$$P(x_1 \leq X < x_2, y_1 \leq Y < y_2) = [F(x_2, y_2) - F(x_1, y_2)] - [F(x_2, y_1) - F(x_1, y_1)]$$

Визначення. Щільністю спільного розподілу ймовірностей $f(x, y)$ двовимірної неперервної випадкової величини (X, Y) називають другу змішану частинну похідну від функції розподілу:

$$f(x, y) = \frac{\partial^2 F(x, y)}{\partial x \partial y}.$$

Знаючи щільність спільного розподілу, можна знайти функцію розподілу за формулою:

$$F(x, y) = \int_{-\infty}^y \int_{-\infty}^x f(x, y) dx dy.$$

Лема. Для обчислення ймовірності потрапляння випадкової точки (X, Y) в область D використовують формулу:

$$P((X, Y) \in D) = \iint_{(D)} f(x, y) dx dy.$$

Властивості щільності розподілу ймовірностей:

1). $f(x, y) \geq 0$, оскільки $F(x, y)$ - неспадна.

$$2). \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f(x, y) dx dy = 1.$$

Визначення. Щільність розподілу однієї зі складових дорівнює невластивому інтегралу з нескінченними границями від щільності розподілу системи за іншою складовою, тобто:

$$f_1(x) = \int_{-\infty}^{+\infty} f(x, y) dy, \quad f_2(y) = \int_{-\infty}^{+\infty} f(x, y) dx.$$

Визначення. Математичним сподіванням двовимірної випадкової величини (X, Y) називається точка (M_X, M_Y) – центр двовимірного розподілу:

$$M(X) = \int_{-\infty}^{+\infty} x f_1(x) dx = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} x f(x, y) dx dy,$$

$$M(Y) = \int_{-\infty}^{+\infty} y f_2(y) dy = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} y f(x, y) dx dy.$$

Визначення. Умовним розподілом складової X при $Y = y_j$ називають сукупність умовних ймовірностей $P(x_1/y_j), \dots, P(x_n/y_j)$, обчислених в припущенні, що подія $Y = y_j$ вже настала.

У загальному випадку умовні закони розподілу складової X визначаються співвідношенням:

$$P(x_i/y_j) = P(x_i, y_j) / P(y_j)$$

Визначення. Умовною щільністю розподілу $\varphi(x/y)$ складової X при даному значенні $Y=y$ називається відношення щільності спільного розподілу $f(x,y)$ до щільності $f_2(y)$:

$$\varphi(x/y) = f(x,y)/f_2(y).$$

Аналогічно визначається $\psi(y/x)$:

$$\psi(y/x) = f(x,y)/f_1(x).$$

Якщо відома щільність спільного розподілу, тоді:

$$\varphi(x/y) = f(x,y) / \int_{-\infty}^{+\infty} f(x,y) dx ,$$

$$\psi(y/x) = f(x,y) / \int_{-\infty}^{+\infty} f(x,y) dy .$$

Теорема. Для того, щоб випадкові величини X і Y були незалежними, необхідно і достатньо щоб функція розподілу системи дорівнювала добутку функцій розподілу складових:

$$F(x,y)=F_1(x) \cdot F_2(y) .$$

Наслідок. Для того, щоб випадкові величини X і Y були незалежні необхідно і достатньо, щоб виконувалася рівність:

$$f(x,y) = f_1(x) \cdot f_2(y).$$

Визначення. Початковим моментом порядку k, s системи (X,Y) називається математичне сподівання добутку X^k на Y^s :

$$\mu_{k,s}=M(X^k \cdot Y^s) .$$

Визначення. Центральним моментом порядку k, s системи (X,Y) називається математичне сподівання добутку $(X-M(x))^k$ на $(Y-M(y))^s$:

$$\nu_{k,s}=M[(X-M(x))^k \cdot (Y-M(y))^s] .$$

Напишемо формули, які служать для безпосереднього підрахунку моментів. Для дискретних випадкових величин:

$$\mu_{k,s} = \sum_i \sum_j x_i^k y_j^s p_{ij} , \quad \nu_{k,s} = \sum_i \sum_j (x_i - m_x)^k \cdot (y_j - m_y)^s \cdot p_{ij} ,$$

де $p_{ij} = P((X = x_i), (Y = y_j))$

Для неперервних випадкових величин:

$$\mu_{k,s} = \int \int_{-\infty}^{+\infty} x^k y^s f(x,y) dx dy ,$$

$$\nu_{k,s} = \int \int_{-\infty}^{+\infty} (x - m_x)^k (y - m_y)^s f(x,y) dx dy .$$

Перші початкові моменти представляють собою математичні сподівання величин X і Y :

$$m_x = \mu_{1,0} = M(X^1 Y^0) = M(X),$$

$$m_y = \mu_{0,1} = M(X^0 Y^1) = M(Y).$$

На практиці також широко застосовуються центральні моменти системи другого порядку:

$$D_x = \nu_{2,0} = M[(x - m_x)^2 (y - m_y)^0] = M[(x - m_x)^2],$$

$$D_y = \nu_{0,2} = M[(x - m_x)^0 (y - m_y)^2] = M[(y - m_y)^2].$$

Особливу роль грає другий змішаний центральний момент.

Визначення. *Кореляційним моментом* або *коваріацією* випадкових величин X і Y називають математичне сподівання добутку відхилень цих величин:

$$K_{xy} = \nu_{1,1} = M[(x - m_x)(y - m_y)]$$

На практиці для обчислення коваріації використовуються наступні формули:

а) для дискретних випадкових величин:

$$K_{xy} = \sum_i \sum_j (x_i - m_x)(y_j - m_y) p_{ij};$$

б) для неперервних випадкових величин:

$$K_{xy} = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} (x - m_x)(y - m_y) f(x, y) dx dy.$$

Кореляційний момент служить для характеристики зв'язку між величинами X і Y . Легко переконатися, що кореляційний момент можна записати в наступному вигляді:

$$K_{xy} = M(XY) - M(X)M(Y)$$

Теорема. Кореляційний момент двох незалежних випадкових величин дорівнює нулю.

За визначенням кореляційного моменту слідує, що він має розмірність, що дорівнює добутку розмірностей X і Y , що незручно, тому що важко порівнювати різні системи випадкових величин. Тому вводять нову числову характеристику – коефіцієнт кореляції.

Визначення. *Коефіцієнтом кореляції* випадкових величин X і Y називають відношення кореляційного моменту до добутку середніх квадратичних відхилень, тобто:

$$r_{xy} = K_{xy} / (\sigma_x \sigma_y)$$

Теорема 1. Абсолютна величина кореляційного моменту двох випадкових величин X і Y не перевищує середнього геометричного їх дисперсій:

$$|K_{xy}| \leq \sqrt{D(X)D(Y)}$$

Наслідок. Абсолютна величина коефіцієнта кореляції не перевищує одиниці: $|r_{xy}| \leq 1$.

Теорема 2. Коефіцієнт кореляції між випадковими величинами X і Y дорівнює ± 1 тоді і тільки тоді, коли X і Y пов'язані лінійною залежністю $Y = aX + b$, причому $r_{xy} = 1$ при $a > 0$, $r_{xy} = -1$ при $a < 0$.

Теорема 3. Якщо випадкові величини X і Y незалежні, то $r_{xy} = 0$.

Наслідок. Якщо $r_{xy} \neq 0$, то X і Y залежні випадкові величини.

Визначення. Дві випадкові величини називають *корельованими*, якщо їх коефіцієнт кореляції відмінний від нуля. У протилежному випадку – *некорельованими*.

З некорельованості двох випадкових величин, в загальному випадку, не випливає їх незалежність. Але ж, для нормально розподілених випадкових величин некорельованість тотожна незалежності.

Для визначення взаємного впливу декількох змінних будується **кореляційна матриця**, у якій вказуються коефіцієнти попарно для усіх змінних. Відносно головної діагоналі елементи матриці симетричні, а всі елементи головної діагоналі матриці дорівнюють 1, бо відображають зв'язок самої величини із собою. Симетричність матриці дозволяє у програмах обробки статистичної інформації виводити лише половину матриці, наприклад, нижче головної діагоналі.

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1k} \\ r_{21} & r_{22} & \dots & r_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ r_{k1} & r_{k2} & \dots & r_{kk} \end{bmatrix} = \begin{bmatrix} 1 & r_{12} & \dots & r_{1k} \\ r_{21} & 1 & \dots & r_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ r_{k1} & r_{k2} & \dots & 1 \end{bmatrix}$$

При вивченні явищ навколишнього світу ми часто маємо справу із процесами, розвиток яких заздалегідь передбачити неможливо. Така непередбачуваність пояснюється впливом на хід процесів випадкових факторів. Строго кажучи, у природі немає не випадкових явищ, але є процеси, на які випадковість впливає несуттєво, і при їх вивченні цей вплив можна не брати до уваги, але є і такі, де випадковість відіграє основну роль. Між цими двома полюсами перебуває багато процесів, на перебіг яких випадковість впливає більшою або меншою мірою.

Визначення. *Теорія випадкових (стохастичних) процесів* – це математична наука, яка вивчає закономірності випадкових явищ у динаміці їх розвитку.

Визначення. *Випадковою функцією (ВФ)* називається функція, яка внаслідок випробування може набути того чи іншого конкретного вигляду, заздалегідь невідомо, якого саме. Випадкові функції аргументу t позначають $X(t)$, $Y(t)$.

Конкретний вигляд, набутий ВФ внаслідок випробування, називається *реалізацією ВФ*.

На практиці нерідко спостерігаються ВФ, аргументом яких є час. Такі ВФ називаються випадковими процесами. Крім того, ВФ можуть залежати від кількох аргументів.

Розглянемо ВФ $X(t)$. Нехай унаслідок n випробувань отримано її реалізації $x_1(t)$, $x_2(t)$, ..., $x_n(t)$. Конкретне випробування перетворює ВФ $X(t)$ на не випадкову функцію, тобто на одну зі своїх реалізацій. Зафіксуємо деяке значення аргументу: $t=t_0$, тоді ВФ перетворюється на випадкову величину $X(t_0)$, яка називається розрізом ВФ для аргумента: $t=t_0$. Тобто ВФ $X(t)$ об'єднує в собі властивості випадкової величини й функції.

Випадковий процес називається процесом із дискретним часом, коли система, в якій він відбувається, змінює свій стан тільки у моменти $t_1, t_2, \dots, t_j$, число яких скінченне або лічильне.

Випадковий процес $X(t)$ називається процесом, що характеризується неперервним часом, якщо переходи системи від одного стану до іншого можуть відбуватися в будь-який момент часу t протягом спостережуваного періоду τ .

Випадковий процес називається процесом, який характеризується неперервними станами, коли його розрізом у будь-який момент часу t є неперервна випадкова величина.

Таким чином, усі випадкові процеси ми можемо поділити на такі чотири класи:

1. Процеси, характерні дискретними станами і дискретним часом.
2. Процеси, характерні дискретними станами і неперервним часом.
3. Процеси, характерні неперервними станами і дискретним часом.

4. Процеси, характерні неперервними станами і неперервним часом.

Що ж являє собою закон розподілу ВФ?

Очевидно, що набір m розрізів буде являти собою систему m випадкових величин і ця система повинна описуватися m -вимірним законом розподілу ймовірностей.

Розглянемо деякий розріз ВФ $X(t)$ для аргументу t . Ця випадкова величина в загальному випадку залежить від значення t і має закон розподілу $f(x, t)$. Функція $f(x, t)$ називається одновимірним законом розподілу ВФ. Вона повністю описує кожний окремий розріз, але не дає повного опису випадкової функції.

Із цього погляду більш повною характеристикою ВФ є двовимірний закон розподілу $f(x_1, x_2, t_1, t_2)$, який виступає результатом сумісного розгляду двох розрізів ВФ $X(t)$: $X_1=X(t_1)$ та $X_2=X(t_2)$, тобто вивчення системи двох випадкових величин (X_1, X_2) . Але цей закон розподілу також не дає повного опису будь-якого випадкового процесу (він достатній для опису нормально розподіленої випадкової функції або марковського випадкового процесу). Тоді виникає необхідність розглядати тривимірні $f(x_1, x_2, x_3, t_1, t_2, t_3)$, чотиривимірні й т. д. функції розподілу. Взагалі, щоб повністю описати випадкову функцію (або випадковий процес) необхідно розглядати нескінченновимірний закон розподілу або хоч би n -вимірний.

Оскільки такий спосіб вивчення випадкових функцій дуже громіздкий, то на практиці він застосовується лише в окремих випадках. Звичайно ж опис ВФ виконується за допомогою найпростіших її характеристик (моментів), аналогічних характеристикам випадкової величини.

Визначення. Математичним сподіванням ВФ $X(t)$ називається не випадкова функція $m_x(t)$, яка для кожного значення аргументу t дорівнює математичному сподіванню відповідного розрізу випадкової функції: $m_x(t) = M[X(t)]$.

Геометрично математичне сподівання випадкової функції можна розуміти як “середню криву”, навколо якої розташовані інші криві, що відображають реалізації ВФ.

Визначення. Дисперсією випадкової функції $X(t)$ називається не випадкова функція $D_x(t)$, значення якої для кожного t дорівнює дисперсії відповідного розрізу ВФ, тобто $D_x(t) = D[X(t)]$.

Дисперсія для кожного значення аргументу t характеризує розсіювання реалізацій ВФ відносно математичного сподівання $m_x(t)$.

Аналогічно, середнє квадратичне відхилення ВФ визначимо за такою формулою:

$$\sigma_x(t) = \sqrt{D_x(t)}$$

Так само як і математичне сподівання, дисперсія випадкового процесу $X(t)$ визначається його одновимірним законом розподілу.

Кореляційною функцією ВФ $X(t)$ називається не випадкова функція двох аргументів $K_x(t, t')$, яка для кожної пари значень t й t' дорівнює кореляційному моменту відповідних розрізів ВФ.

На практиці при побудові кореляційної функції $K(t, t')$ випадкової функції $X(t)$ застосовують такий спосіб: задаються рядом рівновіддалених значень аргументу й будують кореляційну матрицю отриманої системи випадкових величин $X(t_1) \dots X(t_m)$

Окрім кореляційної функції, на практиці для оцінювання ступеня залежності між розрізами ВФ часто використовують ще одну характеристику, а саме, нормовану кореляційну функцію ВП, яка є коефіцієнтом кореляції відповідних розрізів випадкової функції і обчислюється за такою формулою:

$$r_x(t, t') = \frac{K_x(t, t')}{\sigma_x(t)\sigma_x(t')}$$

Абсолютна величина нормованої кореляційної функції не перевищує одиниці. Крім того, очевидно, що коли $t=t'$, то $r_x=1$.

Імовірнісний сенс нормованої кореляційної функції такий самий, що й коефіцієнта кореляції для ВВ, а саме: чим ближче значення нормованої кореляційної функції до одиниці, тим сильніша лінійна залежність між розрізами випадкової функції.

Вся множина об'єктів (спостережень), яка підлягає вивченню, називається *генеральною сукупністю*. Це сукупність усіх мислимих спостережень, які могли б бути зроблені при даному комплексі умов. ГС може бути скінченною або нескінченною. Та частина об'єктів з генеральної сукупності, яка випадковим чином відібрана для безпосереднього вивчення, називається *вибіркою*. Число об'єктів у вибірці називається її *об'ємом*. Властивості і характеристики генеральної сукупності зазвичай невідомі.

У загальному випадку розглядають три так звані основні завдання математичної статистики:

- Визначення виду закону розподілу досліджуваної випадкової величини (задача згладжування експериментальних залежностей);
- Визначення невідомих параметрів розподілу;
- Перевірка правдоподібності гіпотез.

Варіаційний ряд. Статистичні розподіли

Різні значення ознаки (випадкової величини X) називаються *варіантами x* . Отримана в результаті безпосереднього статистичного спостереження вибірка з n варіант досліджуваної кількісної ознаки X утворює *варіаційний ряд*. *Ранжируваний* варіаційний ряд отримують, розташувавши варіанти x_j (де $j = 1, 2, \dots, n$) в порядку зростання значень, тобто $x_1 \leq x_2 \leq \dots \leq x_j \leq \dots \leq x_n$. Досліджувана ознака X може бути *дискретною* або *неперервною*.

Частотою m_i в разі дискретного ознаки X називають число однакових варіант x_i , які містяться в вибірці. У ранжируваному варіаційному ряді однакові варіанти очевидно розташовані поспіль:

$$\overbrace{\underbrace{x_1, x_1, \dots, x_1}_{m_1}, \dots, \underbrace{x_i, x_i, \dots, x_i}_{m_i}, \dots, \underbrace{x_k, x_k, \dots, x_k}_{m_k}}^n.$$

Варіаційний ряд для дискретної ознаки X прийнято наочно і компактно представляти у вигляді таблиці, в першому рядку якої вказані k різних значень x_i досліджуваної ознаки, а у другому рядку – відповідні цим значенням частоти m_i , де $i = 1, 2, \dots, k$. Таку таблицю називають *статистичними (вибірковим) розподілом*.

Приклад. Варіаційний ряд, який отримано в результаті статистичного

спостереження: 7, 17, 14, 17, 10, 7, 7, 14, 7, 14.

Ранжируваний варіаційний ряд:

$$x_j: \underbrace{7, 7, 7, 7}_{m_1=4}, \underbrace{10}_{m_2=1}, \underbrace{14, 14, 14}_{m_3=3}, \underbrace{17, 17}_{m_4=2}, \text{ де } j=1, 2, \dots, n, \quad n=10.$$

Відповідний статистичний розподіл ($i=1, 2, 3, 4$):

x_i	7	10	14	17
m_i	4	1	3	2

Модою M_o називається варіанта, що має найбільшу частоту. Медіаною M_e називається варіанта, яка ділить навпіл варіаційний ряд на дві частини з однаковим числом варіант в кожній. Якщо об'єм вибірки є непарним числом, то медіана дорівнює серединному значенню, а для ряду з парним числом варіант – напівсумі двох серединних значень: $M_e = \frac{x_j + x_{j+1}}{2}$. Розмахом варіювання називається різниця між максимальною і мінімальною варіантами або довжина інтервалу, якому належать всі варіанти вибірки: $R = x_{\max} - x_{\min}$.

Статистичний розподіл для неперервної ознаки X прийнято представляти інтервальним рядом – таблицею, в першому рядку якої вказані k інтервалів значень досліджуваної ознаки X виду $(x_{i-1} - x_i)$, а у другому рядку – відповідні цим інтервалам частоти m_i , де $i=1, 2, \dots, k$. Позначення $(x_{i-1} - x_i)$ – вказує не на різниці, а на всі значення ознаки X від x_{i-1} до x_i , крім правої границі інтервалу: $[x_{i-1}; x_i)$. Для неперервної ознаки X частота m_i – кількість різних x_j , які потрапили до відповідного інтервалу: $x_j \in [x_{i-1}; x_i)$.

Приклад. Варіаційний ряд, який отримано в результаті статистичного спостереження (одиниці вимірювання опускаємо) – 3,14; 1,41; 2,87; 3,62; 2,71; 3,95. Ранжируваний варіаційний ряд – x_j : 1,41; 2,71; 2,87; 3,14; 3,62; 3,95; де $j=1, 2, \dots, n$, $n=6$. Відповідне статистичний розподіл ($i=1, 2, 3$):

x_i	1–2	2–3	3–4
m_i	1	2	3

Якщо число різних значень дискретної ознаки дуже велике, то для зручності подальших обчислень і наочності статистичний розподіл такої дискретної ознаки також може бути представлений у вигляді інтервального ряду.

Замість частот m_i у другому рядку можуть бути вказані відносні частоти $w_i = \frac{m_i}{n}$. Очевидно, що сума частот дорівнює об'єму вибірки (вибіркової сукупності) n , а сума відносних частот дорівнює одиниці: $\sum_{i=1}^k m_i = n$, $\sum_{i=1}^k w_i = \sum_{i=1}^k \frac{m_i}{n} = 1$.

Емпірична функція розподілу

Нехай m_x – число спостережень, при яких спостерігалось значення ознаки X , менше за x . При об'ємі вибірки, рівному n , відносна частота події $X < x$ дорівнює m_x/n . Функція, що визначає для кожного значення x відносну частоту події $X < x$ називається *емпіричною функцією розподілу*, або функцією розподілу вибірки $F^*(x) = \frac{m(X < x)}{n}$. На відміну від емпіричної функції розподілу $F^*(x)$ вибірки функція розподілу $F(x)$ генеральної сукупності називається *теоретичною функцією розподілу*. Функція $F(x)$ визначає ймовірність події $X < x$: $F(x) = P(X < x)$, а $F^*(x)$ – відносну частоту цієї події. Емпіричну функцію розподілу можна використовувати для оцінки теоретичної функції розподілу генеральної сукупності. Емпірична функція розподілу $F^*(x)$ задається рядом накопичених відносних частот: $F^*(x_1) = 0$; $F^*(x_i) = \frac{m(X < x_i)}{n} = \sum_{l=1}^i w_l$, де $i = 2, 3, \dots, k$; $F^*(x_{k+1}) = 1$. За визначенням: $F^*(x_{\text{ме}}) = \frac{1}{2}$.

Якщо випадкова величина неперервна і її вибіркові значення представлені у вигляді інтервального варіаційного ряду, то вибірккову функцію розподілу будують, підрахувавши на кінцях інтервалів значення функції $F^*(x)$ у вигляді «накопиченої відносної частоти». При графічному зображенні такої функції її довизначають, з'єднавши точки графіка, що відповідають кінцям інтервалу, відрізками прямої.

Графічне представлення статистичних розподілів

Для наочності прийнято використовувати такі форми графічного подання

статистичних розподілів:

- дискретний ряд зображують у вигляді полігону. *Полігон частот* – ламана лінія, відрізки якої з’єднують точки з координатами $(x_i; m_i)$; аналогічно, *полігон відносних частот* – ламана, відрізки якої з’єднують точки з координатами (x_i, w_i) ;

- інтервальний ряд зображують у вигляді гістограми. *Гістограма частот* є ступінчаста фігура, що складається з прямокутників, основи яких – інтервали довжиною h_i , а висоти – щільності частот $\frac{m_i}{h_i}$. У разі *гістограми відносних частот*

висоти прямокутників – щільності відносних частот $\frac{w_i}{h_i} = \frac{m_i}{n \cdot h_i}$. Тут в загальному

випадку $h_i = x_i - x_{i-1}$, однак на практиці частіше вважають величину h однаковою для всіх інтервалів: $h_i = h = (x_k - x_0)/k$, де $i = 1, 2, \dots, k$. Площа гістограми є сума площ

її прямокутників: $S_{\text{ч}} = \sum_{i=1}^k S_i = \sum_{i=1}^k h \cdot \frac{m_i}{h} = \sum_{i=1}^k m_i = n$, $S_{\text{отн.ч}} = \sum_{i=1}^k S_i = \sum_{i=1}^k h \cdot \frac{m_i}{n \cdot h} = \frac{1}{n} \cdot \sum_{i=1}^k m_i = 1$,

таким чином, площа гістограми частот $S_{\text{ч}}$ дорівнює об’єму вибірки, а площа гістограми відносних частот $S_{\text{отн.ч}}$ дорівнює одиниці.

В теорії ймовірностей гістограмі відносних частот відповідає графік щільності розподілу ймовірностей $f(x)$. Тому гістограму можна використовувати для підбору закону розподілу генеральної сукупності. Графічним шляхом за допомогою гістограми найпростіше знайти моду.

Крива накопичених частот називається *кумулятою*.

Візуалізація даних – це зображення числової і текстової інформації у вигляді графіків, діаграм, структурних схем, таблиць, малюнків, карт і т. д.

Одна з труднощів, яка істотно сповільнює складання звітів і аналітичну роботу, полягає в підборі правильного типу діаграми.

1. Вам потрібно порівнювати величини?

Графіки ідеально підходять для порівняння одного або декількох наборів величин, і вони можуть легко відображати самі низькі і високі показники.

Для створення порівняльної діаграми використовуйте наступні типи: гістограма, кругова діаграма, точкова діаграма, шкала зі значеннями.

2. Ви хочете показати структуру чогось?

Наприклад, ви хочете розповісти про типи мобільних пристроїв, які використовують відвідувачі сайту, або загальний обсяг продажів, розбитий на сегменти.

Щоб показати структуру, використовуйте наступні діаграми: кругова діаграма, гістограма з накопиченням, вертикальний стек, обласна діаграма, діаграма-водоспад.

3. Ви хочете зрозуміти, як розподіляються дані?

Таблиці з розподілом допомагають зрозуміти основні тенденції і відзначити, що виходить за рамки.

Використовуйте ці діаграми: точкова діаграма, лінійна діаграма, гістограма.

4. Ви зацікавлені в аналізі тенденцій у певному наборі даних?

Якщо ви хочете дізнатися більше про те, як цифри поведуться протягом конкретного часового періоду, є типи діаграм, які дуже добре це відображають.

Вам знадобляться: лінійна діаграма, подвійна вісь (стовпець і лінія), гістограма.

5. Хочете краще зрозуміти взаємозв'язок між встановленими значеннями?

Взаємопов'язані графіки підходять для того, щоб показати, як одна змінна відноситься до іншої або декількох різних змінних. Це можна використовувати, щоб показати позитивний, негативний або нульовий вплив на іншу цифру.

Використовуйте для цього такі діаграми: точкова діаграма, бульбашкова діаграма, лінійна діаграма.

Точкова діаграма

Точкова діаграма або діаграма розсіювання – це тип діаграми для відображення значень, як правило, для двох змінних для набору даних з використанням декартових координат (рис. 1). Точкова діаграма показує взаємозв'язок між двома різними змінними або демонструє розподіляючі тенденції. Вона підходить, якщо у вас багато різних

точкових даних, і ви хочете знайти загальне в наборі даних. Така візуалізація добре працює в пошуку винятків або закономірності розподілу даних.

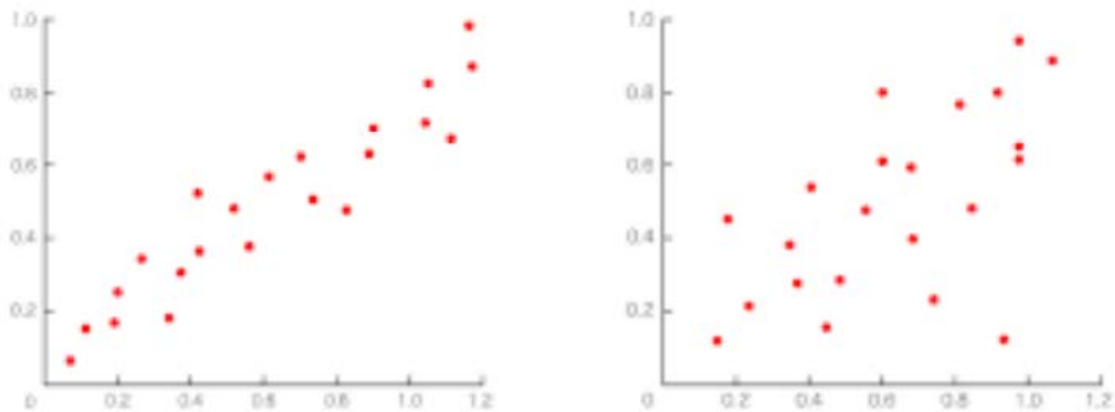


Рисунок 1 – Діаграма розсіювання

Дані відображаються, як набір точок, кожна з яких має значення однієї змінної, яка визначає розташування на горизонтальній осі і значення іншої змінної, що визначає положення по вертикальній осі. Цей тип графіка дозволяє нам проводити дослідницький аналіз відповідних залежностей змінних для того, щоб визначити послідовність кроків обробки даних. Слід зазначити, що точки, які відповідають одним і тим же об'єктам, можуть бути позначені як кольором, так і розміром. У другому випадку ми отримуємо бульбашкову діаграму, яка є різновидом діаграми розсіювання, в якій точки даних замінюються бульбашками, та додатковий вимір даних представлений у розмірі бульбашок. Діаграми розсіювання допомагають вирішувати проблеми, показуючи взаємозв'язок між змінними.

Рекомендації щодо дизайну для точкових діаграм:

1. Використовуйте більше змінних, таких як різні розміри, щоб об'єднати більше даних.
2. Почніть вісь Y з 0 для точного розподілу даних.
3. Якщо ви використовуєте лінії тенденцій, необхідно обмежитися максимум двома, щоб графік був зрозумілий.

Гістограма

Гістограма використовується, щоб показати порівняння між різними елементами, також вона може порівняти елементи за певний проміжок часу. Одна вісь гістограми показує конкретні категорії, що порівнюються, а інша вісь, являє собою виміряне значення. Гістограми відображають статистичну інформацію, яка використовує прямокутники, щоб показати частоту елементів даних в послідовні числові інтервали рівного розміру (рис. 2).

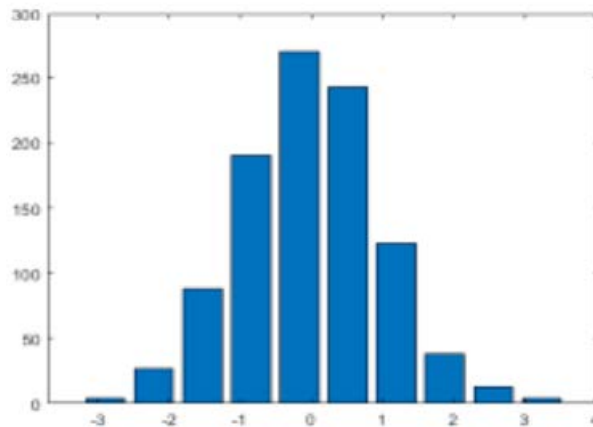


Рисунок 2 – Гістограма

У деяких випадках гістограми представляють собою стовпці, згруповані в декілька стовпців, показуючи значення більш ніж одної вимірюваної змінної (рис. 3).

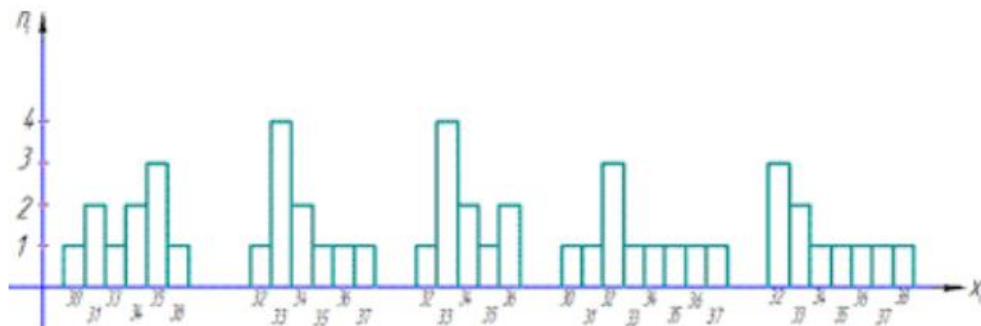


Рисунок 3 – Гістограма зі згрупованими стовпцями

Створення гістограми передбачає сортування аналізованої змінної по зростанню, потім вони діляться на кластери з деякими інтервалами. В результаті отримана гістограма виглядає, як набір прямокутних смуг, ширина яких відповідає відповідним значенням інтервалу, а довжина смуг пропорційна частоті елементів даних у відповідних інтервалах.

Рекомендації щодо дизайну для гістограм

1. Підбирайте єдину кольорову гаму і акцентуйте кольором місця, які хочете виділити як значущі точки перелому або зміни з плином часу.
2. Використовуйте горизонтальні мітки, щоб поліпшити читаність.
3. Почніть вісь Y з 0, щоб правильно відобразити значення на графіку.

Стовпчаста діаграма

Стовпчаста діаграма - це діаграма, яка дозволяє відобразити категоричні дані даних прямокутними смугами з довжиною, пропорційною значенням, які вони представляють. Смуги можуть бути нанесені вертикально або горизонтально (рис. 4).

У звичайній стовпчастій діаграмі, призначеній для порівняння кількох значень, значущою є лише одна вісь, а вздовж іншої розташовано стовпчики зручної для

сприйняття ширини, висота яких кодує значення. Стовпчасті діаграми часто будують не від нуля.

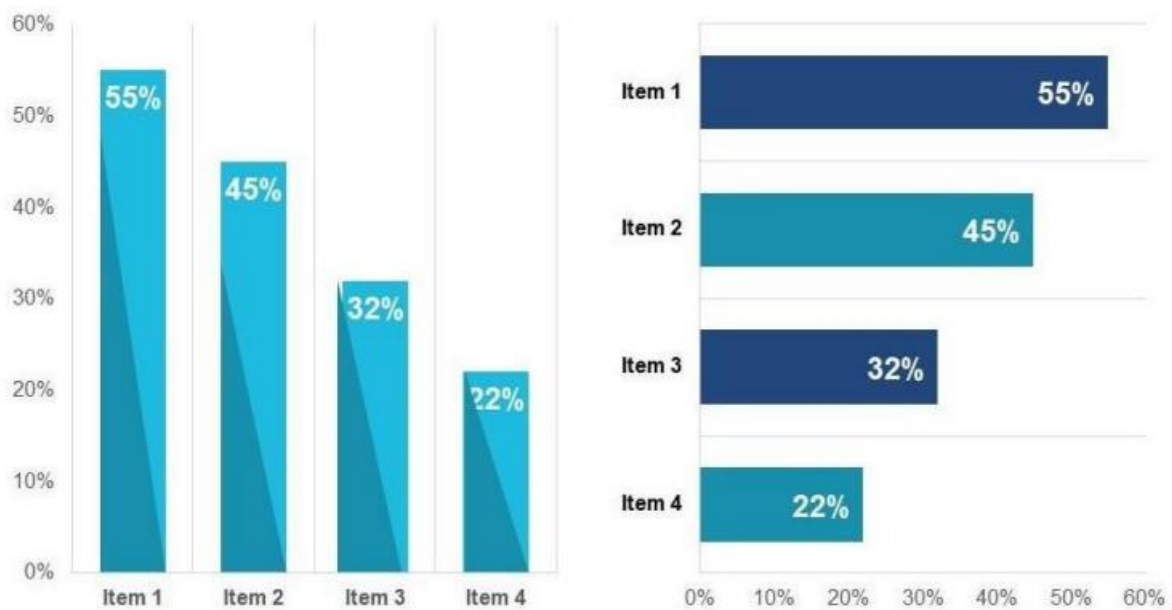


Рисунок 4 – Вертикальна та горизонтальна стовпчаста діаграма

Складнішим різновидом стовпчастої діаграми є стовпчаста діаграма зі змінною шириною стовпчиків, де ширина стовпчиків не є фіксованою, а кодує іншу змінну. Проблемою такої діаграми можна назвати те, що якщо співвідношення за одною змінною оцінювані легко, то другу ми сприймаємо опосередковано площею стовпчиків. Відповідно, якщо обидві змінні дають спільний ефект, цей тип діаграми годиться, а якщо змінні та їхній вплив незалежні, то ні.

Ще складніший варіант стовпчастої діаграми – стосова (рис. 5). В ній кожен стовпчик складається з шарів, товщина кожного з яких кодує змінну, сума яких становить певне значущє ціле.

Існує форма стосової діаграми, коли всі стовпчики мають рівну висоту – тоді вона показує лише різний розподіл між змінними між об'єктами-стовпчиками.



Рисунок 5 – Стосова діаграма

Рекомендації щодо дизайну для стовпчастих діаграм

1. Підбирайте єдину кольорову гаму та акцентуйте кольором місця, які хочете виділити як значущі моменти перелому або зміни з плином часу.
2. Використовуйте горизонтальні мітки, щоб поліпшити читаність.
3. Почніть вісь Y з 0, щоб правильно відобразити значення на графіку.

Кругова діаграма

Кругова діаграма відображає статичне число і те, як частини складаються в ціле – склад чого-небудь. Кругова діаграма показує числа у відсотках, і загальна сума всіх сегментів повинна дорівнювати 100%. Аналогічно до поділу на частини стовпчиків в стосовій діаграмі, кола можна ділити на сектори, щоби показати частки у цілому. Така «тортова» діаграма добре передає співвідношення кількох значних часток, але стає неефективною, якщо часток дуже багато. Тоді варто об'єднати дрібні частки в групу «інші».

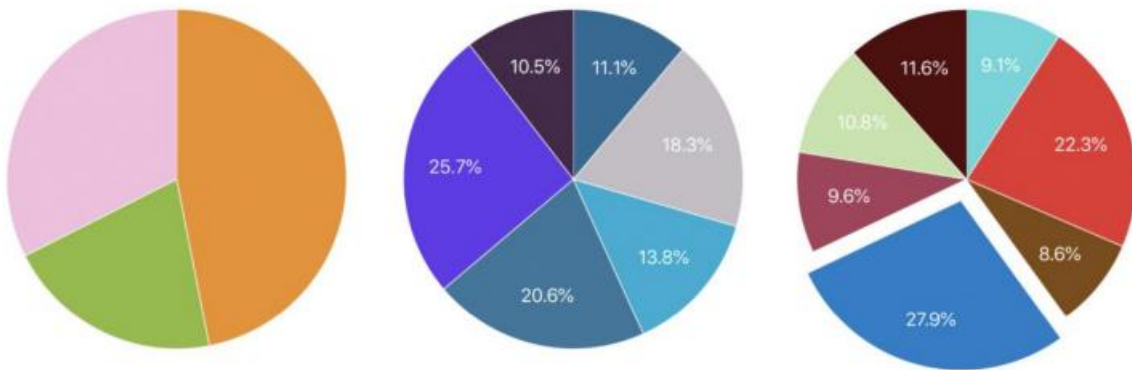


Рисунок 6 – Кругові діаграми

Рекомендації щодо дизайну для кругових діаграм

1. Не додавайте занадто багато категорій, щоб різниця між зрізами була добре помітна.
2. Переконайтеся, що загальна сума всіх частин становить 100%.
3. Необхідно впорядкувати частини відповідно до їхніх розмірів.

Оцінки параметрів розподілу

Більшість випадкових величин, розглянутих в курсі теорії ймовірностей, мали розподіли, які залежать від одного або декількох параметрів. Так, біноміальний розподіл залежить від параметрів p і n , нормальний – від параметрів a і σ , розподіл Пуассона – від параметра λ . Однією з основних задач математичної статистики є оцінювання цих параметрів по спостережуваним даними, тобто по вибірковій сукупності. Статистичні оцінки можуть бути точковими і інтервальними.

Точковою оцінкою невідомого параметра називають число (точку на числовій осі), яке приблизно дорівнює оцінюваному параметру і може замінити його з достатнім ступенем точності в статистичних розрахунках. Нехай θ^* – точкова оцінка невідомого параметра θ . Оскільки X – випадкова величина, то і оцінка θ^* (на відміну від оцінюваного параметра θ – величини не випадковою, детермінованою) є випадковою величиною, що залежить від закону розподілу випадкової величини X і числа n .

Для того щоб точкові статистичні оцінки забезпечували «хороші» наближення невідомих параметрів, вони повинні бути незміщеними, спроможними і ефективними.

Незміщеною називають таку точкову статистичну оцінку θ^* , математичне очікування якої дорівнює дійсному значенню оцінюваного параметра: $M(\theta^*) = \theta$ (1). В іншому випадку оцінка називається *зміщеною*. Якщо рівність (1) не виконується, то оцінка, отримана за різними вибірками, буде в середньому або завищувати значення параметра, або занижувати його. Таким чином, вимога незсуненості гарантує відсутність систематичних помилок при оцінюванні.

Спроможною називають таку точкову статистичну оцінку, яка при необмеженому збільшенні об'єму вибірки прагне за ймовірністю до оцінюваного параметру, тобто задовольняє закону великих чисел. Це означає: при досить великому об'ємі вибірки з практичною вірогідністю (з імовірністю, близькою до одиниці) θ^* практично збігається з дійсним значенням θ . У разі використання спроможних оцінок виправдовується збільшення обсягу вибірки, так як при цьому

стають малоймовірними значні помилки при оцінюванні. Якщо дисперсія незміщеної оцінки при $n \rightarrow \infty$ прямує до нуля, то така оцінка виявляється і спроможною.

Ефективною називають таку точкову статистичну оцінку, яка при фіксованому n має найменшу дисперсію (найменший розкид щодо θ) в порівнянні з іншими незміщеними оцінками параметра θ .

Математичне очікування $M(X)$ генеральної сукупності X назовемо *генеральною середньою* $\bar{x}_{\text{ген}}$. Точковою оцінкою генеральної середньої $\bar{x}_{\text{ген}}$ служить вибіркова середня $\bar{x}_b$. Ця оцінка є спроможною, ефективною і незміщеною. Вибіркова дисперсія D_b є спроможною, але зміщеною оцінкою генеральної дисперсії $D_{\text{ген}} = \sigma^2$. Незміщеною спроможною оцінкою генеральної дисперсії може служити при малих обсягах вибірки, *виправлена вибіркова дисперсія*: $S^2 = \frac{n}{n-1} \cdot D_b$. Ця формула застосовується при малих обсягах вибірки n , тому що для вибірок великого об'єму множник $\frac{n}{n-1}$ близький до 1, і випадкові величини S^2 і D_b мало відрізняються одна від одної. Обидві ці оцінки не є ефективними. Точковими оцінками для генерального середньоквадратичного відхилення $\sigma_{\text{ген}} = \sigma$ служать $\sigma_b = \sqrt{D_b}$ – вибіркове середнє квадратичне відхилення і $S = \sqrt{S^2}$ – виправлене вибіркове середнє квадратичне відхилення.

Відносна частота події A служить спроможною, незміщеною і ефективною оцінкою ймовірності $p(A)$.

Інтервальне оцінювання параметрів розподілу

Точкова оцінка при малому обсязі вибірки може істотно відрізнятися від оцінюваного параметра, тому важливо знати, наскільки справжнє значення параметра може відхилятися від знайденої точкової оцінки. В такому випадку краще користуватися інтервальними оцінками, тобто вказувати інтервал, в який із заданою ймовірністю потрапляє істинне значення оцінюваного параметра. Чим менше довжина цього інтервалу, тим точніше оцінка параметра.

Інтервал виду $|\theta - \theta^*| < \Delta$, де θ – справжнє значення оцінюваного параметра, а

θ^* – його точкова оцінка, називається *довірчим інтервалом*.

Імовірність того, що довірчий інтервал $(\theta^* - \Delta, \theta^* + \Delta)$ включає в себе (покриває) невідомий параметр θ дорівнює γ і називається *довірчою ймовірністю* $P(|\theta - \theta^*| < \Delta) = \gamma$ або *надійністю* інтервальної оцінки. Величину Δ називають *точністю* оцінки.

Довірчий інтервал не завжди, але дуже часто, вибирають симетричним щодо параметра θ . Величину Δ найбільшого відхилення оцінки від оцінюваного параметра, називають також *граничною помилкою* вибірки. Ця помилка є помилкою репрезентативності (представництва) вибірки. Вона виникає внаслідок того, що використовується не вся сукупність, а лише частина її (вибірка), відібрана випадково.

Для побудови довірчого інтервалу потрібно знати закон розподілу досліджуваної випадкової величини. Нехай ознака X генеральної сукупності розподілена по нормальному закону, тобто теоретичне розподіл має вигляд:

$$\varphi(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-a)^2}{2\sigma^2}} \quad \text{з параметрами: } a = M(X) = \bar{x}_{\text{ген}} - \text{математичне очікування}$$

$$\text{ознаки } X; \sigma = \sqrt{M((X - M(X))^2)} = \sigma_{\text{ген}} - \text{середньоквадратичне відхилення ознаки}$$

X . Побудуємо інтервальну оцінку параметра $a = \bar{x}_{\text{ген}}$ для двох випадків:

1) параметр σ нормального закону розподілу ознаки X генеральної сукупності *відомий*. В цьому випадку інтервальна оцінка параметра $a = \bar{x}_{\text{ген}}$ із заданою надійністю γ визначається формулою: $\bar{x}_B - \Delta < a < \bar{x}_B + \Delta$, де $\Delta = \frac{t \cdot \sigma}{\sqrt{n}}$, t – аргумент функції Лапласа: $\Phi(t) = \frac{\gamma}{2}$.

2) параметр σ нормального закону розподілу ознаки X генеральної сукупності *невідомий*. В цьому випадку інтервальна оцінка параметра $a = \bar{x}_{\text{ген}}$ із заданою надійністю γ визначається формулою: $\bar{x}_B - \Delta < a < \bar{x}_B + \Delta$, де $\Delta = \frac{t_\gamma \cdot S}{\sqrt{n-1}}$, S – точкова оцінка параметра σ , $t_\gamma = t(\gamma, n-1)$ – значення розподілу Стюдента, які знаходимо по таблиці.

Для оцінки генерального середнього квадратичного відхилення σ при заданій надійності γ можна побудувати довірчий інтервал виду $s \cdot (1 - q) < \sigma < s \cdot (1 + q)$ при $q < 1$ і $0 < \sigma < s \cdot (1 + q)$ при $q > 1$, де $q = q(n, \gamma)$ – значення, яке визначається з таблиць.

Заміна невідомої величини σ обчислюваною величиною S призводить до зменшення точності інтервальної оцінки і збільшення довжини довірчого інтервалу.

Перевірка статистичних гіпотез

Під *статистичною гіпотезою* розуміється всяке висловлювання про генеральну сукупність (випадкову величину X), що перевіряється по вибірковій сукупності (за результатами спостережень). Це може бути припущення про вид невідомого розподілу випадкової величини або про параметри відомого розподілу. Поряд з гіпотезою, що перевіряється (*нульовою*, або *основною*) H_0 формулюється і гіпотеза, що їй суперечить (*конкуруюча*, або *альтернативна*) H_1 , яка приймається, якщо відкинута нульова гіпотеза. *Наприклад.* Нехай H_0 полягає в тому, що математичне очікування генеральної сукупності $a = 3$. Тоді можливі варіанти H_1 : а) $a \neq 3$; б) $a > 3$; в) $a < 3$. Гіпотези поділяються на *прості* (містять лише одне припущення) і *складні* (що містять більше одного припущення). Для розглянутого прикладу $a = 3$ – проста гіпотеза, $a > 3$ – складна, що складається з нескінченного числа простих виду $a = c$, де c – будь-яке число, більше ніж 3.

При перевірці гіпотези можуть бути допущені помилки двох видів: *помилка першого роду*, якщо відхилена вірна нульова гіпотеза, і *помилка другого роду*, якщо прийнята невірна нульова гіпотеза. *Зауваження.* Яка з помилок є на практиці більш небезпечною, залежить від конкретного завдання. Наприклад, якщо перевіряється правильність вибору методу лікування хворого, то помилка першого роду означає відмову від правильної методики, що може уповільнити лікування, а помилка другого роду (застосування неправильної методики) чревата погіршенням стану хворого і є більш небезпечною. Імовірність помилки першого роду називається *рівнем значущості* α . Імовірність α задається малим числом, оскільки це ймовірність помилкового висловлювання, при цьому зазвичай використовуються стандартні значення: 0,05; 0,01. Наприклад, $\alpha = 0,05$ означає наступне: якщо гіпотезу H_0 перевіряти по кожній з 100 вибірок однакового обсягу, то в середньому в 5 випадках з 100 зробимо помилку першого роду.

Для перевірки статистичної гіпотези використовується спеціально підібрана випадкова величина K з відомим законом розподілу, яка називається *статистичним критерієм*. Множина її можливих значень розбивається на дві непересічних підмножини: одна з них (*критична область*) містить значення

критерію, при яких нульова гіпотеза відхиляється, друга (область прийняття гіпотези) – значення K , при яких вона приймається. Значення K , яке відокремлює критичну область від області прийняття гіпотези, називаються *критичною точкою* $k_{кр}$. Критична область може бути *правосторонньою* (якщо вона задається нерівністю $K > k_{кр}$), *лівосторонньою* ($K < k_{кр}$) або *двосторонньою* ($K < (k_{кр})_1$, $K > (k_{кр})_2$). Для її знаходження потрібно задати *рівень значущості*; тоді, наприклад, правостороння критична область задається умовою $p(K > k_{кр}) = \alpha$.

(Див. рисунок)

Порядок перевірки статистичної гіпотези:

1) задається рівень значущості α , вибирається статистичний критерій K , визначається вид критичної області та обчислюється (зазвичай за таблицями для закону розподілу K) значення $k_{кр}$;

2) за вибіркою обчислюється спостережуване значення критерію $K_{спост}$;

3) якщо $K_{спост}$ потрапляє в критичну область, нульова гіпотеза відкидається; при попаданні $K_{спост}$ в область прийняття гіпотези нульова гіпотеза приймається.

Потужністю критерію називають імовірність потрапляння критерію в критичну область за умови, що вірна конкуруюча гіпотеза. Якщо позначити ймовірність помилки другого роду (прийняття неправильної нульової гіпотези) β , то потужність критерію дорівнює $1 - \beta$. Отже, чим більше потужність критерію, тим менше ймовірність зробити помилку другого роду. Тому після вибору рівня значущості слід будувати критичну область так, щоб потужність критерію була максимальною.

Перевірка гіпотез про однорідність вибірок

Гіпотези про однорідність вибірок – це гіпотези про те, що аналізовані вибірки вилучені з однієї й тієї генеральної сукупності.

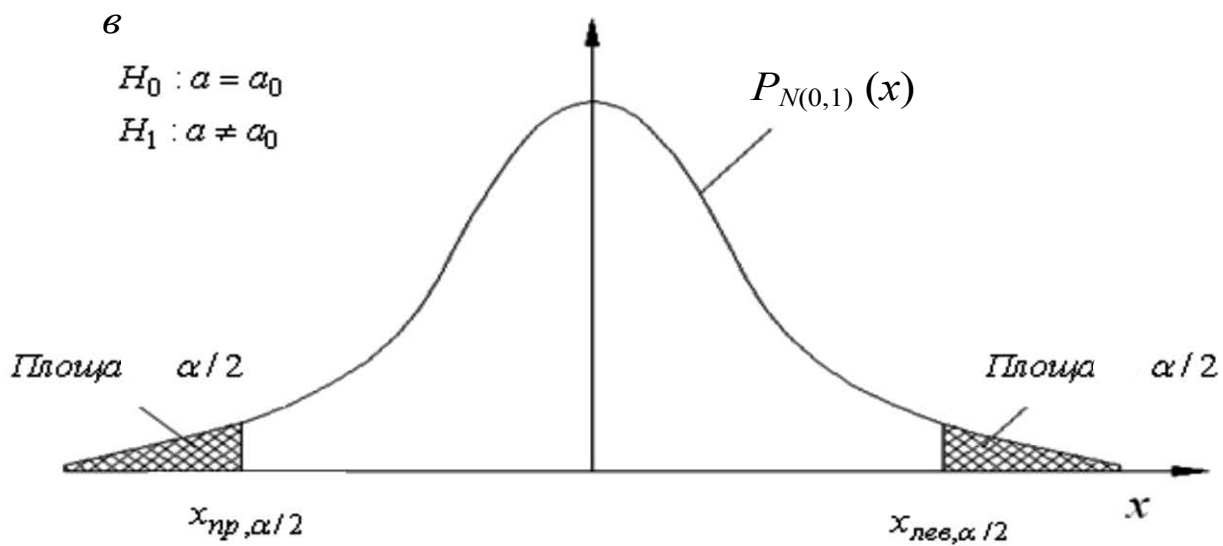
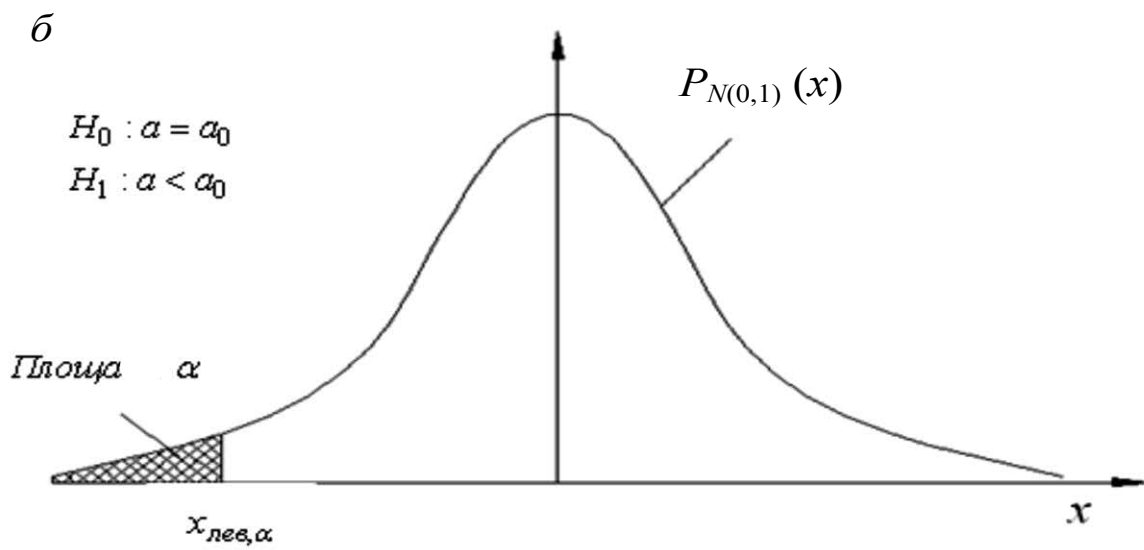
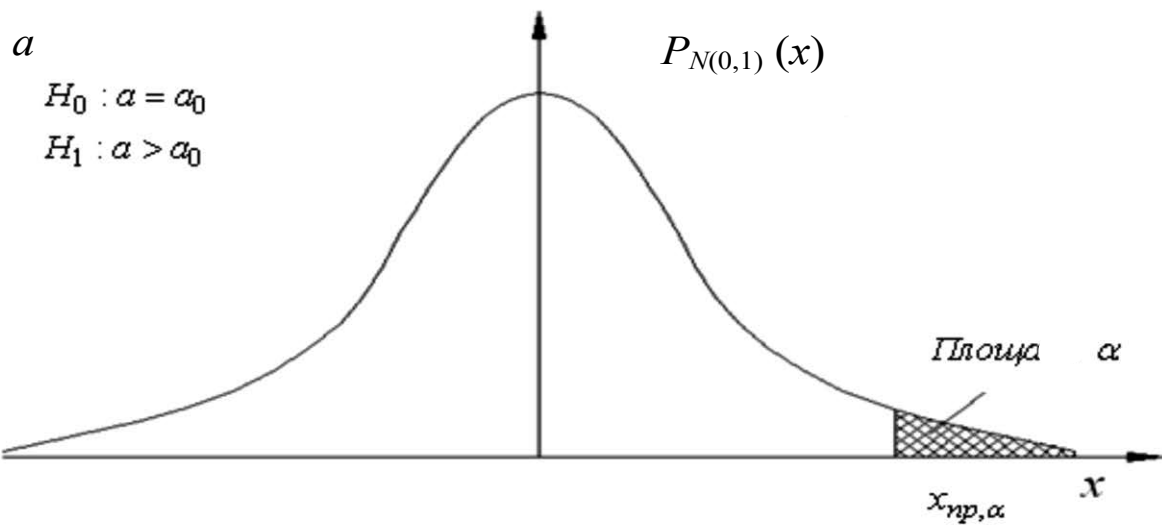
Нехай є дві незалежні вибірки, зроблені з генеральних сукупностей з невідомими теоретичними функціями розподілу $F_1(x)$ і $F_2(x)$. Нульова гіпотеза, що перевіряється, має вигляд $H_0: F_1(x) = F_2(x)$ проти конкуруючої $H_1: F_1(x) \neq F_2(x)$. Будемо вважати, що функції $F_1(x)$ і $F_2(x)$ неперервні.

В критерії Колмогорова-Смірнова порівнюються дві функції емпіричного розподілу.

Статистика критерію має вигляд:
$$\lambda' = \sqrt{\frac{n_1 n_2}{n_1 + n_2}} \cdot \max |F_{n_1}(x) - F_{n_2}(x)|$$
, де $F_{n_1}(x)$ і $F_{n_2}(x)$ –

емпіричні функції розподілу, побудовані за двома вибірками обсягів n_1 і n_2 . Гіпотеза H_0 відкидається, якщо значення статистики, що фактично спостерігається λ' більше за критичне, тобто $\lambda' > \lambda'_{\text{крит}}$ і приймається у протилежному випадку.

При малих обсягах вибірок ($n_1 \leq 20$, $n_2 \leq 20$) критичні значення $\lambda'_{\text{крит}}$ для заданих рівнів значущості критерію можна знайти в спеціальних таблицях. При $n_1 \rightarrow \infty$, $n_2 \rightarrow \infty$ (а практично при $n_1 \geq 50$, $n_2 \geq 50$) розподіл статистики λ' знаходиться за таблицями розподілу Колмогорова.



для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

ОПЕРАЦІЙНІ СИСТЕМИ

8.1.1 Різновиди операційних систем (однокористувацькі, багатокористувацькі, реального часу). ОС для вбудованих систем.

Різновиди операційних систем включають:

1. Однокористувацькі операційні системи: Ці ОС призначені для роботи на комп'ютерах, де один користувач виконує всі операції. Приклади включають Windows, macOS та більшість десктопних та ноутбуків версій Linux.
2. Багатокористувацькі операційні системи: Ці ОС дозволяють кільком користувачам одночасно працювати на одному комп'ютері. Вони зазвичай використовуються в корпоративному або мережному середовищі, де кілька користувачів можуть обмінюватися ресурсами комп'ютера. Приклади включають Windows Server, Linux Server та macOS Server.
3. Операційні системи реального часу: Ці ОС призначені для систем, де важливо забезпечити виконання завдань у строго визначені часові рамки. Вони використовуються в авіоніці, медичних пристроях, автомобілях та інших додатках, де потрібна швидка реакція на зовнішні події. Приклади включають QNX, VxWorks та FreeRTOS.
4. ОС для інтегрованих систем Ці ОС спеціально розроблені для роботи на вбудованих системах, таких як мобільні пристрої, автомобільні інформаційно-розважальні системи, розумні пристрої для дому та інші вбудовані пристрої. Вони зазвичай мають невеликий розмір, низьке споживання ресурсів та оптимізовані для конкретних завдань. Приклади включають Android, Embedded Linux та FreeRTOS.

8.1.2 Основні функції операційних систем.

Операційні системи виконують низку ключових функцій, включаючи:

1. Керування ресурсами: Операційна система контролює доступ до ресурсів комп'ютера, таких як процесор, пам'ять, пристрої вводу-виводу та файлова система. Вона забезпечує ефективний розподіл ресурсів між різними програмами та користувачами.
2. Управління процесами: Операційна система управляє запуском, виконанням та завершенням процесів, які є програмами, що виконуються. Вона контролює їхні пріоритети, планує їх виконання та забезпечує захист між процесами.
3. Керування пам'яттю: Операційна система керує фізичною та віртуальною пам'яттю комп'ютера, забезпечуючи виділення, звільнення та захист пам'яті для програм. Вона також відповідає за обмін даними між оперативною та постійною пам'яттю.
4. Управління файловою системою: Операційна система забезпечує організацію та доступ до файлів та каталогів на пристроях зберігання даних. Вона дозволяє користувачам створювати, копіювати, переміщати, видаляти та перейменовувати файли, а також керувати їх доступом та безпекою.
5. Керування введенням-виводом: Операційна система контролює взаємодію комп'ютера із зовнішніми пристроями введення-виводу, такими як клавіатура, миша, принтери, дисководи та мережні пристрої. Вона забезпечує обробку запитів на введення-виведення та керує буферизацією даних.
6. Керування користувачами та безпекою: Операційна система контролює доступ користувачів до ресурсів комп'ютера та забезпечує захист від несанкціонованого доступу та шкідливих програм. Вона керує обліковими записами користувачів, правами доступу та автентифікацією.

Ці функції допомагають забезпечити ефективну роботу комп'ютерної системи та забезпечити її стабільність, продуктивність та безпеку.

8.1.3. Вимоги до операційних систем, поняття відмовостійкості.

Вимоги до операційних систем залежать від конкретного застосування та контексту використання. Однак, загальні вимоги включають наступне:

1. Надійність: Операційна система має бути надійною та стабільною, щоб забезпечити безперебійну роботу комп'ютерної системи. Це означає, що вона повинна бути здатна запобігати збоєм та відновлюватися після них.
2. Продуктивність: Операційна система повинна забезпечувати швидку та ефективну роботу комп'ютера, забезпечуючи мінімальний час відгуку на запити користувача та ефективне використання ресурсів.
3. Безпека: Операційна система повинна забезпечувати захист від несанкціонованого доступу та шкідливих програм. Це включає реалізацію механізмів аутентифікації, авторизації, шифрування даних і контролю доступу.
4. Гнучкість і масштабованість: Операційна система повинна бути гнучкою і здатною адаптуватися до різних потреб і умов використання, що змінюються. Вона також має бути масштабованою, щоб підтримувати роботу на різних типах обладнання та у різних середовищах.

Відмовостійкість операційної системи - це її здатність продовжувати роботу та забезпечувати доступність сервісів навіть у разі виникнення збоїв чи відмов. Для забезпечення відмовостійкості операційні системи можуть використовувати різні методи, такі як резервування ресурсів, механізми відновлення після збоїв, реплікація даних та дублювання обладнання. Це дозволяє мінімізувати простой та забезпечувати безперервну роботу комп'ютерної системи навіть у несприятливих умовах.

8.2.1 Типи архітектур ядра операційної системи.

Архітектура ядра операційної системи визначає внутрішню структуру та організацію її основних компонентів. Існує кілька типів архітектур ядра операційної системи:

1. Монолітне ядро (Monolithic Kernel): У монолітному ядрі всі основні функції операційної системи реалізовані як один великий модуль, який виконується у привілейованому режимі. Цей модуль обробляє управління процесами, управління пам'яттю, введення-виведення та інші аспекти системи. Приклади монолітних ядер включають Linux та UNIX.
2. Мікроядро (Microkernel): У мікроядрі базові функції операційної системи, такі як планування процесів та управління пам'яттю, реалізовані як окремі модулі, які працюють у привілейованому режимі. Інші компоненти, такі як драйвери пристроїв і файлові системи, виконуються в режимі користувача і взаємодіють з ядром через набір системних викликів. Приклади мікроядерних операційних систем включають QNX та MINIX.
3. Гібридне ядро (Hybrid Kernel): Гібридне ядро поєднує деякі риси монолітного та мікроядерного підходів. Воно містить основні функції операційної системи у вигляді невеликого мікроядра, а інші компоненти реалізовані як модулі, що динамічно завантажуються, які можуть виконуватися в привілейованому або користувальницькому режимі в залежності від потреб. Прикладом гібридного ядра є Windows NT.

Кожен із цих типів архітектур ядра операційної системи має свої переваги та недоліки, і вибір конкретної архітектури залежить від вимог до конкретної системи та її завдань.

8.2.2 Привілейований режим і режим користувача.

Привілейований режим і режим користувача - це два основних режими роботи процесора, які використовуються операційною системою для забезпечення захисту та безпеки системи. Їхні основні характеристики:

Привілейований режим (Privileged Mode):

Також відомий як режим ядра (Kernel Mode) або системний рівень.

У цьому режимі процесор має повний доступ до всіх апаратних ресурсів та привілеїв керування, таких як доступ до пам'яті, керування введенням-виводом та виконання привілейованих інструкцій.

Операційна система працює в привілейованому режимі для виконання завдань, які вимагають прямого доступу до апаратних ресурсів або виконання привілейованих операцій, таких як зміна режиму роботи процесора або управління пам'яттю.

Режим користувача (User Mode):

Також відомий як режим користувача програми.

У цьому режимі процесор обмежує доступ до апаратних ресурсів та привілеїв управління.

Користувальницькі програми працюють у режимі користувача, де доступ до апаратних ресурсів обмежений, і всі операції виконуються через системні виклики, що надаються операційною системою.

Це забезпечує ізоляцію між процесами і захищає ядро операційної системи від помилок або зловмисних дій додатків користувача.

Привілейований режим використовується операційною системою для виконання привілейованих операцій, а режим користувача забезпечує безпечне виконання користувальницьких застосунків і ізоляцію між процесами.

8.2.3 Системні виклики.

Системні виклики - це механізм, який дозволяє користувальницьким застосункам взаємодіяти з операційною системою та отримувати доступ до її функцій та ресурсів. Ось основні характеристики системних викликів:

1. Використання: Програма користувача може викликати системні виклики для виконання певних завдань, таких як читання або запис файлів, управління процесами, робота з мережею і т.д.
2. Інтерфейс: Системні виклики надають інтерфейс між додатком користувача і операційною системою. Програма робить запит через системний виклик, який потім обробляється ядром операційної системи.
3. Процес виконання: При виклику системного виклику відбувається перемикання режимів роботи процесора з режиму користувача на привілейований режим (режим ядра). Це дозволяє операційній системі виконати необхідну операцію з повним доступом до апаратних ресурсів.
4. Безпека: Системні виклики забезпечують безпечний спосіб взаємодії між програмами користувача та операційною системою. Користувальницькі програми не мають прямого доступу до апаратних ресурсів та функцій операційної системи, що допомагає запобігти помилкам та забезпечити безпеку системи.

Приклади системних викликів: Деякі типові системні виклики включають **read()** і **write()** для роботи з файлами, **fork()** і **exec()** для управління процесами, **socket()** для створення мережних з'єднань та багато інших, які дозволяють програмам взаємодіяти з різними аспектами операційної системи.

8.3.1 Мультизадачність.

Мультизадачність - це здатність операційної системи виконувати одночасно кілька завдань чи процесів. Вона забезпечує у користувача враження одночасної роботи кількох програм чи завдань на комп'ютері, навіть якщо фактично ресурси процесора та інших пристроїв розподіляються між ними.

Основні характеристики мультизадачності:

1. Паралельність виконання завдань: Операційна система ефективно керує часовими інтервалами виконання різних завдань, таким чином, що користувач може сприймати їх виконання як паралельне, хоча процесор перемикається між ними дуже швидко.
2. Поділ ресурсів: Мультизадачність дозволяє різним застосункам та процесам розділяти ресурси комп'ютера, такі як процесорний час, пам'ять, пристрої вводу-виводу тощо. Це дозволяє підвищити використання ресурсів та ефективність роботи комп'ютерної системи.
3. Планування та управління задачами: Операційна система використовує різні алгоритми планування визначення порядку виконання завдань і розподілу ресурсів з-поміж них. Це дозволяє оптимізувати продуктивність системи та забезпечувати «чуйність» для користувальницьких застосунків (швидке реагування на введення користувача, миттєве відкриття та закриття застосунків тощо).

4. Підтримка досвіду користувача: Завдяки мультизадачності користувач може одночасно виконувати кілька завдань, таких як робота з текстовим процесором, перегляд веб-сторінок і прослуховування музики, що підвищує ефективність та зручність використання комп'ютера.

8.3.2 Мультипроцесорність.

Мультипроцесорність - це можливість операційної системи ефективно використовувати кілька процесорів (чи ядер процесора) для виконання завдань. У мультипроцесорній системі кілька процесорів працюють паралельно, що дозволяє збільшити продуктивність і розподілити навантаження виконання різних завдань.

Операційна система має вміти ефективно розподіляти завдання між процесорами, контролювати доступ до загальних ресурсів та забезпечувати узгодженість даних. В результаті мультипроцесорність дозволяє покращити чуйність системи, збільшити швидкість виконання завдань та підвищити її стійкість до відмови.

8.3.3 Мультипроцесорність.

В операційних системах паралельність забезпечує можливість виконання кількох завдань чи процесів одночасно, що підвищує ефективність використання ресурсів комп'ютерної системи. Це особливо важливо на багатоядерних процесорах, де кожне ядро може виконувати свої завдання незалежно один від одного. Паралельність може бути досягнута різними способами:

1. Паралельні процеси: Операційна система може одночасно запускати кілька процесів, кожен із яких виконує своє завдання. Ці процеси можуть бути незалежними або взаємодіяти один з одним через механізми міжпроцесної взаємодії.
2. Паралельні потоки: Всередині одного процесу можуть існувати кілька паралельних потоків виконання, які можуть виконувати різні завдання. Це дозволяє процесу ефективно використовувати багатопроцесорні системи чи багатозадачні середовища.
3. Розпаралелювання задач: Деякі операції можуть бути розподілені між кількома процесорами або ядрами процесора для прискорення виконання. Це може містити паралельну обробку даних, паралельний доступ до пам'яті або навіть паралельне виконання інструкцій.

Паралельність дозволяє покращити чуйність системи, прискорити виконання завдань та підвищити загальну продуктивність комп'ютерної системи.

8.4.1 Керування процесами і потоками. Модель процесу і потоку. Блок керування процесом. Контекст процесу. Блок керування потоком. Стани процесу (потoku).

Управління процесами та потоками в операційній системі відіграє ключову роль у забезпеченні ефективного використання ресурсів та управлінні виконанням завдань. Ось основні концепції та елементи управління:

1. **Модель процесу та потоку:** Процес є екземпляром програми, який виконується в операційній системі. Він включає код програми, дані, стек викликів, відкриті файли та інші ресурси. Потік – це підпроцес усередині процесу, який може виконувати свої інструкції незалежно від інших потоків у тому самому процесі.
2. **Блок управління процесом (PCB - Process Control Block):** PCB - це структура даних, яка зберігає інформацію про кожен виконуваний процес в операційній системі. Цей блок містить всю необхідну інформацію про процес, таку як його стан, ідентифікатор, реєстри процесора, інформацію про пам'ять, лічильники та інші параметри, які операційна система використовує для керування виконанням процесів. Коли процесор перемикає контекст між процесами, він зберігає стан поточного процесу в його PCB і завантажує стан наступного процесу з його PCB.
PCB є ключовим інструментом операційної системи для управління та відстеження стану процесів, а також для координації їх виконання.
3. **Контекст процесу:** Контекст процесу (або контекст виконання процесу) – це набір даних, який операційна система зберігає та відновлює при перемиканні між виконанням різних процесів на центральному процесорі (CPU). Цей контекст включає інформацію про стан процесу в певний момент часу, яка потрібна для продовження його виконання з точки зупинки або при відновленні роботи після перемикавання.

Контекст процесу зазвичай включає такі елементи:

3.1 Регістри процесора: це значення регістрів центрального процесора, такі як лічильник команд (program counter), показчик стека та регістри загального призначення. Ці значення визначають стан виконання процесу у конкретний час.

3.2 Стан процесу: це інформація про стан процесу, така як його поточний стан (активний, очікуваний, зупинений тощо), ідентифікатор процесу, пріоритет, обмеження часу CPU та інші атрибути.

3.3 Інформація про пам'ять: це інформація про розташування програмного коду, даних та стек процесу в пам'яті. Вона може включати адреси початку і кінця сегментів пам'яті, використовуваних процесом.

Коли операційна система перемикає виконання між процесами, вона зберігає поточний контекст процесу, а потім завантажує контекст наступного процесу продовження виконання. Цей механізм дозволяє операційній системі ефективно керувати безліччю процесів та забезпечувати багатозадачність..

4. **Блок управління потоком** (TCB - Thread Control Block): це структура даних, що використовується операційною системою для управління та відстеження стану виконання конкретного потоку (або нитки) у багатопотоковому середовищі. Кожен потік в операційній системі має власний TCB, який містить інформацію, необхідну для його управління та виконання.

Зазвичай TCB включає такі елементи:

4.1 Ідентифікатор потоку: унікальний ідентифікатор, який дає змогу операційній системі однозначно ідентифікувати потік серед інших потоків.

4.2 Стан потоку: це інформація про поточний стан виконання потоку, такий як активний, очікуваний, зупинений і т.д.

4.3 Контекст виконання: це дані, необхідні для продовження виконання потоку з точки зупинки, включаючи регістри процесора, показчик стека та інші релевантні регістри.

4.4 Стек потоку: це область пам'яті, яка використовується для зберігання локальних змінних та часових даних під час виконання функцій та методів потоку.

4.5 Пріоритет виконання: значення, що визначає відносний пріоритет потоку, порівняно з іншими потоками в системі. Він використовується для керування плануванням виконання потоків.

4.6 Інформація про блокування: інформація про будь-які блокування або синхронізацію, пов'язану з потоком, таких як м'ютекси, семафори та інші механізми синхронізації.

TCB є основним інструментом операційної системи для управління потоками та забезпечення їх безпечного та ефективного виконання. Кожна зміна стану потоку відбивається у його TCB, що дозволяє операційній системі правильно управляти ресурсами та виконанням завдань у багатопотоковому середовищі.

5. **Стан процесу** (потіку): Процес (потік) може перебувати в різних станах (див. слайд).

Стани процесу

1. Процес виконується в режимі задачі.
2. Процес виконується в режимі ядра.
3. Процес не виконується, але готовий до запуску під управлінням ядра.
4. Процес призупинено і він знаходиться в оперативній пам'яті.
5. Процес готовий до запуску, але програма підкачки (нульовий процес) повинна ще завантажити процес в оперативну пам'ять, перш ніж він буде запущений під керуванням ядра.
6. Процес призупинено і програма підкачки вивантажила його в зовнішню пам'ять, щоб в оперативній пам'яті звільнити місце для інших процесів.
7. Процес повернуто з привілейованого режиму (режиму ядра) в непривілейований (режим задачі), ядро резервує його і перемикає контекст на інший процес. Про відміну цього стану від стану 3 (готовність до запуску) буде сказано нижче.
8. Процес знову створений і знаходиться в перехідному стані – процес існує, але не готовий до виконання, хоча і не припинений. Цей стан є початковим станом всіх процесів, крім нульового.
9. Процес викликає системну функцію exit і припиняє існування.

Керування переходами між цими станами здійснюється операційною системою залежно від подій та запитів, що надходять від додатків та зовнішніх пристроїв.

В цілому, управління процесами та потоками дозволяє операційній системі ефективно використовувати ресурси комп'ютера, забезпечувати чуйність системи та керувати виконанням застосунків.

8.4.2 Розподіл пам'яті (типи адрес, методи розподілу пам'яті).

Розподіл пам'яті - це процес операційних системах, у якому виділяється і керується фізичним чи віртуальним простором пам'яті програм і процесів. Цей процес включає виділення, звільнення та управління блоками пам'яті для програмного забезпечення.

У межах розподілу пам'яті операційна система відстежує вільні та зайняті області пам'яті, і навіть контролює доступом до них. Це дозволяє ефективно використовувати ресурси системи та запобігати конфліктам між різними процесами, які можуть намагатися використовувати одні й ті самі області пам'яті.

Розподіл пам'яті може здійснюватися різними способами, включаючи використання різних алгоритмів виділення пам'яті, таких як стратегії First Fit, Best Fit і Worst Fit, а також використання різних методів управління віртуальною пам'яттю, таких як сегментація та сторінкова пам'ять. Ці методи дозволяють ефективно керувати пам'яттю та забезпечувати безпечне та надійне функціонування операційної системи.

Трохи детальніше про стратегії:

Діаграма станів процесу



First Fit (перший, що підходить): Це алгоритм розподілу пам'яті, коли при запиті на виділення блоку пам'яті система вибирає перший відповідний блок пам'яті з доступних блоків. Цей метод є простим у реалізації, але може призвести до фрагментації пам'яті, коли між блоками може залишатися нерозподілений простір.

Best Fit (найкращий, що підходить): Цей метод полягає у виборі блоку пам'яті, який найкраще відповідає розміру запиту. Система шукає найменший відповідний блок доступних блоків. Цей метод може зменшити фрагментацію пам'яті, але потребує додаткових обчислювальних ресурсів для пошуку відповідного блоку.

Worst Fit (найгірший, що підходить): Цей метод вибирає найбільший підходящий блок з доступних блоків пам'яті для виділення. Він може призвести до більшої фрагментації пам'яті, оскільки між блоками може залишатися великий нерозподілений простір.

8.4.3 Віртуальна пам'ять (сторінкова, сегментна, сегментно-сторінкова організація пам'яті, свопінг).

Віртуальна пам'ять - це механізм, який дозволяє програмам використовувати більше пам'яті, ніж доступно на комп'ютері.

Сторінкова пам'ять: Пам'ять ділиться на блоки фіксованого розміру, що називаються сторінками, які завантажуються в оперативну пам'ять у міру необхідності. Процес також може бути поділено на такі ж сторінки. Це забезпечує ефективніше використання пам'яті та дозволяє завантажувати лише необхідні сторінки.

Сегментна організація пам'яті: Пам'ять поділяється на логічні сегменти різних розмірів, кожен із яких відповідає певному логічному блоку програми чи даних. Сегменти описуються сегментними дескрипторами.

Це забезпечує гнучкість управління пам'яттю, але може призвести до фрагментації пам'яті.

Сегментно-сторінкова організація пам'яті: Комбінує переваги як сторінкової, так і сегментної організації пам'яті, розділяючи пам'ять на сегменти і далі подальшим поділом кожного сегмента на сторінки.

Свопінг: Процес переміщення даних між оперативною пам'яттю та зовнішнім носієм, таким як жорсткий диск, для звільнення оперативної пам'яті та управління процесами, коли доступна оперативна пам'ять вичерпана.

8.5.1 Основні поняття про файли та файлові системи

Основні поняття про файли та файлові системи включають:

Файл: Іменована колекція даних на диску, яка зберігається як єдине ціле та має унікальне ім'я.

Шлях до файлу: Унікальний шлях, що вказує на розташування файлу у файловій системі.

Розширення файлу: Частина імені файлу, яка вказує на тип або формат даних.

Каталог (директорія): Спеціальний тип файлу, який містить посилання на інші файли або каталоги.

Файлова система: Метод організації файлів та каталогів на носії даних, що забезпечує доступ до них та їх керування.

Файлові атрибути: Властивості файлу, такі як права доступу (наприклад, права на читання, запис або виконання), розмір, дата створення та дата останнього доступу або зміни та ін.

Файлові операції: Дії, які можна виконати з файлами, такі як читання, запис, створення, видалення та переміщення.

Файлові дескриптори: Спеціальні об'єкти в операційній системі, які використовуються для представлення відкритих файлів та керування ними.

8.5.2 Логічна та фізична організація файлів.

Логічна організація файлів визначає, як дані всередині файлу представлені для програмного забезпечення, включаючи форматування, структуру та доступ до даних. Це може включати різні методи організації даних, такі як послідовний запис, індексування, хешування або деревоподібні структури.

Фізична організація файлів визначає, як дані на диску або іншому носії фізично розташовуються та керуються операційною системою. Це включає структуру файлової системи, методирозподылу простору на диску, такі як блокове виділення або контроль вільного простору, а також управління доступом до даних на рівні носія.

Для кращого розуміння відмінностей між логічною та фізичною організацією файлів розглянемо приклад *текстового файлу*.

Логічна організація файлу визначає, як дані у цьому файлі структуровані та доступні програмам. Наприклад, у текстовому файлі дані можуть бути організовані як рядки, що розділені символами нового рядка. Кожен рядок може бути окремим записом або частиною інформації.

Фізична організація файлу включає інформацію про те, як дані зберігаються на фізичному носії, такому як жорсткий диск. Це може включати інформацію про сектори, доріжки, голівки та інші аспекти, пов'язані з фізичним розташуванням даних на диску. Крім того, фізична організація може включати методи управління вільним простором на диску.

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка"
ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)

10. ШТУЧНИЙ ІНТЕЛЕКТ

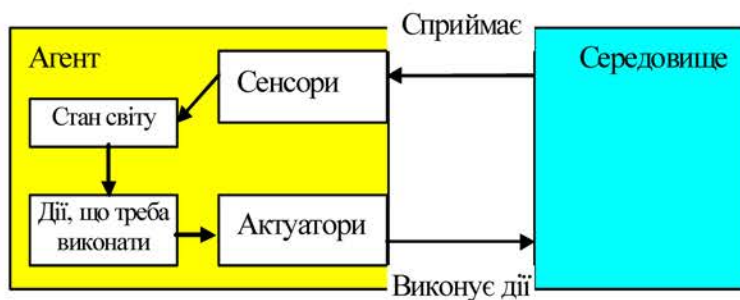
10.1. Фундаментальні поняття

10.1.1. Інтелектуальна система, поняття агента і середовища, задачі штучного інтелекту, сильний і слабкий штучний інтелект, ризики штучного інтелекту.

4 інтелектуальна система

Система, яка забезпечує розв'язування неформалізованих задач користувача в деякій предметній галузі та організовує його взаємодію з комп'ютером у звичних поняттях, термінах, образах

Агент – це сутність (об'єкт), що знаходиться у середовищі, здатна сприймати його та реагувати на нього виконуючи дії відповідно до мети.



Середовище - це залишок Всесвіту, який лежить поза межами агента.

Повністю спостережне – середовище, у якому датчики агента надають йому доступ до повної інформації про стан середовища в кожний момент часу, необхідної для вибору агентом дії.	Частково спостережне – середовище, у якому через датчики, що створюють шум і є неточними, або через те, що окремі характеристики його стану просто відсутні в інформації, отриманій від датчиків, агент не може мати доступ до повної інформації про стан середовища в кожний момент часу.
--	---

Детерміноване – середовище, наступний стан якого цілком визначається поточним станом і дією, виконаною агентом.	Стохастичне – середовище, наступний стан якого цілком не визначається поточним станом і дією, виконаною агентом.
--	---

Епізодичне середовище: досвід агента складається з нерозривних епізодів, кожний з яких містить сприйняття середовища агентом, а потім виконання однієї дії, наступний епізод не залежить від дій у попередніх епізодах.	Послідовне середовище: поточне рішення може вплинути на всі майбутні рішення.
--	--

Одноагентне – середовище, де діє тільки один агент	Мультиагентне – середовище, де діють два агенти і більше.
---	--

Динамічне – середовище, що може змінитися в ході того, як агент обирає чергову дію.	Напівдинамічне – середовище, що з часом не змінюються, а змінюються тільки показники продуктивності агента в середовищі.	Статичне – середовище, що не може змінитися в ході того, як агент обирає чергову дію.
--	---	--

Дискретне або неперервне середовище: за станом середовища, способом обліку часу, а також сприйняттям і діям агента.

Сильний та слабкий штучний інтелект

це філософська концепція, що стосується штучного інтелекту, яка стверджує, що певні типи штучного інтелекту можуть ефективно формулювати та вирішувати проблеми.

Сильний (універсальний)

штучний інтелект: штучна система може отримати здатність мислити і усвідомлювати себе як окрему особистість (зокрема, розуміти власні думки), хоча і не обов'язково, що її розумовий процес буде подібний до людського.

Слабкий (прикладний або вузький)

штучний інтелект - призначений для вирішення будь-якої однієї інтелектуальної задачі або їх невеликої множини (наприклад, системи для гри в шахи, розпізнавання образів, мови, кредитний скоринг і т. д.), які не мають на увазі наявності у комп'ютера справжньої свідомості.

Задачі штучного інтелекту

Головним завданням штучного інтелекту є комп'ютерне вирішення когнітивних задач, властивих людському мозку. Такі задачі традиційно вирішуються людьми в умовах неповноти, неточності та суперечливості знань про об'єкт дослідження, для розв'язання яких немає чітко заданого алгоритму. Основні напрями досліджень в галузі штучного інтелекту виділяють такі:

– **подання знань і їх використання для прийняття рішень** – створення спеціалізованих моделей і мов для подання знань в ЕОМ, а також програмних і апаратних засобів для їхнього перетворення (поповнення, логічної обробки і т. д.).

– **планування доцільного поведіння** – дослідження зі створення методів формування цілей і вирішення задач планування дій агента, що функціонує в складному зовнішньому середовищі.

– **спілкування людини з ЕОМ** (обробка природної мови) – задачі створення мовних засобів, що дозволяють ефективно взаємодіяти з ЕОМ непрограмуємому користувачу (машинний переклад, обробка текстів, синтез мовлення, відповіді на питання).

– **розпізнавання образів і навчання** – дослідження зі сприйняття зорової, слухової і інших видів інформації, методів її обробки, формування відповідних реакцій на впливи зовнішнього середовища і способів адаптації штучних систем до середовища шляхом навчання

– **моделювання (апроксимація) залежностей**, прогнозування за наборами точкових спостережень (зразками), шляхом адаптації експертних знань та гібридним способом.

Застосування штучного інтелекту: допомагати / автоматизувати прийняття рішень, надавати рекомендації у різних галузях промисловості, включаючи: сільське господарство, транспортну індустрію; банківсько-фінансові технології; оборону і безпеку; освіту; енергетику; охорону здоров'я; право; виробництво; ЗМІ та розваги; змішану реальність; державний сектор; роздрібну торгівлю та маркетинг; космічну техніку; телекомунікації. Окремі приклади застосувань штучного інтелекту:

- Виявлення шахрайства (використання обману з метою наживи): підроблених грошей і документів; викрадених кредитних карток та документів; в особистому спілкуванні, наприклад електронній пошті; підроблених або викрадених осіб.

- Автоматизовані транспортні засоби: оптимізований маршрут (наприклад, пошук найшвидшого маршруту з огляду на поточні умови руху); автоматизована зміна смуги руху; уникнення об'єктів (наприклад, автоматичні маніпуляції гальмами, дроселем і рульовим управлінням на основі інтерпретації сигналів від камер і датчиків виявлення світла та визначення дальності); повністю автоматизована подорож з пункту А в пункт Б.

- Прогнозне технічне обслуговування (передбачає обслуговування або заміну компонентів на основі спостережень за їхньою поточною поведінкою чи продуктивністю та очікуваним терміном служби компонентів): виявлення пустот під залізничними коліями (що може призвести до сходу з рейок); виявлення тріщин або пошкоджень асфальту; виявлення несправності підшипників в електродвигунах; виявлення аномальних коливань потужності в системах електропостачання.

Ризики штучного інтелекту

Втрата роботи людьми через автоматизацію завдяки використанню штучного інтелекту.

Діпфейки – підробки зображень, відео та голосу людей може бути використане для поширення дезінформації або шкідливого вмісту, завдаючи шкоди репутації окремих осіб, впливаючи на політичний дискурс і навіть загрожуючи національній безпеці.

Порушення конфіденційності при обробці даних системами штучного інтелекту, зокрема неправомірне використання персональних даних та нав'язливого стеження за людьми.

Алгоритмічне зміщення, спричинене неправильними даними: алгоритм настільки хороший, наскільки хороші дані, на яких він навчається. Тому упереджені дані призведуть до упереджених результатів.

Соціально-економічна нерівність: переваги доступу до ШІ мають більш забезпечені суб'єкти, посилюючи соціально-економічну нерівність та несправедливість у суспільстві.

Небезпека для людини: системи ШІ, що призначені для взаємодії з фізичним світом, можуть становити загрозу безпеці людей, спричиняти аварії, завдавати шкоди людям.

Соціальна маніпуляція: здатність штучного інтелекту аналізувати величезні масиви даних і робити прогнози щодо поведінки людей може бути використана для соціального маніпулювання, зокрема для доставки персоналізованої реклами чи політичної пропаганди, щоб вплинути на думку чи поведінку людей. Така маніпуляція може підірвати демократичні процеси та позбавити окрему людину автономії.

Вторгнення в приватне життя та соціальне оцінювання: ШІ має потенціал для втручання в особисту конфіденційність і забезпечення соціального оцінювання, наприклад, шляхом аналізу активності людини у соціальних мережах, фінансових операцій та ін., щоб створити «соціальний бал» для кожної людини. Ця оцінка потім може бути використана для прийняття рішень щодо особи, наприклад, чи має вона право на отримання кредиту чи отримання пропозиції про роботу.

Невідповідність між цілями людей та цілями ШІ: Системи штучного інтелекту розроблені для досягнення конкретних цілей, які визначають їхні творці. Однак, якщо цілі системи штучного інтелекту не повністю узгоджуються з людськими, це може призвести до проблем. Наприклад, торговий алгоритм може бути запрограмований на максимізацію прибутку. Але якщо він робить це шляхом здійснення ризикованих угод, які ставлять під загрозу довгострокову життєздатність компанії, це буде невідповідністю цілей.

Відсутність прозорості: системи ШІ працюють як «чорні ящики», приймаючи рішення у спосіб, який люди не можуть легко зрозуміти або пояснити, що може призвести до недовіри та ускладнити притягнення до відповідальності систем ШІ.

Втрата контролю: люди можуть втратити контроль над ШІ: поступово, коли ми делегуємо більше рішень і завдань штучному інтелекту, або раптово, якщо система штучного інтелекту стане несправною. У поступовому сценарії ми можемо виявитися надмірно залежними від ШІ, нездатними функціонувати без нього та вразливими до будь-яких збоїв або упереджень у системах ШІ. У раптовому сценарії штучний інтелект, який вийшов з-під контролю, може завдати катастрофічної шкоди ще до того, як люди зможуть втрутитися.

Уведення програмної упередженості в процес прийняття рішень: Упередження в системах штучного інтелекту можуть виникати через дані, на яких вони навчаються, і спосіб їх програмування. Якщо штучний інтелект запрограмовано з певними упередженнями, свідомо чи несвідомо, це може призвести до несправедливого прийняття рішень.

Техносолюціонізм: тенденція розглядати ШІ як панацею, яка може вирішити всі проблеми може призвести до надмірної залежності від технологій і нехтування іншими важливими факторами.

Нечітке правове регулювання: законодавчі норми часто не встигають за розвитком ШІ, що може призвести до невизначеності та можливого неправильного використання штучного інтелекту, оскільки закони та нормативні акти не зможуть належним чином вирішити нові виклики, пов'язані з технологіями штучного інтелекту.

10.2. Пошук у просторі станів

10.2.1. Стратегії пошуку у просторі станів: пошук в ширину, пошук в глибину, прямий, зворотний та двонаправлений пошук, жадібний алгоритм.

Пошук – процес формування системою послідовності дій, що дозволяють їй досягти своїх цілей.

Стан – це колекція характеристик, що можуть використовуватися для визначення стану або статусу об'єкта (варіант рішення задачі).

Простір станів – це множина станів, за допомогою якої подаються переходи між станами, у яких може бути об'єкт. У результаті переходу об'єкт попадає з одного стану до іншого.

112 простір станів

Сукупність станів, у яких може перебувати інтелектуальна система, і операторів переходу з одного стану до іншого

Стратегія пошуку в просторі станів – порядок, у якому відбувається розгортання станів. Стратегія пошуку повинна бути виражена у вигляді функції, що вибирає певним чином з периферії наступний вузол, який підлягає розгортанню.

Стратегія прямого пошуку (forward chaining) – відповідає руху від вихідних вершин графа до цільової вершини.

17 пряме виведення

Стратегія виведення, за якою виведення виконується від вихідних посилок до цільового висновку

Стратегія зворотного пошуку (backward chaining) – відповідає руху від цільової вершини до вихідних вершин.

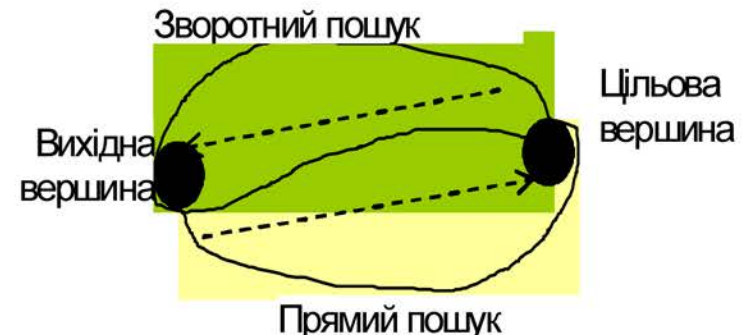
18 зворотне виведення

Стратегія виведення, при якій виведення виконується шляхом добору відповідних посилок під наявний висновок

Стратегія двонаправленого пошуку (bi-directional chaining) – поєднує прямий пошук (рух від вихідних вершин до цільової) і зворотний пошук (рух від цільової вершини до вихідної) та намагається досягти деякого загального для обох пошуків стану, зупиняючись після того, як два процеси пошуку зустрінуться на середині

116 двонаправлений пошук

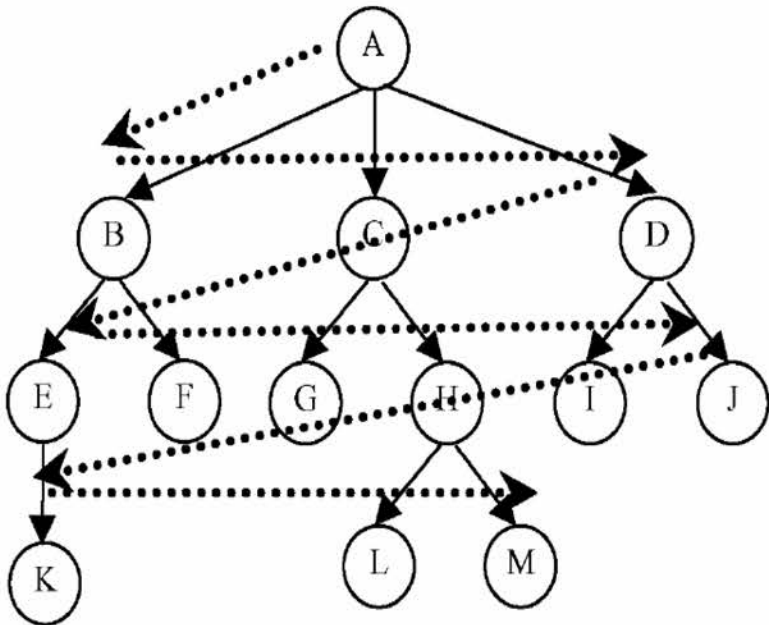
Спосіб пошуку, за яким обхід простору пошуку йде одночасно від початкових заданих вузлів до цільових вузлів і від цільових вузлів до початкових заданих вузлів



113 пошук вшир

Спосіб обходу простору пошуку, за яким спочатку аналізуються структури на всіх вузлах одного рівня, а потім — на вузлах наступних рівнів

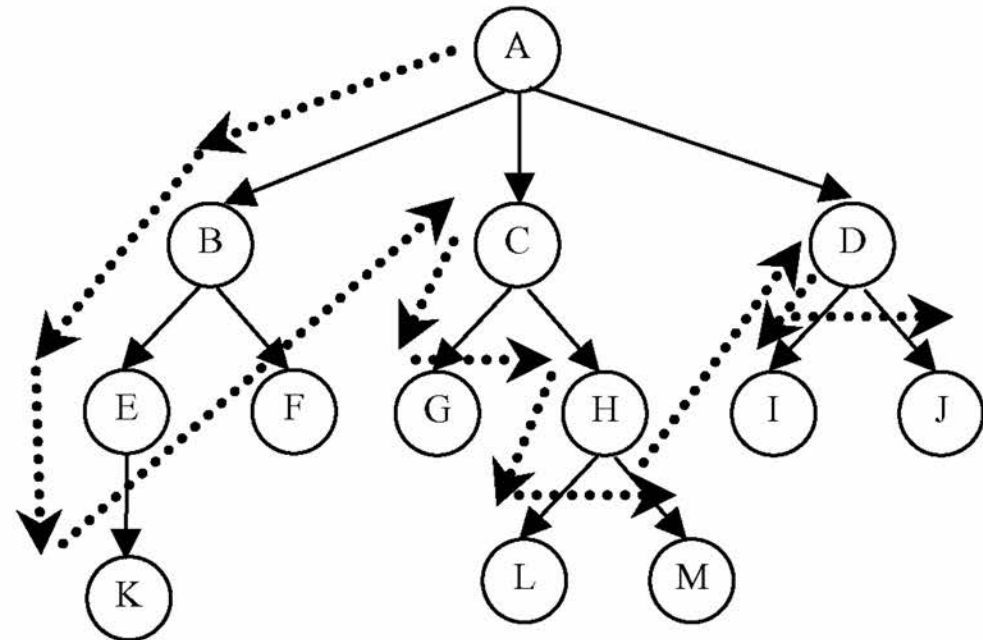
Метод пошуку в ширину (breadth-first search): простір станів послідовно проглядається по рівнях: на кожному рівні, у свою чергу, послідовно проглядаються стани, подані вузлами цього рівня один за іншим, і тільки якщо станів на даному рівні більше немає, метод переходить до наступного рівня.



114 пошук вглиб

Спосіб обходу простору пошуку, за яким спочатку аналізується структура, що починається в певному вузлі, а потім — структура в непроаналізованих вузлах того ж рівня

Метод пошуку в глибину (depth-first search): переглядаються стани на одному шляху – розгортається найглибший вузол у поточній периферії дерева пошуку, у результаті чого пошук безпосередньо переходить на найглибший рівень дерева пошуку, на якому вузли не мають нащадків. У міру того, як ці вузли розгортаються, вони видаляються з периферії, тому надалі пошук переходить до наступного найповерхневого вузлу, що усе ще має недосліджених нащадків.



Жадібний алгоритм (Greedy algorithm) — простий і прямолінійний евристичний алгоритм, який приймає найкраще рішення, виходячи з наявних на кожному етапі даних, не зважаючи на можливі наслідки, сподіваючись урешті-решт отримати оптимальний розв'язок.

Він базується на п'яти складових:

1. Набір можливих варіантів, з яких робиться вибір;
2. Функція вибору, за допомогою якої знаходиться найкращий варіант, який буде додано до розв'язку;
3. Функція придатності, яка визначає придатність отриманого набору (Придатний набір варіантів — такий, що обіцяє не просто отримання розв'язку, а отримання оптимального розв'язку задачі);
4. Функція цілі, передає значення розв'язку або частковому розв'язку;
5. Функція розв'язку, яка вказує на те, що кінцевий розв'язок знайдено.

Властивість оптимальної підструктури:

Задача має оптимальну підструктуру, якщо оптимальний розв'язок задачі містить оптимальний розв'язок для підзадач. Інакше кажучи, задача має оптимальну підструктуру, якщо кожен наступний крок веде до оптимального розв'язку.

Принцип жадібного вибору

До оптимізаційної задачі можна застосувати принцип жадібного вибору, якщо послідовність локально оптимальних виборів дає глобально оптимальний розв'язок. В типовому випадку доведення оптимальності здійснюється за такою схемою: спочатку доводиться, що жадібний вибір на першому етапі не унеможливорює шляху до оптимального розв'язку: для будь-якого розв'язку є інший, узгоджений із жадібним і не гірший від першого. Далі доводиться, що підзадача, яка виникла після жадібного вибору на першому етапі, аналогічна початковій, і міркування закінчується за індукцією. Інакше кажучи, за жадібного алгоритму ніколи не переглядаються попередні вибори для здійснення наступного, на відміну від динамічного програмування.

Метод пошуку за першим найкращим збігом (жадібний пошук bestfirst search) — форма евристичного пошуку, що використовує оцінну функцію, за допомогою якої можна порівнювати стани в просторі станів. Кожному стану n відповідає значення оцінної функції $f = g(n) + h(n)$, де $g(n)$ — глибина стану n у просторі пошуку, оцінку якого отримує метод від завзятого проходження по невірному шляху, $h(n)$ — деяка евристична оцінка відстані від стану n до цілі, що веде метод пошуку до евристично найбільш перспективних станів. Метод порівнює не тільки ті стани, у які можливий перехід з поточного стану, але і всі, до яких можна дістати. Такий метод вимагає великих обчислювальних ресурсів, але його ідея полягає в тому, щоб брати до уваги не тільки найближчі стани, тобто локальну обстановку, а переглянути як можна більшу ділянку простору станів і бути готовим, у разі потреби, повернутися туди, де він уже був, і піти іншим шляхом, якщо найближчі претенденти не обіцяють істотного прогресу в досягненні мети. Саме ця можливість відмовитися від частини пройденого шляху заради глобальної мети і дозволяє знайти більш ефективний шлях

10.3. Основи подання знань

10.3.1 Факти, знання, властивості знань. Моделі подання знань: семантичні мережі, фрейми, логічні моделі, продукційні моделі.

Факт - подія, що відбулася в дійсності, певним чином зафіксована і істинність якої може бути перевірена чи не викликає сумніву в межах дискурсу.

Факт - фраза без умов, що містять твердження, які завжди є абсолютно вірними.

2 знання

Сукупність фактів, закономірностей, відношень та евристичних правил, що відображає рівень обізнаності з проблемами деяких предметних галузей

Знання – це формалізована система суджень із принциповою і єдиною організацією, заснована на об'єктивній закономірності, що спостерігається у визначеній предметній області (принципи, зв'язки, закони), встановленій в результаті розумової діяльності людини, спрямованої на узагальнення досвіду, отриманого нею у результаті практичної діяльності, яка дозволяє ставити і вирішувати задачі в цій області.

Властивості знань:

- **Автономність:** ступінь можливості використання знань незалежно від інших знань.
- **Активність:** здатність породжувати нові знання.
- **Актуальність:** відповідність знань поточному стану предметної галузі.
- **Відтворюваність та переносимість:** здатність знань бути відтворюваними та переносимими для їх використання.
- **Внутрішня інтерпретованість:** можливість змістовної інтерпретації знань без використання відповідної програми, що відрізняє їх від даних, які у відриві від програми не несуть ніякої змістовної інформації і можуть змістовно інтерпретуватися лише відповідною програмою.
- **Зв'язність:** Можливість встановлення між інформаційними одиницями зв'язків різного типу, які можуть мати як декларативний («одночасно», «причина - наслідок»), так і процедурний характер («аргумент - функція»).
- **Зрозумілість (логічна прозорість):** властивість знань бути зрозумілими для сприйняття
- **Ієрархічність:** властивість знань мати ієрархічну структуру, де загальні поняття перебувають у верхніх рівнях, а більш специфічні - на нижніх.
- **Контекстуальність:** залежність знань від контексту.
- **Мінливість (динамічність):** властивість знань змінюватися з часом.
- **Модульність:** можливість поділу знань на окремі блоки чи модулі, які можна незалежно використовувати один від іншого.
- **Надійність:** властивість, що визначає ступінь довіри до знань.
- **Об'єктивність (суб'єктивність):** ступінь незалежності/залежності знань від суб'єкта (особистого досвіду та поглядів експерта).
- **Повнота:** ступінь вичерпного охоплення знань певної предметної галузі, кількість засвоєної інформації, порівняно із загальним обсягом доступної інформації.
- **Релевантність:** ступінь відповідності знань для завдання, що розв'язується.
- **Розширюваність:** здатність знань розширюватися для адаптації до нових ситуацій та завдань.
- **Семантична метрика:** Це відношення, яке характеризує ситуаційну відстань, або силу асоціативного зв'язку між інформаційними одиницями. Його називають також відношенням релевантності для інформаційних одиниць. Відношення релевантності дозволяє знаходити знання близькі до тих, які вже знайдені.
- **Складність (Простота):** характеристика знань, що відображає кількість їх складових
- **Стійкість до помилок:** властивість знань бути стійкими до неточностей та помилок у даних.
- **Структурованість:** властивість знань мати структуру (вкладеність одних інформаційних одиниць в інші), класифікуючі відношення типу «частина - ціле», «рід - вид», «елемент - клас».
- **Сумісність:** властивість знань бути сумісними, щоб вони могли бути інтегровані в одну систему різних джерел.
- **Узгодженість (несуперечність):** властивість знань мають бути узгодженими, щоб уникнути протиріч та конфліктів між різними частинами знань.
- **Формалізованість:** властивість знань бути представленою у певній формі.
- **Чіткість:** ступінь ясності та визначеності інформації, якою володіє індивід або система.

3 подання знань

Сукупність методів, способів, форм і моделей структурування, відображення та формалізації знань

63 система подання знань

Сукупність методів та засобів опису, розміщення, зберігання та модифікації знань в інтелектуальних системах

64 модель подання знань; модель знань

Сукупність правил подання, опису та породження знань у базі знань

83 семантична мережа

Модель подання знань за допомогою мережі вузлів, зв'язаних дугами, де вузли відповідають поняттям чи об'єктам, а дуги - відношенням між вузлами

Операції модифікації бази знань на семантичних мережах зводяться до видалення і додавання нових вершин і ребер.

Пошук рішення в базі знань типу семантичної мережі зводиться до пошуку фрагмента мережі-бази знань, відповідного деякій підмережі, що відображає поставлений запит (**зразок**) до бази знань.

93 фрейм

Модель подання знань, яка під час заповнення її елементів-слотів певними значеннями перетворюється в опис конкретного факту, події, процесу

95 фрейм-прототип; прото-фрейм

Фрейм, у якого в частині слотів чи в усіх слотах відсутні константні значення

Табличне подання фрейму

Ім'я фрейму					
Ім'я слота	Показчик спадкування	Показчик типу	Значення слота	Приєднана процедура	
				Слуга	Демон
слот 1					
слот 2					
.....					
слот N					

Показчик спадкування показує, яку інформацію про атрибути слотів у фреймі верхнього рівня успадковують слоти з тими ж іменами у фреймі нижнього рівня.

Приєднані до слоту процедури (зв'язані процедури):

процедура-демон – це прихована або віртуальна приєднана процедура, яка автоматично виконується при наявності певних умов, при певних змінах бази знань, наприклад, коли не встановлене значення слоту, до якого відбувається звертання; коли значення слоту стирається і коли в слоті підставляється його значення. Процедури-демони активізуються при кожній спробі додавання чи видалення даних зі слоту (за замовчуванням).

процедура-слуги активізуються тільки при виконанні умов, визначених користувачем при створенні фрейму.

Пошук за зразком є основною операцією для фреймових моделей. **Зразок** являє собою фрейм, у якому заповнено не всі структурні одиниці, а тільки ті, заякими серед фреймів, що зберігаються в пам'яті ЕОМ, будуть відшукуватися потрібні фрейми.

Процес зіставлення – процес, у ході якого перевіряється правильність вибору фрейму. Звичайно цей процес здійснюється відповідно до поточної мети та інформації (значень), що міститься в даному фреймі. Іншими словами, фрейм містить умови, що обмежують значення слота, а мета використовується для визначення, яка з цих умов, маючи відношення до даної ситуації, є релевантною.

98 слот

Складова частина фрейму, яка повинна бути заповнена елементом даних визначеного типу

91 мережа фреймів

Модель подання знань про предметну галузь, що ґрунтується на використанні взаємопов'язаних фреймів

94 фрейм-екземпляр

Фрейм-прототип, у якого значення всіх слотів є константами

Формульне подання фрейму: $(f \{ \langle S \rangle, \langle v1, g1, [p1] \rangle, \langle v2, g2, [p2] \rangle, \dots, \langle vk, gk, [pk] \rangle \})$, де f – ім'я фрейму, S – слот з родовим або суперпоняттям, vi – ім'я слота, gi – значення слота, pi – процедура. Слоти $\langle vi, gi, [pi] \rangle$ описують властивості, що відрізняють даний об'єкт від родового поняття.

67. логічна модель знань; логічна модель

Модель подання знань, в основі якої лежить формальна система

Формальна система задається четвіркою виду $M = \langle T, P, A, B \rangle$, де T – множина базових елементів різної природи, P – множина синтаксичних правил, за допомогою яких з елементів T утворюють синтаксично правильні сукупності, у множині яких виділяється деяка підмножина A , елементи якої називаються аксіомами, B – множина правил виведення, застосовуючи які до елементів A , можна одержувати нові синтаксично правильні сукупності, до яких знову можна застосовувати правила з B . Для множини T існує деякий спосіб визначення належності чи неналежності довільного елемента до цієї множини. Процедура такої перевірки $\Pi(T)$ може бути будь-якою, але за кінцеве число кроків вона повинна давати позитивну чи негативну відповідь на питання, чи є x елементом множини T .

За допомогою процедури $\Pi(P)$ за кінцеве число кроків можна одержати відповідь на питання, чи є сукупність X синтаксично правильною.

За допомогою процедури $\Pi(A)$ для будь-якої синтаксично правильної сукупності можна одержати відповідь на питання про належність її до множини A .

Для знань, що входять у базу знань, можна вважати, що множина A утворює всі інформаційні одиниці, що введені в базу знань ззовні, а за допомогою правил виведення з них виводяться нові похідні знання. Іншими словами, формальна система являє собою генератор породження нових знань, що утворюють множину виведених у даній системі знань.

Логічні моделі, як правило, засновані на численні предикатів першого порядку, яке, у свою чергу, засновано на численні висловлень.

Висловлення – речення, зміст якого можна виразити значеннями: істина (1) або хибність (0).

Логічні зв'язування (пропозиціональні зв'язування) – k -арні логічні операції ($k \geq 0$), які перетворюють той чи інший набір з k висловлень у висловлення, а набір із пропозиціональних (висловлювальних) форм від деяких змінних (тобто виразів, що містять змінні і перетворюються у висловлення при заміні змінних конкретними висловленнями) – у пропозиціональну форму від цих змінних.

Числення висловлень – логічне числення, яке визначає логічні закони, що характеризують логічні зв'язування. Воно характеризується двозначною інтерпретацією і в ньому доказові всі тотожньо-істинні формули даного числення і тільки вони. Числення висловлень входять як частина до числення предикатів.

Предикат від n змінних (n -арний предикат) на множині A – n -місцева функція, визначена на множині A зі значеннями в множині {істина, хибність}. Під предикатом розуміють деякий зв'язок, що задано на наборі змінних.

Логіка предикатів першого порядку – розділ математичної логіки, що вивчає логічні закони, загальні для будь-якої непорожньої області об'єктів із заданими в цій області предикатами. Ці закони формулюються в спеціальній формальній логічній мові першого порядку, основними синтаксичними одиницями якої є константи, змінні, функції, предикати, квантори і логічні зв'язування.

81 продукція

Правило, яке визначає певні ситуації та відповідні їм дії

Продукція - вираз виду:

$(i); Q; P; A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_k; N$,
де i – ім'я продукції, у якості котрого може виступати деяка лексема, що відбиває суть даної продукції або її порядковий номер, Q – елемент, що характеризує сферу застосування продукції (визначає для яких випадків може бути застосована продукція, тобто задає множину елементів для якої продукція є застосовною), $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_k$ – ядро продукції, « $\rightarrow$ » – знак секвенції, A_i – i -та передумова (умова) правила, B_j – j -ий висновок (наслідок, дія) правила, P – умова застосовності ядра продукції (визначає при якій умові продукція може бути виконана), N – постумови продукції (описують дії і процедури, які необхідно виконати після реалізації наслідків продукції й актуалізуються тільки в тому випадку, якщо ядро продукції реалізувалося).

Ядро продукції складається з двох частин:
– **антецедент** – комбінація умов правила (припущень про наявність деяких властивостей, що приймають значення істина або хибність з визначеним ступенем вірогідності), з'єднаних логічними зв'язуваннями (ТА, АБО і т. д.), призначена для розпізнання ситуації, коли правило повинне спрацювати;
– **консеквент** (висновок, наслідки правила)
– містить опис дій, що повинні бути виконані над робочою пам'яттю у випадку виконання відповідних умов.

79 продукційна система

Система, в якій знання подано у вигляді сукупності продукцій та правил їх застосування

Продукційна система складається з **продукційної пам'яті** (production memory) – бази знань у вигляді продукційних правил, **машини логічного виведення**, що послідовно визначає, які продукції можуть бути активовані в залежності від умов, у них що містяться, вибирає одне з застосовних у даній ситуації правил продукцій і виконує дії для обраного правила, а також **робочої пам'яті**, що містить дані (факти), опис мети і проміжні результати, що у сукупності визначають поточний стан проблеми.

Конкретизація правила – сукупність узагальненого формулювання правила і значень використовуваних у ньому змінних.

Активізоване (реалізоване) – правило, всі умови якого задоволено. Всі активізовані правила заносяться до робочого списку правил.

Конфлікти правил виникають у робочому списку правил, якщо одночасно активізується кілька правил.

Конфліктний набір правил (конфліктна множина, conflict set) – набір конкретизованих правил, які активізовані протягом одного циклу обчислень.

Припустимі продукції – продукції, що містяться в конфліктній множині.

Метод роботи машини логічного виведення.

1. **Ініціалізація:** робоча пам'ять ініціалізується зразком – початковим описом задачі в у робочій пам'яті.

2. **Цикл «розпізнавання – дія».**

2.1 **Ідентифікація**– порівняння умов з лівих частин продукцій з фактами робочої пам'яті, що призводить до конкретизації й активізації правил.

2.2 **Вирішення конфліктів правил** – вибір зі сформованого списку заявок (конфліктної множини) єдиного правила, яке має бути виконане в поточній ситуації.

2.3 **Дія** (запуск правил) – зміна вмісту робочої пам'яті за допомогою послідовного виконання дій, зазначених у правій частині активізованого правила, обраного з робочого списку правил. Після виконання дії, відповідне правило видаляється з робочого списку правил або виконується його релаксація.

2.4 **Перевірка умов зупинення.** Якщо хоча б одна з умов зупинення задовольняється, тоді закінчити роботу і повернути рішення, якщо воно знайдено, у противному випадку – повторити кроки 2.1–2.4.

Умови зупинення виділяють такі: переривання роботи користувачем; вичерпання всіх правил із продукційної пам'яті; виконання деякої умови, якій задовольняє вміст робочої пам'яті; невідповідність вмісту робочої пам'яті ніяким умовам.

10.4. Машинне навчання

10.4.1. Навчання з вчителем та без, навчання з підкріпленням, регресійні і класифікаційні задачі.

Навчання - процес, за допомогою якого система покращує свої функціональні характеристики шляхом набуття нових знань чи досвіду або шляхом реорганізації існуючих знань та досвіду.

Навчання з учителем - процес встановлення асоціацій між вибраними значеннями входу і коректними значеннями виходу, заданими учителем.

Навчання без учителя (самонавчання) - процес, через який система вдосконалює своє виконання за допомогою регулювання своїх параметрів у відповідь на послідовність шаблонів входу без указівок учителя.

Самозорганізованість - здатність системи, яка здійснює неконтрольоване самонавчання, регулювати параметри на основі шаблонів входу.

Навчання з підкріпленням - це галузь машинного навчання, що вивчає питання про те, які дії повинні виконувати програмні агенти в певному середовищі задля максимізації деякого уявлення про сукупну винагороду.

Завдання регресії - прогнозування безперервної змінної шляхом вивчення функції, яка найкраще відповідає набору навчальних даних.

У задачі регресії навчена регресійна модель представляє спеціальний простір. Коли навчена модель застосовується до нового екземпляра даних, екземпляр проектується в настроюваний простір, визначений навченою моделлю регресії.

Регресія в основному використовується для прогнозування числових значень процесу реального світу на основі попередніх вимірювань або спостережень того самого процесу.

Регресійна модель - модель машинного навчання, очікуваний вихід для даного вхідного параметра є безперервною змінною.

Ознака (атрибут) – вимірювана властивість об'єкта або події.

Клас - визначена людиною, категорія елементів, які є частиною набору даних і мають спільні атрибути.

Кластер - автоматично створена категорія елементів, які є частиною набору даних і мають спільні атрибути.

Завдання класифікації - передбачення віднесення екземпляра вхідних даних до визначеної категорії або класу. Класифікація може бути двійковою (тобто істинною чи хибною), багатокласовою (тобто однією з кількох можливостей) або багатозначною (тобто будь-яке число з кількох можливостей). Класи, як правило, складаються з дискретної та невпорядкованої множини, так що проблема не може бути формалізована як завдання регресії.

Модель класифікації - модель машинного навчання, очікуваним виходом якої для даного входу є один або більше класів.

10.4.2. Лінійна і логістична регресія: ідентифікація, регуляризація, сфера застосування. Вибір тренувальних та валідаційних даних для навчання.

Регресія - форма зв'язку між випадковими величинами. Закон зміни математичного сподівання однієї випадкової величини залежно від значень іншої.

Крива регресії Y на X - це залежність умовного математичного сподівання величини Y від заданого значення X

Загальна лінійна регресійна модель:

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_K x_K + u,$$

де y — залежна пояснювана змінна, $(x_1, x_2, \dots)$ — незалежні пояснювальні змінні, u — випадкова похибка, розподіл якої в загальному випадку залежить від незалежних змінних, але математичне сподівання якої дорівнює нулеві.

Вектор параметрів $(\beta_0, \beta_1, \dots, \beta_K)$ є невідомим і задача лінійної регресії полягає у пошуку цих параметрів на основі деяких експериментальних значень y і $(x_1, x_2, \dots, x_K)$. Згідно з означенням моделі для кожного експериментального випадку залежність між змінними визначається як

$y_i = \beta_0 + \beta_1 x_{1,i} + \dots + \beta_K x_{K,i} + u_i$, $i = 1, 2, \dots, n$, де n — кількість експериментів (екземплярів), або, у матричних позначеннях: $y = X\beta + u$.

Оцінювання параметрів лінійної регресії здійснюють **методом найменших квадратів**, який приймає за оцінку параметра значення, що мінімізують суму квадратів залишків по всіх спостереженнях:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left| y_i - \beta_0 - \sum_{j=1}^K X_{ij} \beta_j \right|^2 = \arg \min_{\beta} \|y - X\beta\|^2.$$

$$\hat{\beta} = (X'X)^{-1} X'y.$$

Проста лінійна регресія - лінійна регресійна модель з однією незалежною змінною.

Припустимо, що є n точок $\{(x_i, y_i), i = 1, \dots, n\}$. Функція, яка описує зв'язок x і y записується: $y_i = \alpha + \beta x_i + \varepsilon_i$. Завдання полягає в тому, щоб знайти рівняння прямої лінії $y = \alpha + \beta x$, яка б забезпечувала найкраще в сенсі найменшого квадратичного відхилення допасування наявних точок даних (лінії, що мінімізує суму квадратів похибок лінійної регресійної моделі). Оцінювання параметрів методом найменших квадратів:

$$\hat{\alpha} = \bar{y} - \hat{\beta} \bar{x},$$

$$\hat{\beta} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2},$$

$\bar{x}, \bar{y}$ - середні значення x та y , відповідно.

Логістична регресія - статистичний регресійний метод, що застосовують у випадку, коли залежна змінна є бінарною, тобто може набувати тільки двох значень (0 чи 1).

Нехай є деяка випадкова величина Y , що може набувати лише двох значень, які, як правило, позначаються цифрами 0 і 1. Нехай ця величина залежить від деякої множини пояснювальних змінних $x = (1, x_1, \dots, x_n)^T$. Залежність Y , від $x_1, \dots, x_n$. можна визначити ввівши додаткову змінну y^* , де $y^* = \theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n + \varepsilon$.
Тоді:

$$Y = \begin{cases} 0, & y^* \leq 0 \\ 1, & y^* > 0 \end{cases}$$

При визначенні логістичної моделі стохастичний доданок ε вважається випадковою величиною з логістичним розподілом ймовірностей

Навчальна вибірка даних – вибірка, що використовується для навчання (побудови) моделі. Навчальна вибірка має бути **репрезентативною** стосовно генеральної сукупності, тобто відбивати її основні властивості та топологію класів.

Тестова вибірка даних – вибірка, що використовується для перевірки побудованої моделі.

Валідаційна вибірка даних – вибірка, що використовується для перевірки моделі під час та після навчання для налаштування параметрів моделі.

Оцінка параметрів $\theta_0, \theta_1, \dots, \theta_n$ на основі деякої вибірки $(x^{(1)}, Y^{(1)}), \dots, (x^{(m)}, Y^{(m)})$, де $x^{(i)} \in \mathbb{R}^n$ – вектор значень незалежних змінних, а $Y^{(i)} \in \{0, 1\}$ – відповідне їм значення Y , як правило здійснюється за допомогою методу максимальної правдоподібності, згідно з яким вибираються параметри θ , що максимізують значення функції правдоподібності на вибірці:

$$\hat{\theta} = \operatorname{argmax}_{\theta} L(\theta) = \operatorname{argmax}_{\theta} \prod_{i=1}^m \Pr\{Y = Y^{(i)} | x = x^{(i)}\}.$$

Максимізація функції правдоподібності еквівалентна максимізації її логарифма:

$$\log L(\theta) = \sum_{i=1}^m \log \Pr\{Y = Y^{(i)} | x = x^{(i)}\} = \sum_{i=1}^m Y^{(i)} \log \Lambda(\theta^T x^{(i)}) + (1 - Y^{(i)}) \log(1 - \Lambda(\theta^T x^{(i)}))$$

Для максимізації цієї функції може бути застосований, наприклад, метод градієнтного спуску, метод Ньютона чи стохастичний градієнтний спуск.

Тестова та валідаційна вибірки мають складатися з даних, які ніколи не показувалися моделі під час навчання. Якщо для цих цілей використовувати навчальні дані, то модель може «запам'ятати» правильний прогноз без фактичної обробки вибірки даних. Щоб уникнути переоцінки якості моделі, тестові дані також не повинні показуватися моделі під час налаштування її параметрів.

Під час використання перехресної перевірки (кросвалідація) дані поділяються таким чином, що кожна вибірка даних використовувалася як для навчання, так і для перевірки. Цей підхід емулює використання більшого набору даних, що може покращити якість моделі. Іноді недостатньо даних, щоб мати окремі набори для навчання, перевірки та тестування. У таких випадках дані поділяються лише на два набори, а саме: дані навчання або перевірки та дані тестування. Окремі вибірки даних для валідації та навчання генеруються з даних навчання або валідації, наприклад, за допомогою бутстрепа або перехресної перевірки (кросвалідації).

10.4.3. Поняття: штучний нейрон, штучна нейронна мережа, функції активації штучного нейрона

34.01.07 штучний нейрон

Найпростіший елемент оброблення *нейронної мережі* з кількома входами й одним виходом, значенням виходу якого є нелінійна функція лінійної комбінації значень входу зі зваженими коефіцієнтами.

Примітка 1. Штучні нейрони змодельовані згідно з функціонуванням нейронів нервової системи і сполучені між собою для того, щоб обмінюватися повідомленнями.

Примітка 2. Кожен штучний нейрон — це *вузол* нейронної мережі*, який кооперується і взаємодіє з іншими нейронами. Нейронна мережа може також мати вузли входу, які не є штучні нейрони

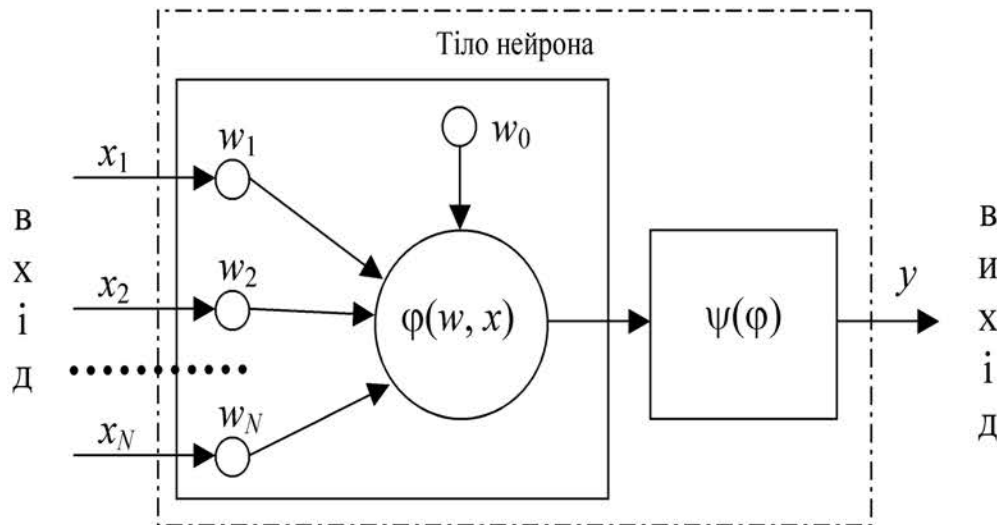


Рисунок – Схема штучного (формального) нейрона

Модель функціонування штучного нейрона:

$$y = \psi (\varphi(w, x)),$$

де x – вектор вхідних аргументів (сигналів);
 y – значення на виході нейрона;

ψ – функція активації;

φ – дискримінантна функція;

$w = \{w_j\}$ – вектор, що містить значення вагових коефіцієнтів w_j і значення зсуву (порогове значення) w_0 . (пам'ять нейрона)

Дискримінантні (вагові, постсинаптичні) функції нейрона:

– *зважена сума*: $\varphi(w, x) = \sum_{j=1}^N w_j x_j + w_0$;

– *зважений добуток*: $\varphi(w, x) = \prod_{j=1}^N w_j x_j = w_0 \prod_{j=1}^N x_j$;

– *евклідова відстань*: $\varphi(w, x) = \sum_{j=1}^N (w_j - x_j)^2$.

34.03.13 функція активування

Функція, яка обчислює значення *виходу** *штучного нейрона* на основі значень його входу і його поточних вагів зв'язків.

Функція активації $\psi(x)$ повинна бути

обмеженою (на інтервалі значень $x \in (-\infty, a]$ приймати значення y^0 , на інтервалі $x \in [b, +\infty)$ – приймати значення y^1 , на інтервалі $x \in (a, b)$ приймати значення y : $y^0 \leq y \leq y^1$, де y^0 та y^1 – деякі постійні мінімальне та максимальне значення, а та b – деякі константи, причому: $a \leq b$)

і монотонною (на інтервалі $x \in (a, b)$ $\Delta\psi(x) = \psi(x+\Delta x) - \psi(x)$) і не повинна змінювати знак при $\Delta x > 0$ та $x, x+\Delta x \in (a, b)$).

Типові функції активування (активації):

– **лінійна** приймає значення в діапазоні $(-\infty; +\infty)$:

$\psi(x) = cx$, де c – константа, як правило, $c = 1$;

– **лінійна біполярна з насиченням:**

$$\psi(x) = \begin{cases} 1, & x > a_2; \\ Kx, & a_1 \leq x \leq a_2; \\ -1, & x < a_1, \end{cases}$$

де K, a_1, a_2 – константи;

– **лінійна уніполярна з насиченням:**

$$f(x) = \begin{cases} 1, & x \geq \frac{1}{2a}; \\ ax + 0,5, & |x| < \frac{1}{2a}; \\ 0, & x \leq -\frac{1}{2a}, \end{cases}$$

де a – константа.

– **порогова (Хевісайда)** приймає значення в діапазоні $[0; 1]$:

$$\psi(x) = \begin{cases} 1, & x \geq 0; \\ 0, & x < 0; \end{cases}$$

– **порогова знакова (біполярна)** приймає значення в діапазоні $[-1; 1]$:

$$\psi(x) = \begin{cases} 1, & x \geq 0, \\ -1, & x < 0; \end{cases}$$

– **сигмоїдна логістична (уніполярна)** приймає значення в діапазоні $(0; 1)$:

$$\psi(x) = \frac{1}{1 + e^{-cx}},$$

де c – константа, як правило, $c = 1$.

– **гіперболічний тангенс (біполярна):**

$$\psi(x) = \tanh(Kx) = \frac{e^{Kx} - e^{-Kx}}{e^{Kx} + e^{-Kx}},$$

де K – константа.

– **косинусоїдальна з насиченням (уніполярна):**

$$\psi(x) = \begin{cases} 1, & x \geq \frac{\pi}{2}; \\ \frac{1}{2} \left(1 + \cos \left(x - \frac{\pi}{2} \right) \right), & |x| < \frac{\pi}{2}; \\ 0, & x \leq -\frac{\pi}{2}; \end{cases}$$

– **синусоїдальна з насиченням (біполярна):**

$$\psi(x) = \begin{cases} 1, & x \geq a; \\ \sin x, & |x| < a; \\ -1, & x \leq -a, \end{cases}$$

де a – константа.

– **радіально-базисна (Гауса)** приймає значення в діапазоні $(0; +\infty)$:

$$\psi(x) = e^{-cx^2},$$

де c – константа;

– **winner-take-all** (WTA – переможець отримує усе):

$$\psi^{(\mu,i)}(\{\varphi^{(\mu,j)}\}) = \begin{cases} 1, & \varphi^{(\mu,i)} \leq \min_j \{\varphi^{(\mu,j)}\}; \\ 0, & \text{інакше.} \end{cases}$$

– **ReLU** (Rectified Linear Units – ненасичуваний лінійний модуль) визначається формулою:

$$\psi(\varphi) = \max(0, \varphi).$$

Функція **Softmax** (нормована експоненційна функція) – це узагальнення логістичної функції, що "стискує" K -вимірний вектор $\mathbf{z}$ із довільним значеннями компонент до K -вимірного вектора $\sigma(\mathbf{z})$ з дійсними значеннями компонент в області $[0, 1]$ що в сумі дають 1:

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, j = 1, \dots, K.$$

Наступна таблиця містить передавальні функції від декількох змінних:

Назва	Рівняння	Похідна(ні)	Область
Softmax	$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}$ for $i = 1, \dots, J$	$\frac{\partial f_i(\vec{x})}{\partial x_j} = f_i(\vec{x})(\delta_{ij} - f_j(\vec{x}))$ ^[7]	$(0, 1)$
Maxout ^[26]	$f(\vec{x}) = \max_i x_i$	$\frac{\partial f}{\partial x_j} = \begin{cases} 1 & \text{for } j = \operatorname{argmax}_i x_i \\ 0 & \text{for } j \neq \operatorname{argmax}_i x_i \end{cases}$	$(-\infty, \infty)$

↑ Тут, δ_{ij} – символ Кронекера.

Назва	Графік	Рівняння	Похідна (по x)	Область
Тотожна		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Двійковий крок		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$
Логістична (a.k.a. Сігмоїда або М'який крок)		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$ [1]	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$
TanH		$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$	$\left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$
Softsign [10][11]		$f(x) = \frac{x}{1 + x }$	$f'(x) = \frac{1}{(1 + x)^2}$	$(-1, 1)$
Inverse square root unit (ISRU) [12]		$f(x) = \frac{x}{\sqrt{1 + \alpha x^2}}$	$f'(x) = \left(\frac{1}{\sqrt{1 + \alpha x^2}} \right)^3$	$\left(-\frac{1}{\sqrt{\alpha}}, \frac{1}{\sqrt{\alpha}}\right)$
Випрямлена лінійна (Rectified linear unit, ReLU) [13]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$
Leaky rectified linear unit (Leaky ReLU) [14]		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$

SoftExponential [24]		$f(\alpha, x) = \begin{cases} \frac{-\ln(1 - \alpha(x + \alpha))}{\alpha} & \text{for } \alpha < 0 \\ x & \text{for } \alpha = 0 \\ \frac{e^{\alpha x} - 1}{\alpha} + \alpha & \text{for } \alpha > 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \frac{1}{1 - \alpha(x + \alpha)} & \text{for } \alpha < 0 \\ 1 & \text{for } \alpha = 0 \\ e^{\alpha x} & \text{for } \alpha > 0 \end{cases}$	$(-\infty, \infty)$
Синусоїда [25]		$f(x) = \sin(x)$	$f'(x) = \cos(x)$	$[-1, 1]$
Sinc		$f(x) = \begin{cases} 1 & \text{for } x = 0 \\ \frac{\sin(x)}{x} & \text{for } x \neq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x = 0 \\ \frac{\cos(x)}{x} - \frac{\sin(x)}{x^2} & \text{for } x \neq 0 \end{cases}$	$[\approx -0.217234, 1]$
Гауссіан		$f(x) = e^{-x^2}$	$f'(x) = -2xe^{-x^2}$	$(0, 1]$

↑ Тут, H це функція Гевісайда.

↑ α є стохастичною змінною вибраною з нормального розподілу під час навчання і зафіксована як очікуване значення розподілу до часу тестування.

↑ ↑ ↑ Тут, σ — логістична функція.

↑ $\alpha > 0$ виконується для всього інтервалу.

Parametric rectified linear unit (PReLU) [15]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$ [2]
Randomized leaky rectified linear unit (RReLU) [16]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ [3]	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Exponential linear unit (ELU) [17]		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(\alpha, x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Scaled exponential linear unit (SELU) [18]		$f(\alpha, x) = \lambda \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ з $\lambda = 1.0507$ та $\alpha = 1.67326$	$f'(\alpha, x) = \lambda \begin{cases} \alpha(e^x) & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\lambda\alpha, \infty)$
S-shaped rectified linear activation unit (SReLU) [19]		$f_{t_l, a_l, t_r, a_r}(x) = \begin{cases} t_l + a_l(x - t_l) & \text{for } x \leq t_l \\ x & \text{for } t_l < x < t_r \\ t_r + a_r(x - t_r) & \text{for } x \geq t_r \end{cases}$ t_l, a_l, t_r, a_r are parameters.	$f'_{t_l, a_l, t_r, a_r}(x) = \begin{cases} a_l & \text{for } x \leq t_l \\ 1 & \text{for } t_l < x < t_r \\ a_r & \text{for } x \geq t_r \end{cases}$	$(-\infty, \infty)$
Inverse square root linear unit (ISRLU) [12]		$f(x) = \begin{cases} \frac{x}{\sqrt{1 + \alpha x^2}} & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \left(\frac{1}{\sqrt{1 + \alpha x^2}} \right)^3 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$\left(-\frac{1}{\sqrt{\alpha}}, \infty\right)$
Adaptive piecewise linear (APL) [20]		$f(x) = \max(0, x) + \sum_{s=1}^S a_s^* \max(0, -x + b_s^*)$	$f'(x) = H(x) - \sum_{s=1}^S a_s^* H(-x + b_s^*)$ [4]	$(-\infty, \infty)$
SoftPlus [21]		$f(x) = \ln(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$	$(0, \infty)$
Bent identity		$f(x) = \frac{\sqrt{x^2 + 1} - 1}{2} + x$	$f'(x) = \frac{x}{2\sqrt{x^2 + 1}} + 1$	$(-\infty, \infty)$
Sigmoid-weighted linear unit (SiLU) [22] (a.k.a. Swiss) [23]		$f(x) = x \cdot \sigma(x)$ [5]	$f'(x) = f(x) + \sigma(x)(1 - f(x))$ [6]	$[\approx -0.28, \infty)$

штучна нейронна мережа, ШНМ

Мережа найпростіших елементів оброблення, пов'язаних зваженими зв'язками зі змінними вагами, в якій кожен елемент виробляє значення, застосовуючи нелінійну функцію до значень його входу, і передає це значення іншим елементам або подає його як *вихідне значення*.

34.02.01 початковий вузол; джерельний вузол

Вузол *нейронної мережі*, який постачає *вхідні сигнали* у *штучні нейрони*

34.02.02 вхідний нейрон

Штучний нейрон, який отримує *сигнали* від зовнішніх джерел

34.02.03 вихідний нейрон

Штучний нейрон, який посиляє *сигнали* зовнішній системі

34.02.04 видимий нейрон

Штучний нейрон, який безпосередньо взаємодіє із зовнішніми системами.

Примітка. Видимий нейрон може бути *вхідним нейроном*, *вихідним нейроном*, або обома

34.02.05 прихований нейрон

Штучний нейрон, який має змогу взаємодіяти із зовнішніми системами лише опосередковано

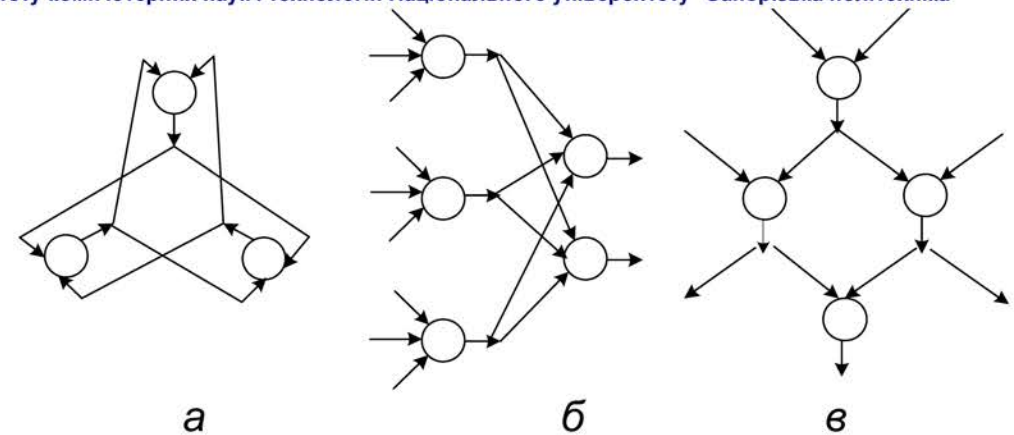


Рисунок – Нейромережі з різною топологією зв'язків:
а – повнозв'язна б – шарувата; в – слабозв'язана

34.03.01 нейронне з'єднання; нейронний зв'язок; *синаптичний взаємозв'язок* (НД); *синаптичний зв'язок* (НД)

Зв'язок між двома *штучними нейронами*, який визначають через нейрон-джерело, нейрон-адресата і вагу зв'язку

34.03.02 вага зв'язку (у нейронних мережах); сила зв'язку; *синаптична вага* (НД)

Коефіцієнт, на який множать значення *входу штучного нейрона*, перш ніж воно комбінується з іншими *вхідними значеннями*

для вступників до магістратури

Освітні програми: "Інженерія програмного забезпечення", "Системи штучного інтелекту"



Національний університет "Запорізька політехніка" ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК І ТЕХНОЛОГІЙ

КАФЕДРА ПРОГРАМНИХ ЗАСОБІВ

Завідувач кафедри:

СУББОТИН Сергій Олександрович
III корпус, 5 поверх, ауд. 53-г
моб. тел. +38 067 394 11 80
email: subbotin@zp.edu.ua

Сайти кафедри:

<https://pz.zp.ua>

 <https://cutt.ly/AGKP8Fi>

 <https://fb.com/groups/pz.zntu/>

 https://t.me/pz_zntu

Деканат факультету:

вул. Жуковського, 64
I корпус, 3 поверх, ауд. 373
тел.: (061) 764-33-61



Приймальна комісія:

тел.: (061) 769-82-26

сайт: <https://pk.zp.edu.ua/>

Спеціальності:

121 «Інженерія програмного
забезпечення»

122 «Комп'ютерні науки»

Освітні рівні:



доктор філософії (PhD)

магістр (M.Sc.)

бакалавр (B.Sc.)